design-develop-database-solutions

Design and implement database objects with SQL

This module covers designing and implementing various database objects including tables with different data types, specialized table types, indexes, constraints, and partitioning strategies. You'll learn how to create and optimize database objects for modern SQL platforms.

Introduction

You'll learn how to design and implement various database objects across SQL Server, Azure SQL Database, Azure SQL Managed Instance, and SQL Database in Microsoft Fabric. Proper database object design is fundamental to building high-performance, scalable, and maintainable SQL solutions across these platforms.

As a SQL Developer you probably noticed that database object design decisions are far more permanent than application code. While you can refactor a C# class or rewrite a microservice with minimal impact, changing a table from rowstore to columnstore, retrofitting temporal history tracking, or switching from identity column to sequence objects requires migrations that can lock tables for hours and disrupt production systems.

The specialized object types you'll learn in this module aren't just performance optimizations you can add later. They fundamentally change how data is stored, queried, and validated at the engine level. Choosing a standard table when you need temporal auditing means manually building triggers and history tables. Selecting IDENTITY when your architecture needs distributed sequences forces workarounds in your application tier.

Understanding these objects upfront lets you design systems that can evolve without painful rewrites, enabling capabilities like blockchain-style verification, millisecond-latency caching, or real-time analytics that can't be easily changed once you've committed to a different foundation.

What you'll learn

You'll explore database object design techniques that apply across Azure SQL Database, SQL Database in Microsoft Fabric, and Azure SQL Managed Instance:

- **Table design and implementation** - Creating tables with appropriate data types, sizes, and structures. Learn how to choose between rowstore and

columnstore indexes for your workload, whether you're building a transactional app on Azure SQL Database or an operational analytics database in Fabric.

- **Specialized table types** - Using in-memory tables for high-throughput scenarios in SQL Managed Instance, temporal tables for audit trails across all platforms, external tables for Fabric lakehouse integration, LEDGER tables for compliance-critical applications, and GRAPH tables for complex relationships.
- **Constraints and validation** - Implementing primary keys, foreign keys, unique constraints, CHECK constraints, and DEFAULT values that ensure data integrity whether your database serves a microservice, an enterprise application, or feeds analytics pipelines.
- **Advanced features** - Working with JSON columns for flexible schemas in cloud-native apps, implementing indexes optimized for your platform's query engine, and using SEQUENCE objects for distributed ID generation patterns.
- **Partitioning strategies** - Designing and implementing table and index partitioning for large-scale databases. Essential for Hyperscale databases in Azure SQL Database, multi-TB databases in SQL Managed Instance, and time-series data in Fabric operational databases.

Why this matters

Effective database object design directly impacts:

- **Performance** - Well-designed tables and indexes reduce query execution times
- **Data integrity** - Proper constraints ensure data consistency and accuracy
- **Maintenance** - Organized object design simplifies database administration
- **AI capabilities** - Proper data structures enable AI feature integration
- **Scalability** - Partitioning enables handling of large datasets efficiently

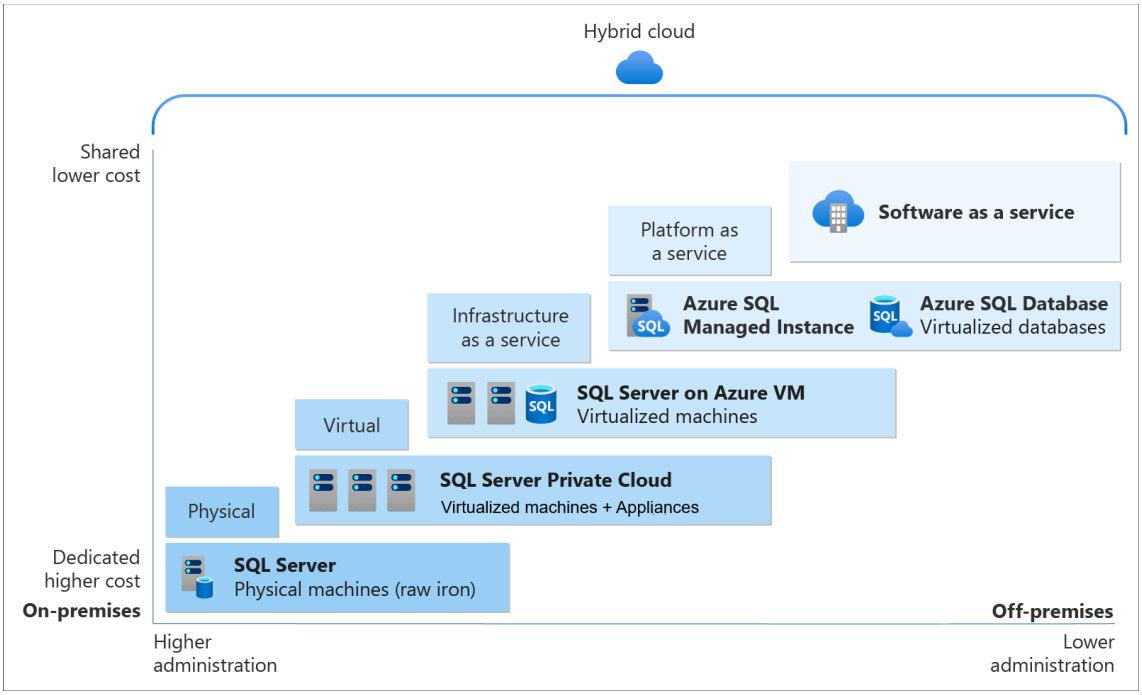
Understand your SQL server-based platform choices

Completed100 XP

- 6 minutes

Choose your preferred content format

Microsoft's SQL platforms serve different scenarios, and the database objects you design must align with your platform's capabilities and use cases. Understanding the control and management trade-offs between Infrastructure-as-a-Service (IaaS) and Platform-as-a-Service (PaaS) helps determine which platform best supports your database design requirements.



The diagram shows how PaaS platforms divide responsibilities: Azure manages everything below the database layer—physical servers, networking, operating system patches, and engine updates—while you control what matters most to your application: tables, indexes, constraints, and data. This separation lets you invest your time in database design rather than infrastructure maintenance.

Develop using Azure SQL Database

Azure SQL Database is a fully managed PaaS database that provides enterprise-grade performance and availability without infrastructure management. Multiple service tiers support different workload patterns, each influencing how you architect your data layer.

The Hyperscale service tier eliminates many of the practical limitations traditionally associated with cloud databases. The resources of a single node constrain most databases, but Hyperscale databases have no such restrictions. With its flexible storage architecture, storage expands as needed, and there's no predefined maximum size. You're billed only for the capacity you use. For read-intensive workloads, Hyperscale offers rapid scale-out by provisioning more replicas to handle read operations.

The serverless compute tier automatically scales compute based on workload demand and pauses when idle—you pay only for storage during inactive periods. When a connection request is made, the database resumes automatically.

Note

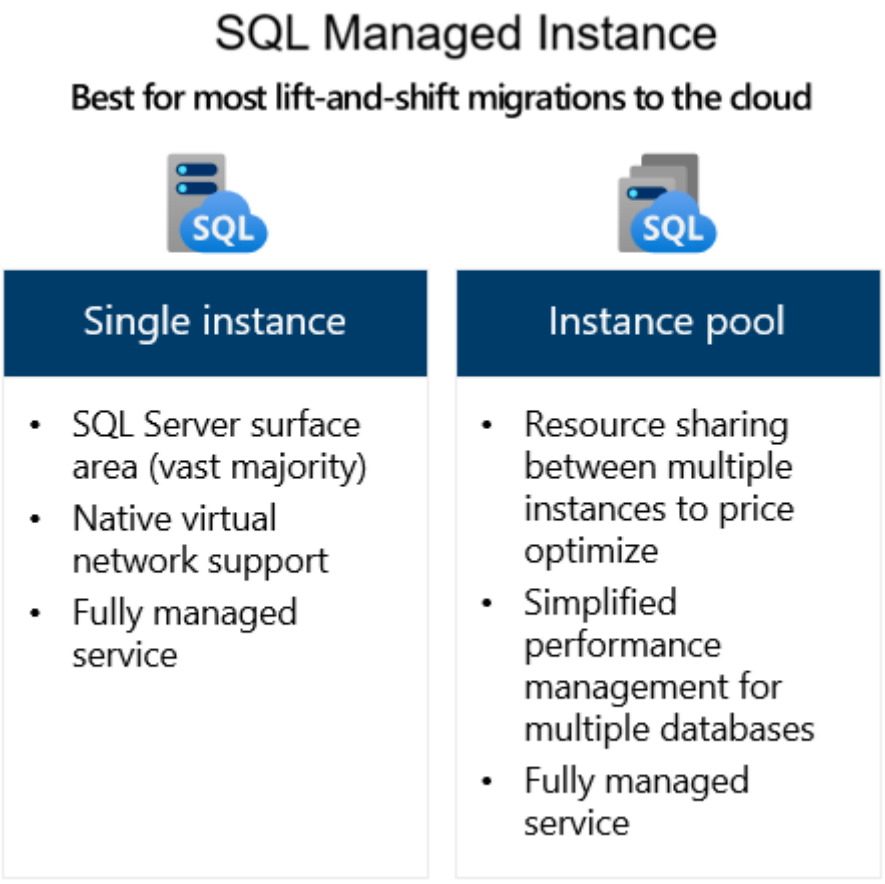
We recommend you design your application with connection retry logic to handle resume delays, and avoid long-running transactions that prevent autopause.

Intelligent query processing and automatic tuning analyze workload patterns to recommend or automatically create indexes. Automatic plan correction detects and fixes query regressions when proper indexing and statistics are in place.

Built-in high availability with a 99.99% uptime Service Level Agreement (SLA) means you can focus on performance and data integrity rather than replication topology.

Migrate for Azure SQL Managed Instance

Azure SQL Managed Instance provides near 100% compatibility with the latest SQL Server Enterprise Edition, always running the most recent database engine version with automatic patching. Native virtual network integration provides security isolation, while PaaS capabilities handle backups, high availability, and maintenance.



Instance-level features include SQL Server Agent, Service Broker, linked servers, cross-database queries with three-part naming, and database mail. The Managed Instance link uses distributed availability groups to synchronize data from SQL Server to Azure in near real-time—enabling hybrid scenarios, read offloading, disaster recovery, and minimal-downtime migrations.

In-Memory OLTP in the Business Critical tier enables memory-optimized tables and natively compiled stored procedures for latency-sensitive workloads.

Use SQL Server on Azure Virtual Machines

SQL Server on Azure Virtual Machines provides Infrastructure-as-a-Service (IaaS) deployment where you control the SQL Server instance, database engine configuration, and underlying Windows or Linux operating system. This deployment option offers maximum compatibility and customization for applications requiring specific SQL Server versions, OS-level access, or configurations not available in PaaS offerings.

The SQL IaaS Agent extension unlocks management capabilities including automated backups, automatic patching during maintenance windows, Azure Key Vault integration, and tempdb configuration through the Azure portal. The SQL best practices assessment validates your configuration against recommended settings,

while I/O performance analysis helps identify storage bottlenecks. For high availability, you can configure Always On availability groups or failover cluster instances with full control over replica placement and failover behavior.

Design for SQL Database in Microsoft Fabric

SQL database in Microsoft Fabric is a developer-friendly transactional database built on Azure SQL Database technology that automatically integrates with Fabric's analytics ecosystem. The platform uses the same SQL Database Engine as Azure SQL Database, combining OLTP capabilities with built-in analytics integration and eliminating the traditional separation between operational and analytical data stores.

Automatic mirroring replicates changes from your operational tables into OneLake as Delta Parquet files. As you insert, update, and delete data, Fabric automatically synchronizes those changes without requiring ETL pipelines, triggers, or extra configuration. This means every table you create instantly becomes available for analytics through the SQL analytics endpoint, which provides a read-only analytical view of your data. You can query across multiple data sources using familiar three-part naming syntax to join your SQL database with other Fabric warehouses, lakehouses, and even other SQL databases in cross-database queries. The key benefit: your analytical queries run against the Delta Parquet copies rather than your live operational tables, so heavy reporting workloads never slow down your transaction processing.

Intelligent performance features work automatically in the background, including automatic index creation that monitors query patterns and creates indexes without manual intervention. The platform also supports AI development with semantic search and retrieval-augmented generation (RAG). Database portability is supported through SqlPackage for .bacpac/.dacpac operations, Fabric source control for git integration, and GraphQL APIs for modern API interfaces.

Throughout this module, you'll learn techniques applicable across all platforms, with callouts for platform-specific capabilities.

Build effective tables

Completed100 XP

- 8 minutes

Choose your preferred content format

Effective table design forms the foundation of any database. Tables structure your data and determine how efficiently your queries access and modify information.

Design and create tables

Tables are the fundamental building blocks of relational databases, organizing data into rows and columns that represent entities and their attributes. In relational systems, tables define the structure for storing transactional data, enforce relationships through foreign keys, and provide the foundation for queries and reporting.

For multidimensional analytics, tables serve as *fact tables* storing measurable events and *dimension tables* providing context for analysis. The design decisions you make when creating tables—data types, column sizing, constraints, and relationships—directly impact storage efficiency, query performance, data integrity, and scalability across both operational and analytical workloads.

Choose appropriate data types

Data types are fundamental decisions that affect your database. The wrong choice can lead to wasted storage, poor performance, data loss, or application errors. Unlike application code that you can refactor easily, changing column data types in production databases often requires table rebuilds, which can mean hours of downtime for large tables.

Select proper data types when you design the initial schema, as this is the easiest time to get it right. Also, consider data types carefully when:

- You're storing data where precision matters
- You're working with high-volume tables where storage costs multiply
- You're defining frequently queried columns that perform faster with smaller types

Explore common data types

Appropriate data types affect storage, performance, and operations:

Type Category	Data Types	Storage Size	Usage Guidelines	Example
Numeric	INT, BIGINT, DECIMAL, FLOAT	4 bytes, 8 bytes, varies	Choose based on range and precision needs	Quantity INT, R DECIMAL(10,2) BIGINT

Type Category	Data Types	Storage Size	Usage Guidelines	Example
String	VARCHAR, CHAR, NVARCHAR	1 byte/char, fixed, 2 bytes/char	Use VARCHAR for variable-length data, CHAR for fixed-length, NVARCHAR for Unicode	Email VARCHA CountryCode C ProductName NVARCHAR(10
Date/Time	DATE, DATETIME2, DATETIMEOFFSET	3 bytes, 6-8 bytes, 10 bytes	DATETIME2 provides better precision than DATETIME	BirthDate DATE OrderTimestam DATETIME2, Ev DATETIMEOFFS
Binary	VARBINARY, IMAGE	varies	For storing binary data like images or documents	ProfilePhoto VARBINARY(MA DocumentCon VARBINARY(MA
Special	UNIQUEIDENTIFIER, XML, JSON	16 bytes, varies, native binary	UNIQUEIDENTIFIER for GUIDs, XML for XML documents, JSON (SQL 2025+) for JSON documents in native binary format	RowGUID UNIQUEIDENT XML, Settings J

Data type nuances require careful attention. For example, using FLOAT for financial data instead of DECIMAL can introduce rounding errors that can't be fixed without recalculating every dependent value. A UNIQUEIDENTIFIER primary key when INT suffices triples your index size and slows every JOIN operation. Most of these decisions affect the performance of the database and can determine whether queries run in milliseconds or minutes.

Estimate table size requirements

Table size isn't just about storage costs; it directly impacts your database operations. Table size affects backup and restore times, index rebuild durations, and query performance.

Important

A poorly designed table that stores 200 bytes per row instead of 100 bytes doubles your storage needs, backup times, and I/O requirements.

Another scenario to plan for table sizing is when you're calculating storage costs for cloud databases, designing for limited disk space, or planning archive strategies. These scenarios all require accurate size estimates to make informed decisions about resources and operations.

For example, a retail company storing 100 million transactions daily with an extra 50 bytes per row wastes 5 GB per day—that's 1.8TB annually of unnecessary storage, plus proportional increases in backup time and costs.

The following example shows how to estimate the size for the Employee table:

SQL

```
-- Estimate row size for a table
-- Fixed-length columns: sum of column sizes
-- Variable-length: estimate average size
-- Example row calculation:
CREATE TABLE Employee (
  EmployeeID INT,          -- 4 bytes
  FirstName NVARCHAR(50),    -- ~2-100 bytes (avg 40)
  LastName NVARCHAR(50),     -- ~2-100 bytes (avg 40)
  HireDate DATE,           -- 3 bytes
  Salary DECIMAL(10,2)       -- 5 bytes
);
-- Estimated row size: 4 + 40 + 40 + 3 + 5 = ~92 bytes
-- Plus row overhead (~7 bytes) = ~99 bytes per row
-- 1 million rows ≈ 94 MB
```

Tip

You can use Copilot to help you generate the table size estimation.

Design effective columns

The following example demonstrates a well-designed Product table that applies the principles covered in this unit:

SQL

```
CREATE TABLE Product (
  ProductID INT PRIMARY KEY IDENTITY(1,1),      -- Auto-incrementing surrogate key (4 bytes)
  ProductName NVARCHAR(100) NOT NULL,          -- Unicode support, appropriate length, enforced
```

```
    Category NVARCHAR(50) NOT NULL,             -- Smaller than ProductName (categorization needs less space)
    Price DECIMAL(10,2) NOT NULL,               -- Exact precision for financial data
    StockQuantity INT NOT NULL DEFAULT 0,       -- Integer sufficient for inventory, default prevents nulls
    LastRestocked DATETIME2 DEFAULT GETUTCDATE()    -- Modern date type with automatic timestamp
);
```

This table demonstrates several best practices:

- **Appropriate data types**: INT for the primary key (smaller than BIGINT or UNIQUEIDENTIFIER), DECIMAL(10,2) for precise financial calculations instead of FLOAT, DATETIME2 for better precision than legacy DATETIME
- **Right-sized columns**: NVARCHAR(100) for product names and NVARCHAR(50) for categories based on expected data length
- **Constraints**: NOT NULL ensures data quality by preventing missing critical values
- **Default values**: Automatic values for StockQuantity (0) and LastRestocked (current UTC time) reduce application code complexity
- **Efficient primary key**: IDENTITY generates sequential keys that cluster efficiently and use minimal storage (4-bytes vs 16 bytes for GUID)

Note

This example uses NVARCHAR (2 bytes per character) for Unicode support. If your data is ASCII-only, VARCHAR (1 byte per character) cuts string storage in half. A ProductName VARCHAR(100) uses ~30 bytes vs ~60 bytes for NVARCHAR(100) on a 30-character name. On 10 million rows, this saves approximately 300 MB. Use NVARCHAR for international data; use VARCHAR when storage efficiency matters and data will remain ASCII-only.

Design best practices

Apply these key principles when designing and implementing tables to ensure performance and maintainability:

- **Use appropriate data types** - Smaller data types reduce storage and improve performance

- **Consider table size early** - Estimate row size and total table size to plan storage and indexing
- **Implement meaningful constraints** - Ensure data quality at the database level
- **Plan for growth** - Design tables to handle future data volume
- **Index strategically** - Index columns used in WHERE, JOIN, and ORDER BY clauses
- **Choose columnstore for analytics** - Use columnstore indexes for large tables with analytical queries
- **Normalize when appropriate** - Balance normalization with query performance needs
- **Monitor row and page compression** - Enable compression for large tables to save storage

Most database performance issues stem from poor design decisions made early in development. Oversized data types waste storage and slow queries. Missing or wrong index types create bottlenecks that resource upgrades can't resolve. Prevent these problems by investing time in proper object design before you create or modify tables. The decisions you make during design—choosing appropriate data types, estimating table sizes, selecting the right index types—have far greater effect on long-term performance and cost than any optimization you can apply later. **Optimize with indexes**

Completed100 XP

- 8 minutes

Choose your preferred content format

Indexes are data structures that accelerate data retrieval by creating optimized lookup paths to table rows. Without indexes, the database engine must scan every row in a table to find matching records, a full table scan that becomes prohibitively slow as tables grow.

An index works like a book's index: instead of reading every page to find an article, you consult the index to jump directly to relevant pages. The database uses indexes similarly, converting potentially millions of row comparisons into a handful of efficient lookups.

However, indexes consume storage space and slow down INSERT, UPDATE, and DELETE operations because the database must maintain the index structure

alongside the data. This trade-off makes index selection a critical design decision that directly impacts both query performance and write throughput.

Different index types serve different purposes.

Use rowstore indexes

Designing efficient indexes is key to achieving good database and application performance. A lack of indexes, over-indexing, or poorly designed indexes are top sources of database performance problems.

Rowstore indexes organize data in row format, storing all columns of a row together on the same page, which makes them optimal for transactional workloads that retrieve complete records or perform frequent updates.

A clustered index sorts and stores the data rows in the table based on their key values. These key values are the columns included in the index definition. There can be only one clustered index per table, because the data rows themselves can be stored in only one order.

A nonclustered index has a structure separate from the data rows. A nonclustered index contains the nonclustered index key values and each key value entry has a pointer to the data row that contains the key value. You can create multiple nonclustered indexes on a table or indexed view.

SQL

```
-- Create clustered index on primary key (defines physical row order)

CREATE CLUSTERED INDEX IX_Product_ProductID

ON Product(ProductID);


-- Create non-clustered index on frequently searched column

CREATE NONCLUSTERED INDEX IX_Product_Category

ON Product(Category)

INCLUDE (ProductName, Price);
```

Clustered indexes are best when you need efficient range queries, stable and narrow keys, or a natural sort order such as identity columns or date fields because they define the physical row order and optimize scans over ordered data.

Nonclustered indexes are ideal when you need fast lookups for specific predicates, joins, or sorting patterns that don't align with the clustered key, or when you want to cover a query by including extra columns to avoid key lookups.

Choosing between them depends on how you access the data: use a clustered index for the primary access path and nonclustered indexes to support alternate, highly selective, or frequently queried patterns while balancing the cost they introduce on write operations.

Understand columnstore indexes

Traditional rowstore indexes store data row-by-row, which is perfect for transactional systems that retrieve individual records. But analytical queries that scan millions of rows to calculate aggregates (SUM, AVG, COUNT) waste time reading columns they don't need. Columnstore indexes aim to solve this by storing data column-by-column, reading only the columns required for your query.

Understand columnstore architecture

A columnstore index organizes data into **rowgroups**, each containing up to 1,048,576 rows. Within each rowgroup, the engine stores each column separately as a **column segment** and compresses it independently. This architecture enables the query optimizer to read only the columns needed for a query, skipping irrelevant data entirely.

When you insert data, small batches first go to a **deltastore**—a temporary rowstore structure using a B+ tree index. Once a delta rowgroup accumulates enough rows (at least 102,400), a background process called the **tuple-mover** compresses it into the columnstore. Rows that arrive through bulk loads of 102,400 or more rows bypass the deltastore and compress directly into the columnstore.

The following table describes the recommendation for columnstore indexes:

Scenario	Recommendation	Reason
Data warehouse fact tables	Use columnstore	Tables with millions+ rows used for analytics benefi columnar storage and compression
Reporting databases	Use columnstore	Read-heavy workloads with aggregate queries perfo column-oriented access
Historical data	Use columnstore	Archived data that you rarely update but frequently achieves high compression ratios
Small tables (<1 million rows)	Avoid columnstore	Overhead outweighs benefits; rowgroups need suffi effective compression
High-frequency updates/deletes	Avoid columnstore	Modifications mark rows as deleted rather than upd place, causing fragmentation

Scenario	Recommendation	Reason
Single-row lookups	Avoid columnstore	Rowstore indexes are faster for retrieving individual

Use Clustered Columnstore Index (CCI)

A Clustered Columnstore Index (CCI) is a type of columnstore index that becomes the primary storage structure for the entire table, replacing any existing clustered rowstore index. Unlike a nonclustered columnstore index (NCCI), which creates a secondary columnar copy alongside the rowstore table, a CCI stores all table data exclusively in columnar format.

This means the table has no traditional row-based storage—the engine compresses and stores every column separately. Both CCI and NCCI use the same columnar compression and batch processing optimizations, but use a CCI when analytics is the primary workload and you don't need row-level transactional access patterns. In contrast, an NCCI allows you to maintain rowstore indexes for transactional queries while providing a columnar structure for analytical queries on the same table.

You can create a clustered columnstore index by using the CREATE CLUSTERED COLUMNSTORE INDEX statement. Here's an example:

SQL

```
-- Create clustered columnstore index (replaces clustered rowstore)

CREATE CLUSTERED COLUMNSTORE INDEX CCI_SalesHistory

ON SalesHistory;


-- Rebuild to improve compression

ALTER INDEX CCI_SalesHistory ON SalesHistory REBUILD;
```

Use Nonclustered Columnstore Index (NCCI)

A Nonclustered Columnstore Index (NCCI) creates a separate columnar copy of selected columns alongside the existing rowstore table, allowing the same table to serve both transactional and analytical workloads efficiently. The table maintains its original clustered rowstore index for fast single-row lookups and updates, while the NCCI provides optimized column-based access for analytical queries. The query optimizer automatically chooses between the rowstore and columnstore structures based on the query pattern.

You can create a nonclustered columnstore index by using the CREATE NONCLUSTERED COLUMNSTORE INDEX statement. Here's an example:

SQL

```
-- Create non-clustered columnstore for analytics

CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI_Product_Analytics

ON Product(Price, StockQuantity, Category, ProductName);
```

Monitor columnstore indexes

You can monitor the health and performance of your columnstore indexes by querying the sys.dm_db_column_store_row_group_physical_stats dynamic management view.

The following query shows rowgroup statistics including state, row counts, deleted rows, and storage size. Open rowgroups are still accepting inserts in the deltastore, closed rowgroups are waiting for the tuple-mover to compress them, and compressed rowgroups store data in columnar format. High deleted row counts or many small rowgroups indicate fragmentation that you can resolve with ALTER INDEX REORGANIZE.

SQL

```
-- Check columnstore health

SELECT

  object_name(object_id) AS TableName,

  state_desc,

  total_rows,

  deleted_rows,

  size_in_bytes / 1024 / 1024 AS SizeMB

FROM sys.dm_db_column_store_row_group_physical_stats

WHERE object_id = OBJECT_ID('SalesHistory');
```

Index selection directly impacts both query performance and writes throughput. Design indexes carefully during initial development to avoid costly rebuilds and performance issues in production.

Use specialized table types

Completed100 XP

- 12 minutes

Choose your preferred content format

SQL Server support specialized table types designed for specific scenarios and workloads beyond standard disk-based tables. These table types, including in-memory, temporal, external, LEDGER, and GRAPH, solve particular performance, compliance, or architectural challenges that standard tables can't address efficiently.

Understanding when and how to use these specialized table types is crucial for designing effective database solutions that meet your application's requirements.

Use in-memory optimized tables

Traditional disk-based tables incur latency from disk I/O, even with caching. For scenarios requiring high speed, such as thousands of transactions per second with millisecond response times, disk latency becomes the bottleneck. In-memory tables eliminate this by keeping data entirely in RAM with lock-free, optimistic concurrency.

Understand when to use in-memory tables

In-memory optimized tables provide significant performance benefits for specific workloads:

- **Session state storage** - Web applications with millions of concurrent sessions
- **Real-time analytics** - Financial trading systems requiring microsecond latency
- **High-frequency OLTP** - Order processing systems handling 10,000+ transactions/second
- **Caching layer** - Frequently accessed reference data (product catalogs, configurations)
- **Staging tables** - ETL processes with intensive insert/update operations

For example, an e-commerce site used in-memory tables for shopping cart data, handling 50,000 concurrent carts with submillisecond response times, reducing checkout latency by 80%.

Consider trade-offs

In-memory tables store the actual table data in RAM for faster access, while traditional tables store data on disk. However, data size is limited by available RAM, and these tables don't support large object types like VARCHAR(MAX), NVARCHAR(MAX), or VARBINARY(MAX).

Even though the table data lives in memory, SQL Server still writes transaction logs to disk to ensure durability. This means you won't lose committed transactions if the server restarts—the data is recovered from the transaction log back into memory.

You can create an in-memory optimized table by using the MEMORY_OPTIMIZED = ON option. Here's an example:

SQL

```
-- Create in-memory optimized table
CREATE TABLE dbo.OrderCache (
  OrderID INT PRIMARY KEY NONCLUSTERED,
  CustomerID INT,
  OrderDate DATETIME2,
  Amount DECIMAL(10,2),
  INDEX IX_CustomerID NONCLUSTERED (CustomerID)
) WITH (MEMORY_OPTIMIZED = ON, DURABILITY = SCHEMA_AND_DATA);
```

Use temporal tables

Temporal tables automatically track the complete history of data changes. When you update or delete a row, SQL Server automatically stores the previous version in a linked history table with timestamps showing when that version was valid. This happens transparently—you modify data using normal INSERT, UPDATE, and DELETE statements, and the database engine handles versioning.

The key benefit is querying data as it existed at any point in time. You can ask "what was this employee's salary on January 1, 2025?" or "show me all products that were in stock last quarter" without maintaining complex audit tables or writing custom versioning logic.

Temporal tables serve compliance, troubleshooting, and analytical needs:

- **Compliance and auditing** - Financial records requiring complete change history
- **Troubleshooting** - Investigating account balances at the time disputed transactions occurred
- **Trend analysis** - Analyzing how product prices changed over quarters
- **Data recovery** - Reverting accidental updates without restoring backups

- **Slowly changing dimensions** - Data warehouse Type 2 dimensions automated

Common business scenarios include applications tracking salary changes and promotions, inventory management analyzing stock trends, healthcare maintaining patient record history for compliance, and insurance tracking policy coverage changes for dispute resolution.

Consider temporal table benefits

Temporal tables require zero application code changes and offer transparent history tracking. Point-in-time queries use simple syntax, and automatic cleanup manages old history data. However, temporal tables roughly double storage requirements.

Temporal tables automatically maintain a complete history of data changes for auditing and point-in-time analysis.

You can create a temporal table by using the SYSTEM_VERSIONING = ON option. Temporal tables require two extra DATETIME2 columns to track the validity period of each row version and a PERIOD FOR SYSTEM_TIME clause to define which columns track these timestamps. Here's an example:

SQL

```
-- Create temporal table with automatic history tracking

CREATE TABLE Employee (

  EmployeeID INT PRIMARY KEY,

  EmployeeName NVARCHAR(100),

  Department NVARCHAR(50),

  SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START,

  SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END,

  PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime)

) WITH (SYSTEM_VERSIONING = ON);


-- Query historical data

SELECT * FROM Employee

FOR SYSTEM_TIME AS OF '2026-01-01'
```

WHERE EmployeeID = 1;

When you create a temporal table, SQL Server automatically creates a history table to store previous row versions and manages both tables transparently.

Use external tables

Modern data architectures often have data scattered across data lakes, blob storage, and multiple systems. Traditionally, you'd have to ETL (extract, transform, load) all data into your database before querying it. External tables enable data virtualization to query data where it lives without moving it, saving storage costs and ETL complexity.

Understand when to use external tables

External tables excel at querying data across distributed storage systems:

- **Data lake integration** - Query Parquet/CSV files in Azure Data Lake Storage without importing
- **Data exploration** - Analyze raw data before deciding what to import
- **Cost optimization** - Avoid duplicating data that's already stored elsewhere
- **Federated queries** - Join database tables with files in external systems
- **Archival storage** - Access historical data stored in cheaper blob storage

Common scenarios include querying years of log files in data lakes alongside transactional data, combining live database records with archived blob storage data, accessing legacy data without full migration, and querying millions of IoT sensor JSON files without importing.

Consider performance constraints

External tables provide unified querying across sources but have limitations:

- No data movement or storage duplication
- Often slower than native tables due to network latency, and file parsing
- Read-only (can't update/delete in most scenarios)
- Limited indexing and optimization

You can create an external table by using the CREATE EXTERNAL TABLE statement with a data source and file format. Here's an example:

SQL

-- Create external table pointing to data lake

```
CREATE EXTERNAL TABLE dbo.ExternalSalesData (
    OrderID INT,
    CustomerID INT,
    OrderAmount DECIMAL(10,2),
    OrderDate DATE
) WITH (
    LOCATION = '/raw/sales/',
    DATA_SOURCE = DataLakeSource,
    FILE_FORMAT = ParquetFormat
);
```

Use ledger tables

In regulated industries, proving data hasn't been tampered with is important. Traditional databases can have data modified by administrators, backdated changes made, or audit logs deleted. Ledger tables use *cryptographic verification* inspired by blockchain technology to create tamper-evident records that can be independently verified, providing cryptographic proof of data integrity.

Understand when to use ledger tables

Ledger tables serve regulatory compliance and forensic auditing needs:

- **Financial transactions** - Banking, payment processing, cryptocurrency exchanges
- **Supply chain** - Tracking product origin, custody, and authenticity
- **Legal records** - Contracts, agreements, legal filings requiring immutability
- **Healthcare** - Prescription records, patient consent forms
- **Government** - Voting records, land registries, permit issuance

For example, a bank can use ledger tables to store transaction records, allowing auditors to verify that no transactions were altered after posting. A supply chain company can track product provenance using ledger tables, providing customers with proof of authenticity.

Choose between updatable and append-only ledgers

Ledger tables come in two types. **Updatable ledger tables** allow INSERT, UPDATE, and DELETE operations while tracking all changes cryptographically. The system

automatically stores previous versions in a history table, similar to temporal tables, but with the added benefit of tamper-proof verification. **Append-only ledger tables** only allow INSERT operations, creating truly immutable records for scenarios requiring absolute data integrity.

You can combine both technologies by creating tables that are both updatable ledger tables and temporal tables, gaining cryptographic verification alongside point-in-time query capabilities.

For example, a pharmaceutical company uses append-only ledger tables for clinical trial data, providing independent auditors cryptographic proof that test results weren't altered after submission.

You can create a ledger table by using the LEDGER = ON option. Here's an example:

SQL

```
-- Create ledger table
CREATE TABLE dbo.FinancialTransaction (
  TransactionID INT PRIMARY KEY IDENTITY,
  AccountNumber NVARCHAR(20),
  Amount DECIMAL(15,2),
  TransactionType NVARCHAR(20)
) WITH (LEDGER = ON);

-- Append-only ledger provides immutability
CREATE TABLE dbo.AuditLog (
  LogID INT PRIMARY KEY IDENTITY,
  EventDescription NVARCHAR(500),
  EventTimestamp DATETIME2
) WITH (LEDGER = ON, APPEND_ONLY = ON);
```

When you create a ledger table, SQL Server automatically adds hidden columns and creates supporting database objects to track the cryptographic chain. Every row modification generates a cryptographic hash that links to previous operations, creating a tamper-evident audit trail. You can verify data integrity using built-in

system views like sys.database_ledger_transactions and procedures such as sp_verify_database_ledger to validate the cryptographic chain remains unbroken.

Use graph tables

Relational databases excel at structured data but struggle with highly connected data requiring many joins. Finding "friends of friends" or "products related through 3 degrees of categories" becomes complex with traditional tables. SQL Graph capabilities natively model *nodes* (entities) and *edges* (relationships), making complex relationship queries simple and performant.

Graph tables simplify relationship modeling but require learning new syntax. They provide intuitive modeling of connected data, simpler queries for relationship traversal, and better performance for multi-hop queries. The flexible schema accommodates evolving relationships. However, graph tables have a learning curve for MATCH syntax and perform best for read-heavy relationship queries.

A database can contain multiple node and edge tables that work together to model your graph data. You define which tables represent nodes and which represent edges based on your data relationships.

Note

Graph tables aren't optimal for every scenario. Avoid them for simple parent-child relationships where foreign keys work fine, primarily transactional data without complex relationships, or highly structured, stable schemas.

Understand graph table structure

SQL Graph uses two types of tables to model relationships. **Node tables** store entities and automatically include a hidden $node_id column that uniquely identifies each node. **Edge tables** store relationships between nodes and include hidden columns $edge_id, $from_id, and $to_id to maintain connections. These special columns enable the MATCH syntax to traverse relationships efficiently.

You can create graph tables by using the AS NODE and AS EDGE syntax. Here's an example:

SQL

```
-- Create graph tables

CREATE TABLE Person AS NODE;

CREATE TABLE Manages AS EDGE;

CREATE TABLE Knows AS EDGE;
```

```
-- Insert nodes
INSERT INTO Person VALUES (1, 'Alice'), (2, 'Bob'), (3, 'Charlie');

-- Insert edges (relationships)
INSERT INTO Manages VALUES (1, 2), (2, 3);

-- Query relationships
SELECT Person1.name, Person2.name
FROM Person AS Person1, Manages, Person AS Person2
WHERE MATCH (Person1-(Manages)->Person2)
AND Person1.id = 1;
```

When you create node and edge tables, SQL Server automatically manages the hidden system columns that enable efficient graph traversal queries.

Each specialized table type comes with trade-offs: in-memory tables need RAM, temporal tables double storage, external tables add network latency, ledger tables prevent deletion, and graph tables require new syntax. We recommend choosing the right table type during design as these decisions are hard to change after deployment.

Enforce data integrity with constraints

Completed100 XP

- 12 minutes

Choose your preferred content format

Constraints and sequence objects are design choices that prevent data problems before they happen. A missing foreign key constraint means orphaned records might already exist in your database. Adding sequence objects later to replace identity columns requires changes across all applications. Application code can validate data, but users can bypass it through bulk imports, direct queries, or new applications that skip validation.

Database constraints enforce rules at the engine level, so they always apply. The decisions you make during design, such as which rules to enforce in the database and whether to use identity columns or sequences, affect your data quality for the life of your application.

Understand when to use constraints

Data quality problems are expensive. Poor data quality leads to incorrect business decisions, failed integrations, and compliance violations. Unlike application-level validation that can be inconsistent across different applications accessing the same database, constraints enforce rules at the database engine level where they can't be bypassed by application code, ad-hoc queries, direct SQL scripts, or bulk imports. Every INSERT, UPDATE, and DELETE operation must satisfy all defined constraints before the database engine commits the change.

Apply database constraints

Constraints prevent data quality problems before they corrupt your database. The following table shows how each constraint type addresses specific data integrity issues:

Problem	Constraint	Example
Orphaned records	FOREIGN KEY	Prevents orders without valid customers
Duplicate data	UNIQUE	Stops duplicate email registrations
Invalid data	CHECK	Rejects negative prices or future birthdates
Missing critical data	NOT NULL	Prevents incomplete records
Referential inconsistency	FOREIGN KEY	Maintains data integrity across tables

Consider a retail company that didn't define a unique constraint on their customer email column. Over time, the same customers were registered multiple times with identical email addresses. When marketing sent promotional campaigns, some customers received three copies of the same email, increasing costs and damaging customer trust. Adding UNIQUE (EmailAddress) to the table definition would have prevented these duplicates from ever being inserted.

Constraints enforce rules at the database engine level, ensuring data quality regardless of how data enters the system. Application validation can be bypassed, varies by application, and is harder to maintain. Database constraints are always enforced, centralized, and provide one source of truth.

Constraints ensure data quality and consistency at the database level.

Use primary key constraints

Primary key constraints guarantee unique data and enforce entity integrity. When you specify a primary key constraint, the Database Engine automatically creates a unique index for the primary key columns. A table can contain only one primary key

constraint, and all columns defined within a primary key constraint must be defined as NOT NULL.

You can create a primary key by using the PRIMARY KEY constraint. Here's an example:

SQL

```
CREATE TABLE Customer (
  CustomerID INT PRIMARY KEY IDENTITY(1,1),
  EmailAddress NVARCHAR(100) NOT NULL
);
```

Use foreign key constraints

Foreign key constraints enforce referential integrity by controlling the data that can be stored in the foreign key table. A foreign key constraint prevents changes to data in the primary key table if those changes invalidate the link to data in the foreign key table.

You can define cascading referential actions such as CASCADE, SET NULL, or SET DEFAULT to specify what happens when a user tries to delete or update a key to which existing foreign keys point. Although manually creating an index on foreign key columns isn't required, it's often useful because foreign key columns are frequently used in join criteria.

You can create a foreign key by using the FOREIGN KEY constraint with a REFERENCES clause. Here's an example:

SQL

```
CREATE TABLE Order (
  OrderID INT PRIMARY KEY IDENTITY,
  CustomerID INT NOT NULL,
  OrderDate DATETIME2,
  FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
);
```

Use unique constraints

Unique constraints ensure no duplicate values are entered in specific columns that don't participate in a primary key. Unlike PRIMARY KEY constraints, UNIQUE constraints allow for the value NULL. However, as with any value participating in a

UNIQUE constraint, only one null value is allowed per column. The Database Engine automatically creates a unique nonclustered index to enforce the uniqueness requirement.

You can create a unique constraint by using the UNIQUE keyword. Here's an example:

SQL

```
CREATE TABLE Product (
  ProductID INT PRIMARY KEY,
  SKU NVARCHAR(50) UNIQUE,
  ProductName NVARCHAR(100)
);
```

Use check constraints

Check constraints enforce domain integrity by limiting the values accepted by one or more columns. You can create a CHECK constraint with any logical expression that returns TRUE or FALSE based on logical operators. You can apply multiple CHECK constraints to a single column or apply a single CHECK constraint to multiple columns.

Because null values evaluate to UNKNOWN, their presence in expressions might override a constraint. For example, a constraint MyColumn = 10 on an INT column still allows NULL to be inserted because NULL doesn't evaluate to FALSE.

You can create a CHECK constraint by using the CHECK keyword with a logical expression. Here's an example:

SQL

```
CREATE TABLE Employee (
  EmployeeID INT PRIMARY KEY,
  HireDate DATE,
  Salary DECIMAL(10,2),
  CHECK (Salary >= 20000),
  CHECK (HireDate <= GETDATE())
);
```

Use default constraints

Default constraints provide default values when no value is specified during INSERT operations. When working with database projects, it's recommended to create constraints with explicit names rather than allowing system-generated names, which differ across environments.

You can create a DEFAULT constraint by using the DEFAULT keyword. Here's an example:

SQL

```
CREATE TABLE Activity (
  ActivityID INT PRIMARY KEY IDENTITY,
  Description NVARCHAR(200),
  CreatedDate DATETIME2 CONSTRAINT DF_Activity_CreatedDate DEFAULT GETUTCDATE(),
  IsActive BIT CONSTRAINT DF_Activity_IsActive DEFAULT 1
);
```

Use sequence objects

A *sequence object* is a user-defined schema-bound object that generates a sequence of numeric values according to the specification with which the sequence was created. Unlike identity columns, sequences aren't associated with specific tables. Applications refer to a sequence object to retrieve its next value, and the relationship between sequences and tables is controlled by the application.

Identity columns work well when you need automatic numbering for a single table. However, they're limited to that one table. You can't share the numbers across multiple tables, get the next value before inserting a row, or easily reset the counter. Sequence objects solve these problems by generating numbers independently of any table.

Understand when to use sequences

Use sequences instead of identity columns in the following scenarios:

- **Shared number series** - The application requires sharing a single series of numbers between multiple tables or multiple columns within a table.
- **Cycling number series** - The application must restart the number series when a specified number is reached. For example, after assigning values 1 through 10, the application starts assigning values 1 through 10 again.

- **Sorted sequence values** - The application requires sequence values to be sorted by another field. The NEXT VALUE FOR function can apply the OVER clause, which guarantees that the values returned are generated in the order of the ORDER BY clause.
- **Reserve multiple numbers** - An application needs to reserve several sequential numbers at once. Requesting identity values could result in gaps if other processes were simultaneously issued numbers. Calling sp_sequence_get_range retrieves several numbers in the sequence at once.
- **Changeable specification** - You need to change the specification of the sequence, such as the increment value, after creation.

Sequence objects can offer more flexibility than identity columns:

Feature	Sequence	Id
Tied to table	No	Y
Shared across tables or columns	Yes	N
Get next value before the insert operation	Yes	N
Custom min/max values	Yes	L
Retrieve multiple numbers at once	Yes	N
Cycle/restart at specified number	Yes	N
Sort values by another field	Yes	N
Change increment after creation	Yes	N

Use an identity column when you need a simple autoincrementing primary key for a single table and don't need to share the same number series across multiple tables or retrieve the next value before inserting the row.

Use a sequence when your application requires a number before the insert is made, needs to share a single series between multiple tables, must restart the number series when a specified number is reached, or needs to reserve multiple sequential numbers at once.

Understand sequence limitations

Unlike identity columns, sequence values aren't automatically protected after insertion into a table. Also, uniqueness isn't automatically enforced for sequence values. If sequence values in a table must be unique, create a unique constraint on the column.

Sequence numbers are generated outside the scope of the current transaction. They're consumed whether the transaction using the sequence number is committed or rolled back.

You can create a sequence object by using the CREATE SEQUENCE statement with optional parameters for start, increment, and range. Here's an example:

SQL

```
-- Create sequence
CREATE SEQUENCE OrderNumber
  START WITH 1000
  INCREMENT BY 1
  MINVALUE 1000
  MAXVALUE 999999
  NO CYCLE;


-- Use sequence in INSERT with NEXT VALUE FOR function
INSERT INTO Order (OrderID, CustomerID, OrderNumber, OrderDate)
VALUES (1, 100, NEXT VALUE FOR OrderNumber, GETDATE());


-- Get next value before INSERT
DECLARE @NextOrderNum INT = NEXT VALUE FOR OrderNumber;
SELECT @NextOrderNum;


-- Get multiple sequence numbers at once for batch processing
DECLARE @FirstSeq INT, @LastSeq INT;
EXEC sp_sequence_get_range
  @sequence_name = N'OrderNumber',
  @range_size = 100,
  @range_first_value = @FirstSeq OUTPUT,
  @range_last_value = @LastSeq OUTPUT;
```

-- Reset sequence

ALTER SEQUENCE OrderNumber RESTART WITH 1000;

This example creates a sequence named OrderNumber that starts at 1000, increments by 1, and stops at 999999 without cycling back. The NEXT VALUE FOR function retrieves the next available number, either inline during an INSERT statement or assigned to a variable before the insert when the application needs to reference the value first. For batch operations that require multiple sequential numbers at once, sp_sequence_get_range reserves a block of 100 numbers, returning the first and last values in the range. The ALTER SEQUENCE statement resets the sequence back to 1000 when needed.

Constraints are architectural decisions that prevent problems before they occur. A missing CHECK constraint allows invalid data to corrupt your database silently. Choosing identity columns when you need cross-table numbering forces application-level workarounds. Constraints defined at the database level protect data quality regardless of which application, tool, or script accesses your database. These decisions shape your data integrity guarantees for the life of your application.

Manage JSON columns and indexes

Completed100 XP

- 8 minutes

Choose your preferred content format

Relational databases work best when every row in a table has the same columns. You define the structure once, and every record follows it. This design works well for data like customers, orders, or invoices where the fields are predictable. But some data varies from record to record. The attributes you need to store depend on the type of item, the source of the data, or choices made by users. Traditional table design forces you to either create many columns that are empty for most rows, or split data across many tables. JSON columns offer another option: store the variable parts as JSON while keeping the predictable parts in regular columns.

For example, an e-commerce product catalog has common fields like product name, price, and category that apply to every item. But a shirt needs size and color, a laptop needs processor speed and screen size, and a book needs author and other attributes. With JSON, you store the common fields as columns and put the category-specific attributes in a JSON column. You can add new product types without changing the table structure.

Understand when to use JSON columns

JSON columns let you query and index semi-structured data using familiar SQL syntax. You don't need a separate NoSQL database to handle flexible data. Consider JSON for these scenarios:

- **User preferences** - Settings like theme, language, and notification options differ per user and change as you add features.
- **API responses** - Data from external services has nested structures that may change when the provider updates their API.
- **Audit logs** - Records that capture before and after states need to adapt as your table schemas evolve.
- **Multi-tenant applications** - Different customers require different custom fields.
- **Flexible metadata** - Tags, labels, and properties that vary by record and don't fit a fixed schema.

Create and query JSON columns

SQL Server 2025 introduces a native **json** data type that stores JSON documents in a binary format optimized for querying and manipulation. The native type provides more efficient reads (the document is already parsed), more efficient writes (updates can modify individual values without rewriting the entire document), and better storage compression compared to storing JSON as NVARCHAR(MAX).

For earlier versions of SQL Server, you store JSON in an NVARCHAR(MAX) column.

To read values from JSON, you use JSON functions like JSON_VALUE to extract a single value or JSON_QUERY to return an object or array. If you query a JSON property frequently, you can create an index on a computed column that extracts that property.

The following example creates a table with a JSON column, inserts documents, queries specific properties, updates values, and creates an index on a frequently accessed field:

SQL

```
-- Create table with native JSON type (SQL Server 2025+)

CREATE TABLE ConfigurationData (

  ConfigID INT PRIMARY KEY,

  ConfigSettings JSON NOT NULL
```

```
);

-- Insert JSON documents
INSERT INTO ConfigurationData (ConfigID, ConfigSettings)
VALUES (1, '{"theme":"dark","language":"en","notifications":true}');

INSERT INTO ConfigurationData (ConfigID, ConfigSettings)
VALUES (2, '{"theme":"light","language":"fr","notifications":false}');

-- Query JSON properties
SELECT ConfigID,
    JSON_VALUE(ConfigSettings, '$.theme') AS Theme,
    JSON_VALUE(ConfigSettings, '$.language') AS Language,
    JSON_QUERY(ConfigSettings, '$') AS FullConfig
FROM ConfigurationData;

-- Update a single property using the modify method (SQL Server 2025+ preview)
UPDATE ConfigurationData
SET ConfigSettings.modify('$.theme', 'light')
WHERE ConfigID = 1;

-- Alternative: JSON_MODIFY works with both JSON and NVARCHAR(MAX) columns
UPDATE ConfigurationData
SET ConfigSettings = JSON_MODIFY(CAST(ConfigSettings AS NVARCHAR(MAX)), '$.notifications', CAST(0 AS BIT))
WHERE ConfigID = 1;

-- Create index on frequently queried JSON property
```

```
ALTER TABLE ConfigurationData
ADD ThemeValue AS JSON_VALUE(ConfigSettings, '$.theme');

CREATE INDEX IX_Theme ON ConfigurationData(ThemeValue);
```

This example creates a table with a JSON column that stores user configuration settings. The INSERT statements add JSON documents as string literals. To read specific values, JSON_VALUE extracts scalar values like the theme and language, while JSON_QUERY returns the entire JSON object. The .modify() method (currently in preview) updates a single property without rewriting the whole document. Because the json type can't be used as an index key column, the example creates a computed column that extracts the theme value, then indexes that computed column.

Combine relational and JSON structure

JSON columns work best for data that varies by record. If every row has the same fields with consistent data types, regular columns are a better fit. You get native data type validation, simpler queries without JSON path syntax, and direct indexing on columns. Use JSON for the parts of your data that need flexibility, and keep the predictable parts in typed columns.

You can combine relational structure with JSON flexibility for products requiring variable metadata. Here's an example:

SQL

```
-- Product with flexible metadata (SQL Server 2025+)
CREATE TABLE ProductMetadata (
  ProductID INT PRIMARY KEY,
  AdditionalAttributes JSON NOT NULL
    CHECK (JSON_PATH_EXISTS(AdditionalAttributes, '$.weight') = 1),
  FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);

-- Store flexible product attributes
INSERT INTO ProductMetadata (ProductID, AdditionalAttributes)
```

```
VALUES (1,
'{"dimensions":{"length":10,"width":5,"height":8},"weight":2.5,"color":"blue"}');


-- Query nested JSON properties
SELECT ProductID,
    JSON_VALUE(AdditionalAttributes, '$.weight') AS Weight,
    JSON_VALUE(AdditionalAttributes, '$.dimensions.length') AS Length
FROM ProductMetadata;
```

Consider JSON design principles

Apply these principles when implementing JSON columns:

- **Use JSON for semi-structured data** - Store flexible data structures that vary by record, not data with consistent schemas.
- **Index frequently queried paths** - Create computed columns with indexes on JSON properties you query often.
- **Validate required properties** - Use CHECK constraints with JSON_PATH_EXISTS to ensure required fields are present.
- **Balance flexibility with structure** - Keep predictable data in regular columns and use JSON only for the variable parts.

JSON columns provide schema flexibility for variable data while maintaining SQL query capabilities, but should complement rather than replace relational design for structured data.

Partition tables for scale

Completed100 XP

- 13 minutes

Choose your preferred content format

Partitioning decisions are nearly permanent. A poorly chosen partition key makes performance worse than an unpartitioned table, and repartitioning a multi-terabyte table requires rebuilding it entirely with hours of downtime. The partition key and index alignment you choose during design determine whether partitioning improves your system or creates maintenance nightmares you can't easily undo.

Table partitioning divides large tables into smaller, more manageable pieces (partitions) while keeping them as a single logical table. Your application sees one

table, but the database engine manages multiple physical segments that can be independently maintained, archived, or queried.

Understand partitioning concepts

Partitioning involves several key components: a *partition function* that defines how data is divided, a *partition scheme* that maps partitions to filegroups, and the *partitioning column* that determines which partition each row belongs to. Understanding these concepts helps you design an effective partitioning strategy.

Evaluate performance and operational benefits

Partitioning provides query performance improvements through partition elimination where queries filter by partition key access only relevant partitions (one month instead of 120 months), parallel processing where multiple partitions are processed simultaneously across CPU cores, faster statistics calculated per partition instead of the entire table, and index seeks where smaller partitions mean shallower B-trees.

Operational benefits include granular maintenance where you rebuild indexes on the current partition while older partitions remain online, fast archival by switching old partitions to archive tables in seconds through metadata operations, improved availability from maintaining partitions independently, and tiered storage by moving older partitions to cheaper, slower storage.

For example, imagine a financial services company with a 1.2-TB transactions table where queries filtering by date (90% of queries) scan the entire table. After implementing monthly partitioning, query performance improves 10-20x through partition elimination, index rebuilds go from 6 hours to 20 minutes per partition, archiving old data reduces from four-hour locks to seconds using partition switching, and storage costs decrease 40% by moving older partitions to cheaper storage.

Understand when to use partitioning

The following table shows when partitioning helps versus when it adds unnecessary complexity:

Scenario	Use partitioning?	Why
Queries filter on a specific column (date, region) 80%+ of the time	Yes	Partition elimination accesses onl partitions
Regular archival of old data (monthly, quarterly)	Yes	Switch partitions out in seconds i DELETE operations

Scenario	Use partitioning?	Why
Need to rebuild indexes on recent data only	Yes	Rebuild current partition while old stay online
Large tables (multi-TB) with tiered storage needs	Yes	Move older partitions to cheaper s
Most queries scan the full table or filter on various columns	No	All partitions scanned—worse per than unpartitioned
Single-row lookups or small range scans are common	No	Partitioning adds overhead withou
No clear column aligns with query patterns	No	Can't choose an effective partitio

Create partitioning components

The following example shows the three components: partition function, partition scheme, and partitioned table:

SQL

```
-- Create partition function based on date ranges
-- Use RANGE RIGHT for datetime columns to keep same-day values together
CREATE PARTITION FUNCTION PF_OrderDate (DATETIME2)
    AS RANGE RIGHT FOR VALUES
    ('2024-01-01', '2024-04-01', '2024-07-01', '2024-10-01');

-- Create partition scheme mapping to a single filegroup (recommended)
-- Use multiple filegroups only for tiered storage or independent backups
CREATE PARTITION SCHEME PS_OrderDate
    AS PARTITION PF_OrderDate ALL TO ([PRIMARY]);

-- Create partitioned table
-- Include partition column in primary key for clustered index alignment
CREATE TABLE Orders (
    OrderID INT NOT NULL,
```

```
    OrderDate DATETIME2 NOT NULL,
    CustomerID INT,
    Amount DECIMAL(10,2),
    CONSTRAINT PK_Orders PRIMARY KEY (OrderID, OrderDate)
) ON PS_OrderDate(OrderDate);
```

This example creates quarterly partitions for an *Orders* table. The partition function defines four boundary values (January, April, July, October) creating five partitions: one for data before 2024, and four for each quarter of 2024. The partition scheme maps all partitions to the PRIMARY filegroup. The *Orders* table uses the *OrderDate* column as the partition key, which must be included in the primary key for proper index alignment.

Choose partitioning strategies

The partition key is your most important decision. Choose poorly, and partitioning hurts more than helps. The ideal partition key appears in the WHERE clause of most queries, creates reasonably balanced partitions, and aligns with your maintenance patterns.

The following key selection criteria help you choose the right partition key:

- **Query patterns**: 80%+ of queries filter by this column
- **Data distribution**: Even distribution across partitions (no single partition with 90% of data)
- **Maintenance alignment**: Matches archival/purge patterns (date columns for time-based archival)
- **Stability**: Value doesn't change after INSERT (avoid partitioning on updateable columns)

Understand range partitioning

Range partitioning divides data based on value ranges—most commonly dates. Each partition contains a specific range (January data, February data, etc.). This is the most widely used strategy.

Here's where range partitioning works best:

- Time-series data (orders, logs, transactions)
- Sequential data (invoice numbers, order IDs)
- Numeric ranges (salary bands, price tiers)

The following table shows common partition patterns:

Pattern	When to Use
Daily	High-volume systems, short retention
Weekly	Medium volume, 6-12 month retention
Monthly	Most common, balances partition count and size
Quarterly	Lower volume, multi-year retention
Yearly	Archive scenarios, long-term historical data

For example, an e-commerce platform partitioning orders monthly enables current month queries to hit one partition, quarterly reports to access 3 partitions, and year-end analysis to use 12 partitions while eliminating older years.

You can create range partitions by defining boundaries in the partition function:

SQL

```
-- RANGE RIGHT creates 5 partitions: <100000, 100000-199999, 200000-299999, 300000-399999, >=400000

CREATE PARTITION FUNCTION PF_InvoiceNumber (INT)

  AS RANGE RIGHT FOR VALUES

  (100000, 200000, 300000, 400000);
```

Partition by categorical values

You can use RANGE partitioning with string or categorical values like regions. The partition function places values based on sort order. This approach works for geographic distribution, multitenant systems, or departmental data where queries frequently filter by category.

The following example partitions data by region:

SQL

```
-- Partition by region

CREATE PARTITION FUNCTION PF_Region (NVARCHAR(50))

  AS RANGE LEFT FOR VALUES ('East', 'North', 'South', 'West');


CREATE PARTITION SCHEME PS_Region
```

```
  AS PARTITION PF_Region ALL TO ([PRIMARY]);


CREATE TABLE RegionalData (
  DataID INT NOT NULL,
  Region NVARCHAR(50) NOT NULL,
  Value DECIMAL(10,2),
  CONSTRAINT PK_RegionalData PRIMARY KEY (DataID, Region)
) ON PS_Region(Region);
```

Implement index partitioning

When you partition a table, indexes can be aligned or nonaligned. Aligned indexes use the same partition scheme as the table, while nonaligned indexes use different partitioning or no partitioning. By default, nonclustered indexes on partitioned tables inherit the table's partition scheme.

Understand aligned versus nonaligned indexes

Aligned indexes use the same partition function as the table. Each index partition matches a table partition, enabling fast partition switching, simplified maintenance, and better partition elimination.

Nonaligned indexes use different partitioning or no partitioning. They can't use partition switching and aren't supported on tables with more than 1,000 partitions.

Use aligned indexes when you need partition switching for archival, want to rebuild specific partitions independently, or when query patterns filter on the partition key.

For example, imagine an *Orders* table partitioned by *OrderDate* with a nonclustered index on *CustomerID*. Using aligned partitioning with the same *OrderDate* scheme allows archiving old months by switching partitions, rebuilding current indexes independently, and dropping old partitions without affecting the entire table.

You can create partitioned indexes using the same partition scheme as the base table:

SQL

```
-- Create partitioned non-clustered index
CREATE NONCLUSTERED INDEX IX_Orders_Customer
  ON Orders(CustomerID)
```

```
    ON PS_OrderDate(OrderDate);


-- Create partitioned columnstore index
CREATE NONCLUSTERED COLUMNSTORE INDEX IX_SalesData_CS
    ON SalesData(Revenue, Region)
    ON PS_SalesDate(SaleDate);
```

Manage partition operations

After creating partitioned tables, you need to manage them over time. Common operations include querying partition metadata, adding new partitions as data grows, and removing old partitions during archival. These operations use the $PARTITION function and ALTER PARTITION FUNCTION statement.

Query partition information

You can view partition information by using the $PARTITION function. Here's an example:

SQL

```
-- View partition information
SELECT
    $PARTITION.PF_OrderDate(OrderDate) AS PartitionNumber,
    MIN(OrderDate) AS MinDate,
    MAX(OrderDate) AS MaxDate,
    COUNT(*) AS RowCount
FROM Orders
GROUP BY $PARTITION.PF_OrderDate(OrderDate)
ORDER BY PartitionNumber;
```

Add new partition boundary

You can split partitions to add new boundary values by using ALTER PARTITION FUNCTION. Here's an example:

SQL

```
-- Split partition to add new boundary
ALTER PARTITION FUNCTION PF_OrderDate()
```

SPLIT RANGE ('2024-11-01');

This statement adds a new boundary value (November 1, 2024) to the partition function, splitting an existing partition into two partitions. The partition containing dates from October through December now becomes two partitions: one for October and another for November through December.

Archive and remove partition

You can merge partitions to archive old data by using ALTER PARTITION FUNCTION with MERGE RANGE. Here's an example:

SQL

-- Merge partitions to archive old data

ALTER PARTITION FUNCTION PF_OrderDate()

MERGE RANGE ('2023-12-31');

Apply partitioning best practices

Following best practices helps you avoid common partitioning mistakes that are difficult to correct after implementation:

- **Align indexes with table partitions**: Use the same partition scheme for tables and indexes to enable partition switching and maintenance
- **Monitor data distribution**: Check partition statistics regularly to identify imbalanced partitions and verify partition elimination
- **Automate partition management**: Schedule jobs to add new partitions before reaching boundaries and archive old partitions
- **Avoid over-partitioning**: Target millions of rows per partition, not thousands—too many partitions create overhead
- **Include partition key in primary key**: Required for clustered index alignment on the partition scheme

Partitioning requires careful planning. The partition key and index alignment you choose determine whether you gain performance or create complexity. When implemented correctly, partitioning transforms large table management through faster queries, efficient archival, and simplified maintenance.

Design and implement database objects with SQL

Estimated Time: 30 minutes

In this exercise you will implement various database objects in SQL Server, including tables, constraints, temporal tables, JSON columns, and indexes. This exercise will help you understand how to create a robust and efficient database schema.

You are a database designer for an e-commerce platform. You need to create a database schema that includes standard tables with appropriate constraints, a temporal table for tracking price changes, JSON storage for product metadata, and partitioning for efficient historical data management.

📝 These exercises ask you to copy and paste T-SQL code. Please verify that the code has been copied correctly, before executing the code.

Setup environment

If your lab virtual machine has been provided and pre-configured, you should find the lab files ready in the **C:\LabFiles** folder. *Take a moment to check, if the files are already there, skip this section*. However, if you're using your own machine or the lab files are missing, you'll need to clone them from *GitHub* to proceed.

⚠ **Important:** This exercise requires **SQL Server 2025** or later.

1. From the lab virtual machine or your local machine if one wasn't provided, start a Visual Studio Code session.
2. Open the command palette (Ctrl+Shift+P) and type **Git: Clone**. Select the **Git: Clone** option.
3. Paste the following URL into the **Repository URL** field and select **Enter**.

code

https://github.com/MicrosoftLearning/mslearn-sql-developer.git

4. Save the repository to the **C:\LabFiles** folder on the lab virtual machine or your local machine if one wasn't provided (create the folder if it does not exist).

Create a new database

1. From the lab virtual machine or your local machine if one wasn't provided, start a SQL Server Management Studio session (SSMS).
2. When SSMS opens, by default the **Connect to Server** dialog will appear. Choose the Default instance and select **Connect**. You might need to check the **Trust server certificate** checkbox.

📝 Note that if you are using your own SQL Server instance, you will need to connect to it using the appropriate server instance name and credentials.

3. Select the **Databases** folder, and then **New Query**.
4. In the new query window, copy and paste the below T-SQL into it. Execute the query to create the database.

sql

```
CREATE DATABASE EcommerceDB;
GO

USE EcommerceDB;
GO
```

5. Under the **Messages** tab, you should see a message indicating that the commands completed successfully.

Create core tables with constraints

Create the foundational tables for the e-commerce system.

1. Select **New Query**. Copy and paste the following T-SQL code into the query window. Select **Execute** to execute this query.

sql

```
USE EcommerceDB;
GO

-- Create Supplier table
CREATE TABLE Supplier (
    SupplierID INT PRIMARY KEY IDENTITY(1,1),
    SupplierName NVARCHAR(100) NOT NULL UNIQUE,
    Country NVARCHAR(50) NOT NULL,
    Email NVARCHAR(100),
    Phone NVARCHAR(20),
```

```
    CreatedDate DATETIME2 DEFAULT GETUTCDATE()
);

-- Create Category table
CREATE TABLE Category (
    CategoryID INT PRIMARY KEY IDENTITY(1,1),
    CategoryName NVARCHAR(100) NOT NULL UNIQUE,
    Description NVARCHAR(500)
);

-- Create Product table with constraints
CREATE TABLE Product (
    ProductID INT PRIMARY KEY IDENTITY(1,1),
    ProductName NVARCHAR(100) NOT NULL,
    CategoryID INT NOT NULL,
    SupplierID INT NOT NULL,
    BasePrice DECIMAL(10,2) NOT NULL,
    StockQuantity INT NOT NULL DEFAULT 0,
    CreatedDate DATETIME2 DEFAULT GETUTCDATE(),
    CHECK (BasePrice > 0),
    CHECK (StockQuantity >= 0),
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID),
    FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID),
);

-- Create indexes
CREATE INDEX IX_Category ON Product(CategoryID);
CREATE INDEX IX_Supplier ON Product(SupplierID);
```

```
GO
```

2. Insert sample data into the tables. Copy and paste the following T-SQL code into a new query window. Select **Execute** to execute this query.

```
sql
USE EcommerceDB;
GO

-- Insert sample suppliers
INSERT INTO Supplier (SupplierName, Country, Email, Phone)
VALUES
    ('Contoso Supplies', 'USA', 'contact@contoso.com', '555-0100'),
    ('Fabrikam Inc', 'Canada', 'sales@fabrikam.com', '555-0200');

-- Insert sample categories
INSERT INTO Category (CategoryName, Description)
VALUES
    ('Electronics', 'Electronic devices and accessories'),
    ('Clothing', 'Apparel and fashion items');

-- Insert sample products
INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice, StockQuantity)
VALUES
    ('Wireless Mouse', 1, 1, 29.99, 100),
    ('Cotton T-Shirt', 2, 2, 19.99, 250);
GO
```

Create a temporal table for price history

Temporal tables automatically track changes over time. This task creates a price history table that maintains a full audit trail of price changes.

1. Select **New Query**. Copy and paste the following T-SQL code into the query window. Select **Execute** to execute this query.

sql

```
USE EcommerceDB;
GO

-- Create Price History table with temporal versioning
CREATE TABLE ProductPrice (
   PriceID INT PRIMARY KEY IDENTITY(1,1),
   ProductID INT NOT NULL,
   CurrentPrice DECIMAL(10,2) NOT NULL,
   EffectiveDate DATE,
   SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START HIDDEN,
   SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END HIDDEN,
   PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime),
   FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
) WITH (SYSTEM_VERSIONING = ON);
GO

-- Insert initial price data
INSERT INTO ProductPrice (ProductID, CurrentPrice, EffectiveDate)
VALUES (1, 99.99, '2025-01-01'), (2, 149.99, '2025-01-01');

-- Update price (creates history entry)
UPDATE ProductPrice SET CurrentPrice = 109.99 WHERE ProductID = 1;
GO
```

2. Query the price history to see changes over time. Copy and paste the following T-SQL code into a new query window. Select **Execute** to execute this query.

sql

```
USE EcommerceDB;
GO

-- Query price history
SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
FROM ProductPrice
FOR SYSTEM_TIME ALL
WHERE ProductID = 1;
```

📝 Note that the temporal table shows both the current price and the previous price with their respective time ranges.

Add JSON columns for metadata

JSON columns store flexible, variable data that differs by product type. This task adds metadata storage and creates an index on a frequently queried property.

1. Select **New Query**. Copy and paste the following T-SQL code into the query window. Select **Execute** to execute this query.

sql

```
USE EcommerceDB;
GO

-- Add metadata column to Product (JSON type requires SQL Server 2025)
ALTER TABLE Product ADD Metadata JSON;
GO

-- Add computed column for indexing
```

```
ALTER TABLE Product ADD MetadataColor AS JSON_VALUE(Metadata, '$.color');
GO

-- Create index on the computed column
CREATE NONCLUSTERED INDEX IX_Product_Metadata_Color
  ON Product (MetadataColor);
GO

-- Update products with metadata
UPDATE Product SET Metadata =
N'{"color":"blue","size":"large","material":"cotton"}'
WHERE ProductID = 1;

UPDATE Product SET Metadata = N'{"color":"red","size":"small","material":"silk"}'
WHERE ProductID = 2;
GO
```

2. Query the JSON data. Copy and paste the following T-SQL code into a new query window. Select **Execute** to execute this query.

```
sql
USE EcommerceDB;
GO

-- Query JSON data
SELECT
  ProductID,
  ProductName,
  JSON_VALUE(Metadata, '$.color') AS Color,
  JSON_VALUE(Metadata, '$.size') AS Size,
  JSON_VALUE(Metadata, '$.material') AS Material
```

```
FROM Product
WHERE JSON_VALUE(Metadata, '$.color') = 'blue';
```

Create a partitioned order table

Partitioning divides large tables into smaller segments for faster queries and easier maintenance. This task creates a partitioned orders table.

1. Select **New Query**. Copy and paste the following T-SQL code into the query window. Select **Execute** to execute this query.

sql

```
USE EcommerceDB;
GO


-- Create partition function for order dates
-- Use RANGE RIGHT for date columns to keep same-day values together
CREATE PARTITION FUNCTION PF_OrderDate (DATE)
    AS RANGE RIGHT FOR VALUES
    ('2025-01-01', '2025-04-01', '2025-07-01', '2025-10-01');


-- Create partition scheme (single filegroup recommended)
CREATE PARTITION SCHEME PS_OrderDate
    AS PARTITION PF_OrderDate ALL TO ([PRIMARY]);


-- Create partitioned Order table
-- Include OrderDate in primary key for clustered index alignment
CREATE TABLE [Order] (
    OrderID BIGINT IDENTITY(1,1),
    OrderDate DATE NOT NULL,
    CustomerName NVARCHAR(100) NOT NULL,
    TotalAmount DECIMAL(12,2) NOT NULL,
```

```
    OrderStatus NVARCHAR(20) DEFAULT 'Pending',
    CONSTRAINT PK_Order PRIMARY KEY (OrderID, OrderDate),
    CHECK (TotalAmount > 0),
    CHECK (OrderStatus IN ('Pending', 'Processing', 'Shipped', 'Delivered',
'Cancelled'))
) ON PS_OrderDate(OrderDate);

-- Create partitioned index
CREATE NONCLUSTERED INDEX IX_Order_Customer
    ON [Order](CustomerName)
    ON PS_OrderDate(OrderDate);
GO

-- Insert sample orders
INSERT INTO [Order] (OrderDate, CustomerName, TotalAmount, OrderStatus)
VALUES
    ('2025-01-15', 'John Smith', 299.97, 'Delivered'),
    ('2025-02-20', 'Jane Doe', 149.99, 'Shipped'),
    ('2025-06-10', 'Bob Johnson', 449.95, 'Processing');
GO
```

2. Query by partition. Copy and paste the following T-SQL code into a new query window. Select **Execute** to execute this query.

```
sql
USE EcommerceDB;
GO

-- Query by partition
SELECT
    $PARTITION.PF_OrderDate(OrderDate) AS PartitionNumber,
```

```
    COUNT(*) AS OrdersInPartition,
    MIN(OrderDate) AS MinDate,
    MAX(OrderDate) AS MaxDate
FROM [Order]
GROUP BY $PARTITION.PF_OrderDate(OrderDate);
```

Create order details with SEQUENCE

Sequences generate unique numbers independently of any table. This task uses a sequence for order line item identifiers.

1. Select **New Query**. Copy and paste the following T-SQL code into the query window. Select **Execute** to execute this query.

sql

```
USE EcommerceDB;
GO

-- Create SEQUENCE for order line items
CREATE SEQUENCE OrderLineSequence
    START WITH 1
    INCREMENT BY 1;

-- Create OrderDetail table
CREATE TABLE OrderDetail (
    OrderLineID INT PRIMARY KEY,
    OrderID BIGINT NOT NULL,
    OrderDate DATE NOT NULL,
    ProductID INT NOT NULL,
    Quantity INT NOT NULL,
    UnitPrice DECIMAL(10,2) NOT NULL,
    LineTotal AS (Quantity * UnitPrice),
```

```
    CHECK (Quantity > 0),
    CHECK (UnitPrice > 0),
    FOREIGN KEY (OrderID, OrderDate) REFERENCES [Order](OrderID, OrderDate),
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);
GO

-- Insert order details using SEQUENCE
INSERT INTO OrderDetail (OrderLineID, OrderID, OrderDate, ProductID, Quantity, UnitPrice)
VALUES
    (NEXT VALUE FOR OrderLineSequence, 1, '2025-01-15', 1, 2, 99.99),
    (NEXT VALUE FOR OrderLineSequence, 1, '2025-01-15', 2, 1, 149.99),
    (NEXT VALUE FOR OrderLineSequence, 2, '2025-02-20', 1, 3, 99.99);
GO
```

2. Verify the data. Copy and paste the following T-SQL code into a new query window. Select **Execute** to execute this query.

```
sql
USE EcommerceDB;
GO

SELECT * FROM OrderDetail;
```

Verify database objects

Run verification queries to ensure all database objects were created correctly.

1. Select **New Query**. Copy and paste the following T-SQL code into the query window. Select **Execute** to execute this query.

```
sql
USE EcommerceDB;
```

GO

-- Verify constraints work

-- This should fail: negative price

INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice, StockQuantity)

VALUES ('Invalid', 1, 1, -50, 10);

📝 This query should fail with a CHECK constraint violation, confirming that the constraint is working correctly.

2. Verify the JSON and partitioning queries. Copy and paste the following T-SQL code into a new query window. Select **Execute** to execute this query.

sql

USE EcommerceDB;

GO

-- Verify JSON queries work

SELECT ProductName, JSON_VALUE(Metadata, '$.color') AS Color

FROM Product

WHERE Metadata IS NOT NULL;

-- Verify partitioning

SELECT $PARTITION.PF_OrderDate(OrderDate) AS Partition, COUNT(*) AS RecordCount

FROM [Order]

GROUP BY $PARTITION.PF_OrderDate(OrderDate);

-- Verify temporal table

SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime

FROM ProductPrice FOR SYSTEM_TIME ALL

```
ORDER BY ProductID, SysStartTime;
```

Cleanup

If you are not using the database or the lab files for any other purpose, you can clean up the objects you created in this lab.

1. From the lab virtual machine or your local machine if one wasn't provided, start a SQL Server Management Studio session (SSMS).
2. When SSMS opens, by default the **Connect to Server** dialog will appear. Choose the Default instance and select **Connect**. You might need to check the **Trust server certificate** checkbox.
3. In **Object Explorer**, expand the **Databases** folder.
4. Right-click on the **EcommerceDB** database and select **Delete**.
5. In the **Delete Object** dialog, check the **Close existing connections** checkbox.
6. Select **OK**.

You have successfully completed this lab.

In this exercise, you've learned how to design and implement various database objects in SQL Server, including tables with constraints, temporal tables for tracking changes, JSON columns for flexible metadata storage, partitioned tables for large data sets, and sequences for generating unique identifiers.

Summary

Database objects form the foundation of every SQL solution. Throughout this module, you explored how to design and implement tables, indexes, constraints, and specialized structures across SQL Server, Azure SQL Database, Azure SQL Managed Instance, and SQL database in Microsoft Fabric.

In this module, you learned to select appropriate data types for storage efficiency and query performance, understanding the trade-offs between precision and space. You explored rowstore indexes for transactional workloads—including clustered indexes that define physical row order and nonclustered indexes for alternate access paths—and columnstore indexes for analytical queries, learning how rowgroups, column segments, and the tuple-mover optimize compression and query performance.

You implemented specialized table types including temporal tables for automatic change tracking, ledger tables for tamper-evident compliance scenarios, graph tables for relationship modeling, and memory-optimized tables for high-throughput OLTP workloads. You learned to enforce data integrity through constraints—PRIMARY KEY, FOREIGN KEY, CHECK, UNIQUE, and DEFAULT—and to generate unique values using both IDENTITY columns and SEQUENCE objects for cross-table numbering.

You explored JSON support for semi-structured data, including the native json data type in SQL Server 2025, and indexing strategies using computed columns. Finally, you designed partitioning strategies for large tables, understanding partition functions with RANGE RIGHT for datetime columns, partition schemes for filegroup placement, and the requirements for clustered indexes on partitioned tables.

Additional resources

To deepen your understanding of database object design and implementation, explore these resources:

- Tables
- Indexes
- Columnstore indexes overview
- Temporal tables
- Ledger overview
- Graph processing with SQL Server
- In-Memory OLTP overview
- JSON data in SQL Server
- Partitioned tables and indexes
- SQL database in Microsoft Fabric

Next steps

With strong database object design skills, you're ready to:

- Explore advanced T-SQL programming with views, functions, and stored procedures
- Implement security and data protection measures
- Optimize query performance using execution plans

- Integrate AI capabilities into your database solutions
- Manage CI/CD and deployment pipelines for database changes

Continue building your expertise in SQL solutions to create robust, scalable, and maintainable database systems.

Implement programmability objects with SQL

Learn how to create and use views, stored procedures, scalar functions, table-valued functions, and triggers to build maintainable, secure, and efficient database solutions.

Introduction

Completed100 XP

- 2 minutes

Choose your preferred content format

SQL Server provides several programmability objects that help you encapsulate logic, improve code reusability, and enforce business rules within your database. These objects—views, stored procedures, functions, and triggers—each serve distinct purposes and offer unique capabilities for database development.

Scenario

You're a database developer at a growing e-commerce company. Your team manages a SQL Server database that handles customer orders, inventory, and reporting. As the application grows more complex, you notice:

- Developers write the same JOIN queries repeatedly across different applications
- Business logic is scattered throughout application code, making it hard to maintain
- Some data modifications need automatic validation and logging
- Complex calculations appear in multiple queries, leading to inconsistencies

You decide to create specific SQL Server objects to centralize logic, improve maintainability, and enhance security across your database applications.

What you'll learn

In this module, you'll explore the core programmability objects in SQL Server:

- **Views** - Virtual tables that simplify data access and provide security boundaries
- **Stored procedures** - Precompiled T-SQL code blocks for complex operations and data modifications
- **Scalar functions** - Reusable calculations that return single values
- **Table-valued functions** - Functions that return result sets for use in queries
- **Triggers** - Automatic responses to data modifications or database events

You'll also learn decision criteria for choosing the right programmability object based on your specific requirements.

Create views

Completed100 XP

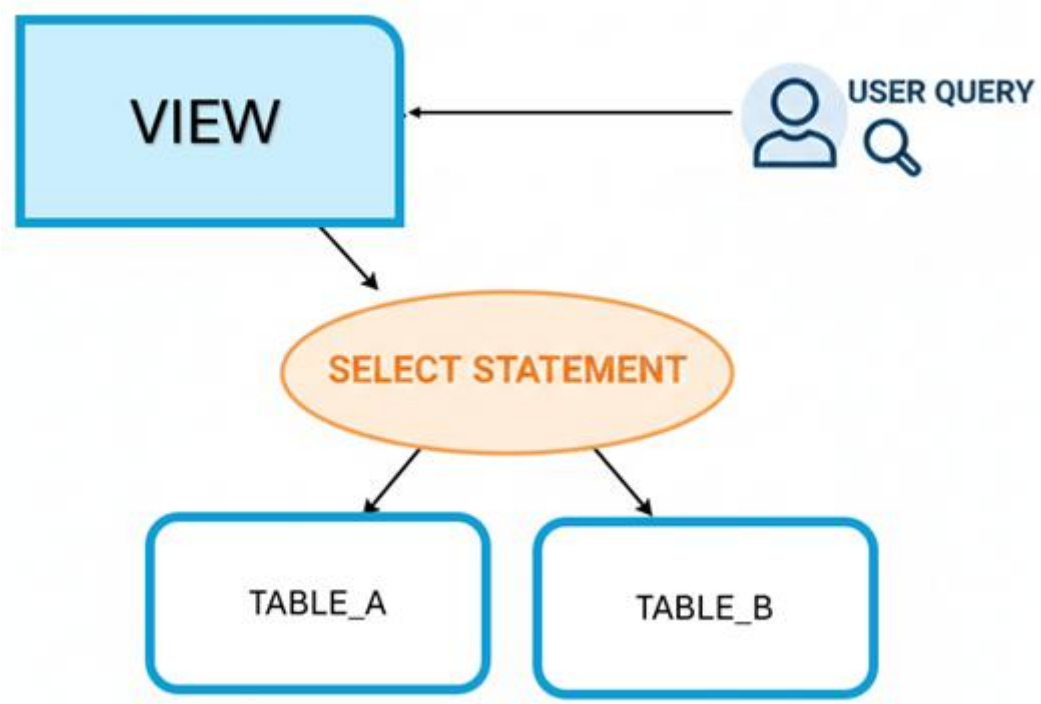
- 6 minutes

Choose your preferred content format

Views simplify how you access and present data in SQL Server. As a SQL developer, you create views to encapsulate complex queries, provide security boundaries, and present data in a format that matches your application's needs.

Understand views in SQL Server

A **view** is a virtual table based on a SELECT statement. Unlike physical tables, views don't store data themselves. Instead, they retrieve data from underlying tables each time you query them.



With views, you can hide the complexity of JOIN clauses, calculations, and filters from application code. For example, if your application frequently needs customer orders with product details, you can create a view that uses JOIN across the Customers, Orders, and Products tables. Your application then queries a single view instead of writing the same complex JOIN repeatedly.

Views also provide a security layer. You can grant users access to specific columns through a view while restricting access to the underlying tables. This approach lets you expose only the data users need without giving them full table access.

Create a basic view

You create a view using the CREATE VIEW statement followed by a SELECT query. The view definition determines what data appears when you query the view.

Here's a simple view that combines customer and order information:

SQL

```
CREATE VIEW Sales.CustomerOrders

AS

SELECT

  c.CustomerID,

  c.CustomerName,

  c.Email,
```

```
    o.OrderID,
    o.OrderDate,
    o.TotalAmount
FROM Sales.Customers c
INNER JOIN Sales.Orders o ON c.CustomerID = o.CustomerID;
```

After creating this view, you can query it like any table:

SQL

```
SELECT * FROM Sales.CustomerOrders
WHERE CustomerName = 'Contoso Ltd';
```

The view executes the underlying SELECT statement and returns results as if they came from a single table. This simplification makes your application code cleaner and easier to maintain.

You can also create views with calculated columns. The following view adds a column that categorizes orders by size:

SQL

```
CREATE VIEW Sales.OrderSummary
AS
SELECT
    OrderID,
    CustomerID,
    OrderDate,
    TotalAmount,
    CASE
        WHEN TotalAmount < 100 THEN 'Small'
        WHEN TotalAmount < 1000 THEN 'Medium'
        ELSE 'Large'
    END AS OrderSize
FROM Sales.Orders;
```

This view handles the categorization logic in one place. Every query against OrderSummary gets the same calculation without duplicating the CASE expression.

Apply design considerations

When designing views, consider how they'll be used and maintained. Well-designed views balance simplicity, performance, and security.

Specify column names explicitly instead of using SELECT *. Explicit columns make views more maintainable and prevent unexpected results when underlying tables change. If someone adds a column to a base table, your view definition stays consistent:

SQL

```
CREATE VIEW Sales.ActiveCustomers
AS
SELECT
    CustomerID,
    CustomerName,
    Email,
    Phone
FROM Sales.Customers
WHERE IsActive = 1;
```

Use the WITH CHECK OPTION clause when views will handle data modifications. This option ensures that INSERT and UPDATE statements through the view only affect rows visible in the view:

SQL

```
CREATE VIEW Sales.HighValueOrders
AS
SELECT
    OrderID,
    CustomerID,
    OrderDate,
    TotalAmount
```

```
FROM Sales.Orders
WHERE TotalAmount > 1000
WITH CHECK OPTION;
```

With this option, you can't insert an order with TotalAmount of 500 through the HighValueOrders view. The database rejects the operation because the new row wouldn't meet the view's WHERE condition.

Keep view definitions focused on a specific purpose. A view that tries to serve multiple purposes often becomes difficult to optimize and understand. Create separate views for different use cases rather than building one complex view.

Determine when to use views

Views excel at specific scenarios where their characteristics provide clear benefits. Understanding these scenarios helps you choose the right tool for each situation.

Use views to simplify complex queries that multiple applications or users need to execute. Instead of requiring everyone to understand the complexity of joining five tables with specific conditions, create a view once. This centralization also means you can optimize or modify the logic in one place.

Consider views when you need to restrict data access at the column or row level. A view can expose only the columns appropriate for a role while hiding sensitive information. Combined with appropriate permissions, views let you grant access to specific data without exposing entire tables.

Views work well for presenting data in different formats for different purposes. You might have a *Products* table with technical details, but your reporting team needs a simplified version with calculated fields. Create a view that transforms and aggregates the data appropriately.

At the same time, recognize when other objects serve better. For performance-critical queries that always return the same results, indexed views (materialized views) store the result set physically. For complex calculations that accept parameters, user-defined functions provide more flexibility. For data modification logic, stored procedures offer better control.

When you need to encapsulate reusable query logic without parameters, and you want to present data in a simplified way, views are your solution. They bridge the gap between the physical database structure and the logical view of data your applications need.

Create stored procedures

Completed100 XP

- 6 minutes

Choose your preferred content format

Stored procedures are one of the most powerful tools in SQL Server for encapsulating business logic and improving application performance. When you create stored procedures, you build reusable code blocks that execute on the server, reducing network traffic and centralizing data access logic.

Understand stored procedures

A stored procedure is a compiled collection of T-SQL statements that SQL Server stores and executes as a single unit. Unlike unplanned queries that you send to the server each time, stored procedures are precompiled and optimized, which means they run faster on subsequent executions.

You use stored procedures to encapsulate complex business logic, enforce data validation rules, and control how applications interact with your database. For example, instead of allowing direct table access, you can create stored procedures that validate input, apply business rules, and log changes before modifying data.

The performance benefits come from query plan caching. With unplanned queries, SQL Server must parse and optimize each query every time. With stored procedures, the execution plan is cached after the first run, reducing overhead for repeated operations.

Create basic stored procedures

Creating a stored procedure starts with the CREATE PROCEDURE statement followed by your T-SQL logic. You specify the procedure name using a schema-qualified identifier, which improves clarity and performance.

SQL

```
CREATE PROCEDURE dbo.GetCustomerOrders
AS
BEGIN
  SET NOCOUNT ON;

  SELECT
```

```
    OrderID,
    CustomerID,
    OrderDate,
    TotalAmount
  FROM dbo.Orders
  ORDER BY OrderDate DESC;
END
```

The SET NOCOUNT ON statement prevents the message about the number of rows affected from being sent to the client. This reduces network traffic and improves performance, especially when the procedure executes multiple statements.

When you create procedures, use the BEGIN and END keywords to clearly define the procedure body. This makes your code more readable and helps prevent errors when adding or modifying logic later.

Work with parameters

Parameters make stored procedures flexible and reusable. You define input parameters to accept values from the calling application, and output parameters to return values back to the caller.

Input parameters use the @ symbol followed by a parameter name and data type. You can provide default values to make parameters optional:

SQL

```
CREATE PROCEDURE dbo.GetCustomerOrdersByDate
  @CustomerID int,
  @StartDate datetime = NULL,
  @EndDate datetime = NULL
AS
BEGIN
  SET NOCOUNT ON;

  SELECT
    OrderID,
```

```
    OrderDate,
    TotalAmount
  FROM dbo.Orders
  WHERE CustomerID = @CustomerID
    AND (@StartDate IS NULL OR OrderDate >= @StartDate)
    AND (@EndDate IS NULL OR OrderDate <= @EndDate)
  ORDER BY OrderDate DESC;
END
```

Output parameters let you return values to the calling application. You define them using the OUTPUT keyword:

SQL

```
CREATE PROCEDURE dbo.CalculateOrderTotal
  @OrderID int,
  @TotalAmount decimal(10,2) OUTPUT
AS
BEGIN
  SET NOCOUNT ON;

  SELECT @TotalAmount = SUM(Quantity * UnitPrice)
  FROM dbo.OrderDetails
  WHERE OrderID = @OrderID;

  RETURN 0;
END
```

When you call a procedure with output parameters, you must declare a variable to receive the value and use the OUTPUT keyword in the EXECUTE statement.

Implement error handling

Robust stored procedures include error handling to manage unexpected conditions and maintain data integrity. You implement error handling using

TRY...CATCH blocks, which work similarly to exception handling in other programming languages.

SQL

```
CREATE PROCEDURE dbo.InsertCustomerOrder
  @CustomerID int,
  @OrderDate datetime,
  @TotalAmount decimal(10,2)
AS
BEGIN
  SET NOCOUNT ON;

  BEGIN TRY
    BEGIN TRANSACTION;

    -- Validate customer exists
    IF NOT EXISTS (SELECT 1 FROM dbo.Customers WHERE CustomerID = @CustomerID)
    BEGIN
      RAISERROR('Customer does not exist.', 16, 1);
    END

    -- Insert order
    INSERT INTO dbo.Orders (CustomerID, OrderDate, TotalAmount)
    VALUES (@CustomerID, @OrderDate, @TotalAmount);

    COMMIT TRANSACTION;
    RETURN 0;
  END TRY
  BEGIN CATCH
```

```
    IF @@TRANCOUNT > 0
      ROLLBACK TRANSACTION;

    DECLARE @ErrorMessage nvarchar(4000) = ERROR_MESSAGE();
    DECLARE @ErrorSeverity int = ERROR_SEVERITY();
    DECLARE @ErrorState int = ERROR_STATE();

    RAISERROR(@ErrorMessage, @ErrorSeverity, @ErrorState);
    RETURN -1;
  END CATCH
END
```

The TRY block contains your main logic, while the CATCH block handles any errors that occur. You can use system functions like ERROR_MESSAGE(), ERROR_SEVERITY(), and ERROR_STATE() to capture error details and pass them to the calling application.

Always check @@TRANCOUNT before rolling back transactions in the CATCH block. This prevents errors if the transaction already completed or was never started.

Apply best practices

Following established best practices when you create stored procedures ensures they're maintainable, secure, and performant.

Use schema-qualified names

Use schema-qualified names for all objects. This eliminates ambiguity and improves performance by avoiding schema resolution overhead:

SQL

```
-- Good
SELECT * FROM dbo.Orders

-- Avoid
SELECT * FROM Orders
```

Implement parameter validation

Implement parameter validation at the start of your procedure. Fail fast when inputs are invalid rather than processing bad data:

SQL

```
IF @CustomerID IS NULL OR @CustomerID <= 0
BEGIN
  RAISERROR('CustomerID must be a positive integer.', 16, 1);
  RETURN -1;
END
```

Avoid SELECT *

Avoid SELECT * in production code. Explicitly list columns to prevent issues when table structures change and to improve query performance:

SQL

```
-- Good
SELECT OrderID, CustomerID, OrderDate FROM dbo.Orders


-- Avoid
SELECT * FROM dbo.Orders
```

Use meaningful names

Use meaningful names that describe what the procedure does. Include a verb that indicates the operation (Get, Insert, Update, Delete, Calculate):

SQL

```
CREATE PROCEDURE dbo.GetActiveCustomersByRegion
CREATE PROCEDURE dbo.UpdateCustomerAddress
CREATE PROCEDURE dbo.DeleteExpiredOrders
```

Avoid the sp_ prefix

Don't use the sp_ prefix for your stored procedures. SQL Server reserves this prefix for system procedures stored in the master database. When you name a procedure with sp_, SQL Server first searches master before checking the current database, adding unnecessary overhead:

SQL

```
-- Good

CREATE PROCEDURE dbo.GetCustomerOrders


-- Avoid

CREATE PROCEDURE dbo.sp_GetCustomerOrders
```

Building on these practices helps you create stored procedures that your team can understand, maintain, and trust to perform reliably in production environments.

Create scalar functions

Completed100 XP

- 6 minutes

Choose your preferred content format

Scalar functions are essential tools in SQL Server that allow you to encapsulate reusable logic and return a single value. You can use them directly in SELECT statements, WHERE clauses, and other T-SQL expressions, making your queries more maintainable and your code more modular.

Understand scalar function fundamentals

A scalar function accepts zero or more parameters and returns a single value of a specified data type. Unlike stored procedures, scalar functions can be embedded directly in SQL expressions wherever you would use a column or variable.

The key characteristics of scalar functions include their ability to accept input parameters, perform calculations or transformations, and return exactly one value. You define the return data type explicitly in the function definition, which SQL Server validates at creation time.

When you create a scalar function, you're creating a reusable piece of logic that other developers can call throughout the database. This promotes code reuse and helps maintain consistency across your applications.

Define scalar function syntax

To create a scalar function, you use the CREATE FUNCTION statement with specific syntax components. The basic structure includes the function name, parameters, return type, and function body.

Here's the fundamental syntax pattern:

SQL

```
CREATE FUNCTION schema_name.function_name
(
  @parameter1 datatype,
  @parameter2 datatype
)
RETURNS return_datatype
AS
BEGIN
  -- Function logic here
  RETURN @result
END
```

The RETURNS clause specifies the data type of the single value the function returns. Within the BEGIN…END block, you write your T-SQL logic and use a RETURN statement to send back the result.

For example, you can create a simple function that calculates sales tax:

SQL

```
CREATE FUNCTION dbo.CalculateSalesTax
(
  @Amount DECIMAL(10,2),
  @TaxRate DECIMAL(5,4)
)
RETURNS DECIMAL(10,2)
AS
BEGIN
  DECLARE @TaxAmount DECIMAL(10,2)
  SET @TaxAmount = @Amount * @TaxRate
  RETURN @TaxAmount
END
```

This function accepts two parameters and returns the calculated tax amount. You can use this function in any SELECT statement.

Implement scalar functions with business logic

Scalar functions excel at encapsulating business rules and calculations that you need to apply consistently across your database. With scalar functions, you centralize logic that might otherwise be duplicated in multiple queries or application code.

Consider a scenario where you need to calculate employee tenure in years. You create a scalar function that accepts a hire date and returns the number of complete years:

SQL

```
CREATE FUNCTION dbo.GetEmployeeTenure
(
  @HireDate DATE
)
RETURNS INT
AS
BEGIN
  DECLARE @Tenure INT
  SET @Tenure = DATEDIFF(YEAR, @HireDate, GETDATE())
  RETURN @Tenure
END
```

You can use this function in queries to display tenure information:

SQL

```
SELECT
  EmployeeName,
  HireDate,
  dbo.GetEmployeeTenure(HireDate) AS YearsOfService
FROM Employees
WHERE dbo.GetEmployeeTenure(HireDate) >= 5
```

This approach ensures consistent tenure calculation throughout your database. If the business rules change, you modify the function once rather than updating multiple queries.

Note

This function uses GETDATE(), which makes it non-deterministic. Non-deterministic functions can't be used in indexed views or indexes on computed columns. For scenarios requiring determinism, pass the current date as a parameter instead.

Apply best practices for scalar functions

When you create scalar functions, several best practices help ensure optimal performance and maintainability. Understanding these practices helps you avoid common pitfalls and create efficient, reliable functions.

First, keep your scalar functions deterministic whenever possible. A deterministic function always returns the same result given the same input parameters. Functions that reference system date/time functions or tables are non-deterministic and may prevent certain query optimizations.

Also, avoid side effects in your functions. Scalar functions shouldn't modify database state or have dependencies on external resources. This restriction exists because SQL Server may execute functions multiple times or in different orders than you expect.

Lastly, be mindful of performance implications. When you use a scalar function in a WHERE clause or SELECT list with large tables, SQL Server may execute the function for every row. This can significantly impact query performance. For such scenarios, consider inline table-valued functions as an alternative.

Here's an example of a well-designed scalar function that follows these practices:

SQL

```
CREATE FUNCTION dbo.FormatPhoneNumber
(
    @PhoneNumber VARCHAR(10)
)
RETURNS VARCHAR(14)
AS
BEGIN
```

```
    DECLARE @FormattedNumber VARCHAR(14)

    IF LEN(@PhoneNumber) = 10
        SET @FormattedNumber = '(' + SUBSTRING(@PhoneNumber, 1, 3) + ') ' +
                               SUBSTRING(@PhoneNumber, 4, 3) + '-' +
                               SUBSTRING(@PhoneNumber, 7, 4)
    ELSE
        SET @FormattedNumber = @PhoneNumber

    RETURN @FormattedNumber
END
```

This function is deterministic, has no side effects, and performs a straightforward transformation. It handles invalid input gracefully by returning the original value when the phone number doesn't match the expected format.

Modify and manage scalar functions

After you create a scalar function, you can modify its definition using the ALTER FUNCTION statement. The ALTER FUNCTION syntax mirrors CREATE FUNCTION but allows you to change the function without dropping and recreating it, which preserves permissions and dependencies.

SQL

```
ALTER FUNCTION dbo.CalculateSalesTax
(
    @Amount DECIMAL(10,2),
    @TaxRate DECIMAL(5,4)
)
RETURNS DECIMAL(10,2)
AS
BEGIN
    DECLARE @TaxAmount DECIMAL(10,2)
    -- Updated logic with rounding
```

```
    SET @TaxAmount = ROUND(@Amount * @TaxRate, 2)
    RETURN @TaxAmount
END
```

To remove a scalar function, you use the DROP FUNCTION statement:

SQL

```
DROP FUNCTION IF EXISTS dbo.CalculateSalesTax
```

The IF EXISTS clause prevents errors if the function doesn't exist, which is useful in deployment scripts. Before dropping a function, verify that no other database objects depend on it by checking system views like sys.sql_expression_dependencies.

Create table-valued functions

Completed100 XP

- 5 minutes

Choose your preferred content format

Table-valued functions let you encapsulate complex query logic into reusable components that return result sets. You can call these functions directly in queries, just like tables or views, making your code more modular and maintainable.

When you build database applications, you often need to retrieve filtered or calculated data sets based on input parameters. Table-valued functions solve this problem by packaging query logic into functions that accept parameters and return tables. Unlike stored procedures, you can use table-valued functions in JOIN clauses and SELECT statements, giving you the flexibility to treat function results as data sources.

Understand table-valued function types

SQL Server provides two types of table-valued functions, each suited for different scenarios.

Inline table-valued function

Contains a single SELECT statement and returns results directly. With inline functions, you don't define the table structure—SQL Server infers it from your SELECT statement. The query optimizer treats inline table-valued functions like views with parameters, often producing better execution plans.

Multi-statement table-valued function

Uses a BEGIN…END block and explicitly declares the structure of the returned table. This type gives you more control when you need to execute multiple statements, perform complex calculations, or build the result set iteratively. However, this flexibility comes with a performance trade-off, as the optimizer treats these functions differently.

The choice between these functions depends on your specific requirements. For simple queries with parameters, inline functions provide better performance. When you need procedural logic or multiple steps to build your result set, multi-statement functions become necessary.

Create inline table-valued functions

Inline table-valued functions offer a concise way to parameterize queries. You define them with a single RETURN statement followed by a SELECT query.

The following example shows an inline function that retrieves orders for a specific customer:

SQL

```
CREATE FUNCTION dbo.GetCustomerOrders
(
  @CustomerID INT
)
RETURNS TABLE
AS
RETURN
(
  SELECT
    OrderID,
    OrderDate,
    TotalAmount,
    Status
  FROM Sales.Orders
  WHERE CustomerID = @CustomerID
);
```

You can now use this function in queries just like a table:

SQL

```
SELECT OrderID, OrderDate, TotalAmount
FROM dbo.GetCustomerOrders(1001)
WHERE OrderDate >= '2024-01-01';
```

The function accepts the customer ID parameter and returns only that customer's orders. You can further filter, JOIN, or aggregate the results as needed. This approach keeps your main query clean while encapsulating the customer filtering logic.

Create multi-statement table-valued functions

Multi-statement table-valued functions provide more flexibility when you need to perform multiple operations to build your result set.

Consider a function that calculates product sales summaries with multiple aggregations:

SQL

```
CREATE FUNCTION dbo.GetProductSalesSummary
(
  @StartDate DATE,
  @EndDate DATE
)
RETURNS @SalesSummary TABLE
(
  ProductID INT,
  ProductName NVARCHAR(100),
  TotalQuantity INT,
  TotalRevenue DECIMAL(18,2),
  AveragePrice DECIMAL(18,2)
)
AS
BEGIN
```

```
    INSERT INTO @SalesSummary
    SELECT
        p.ProductID,
        p.ProductName,
        SUM(od.Quantity) AS TotalQuantity,
        SUM(od.Quantity * od.UnitPrice) AS TotalRevenue,
        AVG(od.UnitPrice) AS AveragePrice
    FROM Production.Products p
    INNER JOIN Sales.OrderDetails od ON p.ProductID = od.ProductID
    INNER JOIN Sales.Orders o ON od.OrderID = o.OrderID
    WHERE o.OrderDate BETWEEN @StartDate AND @EndDate
    GROUP BY p.ProductID, p.ProductName;


    RETURN;
END;
```

Notice how you explicitly declare the table variable @SalesSummary with specific columns and data types. The function body inserts data into this table variable, then returns it. This structure allows you to add additional processing logic, error handling, or conditional statements as needed.

Use table-valued functions in queries

Table-valued functions integrate seamlessly into your queries, enabling powerful data retrieval patterns.

You can join function results with other tables:

SQL

```
SELECT
    c.CustomerName,
    s.ProductName,
    s.TotalRevenue
FROM Customers c
```

```
CROSS APPLY dbo.GetProductSalesSummary('2024-01-01', '2024-12-31') s
WHERE s.TotalRevenue > 10000
ORDER BY s.TotalRevenue DESC;
```

The CROSS APPLY operator calls the function for each row from the Customers table, though in this example, the function parameters are constants. When you pass column values as parameters, CROSS APPLY becomes particularly useful:

SQL

```
SELECT
  c.CustomerName,
  o.OrderID,
  o.TotalAmount
FROM Customers c
CROSS APPLY dbo.GetCustomerOrders(c.CustomerID) o
WHERE o.Status = 'Completed';
```

This query retrieves all completed orders for each customer, demonstrating how table-valued functions enable row-by-row processing within your queries. The function acts as a correlated subquery but with better readability and reusability.

For inline table-valued functions that don't require row-by-row evaluation, you can use INNER JOIN syntax as well:

SQL

```
SELECT
  c.CustomerName,
  o.OrderDate,
  o.TotalAmount
FROM Customers c
INNER JOIN dbo.GetCustomerOrders(c.CustomerID) o ON 1=1
WHERE YEAR(o.OrderDate) = 2024;
```

With these techniques, you can build complex queries from simpler, tested function components, improving both code maintainability and development efficiency.

Create triggers

Completed100 XP

- 6 minutes

Choose your preferred content format

Triggers are special stored procedures that automatically execute when specific events occur in your database. You define triggers to maintain data integrity, enforce business rules, and automate database operations without requiring application-level code.

Understand trigger fundamentals

A trigger responds to data modification or schema changes in your database. When you create a trigger, you specify the event that activates it and the actions it performs.

Triggers execute automatically. Unlike stored procedures that you call explicitly, triggers fire in response to INSERT, UPDATE, DELETE, or DDL statements. This automatic execution makes them powerful for enforcing rules that must apply consistently across all data modifications.

SQL Server supports two main categories of triggers: DML (Data Manipulation Language) triggers and DDL (Data Definition Language) triggers. DML triggers respond to changes in table data, while DDL triggers respond to schema changes like CREATE, ALTER, or DROP statements.

Create DML triggers for data modifications

DML triggers monitor and respond to data changes in tables or views. You define them as either AFTER triggers or **INSTEAD OF** triggers.

AFTER triggers execute after the triggering statement completes. The database first performs the data modification, then runs the trigger code. You use AFTER triggers to validate changes, update related tables, or log modifications:

SQL

```
CREATE TRIGGER tr_UpdateInventory

ON Sales.OrderDetails

AFTER INSERT

AS

BEGIN
```

```
    UPDATE Inventory.Products
    SET QuantityInStock = QuantityInStock - i.Quantity
    FROM Inventory.Products p
    INNER JOIN inserted i ON p.ProductID = i.ProductID;
END;
```

INSTEAD OF triggers replace the original data modification statement. The trigger code executes instead of the INSERT, UPDATE, or DELETE operation. You use INSTEAD OF triggers to modify views that normally wouldn't accept direct modifications or to implement complex business logic:

SQL

```
CREATE TRIGGER tr_UpdateOrderView
ON Sales.OrderSummaryView
INSTEAD OF UPDATE
AS
BEGIN
    UPDATE Sales.Orders
    SET OrderStatus = i.OrderStatus,
        ModifiedDate = GETDATE()
    FROM Sales.Orders o
    INNER JOIN inserted i ON o.OrderID = i.OrderID;
END;
```

With DML triggers, you access the **inserted** and **deleted** pseudo-tables. These temporary tables store copies of the affected rows. INSERT operations populate the **inserted** table, DELETE operations populate the **deleted** table, and UPDATE operations populate both tables with old values in **deleted** and new values in **inserted**.

Implement triggers for specific events

You specify which data modification events activate your trigger. A single trigger can respond to multiple events by combining INSERT, UPDATE, and DELETE in the trigger definition.

For precise control, you create separate triggers for each operation. This approach simplifies your code and makes your triggers easier to maintain:

SQL

```
CREATE TRIGGER tr_LogPriceChanges

ON Products.Catalog

AFTER UPDATE

AS

BEGIN

  IF UPDATE(Price)

  BEGIN

    INSERT INTO Audit.PriceHistory (ProductID, OldPrice, NewPrice, ChangeDate)

    SELECT d.ProductID, d.Price, i.Price, GETDATE()

    FROM deleted d

    INNER JOIN inserted i ON d.ProductID = i.ProductID

    WHERE d.Price <> i.Price;

  END;

END;
```

At the same time, you might combine events when the same logic applies to multiple operations. For example, you could create a single audit trigger that responds to INSERT, UPDATE, and DELETE:

SQL

```
CREATE TRIGGER tr_AuditEmployeeChanges

ON HR.Employees

AFTER INSERT, UPDATE, DELETE

AS

BEGIN

  DECLARE @Operation NVARCHAR(10);


  IF EXISTS (SELECT * FROM inserted) AND NOT EXISTS (SELECT * FROM deleted)
```

```
    SET @Operation = 'INSERT';
  ELSE IF EXISTS (SELECT * FROM inserted) AND EXISTS (SELECT * FROM deleted)
    SET @Operation = 'UPDATE';
  ELSE
    SET @Operation = 'DELETE';

  INSERT INTO Audit.EmployeeLog (EmployeeID, Operation, ChangeDate)
  SELECT COALESCE(i.EmployeeID, d.EmployeeID), @Operation, GETDATE()
  FROM inserted i
  FULL OUTER JOIN deleted d ON i.EmployeeID = d.EmployeeID;
END;
```

The UPDATE() function helps you determine which columns changed. You check specific columns to avoid unnecessary processing when only certain fields matter for your business logic.

Apply trigger best practices

Triggers affect database performance because they execute with every qualifying operation. You write efficient trigger code to minimize impact on transaction throughput.

Keep your trigger logic focused and minimal. Execute only essential operations within the trigger body. For complex or time-consuming operations, consider logging the event details and processing them asynchronously through a separate job:

SQL

```
CREATE TRIGGER tr_QueueLargeOrders
ON Sales.Orders
AFTER INSERT
AS
BEGIN
  INSERT INTO Processing.OrderQueue (OrderID, TotalAmount, QueuedDate)
  SELECT OrderID, TotalAmount, GETDATE()
```

```
    FROM inserted
    WHERE TotalAmount > 10000;
END;
```

Avoid recursive operations where a trigger modification causes the same trigger to fire again. Set the RECURSIVE_TRIGGERS database option appropriately and design your triggers to prevent endless loops.

Handle errors properly within triggers. Transaction behavior depends on your error handling. If a trigger encounters an error and you don't handle it, SQL Server rolls back both the trigger and the original statement:

SQL

```
CREATE TRIGGER tr_ValidateOrderDate
ON Sales.Orders
AFTER INSERT, UPDATE
AS
BEGIN
    IF EXISTS (SELECT * FROM inserted WHERE OrderDate > GETDATE())
    BEGIN
        THROW 50001, 'Order date cannot be in the future', 1;
    END;
END;
```

Document your triggers thoroughly. Other developers need to understand why triggers exist and what they do, since they execute invisibly during normal database operations.

Now that you understand how to create and implement triggers, you're ready to explore how they integrate with other programmability objects to build comprehensive database solutions.

Choose when to use each option

Completed100 XP

- 9 minutes

Choose your preferred content format

SQL Server programmability objects provide different ways to encapsulate and reuse logic in your database. Each object type—views, stored procedures, functions, and triggers—serves distinct purposes and offers unique capabilities.

Compare options

The following table summarizes key capabilities and limitations of each object type:

Capability	Views	Stored Procedures	Functions
Accept parameters	No	Yes	Yes
Modify data	Limited	Yes	No
Return result sets	Yes	Yes	Yes (TVFs)
Use in SELECT/JOIN	Yes	No	Yes
Transaction control	No	Yes	No
Automatic execution	No	No	No
Execution plan caching	No	Yes	Varies

Views can modify data only when changes affect a single base table. Inline table-valued functions benefit from plan caching because the optimizer expands them directly into the query plan. Multi-statement TVFs and scalar functions are treated as "black boxes"—the optimizer can't see inside them, which often leads to inaccurate row estimates and suboptimal plans.

Choose based on your requirements

The right programmability object depends on what you need to accomplish. Use this decision framework to guide your selection:

Choose views when you need to:

- Simplify access to complex joins or commonly filtered data
- Provide a security layer by controlling column and row visibility
- Create a stable interface to underlying tables that might change
- Present data without accepting parameters or modifying values

Choose stored procedures when you need to:

- Execute complex business logic with multiple statements
- Modify data across multiple tables in a single transaction

- Accept input parameters and return output parameters or result sets
- Implement error handling and transaction control

Choose functions when you need to:

- Perform reusable calculations that return values for use in queries
- Return parameterized result sets (table-valued functions)
- Embed logic directly in SELECT, WHERE, or JOIN clauses
- Ensure deterministic results for indexing (for specific function types)

Choose triggers when you need to:

- Automatically respond to data modification events
- Enforce complex business rules that extend beyond constraints
- Maintain audit logs of data changes
- Synchronize related data across tables automatically

Apply decision scenarios

Consider these common scenarios and the recommended approach for each:

Scenario	**Recommended Object**	**Why**
Simplify a 5-table join that multiple reports use	View	Encapsulates complexity; parameters needed
Process an order: validate stock, insert order, update inventory	Stored procedure	Multiple modifications in a
Calculate shipping cost based on weight and destination	Scalar function	Reusable calculation in qu
Return all orders for a customer within a date range	Table-valued function	Parameterized result set us
Log all changes to the Salary column	Trigger	Automatic, transparent au
Provide read-only access to employee data without SSN	View	Security layer hiding sensit

Avoid common mistakes

When choosing programmability objects, watch for these pitfalls:

- **Using scalar functions in WHERE clauses on large tables**— The function executes for every row, degrading performance. Consider inline table-valued functions or rewriting the logic.
- **Creating triggers for logic that stored procedures handle better**— Triggers execute implicitly and can be hard to debug. Use them only when automatic execution is essential.
- **Building complex views that nest other views**— Deeply nested views become difficult to optimize and maintain. Keep view definitions focused and shallow.
- **Choosing stored procedures when a function would integrate better**— If you need the result in a SELECT statement, a function provides cleaner syntax than EXEC with temp tables.

With this understanding of each programmability object's strengths and trade-offs, you can select the appropriate tool for your database design and implementation tasks.

Summary

Completed100 XP

- 2 minutes

In this module, you explored SQL Server programmability objects and learned how to use them effectively in your database solutions.

You learned how to:

- **Create views** to simplify data access, hide complexity, and provide security boundaries by exposing only specific columns or rows from underlying tables.
- **Build stored procedures** to encapsulate complex business logic, handle transactions, implement error handling, and create reusable data modification operations.
- **Develop scalar functions** to create reusable calculations that return single values, making your queries more readable and consistent.
- **Implement table-valued functions** using both inline and multi-statement approaches to return result sets that can be used in FROM clauses like tables.

- **Configure triggers** to automatically respond to DML operations (INSERT, UPDATE, DELETE) or DDL events, enabling audit logging, data validation, and enforcement of complex business rules.
- **Choose the right programmability object** based on your specific requirements, considering factors like whether you need to modify data, return single values or result sets, or respond automatically to database events.

Next steps

Now that you understand SQL Server programmability objects, consider:

- Implementing views in your existing databases to simplify complex queries
- Converting repetitive application logic into stored procedures
- Creating functions to standardize calculations across your organization
- Adding audit triggers to track data changes in critical tables

Learn more

Introduction

Implement programmability objects with SQL

Learn how to create and use views, stored procedures, scalar functions, table-valued functions, and triggers to build maintainable, secure, and efficient database solutions.

Choose your preferred content format

As database applications grow in complexity, basic T-SQL queries often fall short. You might need to calculate running totals across time periods, validate data against complex patterns, find customers with similar names despite misspellings, or traverse hierarchical relationships like organizational charts. Without advanced T-SQL knowledge, developers often resort to processing data in application code—moving large datasets across the network, writing custom logic that duplicates built-in database capabilities, and losing the performance benefits of set-based operations.

Understanding advanced T-SQL capabilities enables you to solve these challenges directly in the database engine, where data processing is most efficient. These skills separate database professionals who can only write basic queries from those professionals who can architect complete data solutions. Whether you're building reporting systems, data pipelines, or application backends, mastering

these techniques reduces code complexity, improves performance, and makes your solutions more maintainable.

What you'll learn

In this module, you learn advanced T-SQL techniques for SQL Server, Azure SQL Database, and SQL databases in Microsoft Fabric. You explore:

- Common Table Expressions (CTEs) for organizing complex queries and traversing hierarchical data
- Window functions for ranking, aggregation, and analytical calculations across row sets
- JSON functions for parsing, constructing, and transforming JSON data
- Regular expressions for pattern matching, validation, and text manipulation
- Fuzzy string matching for finding approximate matches in your data
- Graph queries using the MATCH operator for relationship traversal
- Correlated queries for row-by-row comparisons and calculations
- Error handling patterns for building reliable, production-ready code

By the end of this module, you're able to write T-SQL code that handles complex analytical scenarios, processes modern data formats, and responds gracefully to unexpected situations.

Organize queries with common table expressions

Completed100 XP

- 12 minutes

Choose your preferred content format

When working with complex queries, you often need to break down logic into manageable pieces or reference the same subquery multiple times. Common Table Expressions (CTEs) provide a way to define temporary named result sets that exist only during a single query, making your code more readable and maintainable.

Understand CTE syntax

A Common Table Expression is defined using the WITH clause, followed by the CTE name, an optional column list, and a query that defines the result set. The CTE can

then be referenced in the subsequent SELECT, INSERT, UPDATE, or DELETE statement.

SQL

```
WITH CTE_Name (Column1, Column2)
AS
(
  -- CTE query definition
  SELECT Column1, Column2
  FROM SomeTable
  WHERE SomeCondition = 'Value'
)
SELECT * FROM CTE_Name;
```

CTEs offer several advantages over derived tables and subqueries:

- **Improved readability**: Complex queries become easier to understand when broken into named logical sections
- **Self-referencing capability**: Recursive CTEs can reference themselves, enabling hierarchical data traversal
- **Multiple references**: Reference the same CTE multiple times in the outer query without redefining it
- **Modular design**: Build complex queries incrementally by defining multiple CTEs

Note

CTEs are temporary result sets that exist only during query execution. Unlike traditional tables, they don't persist beyond the statement that uses them and don't require explicit cleanup.

Create nonrecursive CTEs

Nonrecursive CTEs define a result set based on a straightforward query that doesn't reference itself. This pattern is useful for simplifying complex joins, breaking down multi-step calculations, or improving code organization.

The following example uses a CTE to calculate sales metrics before joining with product information:

SQL

```
WITH SalesSummary AS
(
  SELECT
    ProductID,
    SUM(OrderQty) AS TotalQuantity,
    SUM(LineTotal) AS TotalRevenue,
    COUNT(DISTINCT SalesOrderID) AS OrderCount
  FROM SalesLT.SalesOrderDetail
  GROUP BY ProductID
)
SELECT
  p.Name AS ProductName,
  p.ProductNumber,
  p.ListPrice,
  ss.TotalQuantity,
  ss.TotalRevenue,
  ss.OrderCount,
  ss.TotalRevenue / NULLIF(ss.TotalQuantity, 0) AS AverageUnitPrice
FROM SalesLT.Product AS p
INNER JOIN SalesSummary AS ss
  ON p.ProductID = ss.ProductID
ORDER BY ss.TotalRevenue DESC;
```

This query first creates a CTE named SalesSummary that aggregates order details by product, calculating total quantity sold, total revenue, and order count. The main query then joins this CTE with the Product table to display product names alongside their sales metrics. The NULLIF function prevents division by zero when calculating the average unit price.

You can define multiple CTEs in a single WITH clause by separating them with commas. Later CTEs can reference earlier ones, enabling progressive data transformation:

SQL

```
WITH CategorySales AS
(
  SELECT
    p.ProductCategoryID,
    SUM(sod.LineTotal) AS CategoryRevenue
  FROM SalesLT.Product AS p
  INNER JOIN SalesLT.SalesOrderDetail AS sod
    ON p.ProductID = sod.ProductID
  GROUP BY p.ProductCategoryID
),
RankedCategories AS
(
  SELECT
    ProductCategoryID,
    CategoryRevenue,
    RANK() OVER (ORDER BY CategoryRevenue DESC) AS RevenueRank
  FROM CategorySales
)
SELECT
  pc.Name AS CategoryName,
  rc.CategoryRevenue,
  rc.RevenueRank
FROM RankedCategories AS rc
INNER JOIN SalesLT.ProductCategory AS pc
  ON rc.ProductCategoryID = pc.ProductCategoryID
```

```
WHERE rc.RevenueRank <= 5;
```

Tip

When building queries with multiple CTEs, start with the most granular data transformations and progressively aggregate or filter in subsequent CTEs. This approach makes debugging easier since you can test each CTE independently.

Create recursive CTEs

Recursive CTEs reference themselves to process hierarchical or graph-like data structures. A recursive CTE consists of two parts: an anchor member that provides the initial result set, and a recursive member that references the CTE to build upon previous results.

The general syntax for a recursive CTE is:

SQL

```
WITH RecursiveCTE AS
(
  -- Anchor member: starting point
  SELECT columns
  FROM table
  WHERE starting_condition

  UNION ALL

  -- Recursive member: references the CTE
  SELECT columns
  FROM table
  INNER JOIN RecursiveCTE
    ON join_condition
)
SELECT * FROM RecursiveCTE;
```

A common use case is traversing an organizational hierarchy. Consider an employee table where each employee has a manager:

SQL

```
WITH EmployeeHierarchy AS
(
  -- Anchor: Start with top-level managers (no manager)
  SELECT
    EmployeeID,
    FirstName,
    LastName,
    ManagerID,
    0 AS Level,
    CAST(FirstName + ' ' + LastName AS NVARCHAR(500)) AS HierarchyPath
  FROM HumanResources.Employee
  WHERE ManagerID IS NULL

  UNION ALL

  -- Recursive: Find employees who report to previously found employees
  SELECT
    e.EmployeeID,
    e.FirstName,
    e.LastName,
    e.ManagerID,
    eh.Level + 1,
    CAST(eh.HierarchyPath + ' > ' + e.FirstName + ' ' + e.LastName AS
NVARCHAR(500))
  FROM HumanResources.Employee AS e
  INNER JOIN EmployeeHierarchy AS eh
    ON e.ManagerID = eh.EmployeeID
```

```
)
SELECT
  EmployeeID,
  FirstName,
  LastName,
  Level,
  HierarchyPath
FROM EmployeeHierarchy
ORDER BY HierarchyPath;
```

Important

Recursive CTEs can cause infinite loops if the termination condition is never met. SQL Server limits recursion to 100 levels by default. Use OPTION (MAXRECURSION n) to change this limit, where n is the maximum recursion depth (0 for unlimited).

Generate sequences with recursive CTEs

Recursive CTEs excel at generating sequences of numbers or dates without requiring a physical numbers table. This technique is useful for creating date ranges, filling gaps in data, or generating test data:

SQL

```
-- Generate a sequence of dates for the current month
WITH DateSequence AS
(
  -- Anchor: Get the first day of the current month
  SELECT CAST(DATEADD(MONTH, DATEDIFF(MONTH, 0, GETDATE()), 0) AS DATE)
AS DateValue

  UNION ALL

  -- Recursive: Add one day until we reach the end of the month
  SELECT DATEADD(DAY, 1, DateValue)
  FROM DateSequence
```

```
  WHERE DateValue < EOMONTH(GETDATE())
)
-- Output each date with its day name
SELECT DateValue, DATENAME(WEEKDAY, DateValue) AS DayName
FROM DateSequence
OPTION (MAXRECURSION 31);  -- Allow up to 31 iterations (max days in a month)
```

You can combine generated sequences with actual data to identify gaps or create summary reports:

SQL

```
-- Generate a numbers table from 1 to 1000
WITH Numbers AS
(
  -- Anchor: Start with 1
  SELECT 1 AS n
  UNION ALL
  -- Recursive: Increment until we reach 1000
  SELECT n + 1 FROM Numbers WHERE n < 1000
),
-- Convert numbers to dates for the entire year
DateRange AS
(
  SELECT DATEADD(DAY, n - 1, '2024-01-01') AS OrderDate
  FROM Numbers
  WHERE DATEADD(DAY, n - 1, '2024-01-01') <= '2024-12-31'
)
-- Count orders for each date, showing 0 for dates with no orders
SELECT
  dr.OrderDate,
```

```
  COALESCE(COUNT(soh.SalesOrderID), 0) AS OrderCount
FROM DateRange AS dr
LEFT JOIN SalesLT.SalesOrderHeader AS soh
  ON CAST(soh.OrderDate AS DATE) = dr.OrderDate
GROUP BY dr.OrderDate
ORDER BY dr.OrderDate
OPTION (MAXRECURSION 366);  -- Allow up to 366 iterations (leap year)
```

Use CTEs with data modification statements

CTEs can be used with INSERT, UPDATE, and DELETE statements, not just SELECT. This capability is useful when the modification logic requires complex filtering or calculations:

SQL

```
-- Update using a CTE to identify target rows
WITH DiscontinuedProducts AS
(
  SELECT ProductID
  FROM SalesLT.Product
  WHERE SellEndDate < DATEADD(YEAR, -2, GETDATE())
    AND ProductID NOT IN (
      SELECT DISTINCT ProductID
      FROM SalesLT.SalesOrderDetail
      WHERE ModifiedDate > DATEADD(YEAR, -1, GETDATE())
    )
)
UPDATE SalesLT.Product
SET DiscontinuedDate = GETDATE()
WHERE ProductID IN (SELECT ProductID FROM DiscontinuedProducts);
```

This query uses a CTE to identify products that should be marked as discontinued. The CTE finds products where the sold end date is more than two years ago and

that haven't appeared in any order details modified within the last year. The UPDATE statement then sets the DiscontinuedDate for those products. By separating the selection logic into a CTE, the query becomes easier to read and test independently.

For more information about Common Table Expressions, see WITH common_table_expression (Transact-SQL) and Recursive Queries Using Common Table Expressions.

Apply window functions for analytics

Completed100 XP

- 14 minutes

Choose your preferred content format

Analytical queries often require calculations that span multiple rows while still returning individual row details. Traditional aggregate functions collapse rows into groups, losing row-level information. Window functions solve this challenge by performing calculations across a set of rows related to the current row, without collapsing the result set.

Understand window function syntax

Window functions calculate values across a "window" of rows defined by the OVER clause. Unlike regular aggregate functions, window functions don't group rows into a single output row. Instead, they compute values across related rows while preserving all original rows in the result.

The general syntax for a window function is:

```
function_name(arguments) OVER (
  [PARTITION BY partition_expression]
  [ORDER BY order_expression [ASC | DESC]]
  [ROWS | RANGE frame_specification]
)
```

The OVER clause components control how the window is defined:

- **PARTITION BY**: Divides rows into groups (partitions) for the calculation
- **ORDER BY**: Determines the logical order of rows within each partition
- **ROWS/RANGE**: Defines the frame boundaries relative to the current row

The following query demonstrates a simple window function that calculates a running total of order amounts per customer:

SQL

```
SELECT
  CustomerID,
  SalesOrderID,
  OrderDate,
  TotalDue,
  SUM(TotalDue) OVER (PARTITION BY CustomerID ORDER BY OrderDate) AS RunningTotal
FROM SalesLT.SalesOrderHeader
ORDER BY CustomerID, OrderDate;
```

Note

When you specify ORDER BY in the OVER clause without a frame specification, the default frame is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW for aggregate functions. This creates cumulative calculations.

Use ranking functions

Ranking functions assign sequential numbers to rows based on their position within a partition. SQL Server provides four ranking functions. Each function handles ties differently:

ROW_NUMBER() assigns a unique sequential number to each row, with no duplicates even for tied values:

SQL

```
SELECT
  ProductID,
  Name,
  ListPrice,
  ROW_NUMBER() OVER (ORDER BY ListPrice DESC) AS PriceRank
FROM SalesLT.Product
WHERE ListPrice > 0;
```

The result set looks like this:

ProductID	Name	ListPrice	PriceRank
749	Road-150 Red, 62	3578.27	1
750	Road-150 Red, 44	3578.27	2
751	Road-150 Red, 48	3578.27	3
771	Mountain-100 Silver, 38	3399.99	4

This query ranks all products by price from highest to lowest. Each product receives a unique number regardless of whether multiple products share the same price.

RANK() assigns the same rank to tied values, then skips numbers to account for the ties:

SQL

```
SELECT
  ProductID,
  Name,
  ListPrice,
  RANK() OVER (ORDER BY ListPrice DESC) AS PriceRank
FROM SalesLT.Product
WHERE ListPrice > 0;
```

The result set looks like this:

ProductID	Name	ListPrice	PriceRank
749	Road-150 Red, 62	3578.27	1
750	Road-150 Red, 44	3578.27	1
751	Road-150 Red, 48	3578.27	1
771	Mountain-100 Silver, 38	3399.99	4

When two products have identical prices, both receive the same rank. The next product's rank reflects the total number of products ranked higher, creating gaps in the sequence.

DENSE_RANK() assigns the same rank to tied values but doesn't skip numbers:

SQL

```
SELECT
  ProductID,
  Name,
  ListPrice,
  DENSE_RANK() OVER (ORDER BY ListPrice DESC) AS PriceRank
FROM SalesLT.Product
WHERE ListPrice > 0;
```

The result set looks like this:

```
ProductID   Name                    ListPrice   PriceRank
---------   ------------------------   ---------   ---------
749         Road-150 Red, 62          3578.27     1
750         Road-150 Red, 44          3578.27     1
751         Road-150 Red, 48          3578.27     1
771         Mountain-100 Silver, 38     3399.99     2
```

Like RANK(), tied values share the same rank. However, DENSE_RANK() continues with the next consecutive number, so you can use it to count distinct price levels.

NTILE(n) distributes rows into a specified number of roughly equal groups:

SQL

```
SELECT
  ProductID,
  Name,
  ListPrice,
  NTILE(4) OVER (ORDER BY ListPrice DESC) AS PriceQuartile
FROM SalesLT.Product
WHERE ListPrice > 0;
```

The result set looks like this:

ProductID Name ListPrice PriceQuartile

--- --- --- ---

749 Road-150 Red, 62 3578.27 1

771 Mountain-100 Silver, 38 3399.99 1

722 LL Road Frame - Black, 58 337.22 2

859 Half-Finger Gloves, S 24.49 4

This query divides products into four groups based on price. The highest-priced products are in quartile 1, and the lowest-priced are in quartile 4. Use NTILE() for percentile analysis or distributing work evenly.

Combining PARTITION BY with ranking functions enables per-group rankings:

SQL

```
SELECT
  pc.Name AS Category,
  p.Name AS Product,
  p.ListPrice,
  ROW_NUMBER() OVER (
    PARTITION BY p.ProductCategoryID
    ORDER BY p.ListPrice DESC
  ) AS CategoryPriceRank
FROM SalesLT.Product AS p
INNER JOIN SalesLT.ProductCategory AS pc
  ON p.ProductCategoryID = pc.ProductCategoryID
WHERE p.ListPrice > 0;
```

The result set looks like this:

Category Product ListPrice CategoryPriceRank

--- --- --- ---

Road Bikes Road-150 Red, 62 3578.27 1

Road Bikes Road-150 Red, 44 3578.27 2

Mountain Bikes Mountain-100 Silver, 38 3399.99 1

Mountain Bikes Mountain-100 Black, 38 3374.99 2

This query ranks products within each category separately. The ranking restarts at 1 for each category, so you can identify the most expensive product in each category by filtering for CategoryPriceRank = 1.

Tip

Use ROW_NUMBER() when you need exactly one row per rank (such as finding the top N per group). Use RANK() or DENSE_RANK() when you need to preserve tie information for reporting purposes.

Apply aggregate window functions

Standard aggregate functions like SUM, AVG, COUNT, MIN, and MAX can be used as window functions by adding the OVER clause. This allows you to calculate aggregates while retaining individual row details.

The following query demonstrates how to calculate running totals and cumulative aggregates:

SQL

```
SELECT
  SalesOrderID,
  OrderDate,
  TotalDue,
  SUM(TotalDue) OVER (ORDER BY OrderDate, SalesOrderID) AS RunningTotal,
  AVG(TotalDue) OVER (ORDER BY OrderDate, SalesOrderID) AS RunningAverage,
  COUNT(*) OVER (ORDER BY OrderDate, SalesOrderID) AS OrderNumber
FROM SalesLT.SalesOrderHeader
ORDER BY OrderDate, SalesOrderID;
```

The result set looks like this:

SalesOrderID OrderDate TotalDue RunningTotal RunningAverage OrderNumber

---

71774 2008-06-01 972.785 972.785 972.785 1

71776 2008-06-01 87.083 1059.868 529.934 2

71780 2008-06-01 42452.65 43512.518 14504.172 3

71782 2008-06-01 43962.79 87475.308 21868.827 4

Important

When using aggregate window functions without ORDER BY in the OVER clause, the function calculates across the entire partition. Adding ORDER BY creates a running calculation from the partition start to the current row.

Define window frames with ROWS and RANGE

Window frames let you specify exactly which rows relative to the current row should be included in the calculation. The ROWS clause counts physical rows, while RANGE groups rows with equal values.

Frame boundaries can be specified using:

- UNBOUNDED PRECEDING: From the partition start
- n PRECEDING: n rows before current row
- CURRENT ROW: The current row
- n FOLLOWING: n rows after current row
- UNBOUNDED FOLLOWING: To the partition end

The following query calculates a moving average over the last three orders:

SQL

```
SELECT
  SalesOrderID,
  OrderDate,
  TotalDue,
  AVG(TotalDue) OVER (
    ORDER BY OrderDate
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  ) AS MovingAvg3Orders
FROM SalesLT.SalesOrderHeader
ORDER BY OrderDate;
```

The result set looks like this:

SalesOrderID OrderDate TotalDue MovingAvg3Orders

--- --- --- ---

71774 2008-06-01 972.785 972.785

71776 2008-06-01 87.083 529.934

71780 2008-06-01 42452.65 14504.172

71782 2008-06-01 43962.79 28834.174

This query calculates a 3-order moving average by including the current row and the two rows before it. For the first row, only one value is available, so the average equals TotalDue. By the third row, the window includes all three rows.

Use analytical functions

Analytical functions let you access data from other rows without using self-joins or subqueries. These functions are useful for time-series analysis, trend detection, and comparing current values against historical or future values. Unlike aggregate window functions that compute summaries, analytical functions retrieve specific values from specific rows in the window.

LAG() and **LEAD()** access values from previous or subsequent rows, like this:

SQL

```
SELECT
  SalesOrderID,
  OrderDate,
  TotalDue,
  LAG(TotalDue, 1, 0) OVER (ORDER BY OrderDate) AS PreviousOrderTotal,
  LEAD(TotalDue, 1, 0) OVER (ORDER BY OrderDate) AS NextOrderTotal,
  TotalDue - LAG(TotalDue, 1, 0) OVER (ORDER BY OrderDate) AS ChangeFromPrevious
FROM SalesLT.SalesOrderHeader
ORDER BY OrderDate;
```

The result set looks like this:

SalesOrderID OrderDate TotalDue PreviousOrderTotal NextOrderTotal ChangeFromPrevious

--- --- --- --- --- ---

71774 2008-06-01 972.785 0 87.083 972.785

71776 2008-06-01 87.083 972.785 42452.65 -885.702

71780 2008-06-01 42452.65 87.083 43962.79 42365.567

71782 2008-06-01 43962.79 42452.65 0 1510.14

LAG() retrieves a value from a previous row, while LEAD() retrieves from a following row. The second parameter specifies how many rows to look back or ahead (default is 1), and the third parameter provides a default value when no row exists (such as for the first row with LAG()). Use these functions to calculate period-over-period changes, identify trends, or detect anomalies in sequential data.

FIRST_VALUE() and **LAST_VALUE()** return values from the first or last row in the frame:

SQL

```
SELECT
  ProductID,
  Name,
  ListPrice,
  ProductCategoryID,
  FIRST_VALUE(Name) OVER (
    PARTITION BY ProductCategoryID
    ORDER BY ListPrice DESC
  ) AS MostExpensiveInCategory,
  LAST_VALUE(Name) OVER (
    PARTITION BY ProductCategoryID
    ORDER BY ListPrice DESC
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
  ) AS LeastExpensiveInCategory
FROM SalesLT.Product
WHERE ListPrice > 0;
```

The result set looks like this:

ProductID Name ListPrice ProductCategoryID MostExpensiveInCategory LeastExpensiveInCategory

--- --- --- --- --- ---

749 Road-150 Red, 62 3578.27 5 Road-150 Red, 62 LL Road Frame - Red, 58

750 Road-150 Red, 44 3578.27 5 Road-150 Red, 62 LL Road Frame - Red, 58

722 LL Road Frame - Red, 58 337.22 5 Road-150 Red, 62 LL Road Frame - Red, 58

771 Mountain-100 Silver, 38 3399.99 6 Mountain-100 Silver, 38 Mountain-500 Black, 52

FIRST_VALUE() returns the value from the first row in the ordered window, which in this case is the most expensive product per category. LAST_VALUE() returns the least expensive, but requires an explicit frame to include all rows. These functions help you compare each row against benchmark values like the highest, lowest, or baseline value in a group.

Note

LAST_VALUE() requires an explicit frame specification to include rows after the current row. Without it, the default frame only includes rows up to the current row, making LAST_VALUE() return the current row's value.

PERCENT_RANK() and **CUME_DIST()** calculate relative position within a partition:

SQL

```
SELECT
  Name,
  ListPrice,
  PERCENT_RANK() OVER (ORDER BY ListPrice) AS PercentRank,
  CUME_DIST() OVER (ORDER BY ListPrice) AS CumulativeDistribution
FROM SalesLT.Product
WHERE ListPrice > 0
ORDER BY ListPrice;
```

The result set looks like this:

Name ListPrice PercentRank CumulativeDistribution

Patch Kit/8 Patches	2.29	0.0	0.0081
Road Tire Tube	3.99	0.0081	0.0162
Touring Tire Tube	4.99	0.0162	0.0243
Road-150 Red, 62	3578.27	0.9919	1.0

PERCENT_RANK() returns a value between 0 and 1 indicating what percentage of rows have lower values (0 means lowest, one means highest). CUME_DIST() shows the cumulative distribution, indicating what percentage of rows have values less than or equal to the current row. Use these functions for percentile analysis, identifying outliers, or creating distribution reports.

For more information about window functions, see Window Functions (Transact-SQL) and Ranking Functions.

Process JSON data with built-in functions

Completed100 XP

- 12 minutes

Choose your preferred content format

Consider a scenario where your e-commerce application stores customer preferences and order metadata as JSON documents. A mobile app sends shopping cart data in JSON format, and your reporting system needs to export product catalogs as JSON for a web API. Working directly with JSON in your database eliminates the need for application-layer transformations and keeps your data processing efficient.

SQL Server, Azure SQL, and SQL databases in Fabric provide built-in JSON support that lets you parse, query, create, and transform JSON data directly in T-SQL. In this unit, you learn how to use JSON functions to extract values, construct JSON output, aggregate data into JSON arrays, and validate JSON content.

Extract values with JSON_VALUE and JSON_QUERY

When working with JSON stored in your database, you need to extract specific values for filtering, joining, or displaying. SQL Server provides two functions for this purpose:

JSON_VALUE() extracts a scalar value (string, number, boolean) from a JSON string:

SQL

```
DECLARE @json NVARCHAR(MAX) = N'{
  "customer": {
    "id": 12345,
    "name": "Contoso Ltd",
    "active": true
  },
  "orderTotal": 1599.99
}';

SELECT
  JSON_VALUE(@json, '$.customer.id') AS CustomerID,
  JSON_VALUE(@json, '$.customer.name') AS CustomerName,
  JSON_VALUE(@json, '$.orderTotal') AS OrderTotal;
```

The result set will be:

```
CustomerID   CustomerName   OrderTotal
----------   ------------   ----------
12345        Contoso Ltd    1599.99
```

The function navigates the JSON structure using the path expression and returns the value as an NVARCHAR(4000) string. You can cast the result to other data types as needed for calculations or comparisons.

JSON_QUERY() extracts a JSON object or array (nonscalar values):

SQL

```
DECLARE @json NVARCHAR(MAX) = N'{
  "customer": {
    "id": 12345,
    "name": "Contoso Ltd"
  },
  "items": [
    {"product": "Widget", "qty": 5},
```

```
    {"product": "Gadget", "qty": 3}
  ]
}';

SELECT
  JSON_QUERY(@json, '$.customer') AS CustomerObject,
  JSON_QUERY(@json, '$.items') AS ItemsArray;
```

The result set will be:

```
CustomerObject                    ItemsArray
----------------------------------  ----------------------------------------------
{"id": 12345,"name": "Contoso Ltd"}    [{"product": "Widget", "qty": 5},{"product": "Gadget", "qty": 3}]
```

Unlike JSON_VALUE(), JSON_QUERY() preserves the JSON structure, returning objects and arrays as valid JSON strings that you can store, pass to other functions, or return to applications.

The path expression uses $ to represent the root element, with dot notation for nested properties and bracket notation for array elements, like in the following example:

SQL

```
-- Access array elements by index (0-based)
SELECT JSON_VALUE(@json, '$.items[0].product') AS FirstProduct;
```

The result will be:

```
FirstProduct
------------
Widget
```

Array indices start at 0, so $.items[0] refers to the first element. Use this syntax to extract specific items when you know the position, or combine with OPENJSON when you need to process all array elements.

Tip

Use JSON_VALUE() when you need a scalar value for comparisons or calculations. Use JSON_QUERY() when you need to preserve the JSON structure of nested objects or arrays.

Parse JSON arrays with OPENJSON

OPENJSON is a table-valued function that transforms JSON data into a relational rowset. Use this function to join JSON data with relational tables or process array elements individually.

The following query parses a JSON array into rows with default schema:

SQL

```
DECLARE @json NVARCHAR(MAX) = N'[
  {"id": 1, "name": "Widget", "price": 29.99},
  {"id": 2, "name": "Gadget", "price": 49.99},
  {"id": 3, "name": "Gizmo", "price": 19.99}
]';

SELECT * FROM OPENJSON(@json);
```

The result set will be:

```
key   value                                    type
---   ----------------------------------------   ----
0     {"id": 1, "name": "Widget", "price": 29.99}   5
1     {"id": 2, "name": "Gadget", "price": 49.99}   5
2     {"id": 3, "name": "Gizmo", "price": 19.99}    5
```

Without a schema, OPENJSON returns three columns: key (the array index or property name), value (the JSON content), and type (a number indicating the JSON data type: 0=null, 1=string, 2=number, 3=boolean, 4=array, 5=object).

The following query defines an explicit schema to extract specific columns with proper data types:

SQL

```
SELECT
  ProductID,
```

```
  ProductName,
  Price
FROM OPENJSON(@json)
WITH (
  ProductID INT '$.id',
  ProductName NVARCHAR(100) '$.name',
  Price DECIMAL(10,2) '$.price'
);
```

The result set will be:

```
ProductID   ProductName   Price
---------   -----------   ------
1           Widget        29.99
2           Gadget        49.99
3           Gizmo         19.99
```

The WITH clause maps JSON properties to typed columns. This approach gives you proper data types for calculations and comparisons, and lets you select only the properties you need.

Combine OPENJSON with table data using CROSS APPLY:

SQL

```
-- Assuming Orders table has a JSON column called OrderDetails
SELECT
  o.OrderID,
  o.CustomerID,
  items.ProductName,
  items.Quantity,
  items.UnitPrice
FROM Orders AS o
CROSS APPLY OPENJSON(o.OrderDetails)
WITH (
```

```
    ProductName NVARCHAR(100) '$.product',
    Quantity INT '$.qty',
    UnitPrice DECIMAL(10,2) '$.price'
) AS items;
```

Note

When the code uses OPENJSON with CROSS APPLY, rows from the main table that have NULL or empty JSON values don't appear in the results. Use OUTER APPLY if you need to include rows with no JSON data.

Construct JSON with JSON_OBJECT and JSON_ARRAY

SQL Server 2022 introduced JSON_OBJECT and JSON_ARRAY functions for intuitive JSON construction:

JSON_OBJECT() creates a JSON object from key-value pairs. The following example shows how to build a JSON object for a product:

SQL

```
SELECT JSON_OBJECT(
    'id': ProductID,
    'name': Name,
    'price': ListPrice,
    'available': CASE WHEN SellEndDate IS NULL THEN 'true' ELSE 'false' END
) AS ProductJson
FROM SalesLT.Product
WHERE ProductID = 680;
```

The result will be:

```
ProductJson
--------------------------------------------------------------------------
{"id":680,"name":"HL Road Frame - Black, 58","price":1431.50,"available":"true"}
```

The function automatically handles data type conversion and proper JSON escaping for special characters in string values.

JSON_ARRAY() creates a JSON array from values. The following example builds a JSON array:

SQL

```
SELECT JSON_ARRAY(
  'SQL Server',
  'Azure SQL Database',
  'SQL Database in Fabric'
) AS Platforms;
```

The result will be:

```
Platforms
-----------------------------------------------------
["SQL Server","Azure SQL Database","SQL Database in Fabric"]
```

You can pass column values, variables, or literal values to JSON_ARRAY(). The function creates a properly formatted JSON array regardless of the input types.

Then, combine these functions to build nested JSON structures. The following example constructs a complete order JSON object with customer and totals information:

SQL

```
SELECT JSON_OBJECT(
  'orderId': soh.SalesOrderID,
  'orderDate': soh.OrderDate,
  'customer': JSON_OBJECT(
    'id': c.CustomerID,
    'name': c.CompanyName
  ),
  'totals': JSON_OBJECT(
    'subtotal': soh.SubTotal,
    'tax': soh.TaxAmt,
    'total': soh.TotalDue
  )
) AS OrderJson
```

```
FROM SalesLT.SalesOrderHeader AS soh
INNER JOIN SalesLT.Customer AS c
  ON soh.CustomerID = c.CustomerID
WHERE soh.SalesOrderID = 71774;
```

The result will be:

```
OrderJson
--------------------------------------------------------------------------------
{"orderId":71774,"orderDate":"2008-06-
01","customer":{"id":29825,"name":"Contoso"},"totals":{"subtotal":880.35,"tax":70.
43,"total":972.79}}
```

Nesting JSON_OBJECT calls creates hierarchical structures that match your application's expected format. This approach is cleaner than string concatenation and ensures valid JSON output.

Aggregate data with JSON_ARRAYAGG

JSON_ARRAYAGG aggregates values from multiple rows into a single JSON array. This function is useful for creating denormalized JSON output from normalized relational data:

SQL

```
SELECT
  c.CustomerID,
  c.CompanyName,
  JSON_ARRAYAGG(soh.SalesOrderID) AS OrderIds
FROM SalesLT.Customer AS c
INNER JOIN SalesLT.SalesOrderHeader AS soh
  ON c.CustomerID = soh.CustomerID
GROUP BY c.CustomerID, c.CompanyName;
```

The result will be:

```
CustomerID   CompanyName        OrderIds
----------   ------------------   ------------------
29825      Contoso Retail      [71774,71776,71780]
```

29847 Adventure Works [71782,71784]

The function collects all matching values from the grouped rows and combines them into a single JSON array. This method is useful for creating denormalized API responses from normalized database tables.

You can combine JSON_ARRAYAGG with JSON_OBJECT to create arrays of complex objects:

SQL

```
SELECT
  pc.Name AS Category,
  JSON_ARRAYAGG(
    JSON_OBJECT(
      'id': p.ProductID,
      'name': p.Name,
      'price': p.ListPrice
    )
  ) AS Products
FROM SalesLT.ProductCategory AS pc
INNER JOIN SalesLT.Product AS p
  ON pc.ProductCategoryID = p.ProductCategoryID
GROUP BY pc.ProductCategoryID, pc.Name;
```

The following result will be:

Category Products

---------- ----------

Road Bikes [{"id":749,"name":"Road-150 Red, 62","price":3578.27},{"id":750,"name":"Road-150 Red, 44","price":3578.27}]

Mountain Bikes [{"id":771,"name":"Mountain-100 Silver, 38","price":3399.99},{"id":772,"name":"Mountain-100 Black, 38","price":3374.99}]

Important

JSON_ARRAYAGG and JSON_OBJECT/JSON_ARRAY functions are available in SQL Server 2022 and later, Azure SQL Database, and SQL databases in Microsoft Fabric. For earlier versions, use FOR JSON PATH for similar functionality.

Validate and check JSON with JSON_CONTAINS

JSON data from external sources can be malformed, missing expected properties, or contain unexpected values. Attempting to extract values from invalid JSON or missing paths can cause query failures or return misleading NULL results that mask data problems.

Robust JSON processing requires defensive coding: validate that the JSON is well-formed before parsing it. Then check that expected paths exist before extracting values, and verify that values match your expectations before using them in business logic. SQL Server provides several functions to help you validate JSON content at each stage of processing.

Understand lax vs strict path modes

You can use JSON path expressions in two modes that control error handling:

SQL

```
DECLARE @json NVARCHAR(MAX) = N'{"name": "Widget", "price": 29.99}';


-- Lax mode (default): Returns NULL for missing paths
SELECT JSON_VALUE(@json, 'lax $.description') AS LaxResult;


-- Strict mode: Raises an error for missing paths
SELECT JSON_VALUE(@json, 'strict $.description') AS StrictResult;
```

The result will be:

```
LaxResult
---------
NULL


-- Strict mode raises: Property cannot be found on the specified JSON path.
```

Use lax mode (the default) when missing properties are expected and should return NULL. Use strict mode when missing properties indicate a data problem that should raise an error.

ISJSON validates whether a string contains valid JSON. The following example shows how to use ISJSON:

SQL

```
SELECT
  ISJSON('{"name": "test"}') AS ValidJson,     -- Returns 1
  ISJSON('not valid json') AS InvalidJson,     -- Returns 0
  ISJSON(NULL) AS NullJson;                    -- Returns NULL
```

The result will be:

```
ValidJson   InvalidJson   NullJson
---------   -----------   --------
1           0             NULL
```

To handle invalid data gracefully, use ISJSON in WHERE clauses to filter rows with valid JSON, or in CASE expressions.

JSON_PATH_EXISTS checks whether a specific path exists in a JSON document, like the following example:

SQL

```
DECLARE @json NVARCHAR(MAX) = N'{"customer": {"name": "Contoso", "tier": "Gold"}}';

SELECT
  JSON_PATH_EXISTS(@json, '$.customer.name') AS HasName,
  JSON_PATH_EXISTS(@json, '$.customer.email') AS HasEmail;
```

The result will be:

```
HasName   HasEmail
-------   --------
1         0
```

This function returns 1 if the path exists, 0 if it doesn't. Use it before calling JSON_VALUE in strict mode, or to conditionally process JSON with varying structures.

Use JSON_CONTAINS to check if a JSON document contains a specific value or object, like the following example:

SQL

```
DECLARE @json NVARCHAR(MAX) = N'{"tags": ["sql", "database", "azure"]}';


SELECT
  JSON_CONTAINS(@json, '"sql"', '$.tags') AS HasSqlTag,
  JSON_CONTAINS(@json, '"python"', '$.tags') AS HasPythonTag;
```

The result will be:

```
HasSqlTag   HasPythonTag
---------   ------------
1           0
```

Optimize JSON queries with computed columns

When you frequently query specific JSON properties, the database engine must parse the JSON document for every row on every query. For tables with thousands or millions of rows, this repeated parsing creates significant overhead. Computed columns let you extract JSON values once and store them in a queryable format that supports indexing.

Why JSON parsing impacts performance

Consider a table with 100,000 product records where each row contains a JSON document with product attributes. A query filtering by category must:

1. Read each row from the table
2. To find the category property, parse the JSON document
3. Extract and compare the value

Without optimization, even simple filters require full table scans with JSON parsing on every row.

Create computed columns for JSON properties

A computed column automatically extracts a JSON property and makes it available as a regular column, like the following example:

SQL

```
-- Add a computed column that extracts a JSON property

ALTER TABLE Products

ADD ProductCategory AS JSON_VALUE(ProductData, '$.category');


-- The column is now available in queries

SELECT ProductID, ProductName, ProductCategory

FROM Products

WHERE ProductCategory = 'Electronics';
```

The result will be:

```
ProductID   ProductName        ProductCategory

---------   ------------------   ---------------

101       Wireless Mouse      Electronics

102       USB Keyboard        Electronics

103       HD Monitor          Electronics
```

By default, computed columns are virtual. The database calculates the value at query time but can optimize the JSON extraction. For even better performance, you can persist the computed column like in the following example:

SQL

```
-- Persisted computed column stores the extracted value physically

ALTER TABLE Products

ADD ProductCategory AS JSON_VALUE(ProductData, '$.category') PERSISTED;
```

Persisted columns store the extracted value on disk, so the JSON is parsed only during INSERT and UPDATE operations, not during SELECT queries.

Add indexes for faster filtering

The real performance gain comes from indexing computed columns:

SQL

```
-- Create an index on the computed column
CREATE INDEX IX_Products_Category ON Products(ProductCategory);

-- Now this query uses an index seek instead of a table scan
SELECT ProductID, ProductName
FROM Products
WHERE ProductCategory = 'Electronics';
```

Without the index, the query scans all 100,000 rows. With the index, the query engine performs an index seek and retrieves only matching rows. This index can reduce query time from seconds to milliseconds.

Index multiple JSON properties

For queries that filter on multiple JSON properties, create computed columns and a composite index:

SQL

```
-- Extract multiple properties
ALTER TABLE Products
ADD ProductCategory AS JSON_VALUE(ProductData, '$.category') PERSISTED,
    ProductBrand AS JSON_VALUE(ProductData, '$.brand') PERSISTED,
    ProductPrice AS CAST(JSON_VALUE(ProductData, '$.price') AS DECIMAL(10,2)) PERSISTED;

-- Create a composite index for common query patterns
CREATE INDEX IX_Products_Category_Brand ON Products(ProductCategory, ProductBrand);

-- Create an index for price range queries
CREATE INDEX IX_Products_Price ON Products(ProductPrice);
```

Now queries filtering by category and brand, or sorting by price, can use these indexes efficiently.

Tip

For frequently accessed JSON properties, computed columns with indexes can improve query performance compared to parsing JSON at query time. Monitor your query patterns and create computed columns for properties used in WHERE, JOIN, or ORDER BY clauses.

Transform relational data to JSON with FOR JSON

For comprehensive JSON output from queries, use FOR JSON PATH or FOR JSON AUTO:

SQL

```
SELECT
   p.ProductID,
   p.Name,
   p.ListPrice,
   pc.Name AS CategoryName
FROM SalesLT.Product AS p
INNER JOIN SalesLT.ProductCategory AS pc
   ON p.ProductCategoryID = pc.ProductCategoryID
WHERE p.ListPrice > 1000
FOR JSON PATH, ROOT('products');
```

The result will be:

{"products":[{"ProductID":749,"Name":"Road-150 Red, 62","ListPrice":3578.27,"CategoryName":"Road Bikes"},{"ProductID":750,"Name":"Road-150 Red, 44","ListPrice":3578.27,"CategoryName":"Road Bikes"}]}

FOR JSON PATH gives you control over the JSON structure through column aliases. Use dot notation in aliases to create nested objects:

SQL

```
SELECT
   p.ProductID AS 'product.id',
   p.Name AS 'product.name',
   pc.Name AS 'product.category'
FROM SalesLT.Product AS p
```

```
INNER JOIN SalesLT.ProductCategory AS pc
   ON p.ProductCategoryID = pc.ProductCategoryID
WHERE p.ProductID = 680
FOR JSON PATH;
```

The result will be:

```
[{"product":{"id":680,"name":"HL Road Frame - Black, 58","category":"Road Frames"}}]
```

The column alias 'product.id' creates a nested product object with an id property. This technique lets you shape the output to match your API's expected format without post-processing.

For more information about JSON functions in SQL Server, see JSON data in SQL Server and JSON Functions.

Match patterns with regular expressions

Completed100 XP

- 14 minutes

Choose your preferred content format

Text processing in databases often requires pattern matching that goes beyond what the LIKE operator can handle. Regular expressions provide a standardized syntax for complex pattern matching, validation, and text transformation. SQL Server 2025 and SQL databases in Microsoft Fabric include regular expression support through new T-SQL functions.

Consider scenarios where LIKE falls short: when someone validates an email address with proper format, extracting phone numbers regardless of formatting variations, finding product codes that follow specific naming conventions, or detecting patterns like consecutive repeated characters. The LIKE operator supports only simple wildcards (% for any characters, _ for a single character), which can't express these complex patterns.

Regular expressions solve these limitations by providing a rich pattern language. With regex, you can match specific character ranges, require exact repetition counts, use alternation (match this OR that), and capture portions of matched text for extraction or replacement. Once you learn regex syntax, you can apply it across

many programming languages and tools—the patterns you write for SQL Server work similarly in Python, JavaScript, and command-line utilities.

Understand regular expression basics

Regular expressions (regex) use a pattern syntax to describe text patterns. Before you learn SQL Server's functions, let's review these common regex components:

Pattern	Description	Example Match
.	Any single character	a.c matches *"abc"*, *"a1c"*
*	Zero or more of preceding	ab*c matches *"ac"*, *"abc"*, *"abbc"*
+	One or more of preceding	ab+c matches *"abc"*, *"abbc"* but not *"ac"*
?	Zero or one of preceding	colou?r matches *"color"*, *"colour"*
^	Start of string	^Hello matches strings starting with *"Hello"*
$	End of string	world$ matches strings ending with *"world"*
[abc]	Character class	[aeiou] matches any vowel
[^abc]	Negated class	[^0-9] matches nondigits
\d	Digit (0-9)	\d{3} matches three digits
\w	Word character	\w+ matches word characters
{n}	Exactly n occurrences	\d{4} matches exactly four digits
{n,m}	Between n and m occurrences	\d{2,4} matches 2 to 4 digits

Note

SQL Server's regular expression functions use the *ECMAScript* standard regex syntax. This script is the same syntax used in JavaScript and many other programming languages, making patterns portable across technologies.

Match patterns with REGEXP_LIKE

REGEXP_LIKE returns 1 (true) if a string matches a regular expression pattern, or 0 (false) if it doesn't. Use this function in WHERE clauses to filter rows based on complex patterns:

SQL

```
-- Find customers with email addresses from specific domains

SELECT CustomerID, FirstName, LastName, EmailAddress
```

```
FROM SalesLT.Customer

WHERE REGEXP_LIKE(EmailAddress, '@(contoso|adventure-works|fabrikam)\.com$') = 1;
```

Validate data formats:

SQL

```
-- Find valid US phone numbers (various formats)

SELECT CustomerID, Phone

FROM SalesLT.Customer

WHERE REGEXP_LIKE(Phone, '^\(?\d{3}\)?[-.\s]?\d{3}[-.\s]?\d{4}$') = 1;


-- Validate product numbers match expected format (XX-XXXX)

SELECT ProductID, ProductNumber, Name

FROM SalesLT.Product

WHERE REGEXP_LIKE(ProductNumber, '^[A-Z]{2}-[A-Z0-9]{4,6}$') = 1;
```

Use case-insensitive matching with the flags parameter:

SQL

```
-- 'i' flag enables case-insensitive matching

SELECT Name

FROM SalesLT.Product

WHERE REGEXP_LIKE(Name, 'frame', 'i') = 1;
```

Tip

Use REGEXP_LIKE for validation and filtering. It's more efficient than extracting substrings when you only need to know whether a pattern exists.

Replace text with REGEXP_REPLACE

REGEXP_REPLACE finds all occurrences of a pattern and replaces them with a specified string. This function is useful for data cleansing and standardization:

SQL

```
-- Standardize phone numbers to (XXX) XXX-XXXX format

SELECT
```

```
  Phone AS OriginalPhone,
  REGEXP_REPLACE(
    REGEXP_REPLACE(Phone, '[^\d]', ''),  -- First remove all non-digits
    '^(\d{3})(\d{3})(\d{4})$',
    '($1) $2-$3'
  ) AS StandardizedPhone
FROM SalesLT.Customer
WHERE Phone IS NOT NULL;
```

The following examples demonstrate how to use capture groups with backreferences:

SQL

```
-- Swap first and last name
DECLARE @name NVARCHAR(100) = 'Smith, John';
SELECT REGEXP_REPLACE(@name, '^(\w+),\s*(\w+)$', '$2 $1') AS SwappedName;
-- Returns: John Smith

-- Mask credit card numbers (show last 4 digits only)
DECLARE @card NVARCHAR(20) = '4532-1234-5678-9012';
SELECT REGEXP_REPLACE(@card, '\d(?=[\d-]{4,})', '*') AS MaskedCard;
-- Returns: ****-****-****-9012
```

The following examples demonstrate how to clean and normalize data:

SQL

```
-- Remove extra whitespace (multiple spaces to single space)
SELECT REGEXP_REPLACE(Description, '\s+', ' ') AS CleanedDescription
FROM Products;

-- Remove HTML tags
SELECT REGEXP_REPLACE(HtmlContent, '<[^>]+>', '') AS PlainText
```

FROM WebPages;

Extract substrings with REGEXP_SUBSTR

REGEXP_SUBSTR extracts the portion of a string that matches a regular expression pattern. Use it to pull specific data elements from unstructured text like the following examples:

SQL

```
-- Extract domain from email address
SELECT
  EmailAddress,
  REGEXP_SUBSTR(EmailAddress, '@(.+)$', 1, 1, '', 1) AS Domain
FROM SalesLT.Customer
WHERE EmailAddress IS NOT NULL;


-- Extract the first number from a string
SELECT
  ProductNumber,
  REGEXP_SUBSTR(ProductNumber, '\d+') AS FirstNumber
FROM SalesLT.Product;
```

The following example demonstrates the function signature, which includes parameters for occurrence and capture groups:

SQL

```
REGEXP_SUBSTR(source, pattern, start_position, occurrence, flags,
capture_group)
```

Find pattern positions with REGEXP_INSTR

REGEXP_INSTR returns the starting position of a pattern match within a string. Returns 0 if no match is found, like the following examples:

SQL

```
-- Find position of first digit in product number
SELECT
  ProductNumber,
```

```
  REGEXP_INSTR(ProductNumber, '\d') AS FirstDigitPosition
FROM SalesLT.Product;

-- Find position of email domain
SELECT
  EmailAddress,
  REGEXP_INSTR(EmailAddress, '@') AS AtPosition,
  REGEXP_INSTR(EmailAddress, '\.[a-z]+$', 1, 1, 0, 'i') AS TldPosition
FROM SalesLT.Customer
WHERE EmailAddress IS NOT NULL;
```

Count pattern occurrences with REGEXP_COUNT

REGEXP_COUNT returns the number of times a pattern appears in a string. The following examples illustrate its use:

SQL

```
-- Count words in a description
SELECT
  Name,
  REGEXP_COUNT(Name, '\w+') AS WordCount
FROM SalesLT.Product;

-- Count vowels in product names
SELECT
  Name,
  REGEXP_COUNT(Name, '[aeiou]', 1, 'i') AS VowelCount
FROM SalesLT.Product;

-- Find products with multiple numbers in their name
SELECT Name
```

```
FROM SalesLT.Product

WHERE REGEXP_COUNT(Name, '\d+') > 1;
```

Split strings with REGEXP_SPLIT_TO_TABLE

REGEXP_SPLIT_TO_TABLE is a table-valued function that splits a string into rows based on a delimiter pattern:

SQL

```
-- Split comma-separated values

DECLARE @tags NVARCHAR(200) = 'sql,database,azure,analytics';

SELECT value AS Tag

FROM REGEXP_SPLIT_TO_TABLE(@tags, ',');


-- Split on multiple delimiters (comma, semicolon, or pipe)

DECLARE @data NVARCHAR(200) = 'apple,banana;cherry|date';

SELECT value AS Fruit

FROM REGEXP_SPLIT_TO_TABLE(@data, '[,;|]');
```

You can combine REGEXP_SPLIT_TO_TABLE with other queries using CROSS APPLY:

SQL

```
-- Assuming Products table has a Tags column with comma-separated values

SELECT

  p.ProductID,

  p.Name,

  t.value AS Tag

FROM Products AS p

CROSS APPLY REGEXP_SPLIT_TO_TABLE(p.Tags, ',\s*') AS t;
```

Return all matches with REGEXP_MATCHES

REGEXP_MATCHES is a table-valued function that returns all pattern matches as separate rows:

SQL

```
-- Find all numbers in a string
DECLARE @text NVARCHAR(200) = 'Order 12345 contains 3 items totaling $99.99';
SELECT match_value, match_id
FROM REGEXP_MATCHES(@text, '\d+\.?\d*');
-- Returns: 12345, 3, 99.99
```

Important

Regular expression functions are available in SQL Server 2025 and SQL databases in Microsoft Fabric. For earlier SQL Server versions, consider using common language runtime (CLR) functions or application-layer processing for complex regex operations.

For more information about regular expression functions, see Regular expressions.

Find approximate matches with fuzzy string functions

Completed100 XP

- 12 minutes

Choose your preferred content format

Real-world data rarely matches perfectly. Customer names might be misspelled, addresses abbreviated differently, or product descriptions entered inconsistently. Fuzzy string matching functions help you find records that are similar but not identical, enabling data quality improvements, duplicate detection, and more flexible search capabilities.

Understand string similarity concepts

Fuzzy matching algorithms measure how similar two strings are by calculating the differences between them. Two primary approaches are commonly used:

Edit distance (Levenshtein distance) counts the minimum number of single-character operations (insertions, deletions, substitutions) needed to transform one string into another. Lower values indicate more similar strings.

Similarity scores express the relationship between strings as a percentage or ratio, where higher values indicate greater similarity.

Consider these examples:

- "color" → "colour": edit distance = 1 (insert 'u')
- "database" → "databaes": edit distance = 2 (swap 'e' and 's')
- "Microsoft" → "Microsft": edit distance = 1 (delete 'o')

Note

Fuzzy matching is computationally expensive compared to exact matching. Use it strategically, typically on candidate sets that are prefiltered using other criteria.

Calculate edit distance with EDIT_DISTANCE

The EDIT_DISTANCE function returns the Levenshtein distance between two strings, which is the minimum number of edits required to transform one string into the other. The goal is to find strings that are similar based on a defined threshold.

The following example demonstrates how to use EDIT_DISTANCE:

SQL

```
SELECT
  EDIT_DISTANCE('color', 'colour') AS ColorVariant,    -- Returns 1
  EDIT_DISTANCE('database', 'databaes') AS Typo,       -- Returns 2
  EDIT_DISTANCE('SQL Server', 'SQL Server') AS Exact,  -- Returns 0
  EDIT_DISTANCE('hello', 'world') AS Different;        -- Returns 4
```

You can use EDIT_DISTANCE to find records that might be duplicates or matches despite slight variations:

SQL

```
-- Find customers with similar names to a search term
DECLARE @searchName NVARCHAR(100) = 'Jon Smith';

SELECT
  CustomerID,
  FirstName,
  LastName,
  FirstName + ' ' + LastName AS FullName,
  EDIT_DISTANCE(@searchName, FirstName + ' ' + LastName) AS EditDistance
```

```
FROM SalesLT.Customer
WHERE EDIT_DISTANCE(@searchName, FirstName + ' ' + LastName) <= 3
ORDER BY EDIT_DISTANCE(@searchName, FirstName + ' ' + LastName);
```

Also, you can find potential duplicate products:

SQL

```
-- Find product pairs with similar names
SELECT
  p1.ProductID AS Product1ID,
  p1.Name AS Product1Name,
  p2.ProductID AS Product2ID,
  p2.Name AS Product2Name,
  EDIT_DISTANCE(p1.Name, p2.Name) AS EditDistance
FROM SalesLT.Product AS p1
INNER JOIN SalesLT.Product AS p2
  ON p1.ProductID < p2.ProductID
WHERE EDIT_DISTANCE(p1.Name, p2.Name) <= 5
ORDER BY EDIT_DISTANCE(p1.Name, p2.Name);
```

Tip

The maximum meaningful edit distance depends on string length. For short strings (5-10 characters), an edit distance of 1-2 indicates similarity. For longer strings, you might allow distances of 3-5.

Measure similarity with EDIT_DISTANCE_SIMILARITY

EDIT_DISTANCE_SIMILARITY returns a normalized similarity score between 0 and 100, where 100 represents identical strings. This percentage-based metric is easier to interpret than raw edit distance, especially when comparing strings of different lengths:

SQL

```
SELECT
  EDIT_DISTANCE_SIMILARITY('color', 'colour') AS ColorSimilarity,    -- ~85
  EDIT_DISTANCE_SIMILARITY('database', 'databaes') AS TypoSimilarity, -- ~75
```

```
    EDIT_DISTANCE_SIMILARITY('SQL', 'SQL Server') AS PartialMatch,     -- ~30
    EDIT_DISTANCE_SIMILARITY('hello', 'hello') AS Exact;              -- 100
```

You can use similarity scores to find approximate matches with a threshold like the following example:

SQL

```
-- Find products similar to a search term (at least 70% similar)
DECLARE @searchTerm NVARCHAR(100) = 'Mountain Bike Frame';

SELECT
    ProductID,
    Name,
    EDIT_DISTANCE_SIMILARITY(@searchTerm, Name) AS SimilarityScore
FROM SalesLT.Product
WHERE EDIT_DISTANCE_SIMILARITY(@searchTerm, Name) >= 70
ORDER BY EDIT_DISTANCE_SIMILARITY(@searchTerm, Name) DESC;
```

Calculate phonetic similarity with JARO_WINKLER_DISTANCE

The Jaro-Winkler algorithm is designed for comparing names and short strings. It gives higher scores to strings that match from the beginning, making it effective for person names where prefixes are more significant:

SQL

```
SELECT
    JARO_WINKLER_DISTANCE('MARTHA', 'MARHTA') AS NameTypo,     -- ~0.96
    JARO_WINKLER_DISTANCE('JONES', 'JOHNSON') AS SimilarNames,  -- ~0.83
    JARO_WINKLER_DISTANCE('JOHN', 'JON') AS NameVariant,       -- ~0.93
    JARO_WINKLER_DISTANCE('SMITH', 'SMYTH') AS SpellingVar;    -- ~0.96
```

The Jaro-Winkler score ranges from 0 to 1, where 1 indicates identical strings. A score above 0.9 typically indicates a strong match for names.

The following example finds customers with names similar to a search input:

SQL

```
-- Find customers with names similar to a search
DECLARE @searchFirst NVARCHAR(50) = 'John';
DECLARE @searchLast NVARCHAR(50) = 'Smythe';

SELECT
  CustomerID,
  FirstName,
  LastName,
  JARO_WINKLER_DISTANCE(@searchFirst, FirstName) AS FirstNameScore,
  JARO_WINKLER_DISTANCE(@searchLast, LastName) AS LastNameScore,
  (JARO_WINKLER_DISTANCE(@searchFirst, FirstName) +
   JARO_WINKLER_DISTANCE(@searchLast, LastName)) / 2 AS CombinedScore
FROM SalesLT.Customer
WHERE JARO_WINKLER_DISTANCE(@searchFirst, FirstName) > 0.85
 AND JARO_WINKLER_DISTANCE(@searchLast, LastName) > 0.85
ORDER BY CombinedScore DESC;
```

Note

Jaro-Winkler is optimized for short strings like names. For longer strings like addresses or descriptions, EDIT_DISTANCE_SIMILARITY often provides better results.

Performance considerations

Fuzzy matching functions examine every character in both strings, making them computationally intensive. Exact string comparison can stop as soon as characters differ, and indexed lookups use efficient B-tree traversal. In contrast, fuzzy algorithms must calculate similarity scores character by character. For a table with 1,000,000 rows, an unoptimized fuzzy search might perform 1,000,000 similarity calculations, each involving dozens of character comparisons.

The key to efficient fuzzy matching is reducing the candidate set before applying the expensive fuzzy functions. Use indexed columns with LIKE patterns, exact matches on related fields, or range filters to narrow results first. Only then apply fuzzy matching to the smaller candidate set.

The following examples show this progressive filtering approach:

SQL

```
-- Not good: Fuzzy match against entire table
SELECT * FROM LargeCustomerTable
WHERE EDIT_DISTANCE_SIMILARITY('John Smith', FullName) > 70;

-- Better: Pre-filter before fuzzy matching
SELECT * FROM LargeCustomerTable
WHERE FullName LIKE 'J%'  -- First letter filter
  AND EDIT_DISTANCE_SIMILARITY('John Smith', FullName) > 70;

-- Best: Use multiple pre-filters
SELECT * FROM LargeCustomerTable
WHERE FirstName LIKE 'Jo%'
  AND LastName LIKE 'Sm%'
  AND JARO_WINKLER_DISTANCE('John', FirstName) > 0.85
  AND JARO_WINKLER_DISTANCE('Smith', LastName) > 0.85;
```

Important

Fuzzy string matching functions like EDIT_DISTANCE, EDIT_DISTANCE_SIMILARITY, and JARO_WINKLER_DISTANCE are available in SQL Server 2025 and later, Azure SQL Database, and SQL databases in Microsoft Fabric. Check your platform's documentation for specific feature availability.

For more information about fuzzy string matching, see String Functions.

Traverse relationships with graph queries

Completed100 XP

- 14 minutes

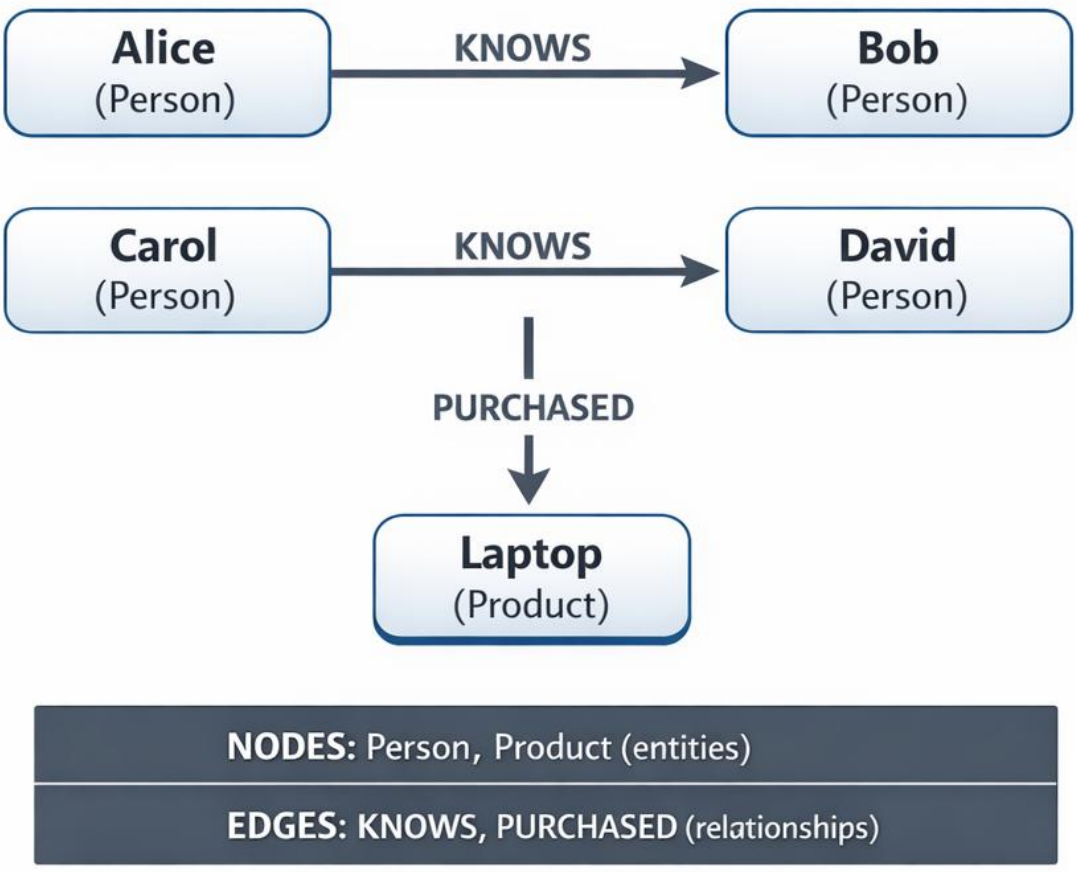
Choose your preferred content format

Some data relationships are naturally represented as networks, including social connections, organizational hierarchies, product recommendations, and fraud detection patterns. While you can model these relationships using foreign keys

and joins, graph queries using the MATCH operator provide a more intuitive and often more efficient way to traverse connected data.

Visualize graph data structures

Before writing graph queries, it helps to visualize how graph data is organized. Consider a simple social network where people know each other and purchase products:



In this model:

- **Nodes** (boxes) represent entities like people and products
- **Edges** (arrows) represent relationships between nodes. Arrow direction indicates the relationship direction (Alice knows Bob, not necessarily Bob knows Alice).

Note

This diagram illustrates graph concepts. The code examples throughout this unit use similar but simplified data to focus on specific features.

Understand graph capabilities

The graph capabilities extend the relational model with dedicated node and edge tables. Nodes represent entities such as people, products, and locations. Edges represent relationships between them, such as "knows," "purchased," or "located in."

The key advantage of graph queries is pattern matching. Instead of writing complex multi-way joins, you express the pattern you're looking for using an ASCII style syntax:

SQL

```
-- Traditional relational approach (multiple joins)
SELECT p1.Name, p2.Name
FROM Person AS p1
INNER JOIN Friendship AS f ON p1.PersonID = f.Person1ID
INNER JOIN Person AS p2 ON f.Person2ID = p2.PersonID;


-- Graph approach (pattern matching)
SELECT Person1.Name, Person2.Name
FROM Person AS Person1, Friendship, Person AS Person2
WHERE MATCH(Person1-(Friendship)->Person2);
```

Note

Graph tables are fully compatible with existing relational features. You can join graph tables with regular tables, use indexes, and apply all standard T-SQL operations.

Create node tables

Node tables store entities in your graph. Create them using CREATE TABLE with the AS NODE clause:

SQL

```
-- Create a Person node table
CREATE TABLE dbo.Person (
  PersonID INT PRIMARY KEY,
  Name NVARCHAR(100) NOT NULL,
  Email NVARCHAR(200),
```

```
  Department NVARCHAR(50)
) AS NODE;

-- Create a Product node table
CREATE TABLE dbo.Product (
  ProductID INT PRIMARY KEY,
  Name NVARCHAR(200) NOT NULL,
  Category NVARCHAR(100),
  Price DECIMAL(10, 2)
) AS NODE;

-- Create a Location node table
CREATE TABLE dbo.Location (
  LocationID INT PRIMARY KEY,
  City NVARCHAR(100) NOT NULL,
  CountryRegion NVARCHAR(100) NOT NULL
) AS NODE;
```

SQL Server automatically adds a $node_id column to node tables that uniquely identifies each node. The system uses this column internally for graph relationships.

The following example inserts four people into the Person node table, then queries the table to show both the business columns and the system-generated $node_id. Notice that the INSERT statement uses only the user-defined columns. SQL Server generates the $node_id automatically for each row:

SQL

```
-- Insert person data using standard INSERT syntax
INSERT INTO dbo.Person (PersonID, Name, Email, Department)
VALUES
  (1, 'Alice Johnson', 'alice@contoso.com', 'Engineering'),
  (2, 'Bob Smith', 'bob@contoso.com', 'Marketing'),
```

```
    (3, 'Carol Davis', 'carol@contoso.com', 'Engineering'),
    (4, 'David Lee', 'david@contoso.com', 'Sales');

-- Query shows the system-generated $node_id alongside user columns
SELECT $node_id, PersonID, Name FROM dbo.Person;
```

Create edge tables

Edge tables represent relationships between nodes. Create them using CREATE TABLE with the AS EDGE clause:

SQL

```
-- Create a "reports to" relationship edge
CREATE TABLE dbo.ReportsTo (
    StartDate DATE,
    ReportType NVARCHAR(50)
) AS EDGE;

-- Create a "purchased" relationship edge
CREATE TABLE dbo.Purchased (
    PurchaseDate DATE NOT NULL,
    Quantity INT NOT NULL,
    TotalAmount DECIMAL(10, 2)
) AS EDGE;

-- Create a "knows" relationship edge (social connection)
CREATE TABLE dbo.Knows (
    ConnectionDate DATE,
    ConnectionStrength INT  -- 1-10 scale
) AS EDGE;
```

SQL Server automatically adds $edge_id, $from_id, and $to_id columns to edge tables. You can insert edges by specifying the $from_id and $to_id values from the connected nodes, like this:

SQL

```
-- Alice reports to Bob
INSERT INTO dbo.ReportsTo ($from_id, $to_id, StartDate, ReportType)
SELECT
  (SELECT $node_id FROM dbo.Person WHERE Name = 'Alice Johnson'),
  (SELECT $node_id FROM dbo.Person WHERE Name = 'Bob Smith'),
  '2023-01-15',
  'Direct';

-- Create social connections
INSERT INTO dbo.Knows ($from_id, $to_id, ConnectionDate, ConnectionStrength)
SELECT
  (SELECT $node_id FROM dbo.Person WHERE Name = 'Alice Johnson'),
  (SELECT $node_id FROM dbo.Person WHERE Name = 'Carol Davis'),
  '2022-06-01',
  8;
```

Tip

Edge tables can store properties about the relationship itself, like dates, weights, or types. This is useful for temporal analysis or weighted graph algorithms.

Query graphs with the MATCH clause

The MATCH clause uses a pattern syntax to specify the relationships you want to find. The basic pattern uses arrows to show relationship direction:

SQL

```
-- Find who reports to whom
SELECT
  Employee.Name AS Employee,
```

```
  Manager.Name AS Manager,
  r.StartDate
FROM dbo.Person AS Employee,
   dbo.ReportsTo AS r,
   dbo.Person AS Manager
WHERE MATCH(Employee-(r)->Manager);
```

The arrow direction matters:

- (Node1)-(Edge)->(Node2): Edge goes from Node1 to Node2
- (Node1)<-(Edge)-(Node2): Edge goes from Node2 to Node1

The following example finds all people who know Alice:

SQL

```
SELECT
  Connector.Name AS PersonWhoKnowsAlice,
  k.ConnectionStrength
FROM dbo.Person AS Connector,
   dbo.Knows AS k,
   dbo.Person AS Target
WHERE MATCH(Connector-(k)->Target)
 AND Target.Name = 'Alice Johnson';
```

Traverse multiple relationships

Single-hop queries find direct connections, but graph databases excel at multi-hop traversals. You can chain multiple edge patterns in a single MATCH clause to follow paths through several relationships. This capability lets you answer questions like "who are my friends' friends?" or "what products did my colleagues purchase?" without writing complex nested subqueries.

The following example finds friends of friends by chaining two *KNOWS* relationships. The pattern Person1-(k1)->Person2-(k2)->Person3 starts at Person1, follows one *KNOWS* edge to Person2, then follows another *KNOWS* edge to reach Person3:

SQL

```
-- Find friends of friends (2-hop connections)
SELECT DISTINCT
   Person1.Name AS Person,
   Person3.Name AS FriendOfFriend
FROM dbo.Person AS Person1,
    dbo.Knows AS k1,
    dbo.Person AS Person2,
    dbo.Knows AS k2,
    dbo.Person AS Person3
WHERE MATCH(Person1-(k1)->Person2-(k2)->Person3)
 AND Person1.Name = 'Alice Johnson'
 AND Person3.Name <> Person1.Name;  -- Exclude self
```

You can also combine different relationship types in a single traversal. The following example crosses from *KNOWS* to *PURCHASED* edges to find what products were bought by people that a given person knows:

SQL

```
-- Find products purchased by people in the same department
SELECT DISTINCT
   p1.Name AS Person,
   p1.Department,
   prod.Name AS Product
FROM dbo.Person AS p1,
    dbo.Knows AS k,
    dbo.Person AS p2,
    dbo.Purchased AS pu,
    dbo.Product AS prod
WHERE MATCH(p1-(k)->p2-(pu)->prod)
 AND p1.Department = p2.Department;
```

Important

Each edge table alias can only appear once in a single MATCH pattern. To traverse the same edge type multiple times, use separate aliases.

Use SHORTEST_PATH for variable-length traversals

You can use SHORTEST_PATH to find the shortest connection across a variable number of relationships. The FOR PATH keyword marks tables that participate in variable-length matching, and quantifiers like + (one or more) or {1,3} (one to three) control the traversal depth.

The following example finds all people reachable from Alice within three hops and counts the distance to each:

SQL

```
SELECT
   StartPerson.Name,
   LAST_VALUE(ReachablePerson.Name) WITHIN GROUP (GRAPH PATH) AS ReachablePerson,
   COUNT(ReachablePerson.Name) WITHIN GROUP (GRAPH PATH) AS Distance
FROM dbo.Person AS StartPerson,
    dbo.Knows FOR PATH AS k,
    dbo.Person FOR PATH AS ReachablePerson
WHERE MATCH(SHORTEST_PATH(StartPerson(-(k)->ReachablePerson){1,3}))
 AND StartPerson.Name = 'Alice Johnson';
```

Choose between graph and relational approaches

Graph queries aren't always the best choice. Consider these guidelines when deciding between graph and traditional relational approaches:

Use graph queries when:

- Relationships are the primary focus of your queries
- You need to traverse variable or unknown depths (friends of friends of friends)
- The data naturally forms a network (social graphs, hierarchies, routes)
- Query patterns would require many self-joins in relational SQL
- You're performing path-finding or connectivity analysis

Use relational queries when:

- Relationships are simple and fixed-depth (parent-child with one level)
- You're primarily filtering and aggregating entity attributes
- The data model is mostly tabular with few relationships
- Performance is critical and indexes on foreign keys are sufficient
- Your team is more familiar with traditional SQL patterns

Troubleshoot common graph query challenges

Graph queries have unique syntax requirements that can cause errors. The following table describes common challenges and how to resolve them.

Challenge	Cause	Solution
Query returns no results	Arrow direction in MATCH pattern doesn't match how edges were inserted	Verify how edges were inserted. If $from_i and $to_id is Manager, the arrow must poi Employee to Manager.
Syntax error with repeated edge	Same edge alias used multiple times in one MATCH pattern	Create separate aliases for each traversal edge type.
SHORTEST_PATH query fails	Edge and node tables not marked with FOR PATH	Add FOR PATH keyword to all tables partic variable-length matching.
Edge references nonexistent nodes	Business key columns used instead of $node_id values	Use subqueries to select $node_id from n when inserting edges.

Note

Graph tables and the MATCH operator are available in SQL Server 2017 and later, and Azure SQL Database. The SHORTEST_PATH function requires SQL Server 2019 or later. Check your platform's documentation for specific feature availability.

For more information about graph features, see Graph processing with SQL Server and MATCH (Transact-SQL).

Compare rows with correlated subqueries

Completed100 XP

- 12 minutes

Choose your preferred content format

Correlated queries are subqueries that reference columns from the outer query, creating a dependency that causes the subquery to execute once for each row

processed by the outer query. While this might sound inefficient, correlated queries are useful for row-by-row comparisons and calculations that are difficult or impossible to express otherwise.

Understand correlated subquery execution

A correlated subquery references one or more columns from the outer query, creating a logical dependency between the two. Unlike a regular subquery that executes once and returns a fixed result, a correlated subquery executes repeatedly, once for each row the outer query processes.

Think of it like a nested loop: for each row in the outer query, the database evaluates the subquery using that row's values. This behavior enables powerful row-by-row comparisons, but it also means you need to understand the execution model to write efficient queries.

Consider how these two queries differ:

SQL

```
-- Non-correlated subquery (executes once)
SELECT ProductID, Name, ListPrice
FROM SalesLT.Product
WHERE ListPrice > (SELECT AVG(ListPrice) FROM SalesLT.Product);


-- Correlated subquery (executes per outer row)
SELECT p1.ProductID, p1.Name, p1.ListPrice
FROM SalesLT.Product AS p1
WHERE p1.ListPrice > (
  SELECT AVG(p2.ListPrice)
  FROM SalesLT.Product AS p2
  WHERE p2.ProductCategoryID = p1.ProductCategoryID  -- References outer query
);
```

In the noncorrelated example, the subquery calculates a single average price across all products. This value is computed once, and then each product's price is compared against that fixed number.

In the correlated example, the subquery references p1.ProductCategoryID from the outer query. This creates a dependency: for each product row, the subquery calculates the average price for that specific category. A product in the "Bikes" category is compared against the bikes average, while a product in "Accessories" is compared against the accessories average.

Note

The query optimizer often transforms correlated subqueries into equivalent joins internally. However, understanding the logical correlated behavior helps you write correct queries, even when the physical execution differs.

Filter with correlated subqueries

Correlated subqueries in the WHERE clause enable row-specific filtering conditions that would be impossible with static comparisons. Instead of comparing against a single fixed value, each row is evaluated against a dynamically calculated value based on that row's attributes.

This pattern is useful when you need to identify outliers within groups, find records that exceed their own category's threshold, or apply business rules that vary by context. The following example finds products priced above their category average, meaning a low-priced accessory might be flagged as expensive while a higher-priced bike might not:

SQL

```
SELECT
    p.ProductID,
    p.Name,
    p.ListPrice,
    pc.Name AS Category
FROM SalesLT.Product AS p
INNER JOIN SalesLT.ProductCategory AS pc
    ON p.ProductCategoryID = pc.ProductCategoryID
WHERE p.ListPrice > (
    SELECT AVG(p2.ListPrice)
    FROM SalesLT.Product AS p2
    WHERE p2.ProductCategoryID = p.ProductCategoryID
```

```
)
ORDER BY pc.Name, p.ListPrice DESC;
```

You can apply the same pattern to identify customers whose behavior differs from their personal baseline.

The following query finds customers who have placed at least one order that exceeds their own average order value, which helps identify unusual purchasing patterns or high-value transactions:

SQL

```
SELECT DISTINCT
  c.CustomerID,
  c.FirstName,
  c.LastName
FROM SalesLT.Customer AS c
INNER JOIN SalesLT.SalesOrderHeader AS soh
  ON c.CustomerID = soh.CustomerID
WHERE soh.TotalDue > (
  SELECT AVG(soh2.TotalDue)
  FROM SalesLT.SalesOrderHeader AS soh2
  WHERE soh2.CustomerID = c.CustomerID
);
```

Use EXISTS with correlated subqueries

The EXISTS operator combined with a correlated subquery tests whether any matching rows exist in a related table, returning a simple true or false result. This pattern is highly efficient because the database engine can stop searching as soon as it finds the first matching row. Unlike subqueries that return actual data, EXISTS only needs to confirm presence or absence.

Use EXISTS when you need to answer questions like "which customers have placed orders?" or "which products have never been sold?" The subquery typically uses SELECT 1 because the actual values don't matter:

SQL

```
-- Find customers who have placed at least one order
```

```
SELECT CustomerID, FirstName, LastName
FROM SalesLT.Customer AS c
WHERE EXISTS (
  SELECT 1
  FROM SalesLT.SalesOrderHeader AS soh
  WHERE soh.CustomerID = c.CustomerID
);

-- Find customers who have never placed an order
SELECT CustomerID, FirstName, LastName
FROM SalesLT.Customer AS c
WHERE NOT EXISTS (
  SELECT 1
  FROM SalesLT.SalesOrderHeader AS soh
  WHERE soh.CustomerID = c.CustomerID
);
```

EXISTS becomes even more valuable when you need to check complex conditions that combine multiple criteria. You can add any filtering logic inside the subquery, and the outer query will include only rows where at least one matching related row exists.

The following examples demonstrate finding products with high-quantity orders and categories where every product meets a price threshold:

SQL

```
-- Find products that have been ordered in quantities greater than 10
SELECT p.ProductID, p.Name
FROM SalesLT.Product AS p
WHERE EXISTS (
  SELECT 1
  FROM SalesLT.SalesOrderDetail AS sod
```

```
  WHERE sod.ProductID = p.ProductID
    AND sod.OrderQty > 10
);

-- Find categories where all products are priced above $100
SELECT pc.ProductCategoryID, pc.Name
FROM SalesLT.ProductCategory AS pc
WHERE NOT EXISTS (
  SELECT 1
  FROM SalesLT.Product AS p
  WHERE p.ProductCategoryID = pc.ProductCategoryID
    AND p.ListPrice <= 100
);
```

Tip

EXISTS typically outperforms IN with subqueries, especially when checking for existence in large tables. The optimizer can stop after finding the first match with EXISTS, while IN may need to retrieve all matching values.

Calculate values with correlated subqueries in SELECT

Correlated subqueries in the SELECT clause calculate a separate value for each row in your result set. This pattern lets you include aggregated or derived values from related tables alongside the main row's details, without collapsing the result into groups.

This approach is useful when you need to display contextual information, such as showing each product alongside its category's average price, or each employee alongside their department's total headcount. The subquery executes once per row, using that row's values to filter the calculation:

SQL

```
-- Show each product with its category's average price
SELECT
  p.ProductID,
  p.Name,
```

```
    p.ListPrice,
    (
        SELECT AVG(p2.ListPrice)
        FROM SalesLT.Product AS p2
        WHERE p2.ProductCategoryID = p.ProductCategoryID
    ) AS CategoryAvgPrice,
    p.ListPrice - (
        SELECT AVG(p2.ListPrice)
        FROM SalesLT.Product AS p2
        WHERE p2.ProductCategoryID = p.ProductCategoryID
    ) AS DifferenceFromAvg
FROM SalesLT.Product AS p;
```

You can also use this pattern to count related records or retrieve specific values from related tables. The following query builds a customer summary that includes each customer's order count and most recent order date, calculated individually for each customer row:

SQL

```
-- Show each customer with their order count
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    (
        SELECT COUNT(*)
        FROM SalesLT.SalesOrderHeader AS soh
        WHERE soh.CustomerID = c.CustomerID
    ) AS OrderCount,
    (
        SELECT MAX(soh.OrderDate)
```

```
    FROM SalesLT.SalesOrderHeader AS soh
    WHERE soh.CustomerID = c.CustomerID
  ) AS LastOrderDate
FROM SalesLT.Customer AS c;
```

Note

Correlated subqueries in the SELECT clause must return exactly one value. If the subquery could return multiple rows, wrap it in an aggregate function like MAX(), MIN(), or SUM().

Find top-N per group with correlated subqueries

One of the most practical applications of correlated subqueries is finding the top N items within each group. This pattern answers questions like "what are the three most expensive products in each category?" or "who are the top five salespeople in each region?"

The correlated subquery examines each row and determines whether it belongs in the top N for its group by checking how many other rows in the same group rank higher. This approach works well when window functions aren't available or when you need complex ranking logic that window functions can't express.

The following query finds the top three most expensive products per category by selecting products whose IDs appear in the top 3 for their category:

SQL

```
SELECT
  pc.Name AS Category,
  p.Name AS Product,
  p.ListPrice
FROM SalesLT.Product AS p
INNER JOIN SalesLT.ProductCategory AS pc
  ON p.ProductCategoryID = pc.ProductCategoryID
WHERE p.ProductID IN (
  SELECT TOP 3 p2.ProductID
  FROM SalesLT.Product AS p2
  WHERE p2.ProductCategoryID = p.ProductCategoryID
```

```
    ORDER BY p2.ListPrice DESC
)
ORDER BY pc.Name, p.ListPrice DESC;
```

An alternative approach counts how many items rank higher than the current row. If fewer than N items have a higher value, the current row is in the top N. This technique handles ties differently and can be useful when you need all items that tie for the Nth position:

SQL

```
-- Find products that are in the top 3 by price within their category
SELECT
    pc.Name AS Category,
    p.Name AS Product,
    p.ListPrice
FROM SalesLT.Product AS p
INNER JOIN SalesLT.ProductCategory AS pc
    ON p.ProductCategoryID = pc.ProductCategoryID
WHERE (
    SELECT COUNT(*)
    FROM SalesLT.Product AS p2
    WHERE p2.ProductCategoryID = p.ProductCategoryID
      AND p2.ListPrice > p.ListPrice
) < 3
ORDER BY pc.Name, p.ListPrice DESC;
```

Compare consecutive rows

Correlated subqueries can access values from previous or subsequent rows based on ordering criteria, enabling period-over-period comparisons and trend analysis. This pattern is useful for calculating changes between consecutive records, such as comparing each order to the previous order or tracking how values evolve over time.

The subquery finds a related row by filtering for rows that come before (or after) the current row in the logical sequence, then orders the results to get the immediately adjacent row:

SQL

```
-- Show each order with the previous order's total
SELECT
  soh.SalesOrderID,
  soh.OrderDate,
  soh.TotalDue,
  (
    SELECT TOP 1 soh2.TotalDue
    FROM SalesLT.SalesOrderHeader AS soh2
    WHERE soh2.CustomerID = soh.CustomerID
     AND soh2.OrderDate < soh.OrderDate
    ORDER BY soh2.OrderDate DESC
  ) AS PreviousOrderTotal
FROM SalesLT.SalesOrderHeader AS soh
ORDER BY soh.CustomerID, soh.OrderDate;
```

Tip

For consecutive row comparisons, window functions like LAG() and LEAD() are typically more efficient and readable than correlated subqueries. Use correlated subqueries when you need more complex conditions than window functions support.

Choose between correlated subqueries and alternatives

Correlated subqueries aren't always the best approach. The following table helps you choose the right technique:

Use this approach	When you need to...
Correlated subqueries	Compare each row against a dynamically calculated value based on that row's attr for existence with EXISTS/NOT EXISTS, or retrieve exactly one related value per row selection logic.
Joins	Retrieve columns from multiple tables, or when relationships are straightforward w row calculations.
Window functions	Calculate running totals, rankings, or access previous/next rows with LAG()/LEAD() efficient than correlated subqueries for these patterns.
CTEs	Reference the same calculated result multiple times, or break complex logic into r readable steps.

Performance considerations

Correlated subqueries can affect performance when not optimized correctly. Because the subquery executes once for each row in the outer query, poorly designed correlated queries can result in thousands or millions of subquery executions on large tables.

Follow these guidelines to optimize correlated subquery performance:

- **Create indexes on correlation columns**: Ensure the columns referenced in the subquery's WHERE clause that links back to the outer query are indexed. For example, if your subquery filters on ProductCategoryID, an index on that column lets the database quickly find matching rows instead of scanning the entire table for each outer row.
- **Include additional columns in indexes**: If your subquery also filters or aggregates on other columns, consider a composite index. An index on (ProductCategoryID, ListPrice) supports both the correlation lookup and price-based filtering or aggregation in a single index seek.
- **Evaluate alternative approaches**: Many correlated subqueries can be rewritten as joins or window functions with better performance. If you're finding the maximum value per group, a window function with ROW_NUMBER() often outperforms a correlated subquery that selects MAX() for each row.
- **Review execution plans**: Use SET STATISTICS IO ON and examine the actual execution plan to understand how the optimizer processes your correlated subquery. The optimizer might transform it into a join internally, or it might execute it row-by-row as written.

- **Test with realistic data volumes**: Correlated subqueries that perform well on small test datasets can become slow with production-sized tables. Always benchmark with representative data before deploying to production.

Important

Always review execution plans when working with correlated subqueries on large tables. The optimizer may transform them efficiently, but complex correlations might benefit from query rewrites.

For more information about subqueries, see Subqueries (Transact-SQL) and EXISTS (Transact-SQL).

Handle errors with TRY...CATCH

Completed100 XP

- 12 minutes

Choose your preferred content format

Production database applications must handle unexpected situations gracefully. Division by zero, constraint violations, connection timeouts, and invalid data can all cause errors. Unhandled errors result in unclear error messages, incomplete transactions, or application crashes. Proper error handling ensures your T-SQL code fails predictably and provides meaningful feedback.

Database operations interact with multiple users, external systems, and unpredictable data inputs simultaneously. Unlike application code that might recover from a failed operation by retrying, database errors can leave data in an inconsistent state, with some rows inserted and others not, or with locks held indefinitely. Error handling transforms these chaotic failure modes into controlled, predictable responses.

Well-designed error handling improves your code in several ways:

- **Data integrity protection**: When an operation fails partway through, proper error handling ensures that either all changes commit together or none of them persist. Without this, a multi-step process might leave your database with orphaned records, mismatched totals, or broken relationships.
- **Debugging efficiency**: Capturing error details like the line number, procedure name, and specific error message makes troubleshooting faster. Instead of searching through logs for vague failures, you can pinpoint exactly where and why an error occurred.

- **User experience**: Applications can display meaningful messages like "The product ID doesn't exist" instead of cryptic database errors. This helps users understand what went wrong and how to fix it.
- **Operational visibility**: Logging errors to a dedicated table creates an audit trail that helps identify patterns, such as recurring constraint violations that indicate a bug or timeout errors that suggest performance problems.
- **Graceful degradation**: When one operation fails, error handling lets the rest of your code continue or take alternative action, rather than crashing the entire batch or stored procedure.

Implement T-SQL error handling

T-SQL provides structured error handling through TRY...CATCH blocks, similar to exception handling in other programming languages. When an error occurs in the TRY block, execution transfers to the CATCH block where you can handle the error appropriately:

SQL

```
BEGIN TRY
  -- TRY block contains code that might cause an error
  -- If an error occurs here, execution jumps to the CATCH block
  SELECT 1/0;  -- This causes a division by zero error
END TRY
BEGIN CATCH
  -- CATCH block handles the error
  -- This code runs only if an error occurred in the TRY block
  PRINT 'An error occurred';
END CATCH;
```

Without error handling, the same code would terminate with an error message:

SQL

```
SELECT 1/0;  -- Msg 8134: Divide by zero error encountered
```

Note

TRY...CATCH can't catch all errors. Compilation errors (syntax errors, missing objects) and errors with severity 20 or higher that close the connection can't be caught within the same session.

Retrieve error information

Within the CATCH block, SQL Server provides the following functions to retrieve details about the error that occurred:

Function	Description
ERROR_NUMBER()	Returns the error number
ERROR_MESSAGE()	Returns the complete error message text
ERROR_SEVERITY()	Returns the error severity (0-25)
ERROR_STATE()	Returns the error state number
ERROR_LINE()	Returns the line number where the error occurred
ERROR_PROCEDURE()	Returns the name of the stored procedure or trigger

The following example demonstrates how to capture error details and log them to a table for later analysis:

SQL

```
BEGIN TRY
  -- Attempt an operation that might fail
  INSERT INTO SalesLT.Customer (CustomerID, FirstName, LastName)
  VALUES (1, 'Test', 'Customer');  -- Duplicate key causes error
END TRY
BEGIN CATCH
  -- Log error details to a table using the ERROR_* functions
  INSERT INTO ErrorLog (
    ErrorTime,
    ErrorNumber,
    ErrorSeverity,
    ErrorState,
    ErrorProcedure,
```

```
    ErrorLine,
    ErrorMessage
  )
  VALUES (
    GETDATE(),
    ERROR_NUMBER(),      -- The error number (e.g., 2627 for duplicate key)
    ERROR_SEVERITY(),    -- Severity level (0-25)
    ERROR_STATE(),       -- Error state for debugging
    ISNULL(ERROR_PROCEDURE(), 'Ad hoc query'),  -- NULL if not in a procedure
    ERROR_LINE(),        -- Line number where error occurred
    ERROR_MESSAGE()      -- Full error message text
  );

  -- Re-raise the error to the calling application
  THROW;
END CATCH;
```

Tip

Always log errors before reraising them. Once you use THROW or RAISERROR, the error functions return NULL if called again.

Handle transactions with TRY...CATCH

When errors occur within transactions, you must explicitly roll back uncommitted work. The @@TRANCOUNT function tells you whether a transaction is active:

SQL

```
BEGIN TRY
  -- Start a transaction to group multiple operations
  BEGIN TRANSACTION;

  -- First operation: update product prices
  UPDATE SalesLT.Product
```

```
    SET ListPrice = ListPrice * 1.05
    WHERE ProductCategoryID = 5;

    -- Second operation: update order totals
    -- If this fails, we want to undo the first update too
    UPDATE SalesLT.SalesOrderHeader
    SET TotalDue = TotalDue * 1.05
    WHERE CustomerID = 12345;

    -- Both operations succeeded, make changes permanent
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    -- Check if a transaction is still active before rolling back
    -- Some errors auto-rollback, so @@TRANCOUNT might be 0
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;  -- Undo all changes from this transaction

    -- Re-raise the error so the caller knows something failed
    THROW;
END CATCH;
```

The @@TRANCOUNT check is important because:

- An error might occur before BEGIN TRANSACTION
- Some errors automatically roll back the transaction before reaching CATCH
- Attempting to rollback without an active transaction causes another error

Important

Always check @@TRANCOUNT before calling ROLLBACK TRANSACTION in a CATCH block. This prevents the error *"ROLLBACK TRANSACTION request has no corresponding BEGIN TRANSACTION."*

Raise custom errors with THROW

The THROW statement raises an exception with a custom error number and message. Use it to signal application-specific error conditions:

SQL

```
CREATE PROCEDURE ProcessOrder
  @OrderID INT,
  @Quantity INT
AS
BEGIN
  BEGIN TRY
    -- Validate input and raise custom errors for invalid data
    IF @Quantity <= 0
      THROW 50001, 'Quantity must be greater than zero.', 1;

    IF NOT EXISTS (SELECT 1 FROM Orders WHERE OrderID = @OrderID)
      THROW 50002, 'Order not found.', 1;

    -- Process the order
    UPDATE Orders
    SET Quantity = @Quantity
    WHERE OrderID = @OrderID;

  END TRY
  BEGIN CATCH
    -- Log the error before reraising
    EXEC LogError;

    -- THROW without parameters reraises the current error
```

```
    THROW;
  END CATCH;
END;
```

Custom error numbers for user-defined errors must be 50000 or higher. The state parameter (1 in the examples) is a user-defined value between 1 and 255 that can help identify where the error was raised.

Use RAISERROR for formatted messages

RAISERROR provides more formatting options than THROW, including printf-style parameter substitution. Including runtime values in error messages makes debugging easier because you can see exactly which data caused the failure without digging through logs or reproducing the issue:

SQL

```
DECLARE @ProductName NVARCHAR(100) = 'Widget Pro';
DECLARE @CurrentStock INT = 5;
DECLARE @RequestedQty INT = 10;

IF @CurrentStock < @RequestedQty
BEGIN
  RAISERROR(
    'Insufficient stock for product "%s". Available: %d, Requested: %d',
    16,  -- Severity
    1,   -- State
    @ProductName,
    @CurrentStock,
    @RequestedQty
  );
END;
```

Note

THROW is the recommended approach for new code because it's simpler and always includes a stack trace. Use RAISERROR when you need formatted messages or compatibility with existing error handling patterns.

Implement nested error handling

Stored procedures that call other procedures need coordinated error handling. Each level should handle its own cleanup and propagate errors appropriately:

SQL

```
CREATE PROCEDURE OuterProcedure
AS
BEGIN
  BEGIN TRY
    -- Outer procedure owns the transaction
    BEGIN TRANSACTION;

    -- First operation in the outer procedure
    UPDATE SomeTable SET Column1 = 'Value';

    -- Call nested procedure - if it fails, error propagates here
    EXEC InnerProcedure;

    -- All operations succeeded, commit the transaction
    COMMIT TRANSACTION;
  END TRY
  BEGIN CATCH
    -- Outer procedure handles rollback for all nested calls
    IF @@TRANCOUNT > 0
      ROLLBACK TRANSACTION;

    -- Propagate error to the application
```

```
    THROW;
  END CATCH;
END;
GO

CREATE PROCEDURE InnerProcedure
AS
BEGIN
  BEGIN TRY
    -- Inner procedure does its work within the outer's transaction
    UPDATE AnotherTable SET Column2 = 'Value';
  END TRY
  BEGIN CATCH
    -- Don't rollback here - let the outer procedure handle it
    -- This keeps transaction management in one place
    THROW;  -- Re-raise error to outer procedure
  END CATCH;
END;
```

Use XACT_ABORT for automatic rollback

You can set XACT_ABORT ON to cause SQL Server to automatically roll back the transaction when any error occurs, even without TRY...CATCH like this:

SQL

```
SET XACT_ABORT ON;

BEGIN TRANSACTION;
  UPDATE Table1 SET Col1 = 'A';
  UPDATE Table2 SET Col2 = 'B';  -- If this fails, entire transaction rolls back
  UPDATE Table3 SET Col3 = 'C';
```

COMMIT TRANSACTION;

Combining XACT_ABORT with TRY...CATCH gives you the benefits of both approaches: XACT_ABORT guarantees immediate rollback for any error, while TRY...CATCH lets you log error details and perform custom cleanup before the error propagates:

SQL

```
-- XACT_ABORT ON ensures automatic rollback on any error
SET XACT_ABORT ON;

BEGIN TRY
  BEGIN TRANSACTION;

  -- Execute multiple procedures as a single unit of work
  EXEC Procedure1;  -- If any of these fail...
  EXEC Procedure2;  -- ...XACT_ABORT automatically rolls back...
  EXEC Procedure3;  -- ...and jumps to the CATCH block

  -- All succeeded, commit the changes
  COMMIT TRANSACTION;
END TRY
BEGIN CATCH
  -- With XACT_ABORT ON, the transaction is usually already rolled back
  -- This check handles edge cases where it might still be active
  IF @@TRANCOUNT > 0
    ROLLBACK TRANSACTION;

  -- Log the error details before re-raising
  EXEC LogError;
```

```
    -- Let the caller know an error occurred
    THROW;
END CATCH;
```

Tip

Using SET XACT_ABORT ON is a best practice for stored procedures, especially those that span multiple operations. It ensures consistent behavior regardless of the specific error that occurs.

For more information about error handling, see TRY...CATCH (Transact-SQL) and THROW (Transact-SQL).

Summary

Completed100 XP

- 2 minutes

In this module, you learned advanced T-SQL techniques that help you write more expressive, efficient, and maintainable database code. These capabilities address common database development scenarios involving complex analytics, hierarchical data, JSON processing, and error handling.

You learned how to:

- Write Common Table Expressions (CTEs) for organizing complex queries and using recursive patterns to traverse hierarchical data structures
- Apply window functions for ranking, running totals, moving averages, and analytical calculations that preserve row-level detail
- Use JSON functions including JSON_OBJECT, JSON_ARRAY, JSON_ARRAYAGG, OPENJSON, and JSON_VALUE to parse, construct, and transform JSON data
- Implement regular expressions with REGEXP_LIKE, REGEXP_REPLACE, REGEXP_SUBSTR, and related functions for pattern matching and text manipulation
- Find approximate matches using fuzzy string functions like EDIT_DISTANCE, EDIT_DISTANCE_SIMILARITY, and JARO_WINKLER_DISTANCE
- Create graph tables and write queries using the MATCH operator and SHORTEST_PATH for relationship traversal
- Write correlated subqueries for row-by-row comparisons, existence checks, and per-row calculations

- Implement structured error handling with TRY…CATCH, error functions, THROW, and proper transaction management

Key takeaways

- Recursive CTEs are the standard approach for traversing hierarchical data like organizational charts or bill-of-materials structures
- Window functions with OVER clauses enable analytical calculations without collapsing rows. Use them instead of self-joins for running totals and rankings
- JSON_OBJECT and JSON_ARRAYAGG construct JSON from relational data, while OPENJSON parses JSON into relational rows
- JARO_WINKLER_DISTANCE is optimized for name matching; EDIT_DISTANCE_SIMILARITY works better for longer strings
- Always check @@TRANCOUNT before ROLLBACK in CATCH blocks to handle cases where no transaction is active
- Combine SET XACT_ABORT ON with TRY…CATCH for full transaction protection

Implement SQL solutions by using AI-assisted tools

Learn how to leverage GitHub Copilot and Fabric Copilot for AI-assisted database development across SQL Server, Azure SQL, and SQL databases in Microsoft Fabric.

Introduction

Imagine you're a database developer working on a complex enterprise application that uses SQL Server, Azure SQL, and Microsoft Fabric. You need to write dozens of stored procedures, optimize query performance, and ensure your database code follows team standards. Traditionally, this means hours spent writing boilerplate code, searching documentation, and reviewing query plans—but AI-assisted development tools can change that workflow entirely.

GitHub Copilot and Fabric Copilot are AI assistants that understand your database context and help you write T-SQL faster and more accurately. These tools can suggest complete queries based on natural language descriptions, explain existing

code, help diagnose performance issues, and even learn your team's coding standards through custom instruction files.

In this module, you'll learn how to:

- Describe AI-assisted development tools available for Microsoft SQL platforms
- Interpret the security impact of using AI-assisted tools in database development
- Enable GitHub Copilot and Fabric Copilot in your development environment
- Configure model and Model Context Protocol (MCP) tool options in chat sessions
- Create and configure GitHub Copilot instruction files for consistent AI behavior
- Connect to MCP server endpoints for SQL Server and Fabric Lakehouse

Prerequisites

- Experience writing T-SQL code for SQL Server or Azure SQL
- Familiarity with SQL Server Management Studio (SSMS) or Visual Studio Code
- Basic understanding of database development concepts (tables, stored procedures, views)
- A GitHub account (Copilot subscription recommended but not required to understand the concepts)

Note

We recognize that different people like to learn in different ways. You can choose to complete this module in video-based format or you can read the content as text and images. The text contains greater detail than the videos, so in some cases you might want to refer to it as supplemental material to the video presentation.

Describe AI-assisted development tools available for Microsoft SQL platforms

Completed100 XP

- 10 minutes

Choose your preferred content format

AI-assisted development tools are transforming how database professionals work with Microsoft SQL platforms. Instead of manually writing every query, debugging

complex T-SQL code, or searching through documentation, you can now leverage AI assistants that understand your database context and provide intelligent suggestions in real time.

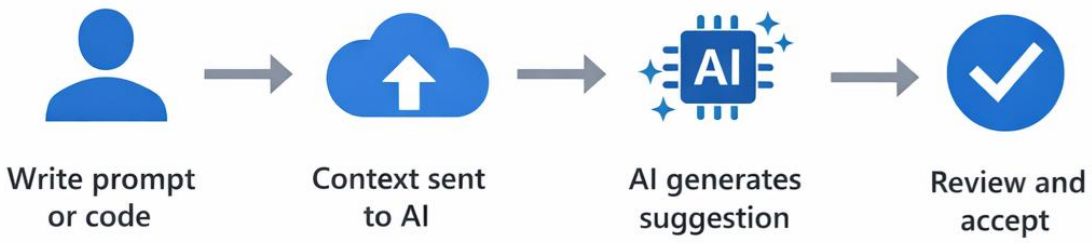
What are AI-assisted development tools?

AI-assisted development tools use large language models (LLMs) to help you write, understand, and optimize code. AI-assisted tools analyze your code context, database schema, and natural language prompts to generate relevant suggestions. For database development on SQL platforms, two primary AI assistants are available:

- **GitHub Copilot** works directly in your development environment—whether that's Visual Studio Code, Visual Studio, or SQL Server Management Studio (SSMS). It provides code completions, answers questions about your database, and helps you write T-SQL queries by understanding your schema and the patterns you use.
- **Fabric Copilot** is integrated into Microsoft Fabric workspaces and provides AI assistance for data engineering, data science, and analytics workloads. When working with SQL databases in Fabric, Copilot can help you explore data, generate queries, and understand your lakehouse structures.

How GitHub Copilot and Fabric Copilot enhance database development

Consider a scenario where you need to write a complex query joining multiple tables with specific filtering conditions. Traditionally, you might spend time reviewing table schemas, checking column names and data types, and testing different query approaches. With AI-assisted tools, you can describe what you need in natural language, and the assistant generates a starting query based on your actual database schema.



GitHub Copilot and Fabric Copilot provide several key capabilities:

- **Code completion**: As you type T-SQL, the assistant suggests complete statements, column names, and table references based on your connected database

- **Natural language to SQL**: Describe what you want to retrieve or modify, and the assistant generates the corresponding T-SQL code
- **Code explanation**: Highlight existing code and ask the assistant to explain what it does—useful when working with unfamiliar stored procedures or complex queries
- **Query optimization**: Get suggestions for improving query performance based on best practices and your specific schema
- **Error assistance**: When queries fail, the assistant can help diagnose issues and suggest fixes

AI assistants across Microsoft SQL platforms

Each Microsoft SQL platform has specific AI integration capabilities:

SQL Server and Azure SQL Database are supported through GitHub Copilot in SSMS and Visual Studio Code. You can install the GitHub Copilot extension in SSMS to get AI assistance while managing databases, writing queries, and troubleshooting performance issues.

Azure SQL Managed Instance also supports GitHub Copilot through the same development tools, allowing you to maintain consistency in your workflow whether you're working with on-premises SQL Server or managed cloud instances.

SQL databases in Microsoft Fabric benefit from both Fabric Copilot within the Fabric portal and GitHub Copilot when connecting from external tools. This dual approach means you can use the tool that best fits your current task—Fabric Copilot for quick data exploration in the portal, or GitHub Copilot for more intensive development work in your preferred IDE.

The role of Model Context Protocol (MCP)

Model Context Protocol (MCP) extends AI assistant capabilities by allowing them to connect directly to your data sources. When you configure an MCP server for your SQL database, the AI assistant can query your actual schema, sample data, and metadata to provide more accurate and contextual suggestions.

With MCP, your AI assistant isn't just working with generic SQL knowledge—it understands your specific tables, columns, relationships, and data types. This contextual awareness significantly improves the relevance and accuracy of generated code. You'll learn how to configure MCP tool options and connect to MCP server endpoints in later units.

Responsible AI considerations

When using AI-assisted tools for database development, you're working with systems that apply responsible AI principles. Both GitHub Copilot and Fabric Copilot are designed with safeguards to help ensure generated code is appropriate and doesn't include harmful content.

Important

AI-generated code suggestions should always be reviewed before execution, especially for queries that modify data or affect database security. The assistant provides helpful starting points, but you remain responsible for validating that generated code meets your requirements and follows your organization's standards.

Understanding GitHub Copilot and Fabric Copilot prepares you to make informed decisions about enabling and configuring them for your database development workflows. The following units guide you through the security considerations, setup process, and advanced configuration options that help you get the most value from AI-assisted development. **Interpret security impact of using AI-assisted tools**

Completed100 XP

- 10 minutes

Choose your preferred content format

Before enabling AI-assisted tools in your database development environment, you need to understand the security implications. GitHub Copilot and Fabric Copilot process code, database schemas, and potentially sensitive data patterns, making security awareness essential for responsible adoption.

How AI assistants process your data

When you use GitHub Copilot or Fabric Copilot, your prompts and code context are sent to cloud-based AI models for processing. Understanding what data flows to these services helps you make informed decisions about where and how to use AI assistance.

GitHub Copilot transmits the following data to generate suggestions:

- The code you're currently writing or editing
- Content from open files in your editor that provide context
- Database schema information when connected to a database
- Your natural language prompts and questions

Fabric Copilot processes similar data within the context of your Fabric workspace, including metadata about your lakehouses, warehouses, and semantic models.

Note

Both GitHub Copilot and Fabric Copilot are designed to protect your data. Prompts and responses are not used to train the underlying AI models, and data is encrypted in transit and at rest.

Organizational policy considerations

Your organization might have specific policies about AI tool usage, especially when working with sensitive data. Before enabling AI-assisted tools, consider these questions:

Data classification: Does your database contain personally identifiable information (PII), financial records, healthcare data, or other regulated information? Some organizations restrict AI tool usage for databases containing sensitive data classifications.

Compliance requirements: Are you subject to any compliance frameworks? Review whether using cloud-based AI services aligns with your compliance obligations.

Intellectual property: Does your organization have policies about code or schema information being processed by external AI services? Some proprietary database designs might be considered trade secrets.

Access controls: Who in your organization should have access to AI-assisted tools? You might need to align AI tool licensing with existing database access permissions.

GitHub Copilot security features

GitHub Copilot includes several security features that help protect your organization:

Content filtering: Copilot filters suggestions to avoid generating potentially harmful code patterns, including SQL injection vulnerabilities and other security anti-patterns.

Organization policies: Enterprise administrators can configure policies that control how Copilot is used across the organization, including which repositories can use Copilot and whether suggestions can include code that matches public repositories.

Audit logging: Organizations using GitHub Enterprise Cloud can track Copilot usage through audit logs, providing visibility into how the tool is being used.

For detailed information about GitHub Copilot's security practices, review the official GitHub Copilot in SSMS documentation.

Protecting credentials and connection strings

One critical security practice is ensuring that AI tools never see your database credentials directly. Follow these guidelines:

- **Never paste credentials into prompts**: Don't ask the AI assistant to help with connection strings that contain actual passwords or keys
- **Use environment variables**: Store connection information in environment variables rather than hardcoding in files the AI can see
- **Review before committing**: Check AI-generated code for any accidentally included sensitive information before committing to source control

SQL

```
-- AVOID: Hardcoded credentials in AI-visible files
-- This pattern should never appear in your code
-- CREATE LOGIN username WITH PASSWORD = 'actual_password';

-- RECOMMENDED: Use parameterized approaches
-- CREATE LOGIN username WITH PASSWORD = $(password);
```

Evaluating AI suggestions critically

AI-generated code requires the same security review as any code entering your database environment. Consider these practices:

Review permissions: When Copilot suggests GRANT or REVOKE statements, verify the permissions align with your least-privilege security model.

Validate dynamic SQL: AI-generated dynamic SQL might be vulnerable to injection if not properly parameterized. Always check for appropriate use of sp_executesql with parameters.

Check data exposure: Review SELECT statements to ensure they don't inadvertently expose more data than intended, especially in views or stored procedures that might be used by applications.

Tip

Treat AI-generated code as a first draft that requires human review. The assistant optimizes for functionality and common patterns, but you know your specific security requirements.

MCP server security considerations

When configuring Model Context Protocol (MCP) servers, you establish direct connections between AI assistants and your databases. This requires careful attention to security:

Authentication: MCP connections should use the principle of least privilege. Create dedicated service accounts with read-only access when the AI only needs to query schema information.

Network security: Consider whether MCP traffic should traverse public networks. For sensitive environments, you might require private endpoints or VPN connections.

Data sampling: Some MCP configurations allow sampling data to improve suggestions. Understand what data might be transmitted and whether this aligns with your data handling policies.

Understanding these security considerations prepares you to implement AI-assisted tools in a way that aligns with your organization's security posture. The next unit guides you through the actual process of enabling GitHub Copilot and Fabric Copilot with security best practices in mind.

Enable GitHub Copilot and Fabric Copilot

Completed100 XP

- 12 minutes

Choose your preferred content format

With a clear understanding of security considerations, you're ready to enable GitHub Copilot and Fabric Copilot for your database development workflows. This unit walks you through the setup process for each tool across different development environments.

Prerequisites for AI assisted tools

Before enabling AI assisted tools, ensure you have the required prerequisites:

For GitHub Copilot:

- A GitHub account with Copilot access (Individual, Business, or Enterprise subscription)

- A supported development environment (Visual Studio Code, Visual Studio, or SQL Server Management Studio 22+)
- An active internet connection for AI model communication

For Fabric Copilot:

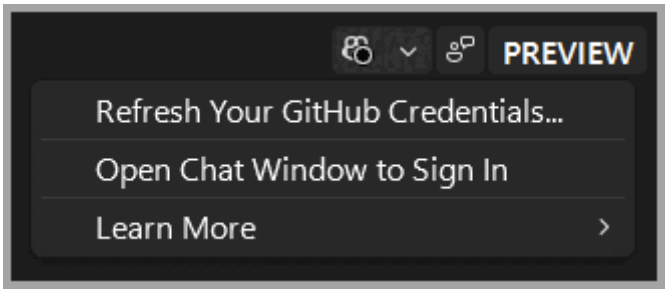
- A Microsoft Fabric capacity (F2 or higher) or Power BI Premium capacity (P1 or higher)
- Appropriate permissions in your Fabric workspace
- Copilot features enabled at the tenant level by your administrator

Enable GitHub Copilot in SQL Server Management Studio

SQL Server Management Studio (SSMS) 22 and later versions include native support for GitHub Copilot. To install GitHub Copilot in SSMS, follow these steps:

1. Launch the **Visual Studio Installer** on your machine
2. Locate your SSMS installation and select **Modify**
3. On the Workloads tab, select **AI Assistance**
4. Select **Modify** to install the GitHub Copilot components

After installation, you'll see a GitHub Copilot status icon in the upper-right corner of SSMS. The icon indicates whether Copilot is active, inactive, unavailable, or not installed.



To sign in and activate Copilot:

1. Select the GitHub Copilot badge in SSMS
2. Choose **Open Chat Window to Sign In**
3. Sign in with your GitHub account that has Copilot access
4. If you don't have a Copilot subscription, select **Sign up for Copilot Free** to get started

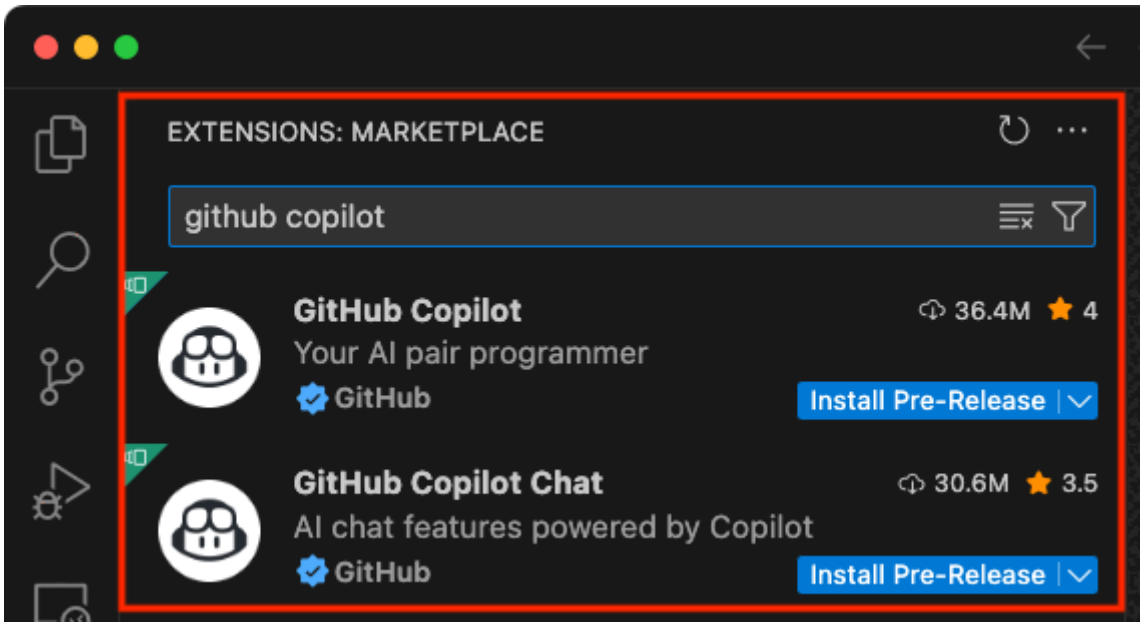
Tip

If you have multiple GitHub accounts, ensure you sign in with the account that has an active Copilot subscription. You can check your subscription status at GitHub's Copilot settings page.

Enable GitHub Copilot in Visual Studio Code

Visual Studio Code provides a flexible environment for database development with the MSSQL extension combined with GitHub Copilot. To set up GitHub Copilot in VS Code:

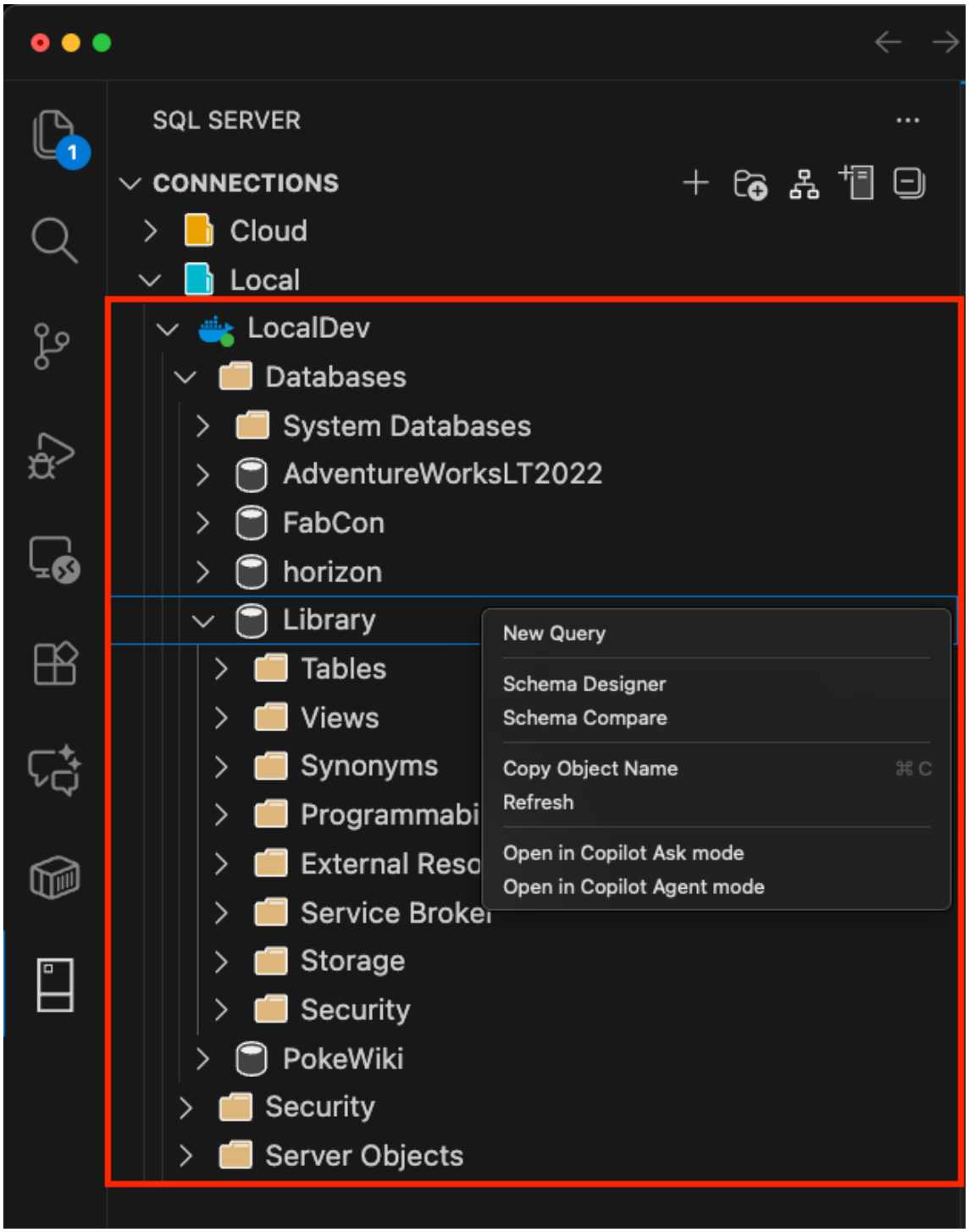
1. Open Visual Studio Code
2. Access the Extensions view (**Ctrl+Shift+X** on Windows/Linux, **Cmd+Shift+X** on macOS)
3. Search for and install the **GitHub Copilot** extension
4. Search for and install the **GitHub Copilot Chat** extension
5. Sign in to your GitHub account when prompted



For SQL database development, you'll also want the MSSQL extension:

1. In the Extensions view, search for mssql
2. Install the **SQL Server (mssql)** extension by Microsoft
3. Connect to your database using the Connections view

With both Copilot and MSSQL extensions installed, you can start conversations about your database directly in the Copilot Chat panel. Right-click on a connected database and select **Chat with this database** to begin a context-aware conversation.



Enable Fabric Copilot

Fabric Copilot is available within Microsoft Fabric workspaces when your organization meets the licensing requirements. Enabling Copilot involves both tenant-level and workspace-level settings.

Tenant-level configuration (requires Fabric Administrator):

1. Navigate to the Fabric Admin portal

2. Go to **Tenant settings**
3. Under the Copilot and Azure OpenAI Service section, enable **Users can use Copilot and other features powered by Azure OpenAI**
4. Configure which security groups have access to Copilot features

Workspace-level usage:

Once enabled at the tenant level, users with appropriate permissions can access Copilot within:

- Data Engineering experiences (notebooks, data pipelines)
- Data Warehouse experiences (SQL queries, semantic models)
- Data Science experiences (notebooks, experiments)

To use Copilot in a SQL database in Fabric:

1. Open your SQL database in the Fabric portal
2. Navigate to the query editor
3. Look for the Copilot button or icon in the toolbar
4. Start typing natural language queries or use the chat interface

Configure Copilot for your team

For enterprise deployments, you'll want consistent configuration across your team. Consider these setup practices:

Standardize subscriptions: Ensure all team members have the same level of Copilot access to avoid confusion about available features.

Document approved uses: Create guidelines about when and how to use AI assistance, especially for production database changes.

Set up shared instruction files: You can create custom instruction files that guide Copilot's behavior for your team's coding standards (covered in a later unit).

Verify your setup

After enabling AI assisted tools, verify everything is working correctly:

In SSMS:

1. Connect to a database in the query editor
2. Open the Copilot Chat window (**Ctrl+\, Ctrl+C**)
3. Ask a simple question like "What tables are in this database?"

4. Verify you receive a contextual response

In VS Code:

1. Connect to a database using the MSSQL extension
2. Open a new .sql file
3. Open the Copilot Chat panel (**Ctrl+Alt+I**)
4. Type a comment describing a query: -- Get all customers from Seattle
5. Press Enter and observe if Copilot suggests relevant SQL

In Fabric:

1. Open a SQL database in your workspace
2. Access the Copilot interface
3. Ask "Describe the tables in this database"
4. Verify the response reflects your actual schema

Note

If Copilot isn't providing contextual suggestions, verify your database connection is active and that your account has the necessary permissions to query schema information.

With GitHub Copilot and Fabric Copilot enabled, you're ready to explore advanced configuration options that customize how the AI assistants behave for your specific workflows.

Configure model and model context protocol (MCP) tool options in a GitHub Copilot or Fabric Copilot chat session

Completed100 XP

- 12 minutes

Choose your preferred content format

Now that you have GitHub Copilot and Fabric Copilot enabled, you can customize how they behave during chat sessions. Selecting the right model and configuring Model Context Protocol (MCP) tools significantly improves the relevance and accuracy of AI suggestions for your database work.

Understanding model selection

GitHub Copilot supports multiple AI models, each with different capabilities and performance characteristics. You can select which model to use for different scenarios based on your needs. Available models typically include:

- **GPT**: Advanced reasoning capabilities, excellent for complex T-SQL generation and database design questions
- **Claude models**: Strong at explaining code and providing detailed documentation
- **Gemini models**: Available in some configurations with different strengths

To change models in a Copilot Chat session in **SSMS**:

1. Open the Copilot Chat window
2. Look for the model selector dropdown at the top of the chat
3. Select the model you want to use for your current session

To change models in a Copilot Chat session in **VS Code**:

1. Open the Copilot Chat panel (**Ctrl+Alt+I**)
2. Use the model picker in the chat interface
3. Switch models as needed for different tasks

Tip

For complex query optimization or database design questions, try more advanced models. For quick code completions, faster models often provide sufficient results with lower latency.

What is Model Context Protocol (MCP)?

Model Context Protocol (MCP) is an open standard that allows AI assistants to connect directly to external data sources and tools. Instead of the AI assistant only seeing the code in your editor, MCP enables it to query your actual database schema, sample data, and metadata in real time.

With MCP configured, when you ask "What columns are in the Customers table?", the assistant can query your database directly rather than relying on whatever code context is visible in your editor. This produces more accurate and reliable suggestions.



MCP follows a client-server architecture:

- **MCP Host**: Your AI environment (GitHub Copilot, Claude, etc.)
- **MCP Client**: The protocol client that connects to MCP servers
- **MCP Server**: A service that exposes specific data sources or tools

Configure MCP tools in GitHub Copilot

GitHub Copilot supports MCP integration through agent mode in VS Code. To use MCP servers in VS Code:

Enable agent mode

Agent mode allows GitHub Copilot to use external tools, including MCP servers, to gather context and perform actions on your behalf.

1. Open the Copilot Chat panel in VS Code
2. Look for the mode selector (Ask, Edit, or Agent)
3. Switch to **Agent** mode to access MCP tools

Add an MCP server

You can add MCP servers to your VS Code workspace using the Command Palette or by manually editing the configuration file.

1. Open the Command Palette (**Ctrl+Shift+P**)
2. Type **MCP: Add Server** and select it
3. Choose the server type (HTTP or Stdio)
4. Enter the server URL or configuration

Manage MCP tools

Once MCP servers are added, you can control which tools are active for each chat session.

1. In Agent mode, select the tools icon in the chat panel
2. View available MCP servers and their tools
3. Enable or disable specific tools as needed

When MCP tools are configured, you'll see them listed when you click the tools icon in Agent mode. The assistant can then use these tools to query your databases and provide contextual suggestions.

MCP server options for SQL databases

Several MCP server options are available for connecting AI assistants to SQL databases:

- **SQL MCP Server**: Connects to SQL Server and Azure SQL databases using Microsoft's open-source solution built on Data API builder. This option provides a secure, managed way to expose database metadata to AI assistants.
- **Microsoft Fabric MCP Server**: Connects to Fabric data agents as MCP servers, enabling AI assistants to query lakehouses, warehouses, and SQL databases within Fabric workspaces.
- **Azure MCP Server**: Provides broader Azure resource integration, including database services.

To configure a SQL MCP server in VS Code:

1. Create a .vscode folder in your workspace (if it doesn't exist)
2. Create a file named mcp.json in the .vscode folder
3. Add your server configuration:

JSON

```
{
 "servers": {
  "sql-server": {
   "type": "http",
   "url": "https://your-mcp-endpoint/mcp"
  }
 }
}
```

4. VS Code detects the configuration and prompts you to start the server
5. Authenticate when prompted

Configure MCP in Fabric Copilot

Fabric Copilot automatically has access to your workspace resources, but you can publish data agents as MCP servers for use in external tools:

1. Create and configure a data agent in your Fabric workspace
2. Publish the data agent

3. Go to the agent's **Settings** and open the **Model Context Protocol** tab
4. Copy the **MCP server URL**
5. Use this URL to configure MCP clients in VS Code or other tools

This approach allows you to use the same data agent from both the Fabric portal and external development environments.

Best practices for model and MCP configuration

- **Start with defaults**: Begin with default model and MCP settings, then adjust based on your experience. Not every scenario requires the most advanced configuration.
- **Match models to tasks**: Use more capable models for complex design decisions, simpler models for routine code completion.
- **Limit MCP scope**: Configure MCP connections with least-privilege access. If your AI workflow only needs schema information, don't grant data read permissions.
- **Test configurations**: After changing model or MCP settings, verify the assistant still provides accurate suggestions. Different models might interpret prompts differently.

Important

MCP servers establish direct connections to your databases. Ensure your network security policies allow these connections and that authentication follows the security best practices covered earlier in this module.

With model selection and MCP tools configured, your AI assistant has the context it needs to provide accurate, database-specific suggestions. The next unit covers how to create custom instruction files that further guide the assistant's behavior.

Create and configure GitHub Copilot instruction files

Completed100 XP

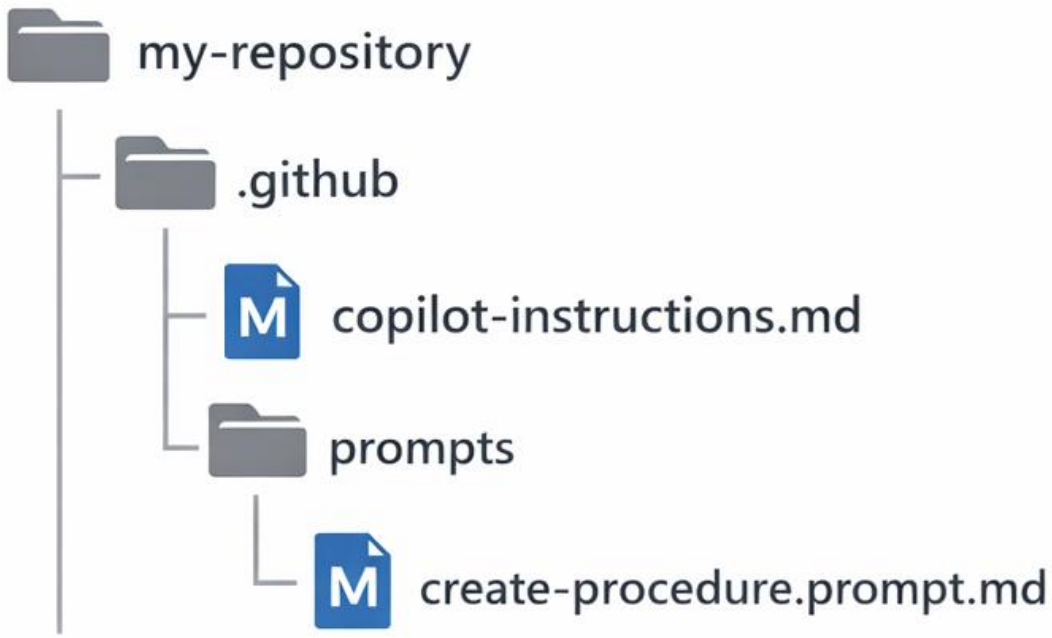
- 12 minutes

Choose your preferred content format

Custom instruction files provide a way to guide GitHub Copilot's behavior consistently across your projects and team. Instead of repeating the same context in every prompt, you define your preferences, coding standards, and project-specific guidance in files that Copilot reads automatically.

What are custom instruction files?

Custom instruction files are markdown documents that contain guidance for GitHub Copilot. When you place these files in specific locations, Copilot includes their contents as context when generating suggestions. This means you can influence how Copilot responds without modifying each prompt.



There are two main types of instruction files:

- **Repository instructions** (copilot-instructions.md): Apply to everyone working in a specific repository. Stored in the .github folder at the repository root.
- **Prompt files** (.prompt.md): Reusable prompts for specific tasks or scenarios. Stored in the .github/prompts folder and can be referenced in chat conversations.

Create a repository instruction file

To create custom instructions for your repository, follow these steps:

1. Navigate to your repository root folder
2. Create a .github folder if it doesn't exist
3. Create a file named copilot-instructions.md
4. Add your instructions in markdown format

Here's an example instruction file for a database project:

markdown

Project Guidelines for Copilot

Database Development Standards

This project uses SQL Server 2022 with the following conventions:

Naming Conventions

- Tables: PascalCase, singular (Customer, OrderDetail)
- Columns: PascalCase (FirstName, OrderDate)
- Stored procedures: usp_*ActionEntity (usp_*GetCustomerOrders)
- Views: vw_*EntityName (vw_*ActiveCustomers)
- Indexes: IX_*TableName*_ColumnName

T-SQL Style

- Use explicit column lists in SELECT statements (avoid SELECT *)
- Always include schema prefix (dbo.TableName)
- Use ANSI JOIN syntax, not comma-separated tables
- Include error handling in all stored procedures
- Use TRY...CATCH blocks for data modification operations

Security Requirements

- Never generate GRANT statements to public
- Use parameterized queries, never concatenate user input
- Avoid dynamic SQL when possible

Performance Guidelines

- Suggest appropriate indexes when creating tables

- Prefer SET NOCOUNT ON in stored procedures

- Use EXISTS instead of COUNT for existence checks

When Copilot generates T-SQL code in this repository, it considers these guidelines and produces suggestions that align with your standards.

Configure instruction file settings

In Visual Studio and VS Code, you can configure how Copilot uses instruction files:

In VS Code:

1. Open Settings (**Ctrl+,**)
2. Search for "GitHub Copilot: Instructions"
3. Configure which instruction files to include automatically

In Visual Studio:

1. Go to **Tools** > **Options**
2. Navigate to **GitHub** > **Copilot**
3. Configure custom instruction preferences

You can also reference instruction files explicitly in chat using the Command Palette (**Ctrl+Shift+P**) and selecting **Chat: Configure Instructions**.

Create reusable prompt files

Prompt files let you define templates for common tasks. For database development, you might create prompts for:

- Generating stored procedure templates
- Creating data migration scripts
- Reviewing query performance
- Documenting existing code

To create a prompt file:

1. Create a .github/prompts folder in your repository
2. Create a file with .prompt.md extension
3. Define the prompt template with any required parameters

Example prompt file for creating a stored procedure (create-procedure.prompt.md):

markdown

Create Stored Procedure

Generate a stored procedure with the following specifications:

- Procedure name: {{procedureName}}

- Purpose: {{description}}

- Parameters: {{parameters}}

Follow these requirements:

- Include TRY...CATCH error handling

- Add SET NOCOUNT ON at the beginning

- Include appropriate comments

- Follow our naming conventions (usp_ prefix)

- Use explicit transactions for data modifications

To use this prompt in VS Code, reference it in the chat panel with the # symbol or through the prompt picker.

Team-wide instruction strategies

For enterprise teams, consider these approaches to instruction file management:

- **Centralized standards**: Maintain a master instruction file in a shared repository that gets copied or linked to project repositories.
- **Layered instructions**: Use organization-level instructions for general standards, with repository-specific additions for project requirements.
- **Version control**: Track instruction file changes in git so you can review how your AI guidance evolves over time.

Tip

Review your instruction files periodically. As your team's practices evolve or as Copilot improves, you might need to adjust your guidance.

Best practices for instruction files

To get the most out of custom instruction files, follow these best practices:

Be specific: Vague instructions produce vague results. Instead of "use good naming," specify your exact naming pattern.

Provide examples: Show Copilot what you want rather than just describing it. Include code snippets that demonstrate your preferred patterns.

Prioritize: Put your most important guidelines first. Copilot considers the full context, but prominent placement helps ensure critical rules are followed.

Test iteratively: After creating instruction files, test various prompts to see if Copilot follows your guidance. Adjust wording if results aren't as expected.

Keep current: Update instructions when your standards change. Outdated guidance leads to inconsistent suggestions.

Example of a specific, example-driven instruction:

markdown

Error handling pattern

Always use this error handling pattern in stored procedures:

```sql
CREATE PROCEDURE dbo.usp_ExampleProcedure
AS
BEGIN
  SET NOCOUNT ON;

  BEGIN TRY
    BEGIN TRANSACTION;

    -- Procedure logic here

    COMMIT TRANSACTION;
  END TRY
  BEGIN CATCH
```

```
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    -- Log error details
    DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
    DECLARE @ErrorSeverity INT = ERROR_SEVERITY();
    DECLARE @ErrorState INT = ERROR_STATE();

    RAISERROR(@ErrorMessage, @ErrorSeverity, @ErrorState);
  END CATCH;
END;
```

This pattern includes explicit transaction handling and proper error propagation.

With well-crafted instruction files, your team gets consistent AI assistance that aligns with your coding standards and project requirements. The next unit covers connecting to MCP server endpoints to enhance this guidance with live database context.

Connect to MCP server endpoints, including Microsoft SQL Server and Fabric lakehouse

Completed100 XP

- 12 minutes

Choose your preferred content format

Model Context Protocol (MCP) servers provide AI assistants with direct access to your data sources, enabling more accurate and contextual suggestions. Building on the MCP concepts from the previous units, this unit covers how to connect to MCP server endpoints for Microsoft SQL Server, Azure SQL databases, and Fabric Lakehouse environments.

Connect to SQL Server MCP endpoints

For SQL Server and Azure SQL databases, you can use the SQL MCP Server or configure direct connections through supported tools.

Using SQL MCP Server:

SQL MCP Server is Microsoft's open-source solution built on Data API builder that enables AI agents to interact with SQL databases. To set up SQL MCP servers in VS Code:

1. Install and configure SQL MCP Server for your database
2. Enable the MCP endpoint in your SQL MCP Server configuration
3. Note the MCP endpoint URL (typically ending in /mcp)
4. Add the server to VS Code using the MCP: Add Server command

Direct SQL Server connection:

As covered in the earlier unit on enabling GitHub Copilot, you can use the MSSQL extension's built-in integration in VS Code. Right-click your connected database and select **Chat with this database** to start schema-aware conversations without configuring a separate MCP server.

Connect to Azure SQL MCP endpoints

Azure SQL databases support the same connection methods as SQL Server. For cloud deployments, consider these additional options:

Using Azure MCP Server:

The Azure MCP Server provides integration across Azure services, including SQL databases:

1. Install the Azure MCP Server extension in VS Code
2. Authenticate with your Azure account
3. Configure access to your Azure SQL resources
4. Use Agent mode to query your databases

Configuring network access:

Azure SQL databases require network configuration for MCP connections:

- Ensure your client IP is allowed through the Azure SQL firewall
- For private endpoints, configure your network to allow MCP traffic
- Consider using service principals for automated or shared MCP access

JSON

```
{
  "servers": {
```

```
  "azure-sql-mcp": {
    "type": "http",
    "url": "https://your-api-endpoint.azurewebsites.net/mcp",
    "headers": {
      "Authorization": "Bearer ${input:azure_token}"
    }
  }
 },
 "inputs": [
  {
    "id": "azure_token",
    "type": "promptString",
    "description": "Azure access token",
    "password": true
  }
 ]
}
```

Connect to Fabric lakehouse MCP endpoints

Microsoft Fabric provides MCP server capabilities through data agents, allowing AI assistants to query lakehouses, warehouses, and semantic models.

Publishing a Fabric data agent as MCP server:

1. Create a data agent in your Fabric workspace
2. Configure the data agent with access to your lakehouse or warehouse
3. Publish the data agent
4. Navigate to the agent's **Settings** > **Model Context Protocol** tab
5. Copy the **MCP server URL** and **tool description**

Configuring VS Code to connect:

Create or update your .vscode/mcp.json file:

JSON

```
{
 "servers": {
  "fabric-lakehouse": {
   "type": "http",
   "url": "https://your-fabric-mcp-endpoint-url"
  }
 }
}
```

When VS Code detects this configuration:

1. Select **Add Server** when prompted
2. Authenticate with your Microsoft account
3. The Fabric MCP server appears in your available tools

Using the Microsoft Fabric MCP Server extension:

For a streamlined experience, install the Fabric MCP Server extension from the VS Code marketplace. This extension simplifies configuration and provides better integration with Fabric workspaces.

Configure authentication for MCP endpoints

MCP servers require authentication to access your data sources. Common authentication methods include:

Interactive authentication: You sign in through a browser prompt when connecting. Best for development environments.

Service principal: Configure a Microsoft Entra ID application with appropriate permissions. Best for automated scenarios and shared environments.

API keys: Some MCP servers support API key authentication. Store keys securely using environment variables or VS Code's input prompts.

Important

Never store credentials directly in configuration files that might be committed to source control. Use environment variables, input prompts, or secure credential stores. For more guidance on protecting credentials, see the security considerations covered earlier in this module.

Test your MCP connections

After configuring MCP endpoints, verify the connection is working:

In VS Code Agent mode:

1. Switch to Agent mode in the Copilot Chat panel
2. Click the tools icon to see available MCP servers
3. Verify your configured servers appear with a green status
4. Ask a question like "What tables are available in the database?"
5. Confirm the response reflects your actual database schema

Troubleshooting common issues:

Issue	Likely Cause	Solution
Server not listed	Configuration file syntax error	Validate JSON in mcp.json
Authentication failed	Expired credentials	Re-authenticate or refresh tokens
Connection timeout	Network/firewall blocking	Check firewall rules and network access
Empty schema results	Insufficient permissions	Verify database permissions for the authent

Best practices for MCP endpoint management

Use least-privilege access: Create dedicated accounts for MCP connections with only the permissions needed (typically read-only schema access).

Separate environments: Configure different MCP endpoints for development, test, and production databases. Avoid connecting AI assistants to production data during routine development.

Monitor connections: Track MCP server usage through logging and monitoring. Understand how often your AI assistant queries your databases.

Keep configurations consistent: For team environments, share MCP configurations through version control so everyone connects to the same endpoints.

Tip

Start with a development or test database when learning MCP configuration. Once you're comfortable with the setup, extend to additional environments as needed.

With MCP endpoints configured, your AI assistant now has real-time access to your database schemas and metadata. Combined with custom instruction files

from the previous unit, you have a fully configured, contextual AI development environment for your Microsoft SQL platforms.

Summary

Completed100 XP

- 2 minutes

In this module, you learned how to:

- Describe AI-assisted development tools (GitHub Copilot and Fabric Copilot) available for SQL Server, Azure SQL, and SQL databases in Microsoft Fabric
- Interpret the security impact of using AI tools, including data processing, organizational policies, and credential protection best practices
- Enable GitHub Copilot in SSMS and VS Code, and configure Fabric Copilot for your workspaces
- Configure model selection and Model Context Protocol (MCP) tools to enhance AI suggestions with real-time database context
- Create custom instruction files that guide Copilot's behavior for your team's coding standards
- Connect to MCP server endpoints for SQL Server, Azure SQL, and Fabric Lakehouse environments

Key takeaways

- AI-assisted tools can significantly accelerate T-SQL development by providing contextual code suggestions, explanations, and optimization recommendations
- Security awareness is essential—never expose credentials to AI tools and always review generated code before execution
- MCP enables AI assistants to access your actual database schema, producing more accurate suggestions than generic AI assistance
- Custom instruction files provide consistent AI behavior across your team by codifying coding standards and project-specific guidelines

Implement data security and compliance with SQL

Learn how to protect sensitive data and meet compliance requirements by implementing encryption, masking, access controls, and auditing across Microsoft's SQL platforms.

Introduction

Completed100 XP

- 3 minutes

Choose your preferred content format

Data security isn't just about preventing external attackers. You must protect sensitive information from unauthorized internal access, maintain compliance with regulations, and create accountability through detailed audit trails. Microsoft's SQL platforms provide built-in security features that address these requirements without requiring extensive application changes.

Modern development practices require you to build security into your database designs from the start. When you create tables that store personal information, you need to consider who should see that data and how to protect it. When you write stored procedures, you must think about what permissions they require and whether they could expose sensitive information. Security decisions made during development are far easier to implement than adding them later.

Modern database security requires a defense-in-depth approach. A single security measure isn't enough. You need encryption to protect data even if storage is compromised, masking to limit exposure of sensitive values, row-level filtering to enforce data boundaries, and auditing to track who accessed what and when. Each layer addresses different threat vectors and compliance requirements.

What you'll learn

In this module, you'll learn how to:

- Design and implement data encryption using Always Encrypted and column-level encryption
- Configure Dynamic Data Masking to protect sensitive data from unauthorized viewers
- Implement Row-Level Security to filter data access based on user context
- Design object-level permissions using roles and schemas

- Implement secure, passwordless database access using Microsoft Entra ID and Managed Identity
- Configure auditing to track database activity and maintain compliance
- Secure model endpoints for AI-enabled database solutions
- Protect GraphQL, REST, and MCP endpoints from unauthorized access

Prerequisites

- Experience writing T-SQL queries and developing databases in SQL Server, Azure SQL, or SQL databases in Microsoft Fabric
- Familiarity with database security concepts such as authentication and authorization
- Understanding of Microsoft Entra ID (formerly Azure Active Directory) basics
- Access to a SQL Server, Azure SQL Database, or SQL database in Microsoft Fabric for testing

Note

We recognize that different people like to learn in different ways. You can choose to complete this module in video-based format or you can read the content as text and images. The text contains greater detail than the videos, so in some cases you might want to refer to it as supplemental material to the video presentation.

Next unit: Protect data with encryption

Protect data with encryption

Completed100 XP

- 12 minutes

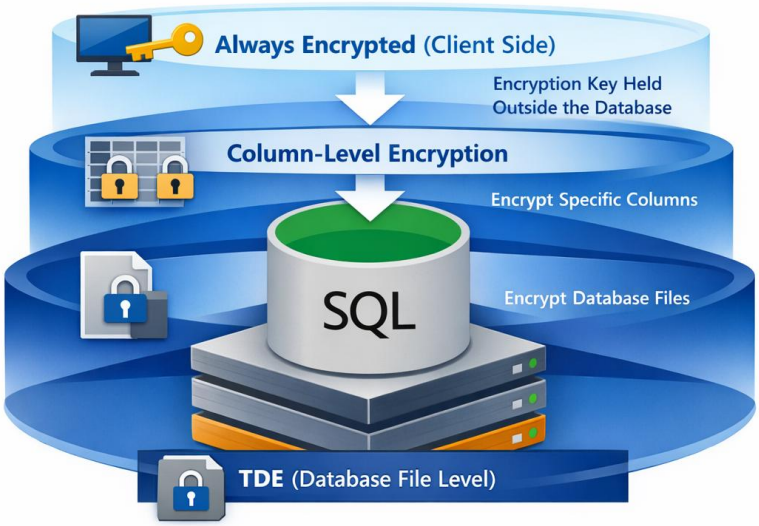
Choose your preferred content format

Data encryption forms the foundation of database security. Even if attackers gain access to your underlying storage, encryption keeps your sensitive information unreadable. Microsoft's SQL platforms offer multiple encryption options, from protecting data at rest to securing data while it's being processed.

Understanding when to use each method helps you balance protection with performance. Let's explore the encryption technologies available in SQL Server, Azure SQL, and SQL databases in Microsoft Fabric.

Understand encryption layers

Database encryption operates at different layers, each solving a specific problem. Transparent Data Encryption (TDE) encrypts data at rest—think of it as protecting your database files on disk. Column-level encryption targets specific sensitive columns, while Always Encrypted goes further by protecting data throughout its lifecycle, even during query processing.



When you enable TDE, SQL Server automatically encrypts database files, transaction logs, and backups. Your applications don't need any code changes—encryption happens transparently behind the scenes. TDE uses a database encryption key protected by a certificate stored in the master database.

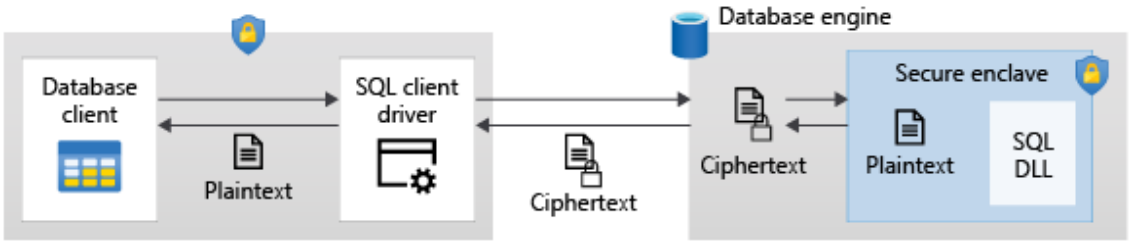
Column-level encryption works differently. You explicitly encrypt and decrypt data in your T-SQL code or application, giving you granular control over which columns contain sensitive data and who can decrypt them.

Always Encrypted takes yet another approach by keeping encryption keys outside the database engine entirely. The database never sees your plaintext data, which means even database administrators with high-level access can't view protected information.

Configure Always Encrypted

Always Encrypted ensures the database engine never processes plaintext values. Your client applications hold the encryption keys and handle all encryption and

decryption. This separation means that even someone with administrative access to the database can't view the protected data.



To get started with Always Encrypted, you first create a column master key (CMK) that protects your column encryption keys. Store the CMK in a secure key store such as Azure Key Vault, Windows Certificate Store, or a hardware security module.

The following T-SQL statement creates a metadata entry pointing to your key in Azure Key Vault. The actual key material remains in the vault, never stored in the database.

SQL

```
CREATE COLUMN MASTER KEY MyCMK

WITH (

  KEY_STORE_PROVIDER_NAME = 'AZURE_KEY_VAULT',

  KEY_PATH = 'https://mykeyvault.vault.azure.net/keys/MyCMK/abc123'

);
```

Next, create a column encryption key (CEK) protected by the column master key:

SQL

```
CREATE COLUMN ENCRYPTION KEY MyCEK

WITH VALUES (

  COLUMN_MASTER_KEY = MyCMK,

  ALGORITHM = 'RSA_OAEP',

  ENCRYPTED_VALUE = 0x01700000016C006F00...

);
```

Notice that the CEK itself is stored in encrypted form. When your application needs to work with encrypted data, it retrieves this value and uses the CMK to decrypt it locally.

When creating or altering tables, you specify the encryption type for sensitive columns:

SQL

```
CREATE TABLE Employees (
  EmployeeID int PRIMARY KEY,
  SSN char(11) COLLATE Latin1_General_BIN2
    ENCRYPTED WITH (
      ENCRYPTION_TYPE = DETERMINISTIC,
      ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256',
      COLUMN_ENCRYPTION_KEY = MyCEK
    ),
  Salary money
    ENCRYPTED WITH (
      ENCRYPTION_TYPE = RANDOMIZED,
      ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256',
      COLUMN_ENCRYPTION_KEY = MyCEK
    )
);
```

You have two encryption types to choose from. Use **deterministic** when you need to perform equality comparisons, joins, or filter with WHERE clauses—the same plaintext always produces the same ciphertext. Use **randomized** for stronger security when you don't need those query operations.

Implement column-level encryption

Column-level encryption using T-SQL functions gives you an alternative when you need more control over the encryption process, or when Always Encrypted isn't the right fit. With this approach, you manage symmetric or asymmetric keys stored within the database.

Start by creating a database master key and certificate:

SQL

```
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'StrongPassword123!';
```

```
CREATE CERTIFICATE SensitiveDataCert
WITH SUBJECT = 'Certificate for sensitive data encryption';
```

Create a symmetric key protected by the certificate:

SQL

```
CREATE SYMMETRIC KEY SensitiveDataKey
WITH ALGORITHM = AES_256
ENCRYPTION BY CERTIFICATE SensitiveDataCert;
```

To encrypt data, open the symmetric key and use the ENCRYPTBYKEY function:

SQL

```
OPEN SYMMETRIC KEY SensitiveDataKey
DECRYPTION BY CERTIFICATE SensitiveDataCert;

INSERT INTO CustomerData (CustomerID, CreditCardNumber)
VALUES (1, ENCRYPTBYKEY(KEY_GUID('SensitiveDataKey'), '4111-1111-1111-1111'));

CLOSE SYMMETRIC KEY SensitiveDataKey;
```

Decryption follows a similar pattern using DECRYPTBYKEY:

SQL

```
OPEN SYMMETRIC KEY SensitiveDataKey
DECRYPTION BY CERTIFICATE SensitiveDataCert;

SELECT CustomerID,
    CONVERT(varchar(20), DECRYPTBYKEY(CreditCardNumber)) AS CardNumber
FROM CustomerData;

CLOSE SYMMETRIC KEY SensitiveDataKey;
```

Yes, this approach requires more work—you're managing keys explicitly in your code. But that complexity comes with flexibility. You can grant or deny permissions to the symmetric key, giving you precise control over who can decrypt your data.

Choose the right encryption approach

Which encryption method should you use? It depends on your security requirements and application constraints.

TDE is your best choice when you need to protect data at rest without touching your application code. It's great for compliance requirements that mandate encryption of database files and backups. Keep in mind, though, that TDE doesn't protect data from users who can connect to the database with the right permissions.

Always Encrypted shines when you need to protect data from database administrators, or when sensitive data must stay encrypted even during query processing. The tradeoff? You need client driver support, and you're limited in what operations you can perform on encrypted columns.

Column-level encryption works well when you need granular control over encryption and decryption, or when you want to encrypt specific columns without the infrastructure overhead of Always Encrypted. It takes more development effort, but you get maximum flexibility in key management.

Tip

You can combine encryption methods. For example, enable TDE for baseline protection of data at rest, then add Always Encrypted for your most sensitive columns.

For SQL databases in Microsoft Fabric, TDE is enabled by default and managed automatically. Focus your encryption design decisions on column-level protection using Always Encrypted or symmetric key encryption based on your application requirements.

Next unit: Configure dynamic data masking

Configure dynamic data masking

Completed100 XP

- 10 minutes

Choose your preferred content format

Dynamic Data Masking provides a way to limit exposure of sensitive data without changing your application code or the underlying data. When users query masked columns, SQL Server returns obfuscated values based on masking rules you define. The actual data remains unchanged in the database, but unauthorized users see masked values in query results.

This approach works well when you need to protect sensitive information from certain users while allowing others to see the complete data. Database administrators, developers, and support staff can work with production data without exposing actual customer information, credit card numbers, or other sensitive values.

Understand masking functions

Dynamic Data Masking supports four masking functions, each designed for different data types and scenarios. Understanding these functions helps you choose the right masking approach for your sensitive columns.

Dynamic Data Masking Functions in SQL Server

Function	Original Value	Masked Value
Default	John Smith	**XXXX**
Email	john.doe@example.com	**jXXX@XXXX.com**
Random	46295378	**84930217**
Partial	206-987-6543	**206-XXX-XX89**

The **default** function replaces the entire value with a fixed string. For strings, you see *"XXXX"* (or fewer X characters for shorter columns). Numbers display as zero, and dates show *"01-01-1900"*. Use this when you want complete obfuscation.

The **email** function reveals just the first character of an email address, replaces the middle with "XXX", and preserves the domain suffix. So *"john.smith@contoso.com"* appears as *"jXXX@XXXX.com"*. The data still looks like a valid email, but the actual address stays hidden.

The **random** function displays numeric values as a random number within a range you specify. Each query returns a different value. This works great for financial or statistical data where you need data that looks genuine.

The **partial** function gives you precise control. You specify how many characters to show at the start, what padding characters to use in the middle, and how many to show at the end. For example, a phone number might appear as *"206-XXX-XX89"*.

Configure column masks

You apply masks when creating tables or by altering existing columns. The MASKED WITH clause specifies which function to use.

Here's a table with several masked columns:

SQL

```
CREATE TABLE Customers (
  CustomerID int PRIMARY KEY,
  FirstName varchar(50),
  LastName varchar(50),
  Email varchar(100) MASKED WITH (FUNCTION = 'email()'),
  Phone varchar(20) MASKED WITH (FUNCTION = 'partial(3, "-XXX-XX", 2)'),
  CreditCardNumber varchar(19) MASKED WITH (FUNCTION = 'partial(0, "XXXX-XXXX-XXXX-", 4)'),
  Income decimal(18,2) MASKED WITH (FUNCTION = 'random(10000, 100000)'),
  SSN char(11) MASKED WITH (FUNCTION = 'default()')
);
```

Notice how each column uses a different masking function based on what makes sense for that data:

- Email uses email masking to preserve the email format
- Phone shows the first three digits and last two digits
- CreditCardNumber reveals only the last four digits
- Income displays a random value between 10,000 and 100,000
- SSN uses default masking for complete obfuscation

To add masking to an existing column, use ALTER COLUMN:

SQL

```
ALTER TABLE Customers
ALTER COLUMN DateOfBirth ADD MASKED WITH (FUNCTION = 'default()');
```

You can remove masking from a column when it's no longer needed:

SQL

```
ALTER TABLE Customers

ALTER COLUMN DateOfBirth DROP MASKED;
```

Control mask visibility with permissions

By default, users see masked data unless they have elevated permissions. The UNMASK permission controls who sees the real values behind the masks.

To allow a user to see all unmasked data in the database:

SQL

```
GRANT UNMASK TO DataAnalyst;
```

This database-level permission reveals all masked columns to the specified user. However, you might want more granular control, allowing users to unmask only specific tables or columns.

Starting with SQL Server 2022, you can grant UNMASK at the schema, table, or even column level:

SQL

```
-- Grant unmask on a specific schema

GRANT UNMASK ON SCHEMA::Sales TO SalesManager;


-- Grant unmask on a specific table

GRANT UNMASK ON Customers TO CustomerService;


-- Grant unmask on a specific column

GRANT UNMASK ON Customers(Phone) TO TelemarketingTeam;
```

This granular approach helps you follow the principle of least privilege—users see only the sensitive data they actually need for their job.

Important

Users with SELECT permission on a table combined with ALTER permission can potentially bypass masking by modifying the column definition. Carefully manage permissions to prevent users from removing masks.

Implement masking strategies

When planning your masking strategy, think about how different user roles interact with your data. Which columns contain sensitive information? Who legitimately needs to see the actual values?

A common pattern is to create database roles for different access levels:

SQL

```
-- Role for users who see masked data

CREATE ROLE MaskedDataViewers;

GRANT SELECT ON Customers TO MaskedDataViewers;


-- Role for users who see unmasked data

CREATE ROLE UnmaskedDataViewers;

GRANT SELECT ON Customers TO UnmaskedDataViewers;

GRANT UNMASK ON Customers TO UnmaskedDataViewers;


-- Add users to appropriate roles

ALTER ROLE MaskedDataViewers ADD MEMBER SupportStaff;

ALTER ROLE UnmaskedDataViewers ADD MEMBER ComplianceOfficer;
```

One thing to keep in mind: Dynamic Data Masking works at the presentation layer, meaning the actual data can still be inferred through certain query patterns. For highly sensitive data requiring complete protection, combine masking with other security measures like encryption or Row-Level Security.

Consider these scenarios where masking is useful:

- Development and testing environments where teams need realistic data structures without actual customer information
- Customer service applications where support staff need to verify partial account details
- Reporting scenarios where aggregate data matters but individual records should remain private
- Audit compliance where specific users require full access while others see limited information

Note

Dynamic Data Masking in SQL databases in Microsoft Fabric works the same way as Azure SQL Database. You configure masks using T-SQL through the SQL analytics endpoint.

Masking adds an important layer of defense, but don't rely on it alone for your most sensitive data. Use it alongside encryption, Row-Level Security, and proper permission management for comprehensive protection.

Next unit: Implement row-level security

Implement row-level security

Completed100 XP

- 11 minutes

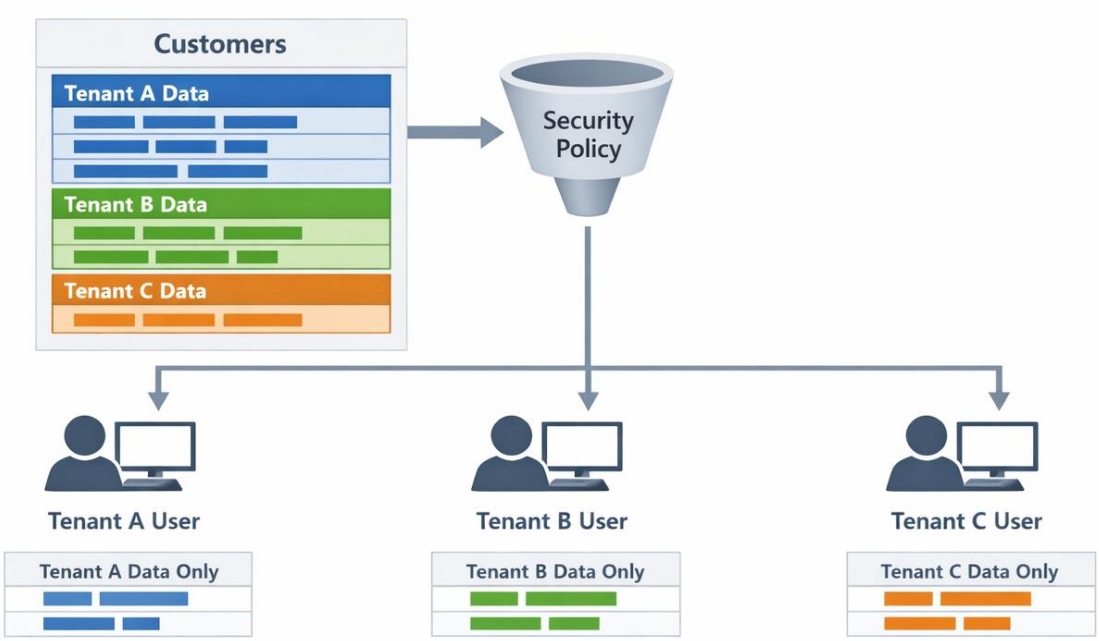
Choose your preferred content format

Row-Level Security (RLS) enables you to control access to rows in a database table based on the characteristics of the user executing a query. Unlike table-level permissions that grant or deny access to entire tables, RLS filters rows dynamically so users see only the data they're authorized to access.

This capability is useful when multiple users or tenants share the same tables but should only see their own data. Sales representatives see their own customer records, managers see their team's data, and regional directors see all data for their region. The filtering happens automatically without requiring application code changes.

Understand RLS components

Row-Level Security uses two components that work together: security predicates and security policies.



A **security predicate** is an inline table-valued function that returns 1 (true) or 0 (false) for each row. It receives the current user context and row values, then decides whether that user should see the row. Think of it as your business logic for data access, packaged as a function.

A security policy binds your predicate functions to tables and specifies the type of filtering. You can create **filter predicates** that silently exclude unauthorized rows from query results, or **block predicates** that prevent unauthorized insert, update, and delete operations.

Filter predicates affect SELECT, UPDATE, and DELETE statements by removing rows the user can't access. Users don't get errors—they just see a filtered result set. Block predicates work differently: they raise an error when users attempt unauthorized changes.

Create filter predicates

Let's start by creating a predicate function that evaluates row access. The function accepts parameters representing the column values to check and returns a table with a single row when access is allowed.

Here's a common scenario: a multitenant application where each row has a TenantID column:

SQL

```
CREATE SCHEMA Security;

GO
```

```
CREATE FUNCTION Security.fn_TenantAccessPredicate(@TenantID int)
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN SELECT 1 AS fn_TenantAccessPredicate_Result
  WHERE @TenantID = CAST(SESSION_CONTEXT(N'TenantID') AS int);
```

This function checks whether the row's TenantID matches the value stored in session context. Your application sets this context after the user authenticates:

SQL

```
EXEC sp_set_session_context @key = N'TenantID', @value = 42;
```

For scenarios based on database users rather than session context, the predicate can reference the current user:

SQL

```
CREATE FUNCTION Security.fn_SalesRepPredicate(@SalesRepID int)
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN SELECT 1 AS fn_SalesRepPredicate_Result
  WHERE @SalesRepID = DATABASE_PRINCIPAL_ID()
    OR IS_MEMBER('SalesManagers') = 1;
```

This predicate allows sales representatives to see their own records while managers in the SalesManagers role can see all records.

Create security policies

Once you've defined your predicate functions, create a security policy that applies them to tables:

SQL

```
CREATE SECURITY POLICY TenantSecurityPolicy
ADD FILTER PREDICATE Security.fn_TenantAccessPredicate(TenantID)
```

```
    ON dbo.Orders,
ADD FILTER PREDICATE Security.fn_TenantAccessPredicate(TenantID)
    ON dbo.OrderDetails
WITH (STATE = ON);
```

This policy filters both the Orders and OrderDetails tables using the same predicate function. When users query these tables, they only see rows matching their tenant context.

You can combine filter and block predicates in a single policy:

SQL

```
CREATE SECURITY POLICY SalesSecurityPolicy
ADD FILTER PREDICATE Security.fn_SalesRepPredicate(SalesRepID)
    ON dbo.CustomerAccounts,
ADD BLOCK PREDICATE Security.fn_SalesRepPredicate(SalesRepID)
    ON dbo.CustomerAccounts AFTER INSERT,
ADD BLOCK PREDICATE Security.fn_SalesRepPredicate(SalesRepID)
    ON dbo.CustomerAccounts AFTER UPDATE
WITH (STATE = ON);
```

Why add block predicates? Without them, a user could insert a row with a different SalesRepID and then lose access to data they just created. The block predicates ensure users can only insert or update rows they'd be able to see.

Implement hierarchical access patterns

Many organizations need hierarchical data access—managers should see their subordinates' data. You can implement this by combining RLS with a management hierarchy table.

Here's a function that traverses the hierarchy:

SQL

```
CREATE FUNCTION Security.fn_HierarchyPredicate(@OwnerID int)
RETURNS TABLE
WITH SCHEMABINDING
AS
```

```
RETURN
  WITH EmployeeHierarchy AS (
    SELECT EmployeeID, ManagerID
    FROM dbo.Employees
    WHERE EmployeeID = DATABASE_PRINCIPAL_ID()

    UNION ALL

    SELECT e.EmployeeID, e.ManagerID
    FROM dbo.Employees e
    INNER JOIN EmployeeHierarchy h ON e.ManagerID = h.EmployeeID
  )
  SELECT 1 AS fn_HierarchyPredicate_Result
  WHERE @OwnerID IN (SELECT EmployeeID FROM EmployeeHierarchy);
```

This recursive common table expression (CTE) builds the complete chain of employees reporting to the current user. The predicate allows access to any row owned by someone in that hierarchy.

Note

Recursive predicates can affect query performance on large datasets. Consider caching hierarchy relationships or limiting recursion depth for better performance.

Manage security policies

Security policies support several management operations for ongoing maintenance. You can disable a policy temporarily without removing it:

SQL

```
ALTER SECURITY POLICY TenantSecurityPolicy
WITH (STATE = OFF);
```

You can add new predicates to existing policies:

SQL

```
ALTER SECURITY POLICY TenantSecurityPolicy
```

```
ADD FILTER PREDICATE Security.fn_TenantAccessPredicate(TenantID)
  ON dbo.Shipments;
```

Or remove predicates when tables no longer need filtering:

SQL

```
ALTER SECURITY POLICY TenantSecurityPolicy
DROP FILTER PREDICATE ON dbo.Orders;
```

To view existing security policies and their predicates:

SQL

```
SELECT p.name AS PolicyName,
    p.is_enabled,
    o.name AS TableName,
    pred.predicate_definition
FROM sys.security_policies p
INNER JOIN sys.security_predicates pred ON p.object_id = pred.object_id
INNER JOIN sys.objects o ON pred.target_object_id = o.object_id;
```

Tip

When troubleshooting RLS issues, temporarily disable the security policy and compare results. This helps determine whether unexpected results come from the RLS predicates or other query logic.

Row-Level Security works transparently with views, stored procedures, and other database objects that query the protected tables. Users accessing data through any path receive consistently filtered results based on their security context.

Next unit: Manage permissions and secure access

Manage permissions and secure access

Completed100 XP

- 12 minutes

Choose your preferred content format

Object-level permissions control what actions users can perform on database objects like tables, views, stored procedures, and functions. While Row-Level Security filters data within objects, object-level permissions determine whether a user can access an object at all and what operations they can perform.

A well-designed permission model goes hand-in-hand with secure authentication. This unit covers both: how to grant the right permissions and how to ensure users authenticate securely, preferably without passwords.

Understand the permission hierarchy

SQL Server uses a hierarchical permission model where permissions granted at higher levels flow down to lower levels. Think of it as a cascade: server → database → schema → individual objects.

At the server level, permissions control sign-in management and server configuration. Database-level permissions govern actions within a specific database. Schema-level permissions apply to all objects within a schema, while object-level permissions target specific tables, views, or procedures.

Here's a powerful pattern: when you grant SELECT permission on a schema, users can select from all tables and views in that schema—including objects created in the future:

SQL

```
-- Grant SELECT on all objects in the Sales schema

GRANT SELECT ON SCHEMA::Sales TO SalesAnalyst;


-- Grant EXECUTE on all procedures in the Reports schema

GRANT EXECUTE ON SCHEMA::Reports TO ReportingUsers;
```

Grant and revoke permissions

Three statements control permissions: GRANT gives users specific permissions, REVOKE removes previously granted permissions, and DENY explicitly blocks permissions—overriding any grants.

SQL

```
-- Allow reading data

GRANT SELECT ON dbo.Customers TO CustomerServiceRole;
```

-- Allow data modifications

GRANT INSERT, UPDATE ON dbo.Orders TO OrderProcessingRole;

-- Explicitly deny access to sensitive data (overrides any grants)

DENY SELECT ON dbo.EmployeeSalaries TO HRAssistants;

-- Remove a permission without explicitly denying it

REVOKE INSERT ON dbo.Customers FROM CustomerServiceRole;

What's the difference between REVOKE and DENY? It matters when users have multiple permission sources. Revoking removes one grant but doesn't prevent access through other grants. Denying blocks access no matter what other grants exist.

Implement role-based access control

Instead of granting permissions directly to users, create database roles that represent job functions. Assign permissions to roles, then add users to the appropriate roles:

SQL

-- Create roles for different job functions

CREATE ROLE DataReaders;

CREATE ROLE DataWriters;

-- Grant appropriate permissions to each role

GRANT SELECT ON SCHEMA::dbo TO DataReaders;

GRANT INSERT, UPDATE, DELETE ON SCHEMA::dbo TO DataWriters;

-- Add users to roles

ALTER ROLE DataReaders ADD MEMBER JohnSmith;

ALTER ROLE DataWriters ADD MEMBER JaneDoc;

You can also nest roles to create hierarchical permission structures—adding someone to a parent role automatically gives them all child role permissions.

Tip

Document your role hierarchy and use a naming convention that makes each role's purpose clear, such as prefixing with the department or function.

Apply schema separation for security

Schemas provide a natural way to organize objects and manage permissions. Group related objects together, then grant permissions at the schema level:

SQL

```
CREATE SCHEMA Sales AUTHORIZATION dbo;

CREATE SCHEMA HR AUTHORIZATION dbo;


-- Sales team can read and write sales data

GRANT SELECT, INSERT, UPDATE ON SCHEMA::Sales TO SalesTeam;


-- HR has full control of their schema

GRANT CONTROL ON SCHEMA::HR TO HRAdministrators;
```

Configure Microsoft Entra authentication

Modern applications should use passwordless authentication whenever possible. Microsoft Entra authentication eliminates the risks associated with password management, including credential theft and the operational burden of rotating secrets.

SQL databases support multiple authentication methods. SQL authentication uses a username and password stored in the database. It's straightforward, but risky if credentials get compromised. Microsoft Entra authentication extends identity integration to cloud scenarios, working with Azure SQL Database, Azure SQL Managed Instance, SQL Server 2022+, and SQL databases in Microsoft Fabric.

Creating database users from Microsoft Entra identities is simple:

SQL

```
-- Create users for Entra accounts, managed identities, or groups

CREATE USER [app-service-identity] FROM EXTERNAL PROVIDER;
```

CREATE USER [developer@contoso.com] FROM EXTERNAL PROVIDER;

CREATE USER [DataAnalystsGroup] FROM EXTERNAL PROVIDER;

-- Grant permissions just like SQL users

ALTER ROLE db_datareader ADD MEMBER [developer@contoso.com];

ALTER ROLE db_datawriter ADD MEMBER [app-service-identity];

Implement managed identity authentication

Managed identities are the most secure option for Azure-hosted applications. Azure handles the identity lifecycle automatically, and there are no credentials that could be leaked or stolen.

For an Azure App Service connecting to Azure SQL Database, enable the system-assigned managed identity, then create a database user:

SQL

CREATE USER [MyWebApp] FROM EXTERNAL PROVIDER;

ALTER ROLE db_datareader ADD MEMBER [MyWebApp];

ALTER ROLE db_datawriter ADD MEMBER [MyWebApp];

Update your application connection string to use managed identity authentication:

Server=tcp:myserver.database.windows.net,1433;Database=mydb;Authentication=Active Directory Managed Identity;Encrypt=True;

Tip

Use user-assigned managed identities when multiple applications need the same database permissions. This reduces the number of database users to manage.

Secure connection strings

No matter which authentication method you use, protect your connection strings with these best practices:

- Store them in Azure Key Vault or environment variables, not in code
- Use managed identities to eliminate credentials from connection strings entirely
- Enable encrypted connections with Encrypt=True;TrustServerCertificate=False
- Limit network access using firewall rules and private endpoints

Important

When using client secrets for service principals, store them in Azure Key Vault and rotate them regularly. Certificate-based authentication provides stronger security than shared secrets.

View and audit permissions

You can query system views to understand current permission assignments:

SQL

```
-- View all permissions for a specific principal
SELECT
  prin.name AS PrincipalName,
  perm.permission_name,
  perm.state_desc AS PermissionState,
  obj.name AS ObjectName
FROM sys.database_permissions perm
INNER JOIN sys.database_principals prin ON perm.grantee_principal_id = prin.principal_id
LEFT JOIN sys.objects obj ON perm.major_id = obj.object_id
WHERE prin.name = 'SalesAnalyst';


-- Check effective permissions for the current user
SELECT * FROM fn_my_permissions('Sales.Orders', 'OBJECT');
```

Make it a habit to audit permission assignments regularly to ensure they still align with current job functions. And remember, object-level permissions work best when combined with other security features like Row-Level Security and Dynamic Data Masking for comprehensive protection.

Next unit: Implement auditing

Implement auditing

Completed100 XP

- 10 minutes

Choose your preferred content format

Auditing provides visibility into database activity, helping you detect security threats, track compliance, and investigate incidents. By recording who accessed what data and when, auditing creates an accountability trail that supports both security monitoring and regulatory requirements.

SQL Server, Azure SQL, and SQL databases in Microsoft Fabric offer built-in auditing capabilities that capture database events without requiring application changes. Understanding how to configure and manage auditing helps you maintain appropriate oversight of your database environments.

Understand auditing options

Auditing captures database events and writes them to an audit log. The events you audit, where you store the logs, and how you analyze them depend on your security requirements and infrastructure.



SQL Server Audit uses the Extended Events infrastructure to record activity. You can write audit records to files, the Windows Security log, or the Windows Application log. This flexibility lets you integrate with existing log management systems.

Azure SQL Database auditing writes to Azure Blob Storage, Log Analytics, or Event Hubs. The managed service handles the infrastructure, so you can focus on deciding what to audit rather than managing storage.

SQL databases in Microsoft Fabric use Fabric's activity logging and Microsoft Purview for audit data. This integration with Microsoft Purview gives you unified auditing across your entire data estate.

Configure SQL Server auditing

SQL Server Audit requires creating a server audit object that defines where to write audit records, then creating audit specifications that define what to capture.

Start by creating a server audit that writes to a file:

SQL

```
CREATE SERVER AUDIT SecurityAudit
TO FILE (FILEPATH = 'C:\AuditLogs\', MAXSIZE = 100 MB, MAX_ROLLOVER_FILES = 10)
WITH (QUEUE_DELAY = 1000, ON_FAILURE = CONTINUE);
```

The QUEUE_DELAY parameter specifies how many milliseconds to buffer events before writing. Lower values provide more real-time logging but can affect performance. The ON_FAILURE setting determines behavior when audit writing fails. Use SHUTDOWN for critical compliance scenarios where missing audit records is unacceptable.

Now enable the audit:

SQL

```
ALTER SERVER AUDIT SecurityAudit WITH (STATE = ON);
```

Next, create a server audit specification to capture server-level events:

SQL

```
CREATE SERVER AUDIT SPECIFICATION ServerAuditSpec
FOR SERVER AUDIT SecurityAudit
ADD (FAILED_LOGIN_GROUP),
ADD (SUCCESSFUL_LOGIN_GROUP),
ADD (SERVER_PERMISSION_CHANGE_GROUP),
ADD (DATABASE_PERMISSION_CHANGE_GROUP)
WITH (STATE = ON);
```

For database-level events, create a database audit specification:

SQL

```
USE MyDatabase;
GO
```

```
CREATE DATABASE AUDIT SPECIFICATION DatabaseAuditSpec
FOR SERVER AUDIT SecurityAudit
ADD (SELECT, INSERT, UPDATE, DELETE ON dbo.SensitiveData BY public),
ADD (EXECUTE ON SCHEMA::dbo BY public),
ADD (DATABASE_ROLE_MEMBER_CHANGE_GROUP)
WITH (STATE = ON);
```

This specification audits all data access on the SensitiveData table, stored procedure executions, and role membership changes.

Configure Azure SQL auditing

Azure SQL Database auditing is configured at the server or database level. Server-level policies apply to all databases on the logical server.

You can enable auditing in the Azure portal or using T-SQL:

SQL

```
-- Enable auditing to blob storage (configured in Azure portal)
ALTER DATABASE AUDIT SPECIFICATION AzureAuditSpec
ADD (SELECT, INSERT, UPDATE, DELETE ON DATABASE::MyDatabase BY public)
WITH (STATE = ON);
```

For more granular control, configure auditing through Azure Policy or ARM templates. Here's an example specifying the storage account, retention period, and audit action groups:

JSON

```
{
  "properties": {
    "state": "Enabled",
    "storageEndpoint": "https://myauditlogs.blob.core.windows.net",
    "retentionDays": 90,
    "auditActionsAndGroups": [
      "SUCCESSFUL_DATABASE_AUTHENTICATION_GROUP",
      "FAILED_DATABASE_AUTHENTICATION_GROUP",
```

```
    "BATCH_COMPLETED_GROUP"
  ]
 }
}
```

Note

Azure SQL auditing to Log Analytics enables powerful query and alerting capabilities using Kusto Query Language (KQL). This integration simplifies security monitoring and compliance reporting.

Select audit actions

Choose audit actions based on your security and compliance requirements. Common action groups include:

Authentication events:

- SUCCESSFUL_DATABASE_AUTHENTICATION_GROUP - Successful logins
- FAILED_DATABASE_AUTHENTICATION_GROUP - Failed sign-in attempts

Permission changes:

- DATABASE_PERMISSION_CHANGE_GROUP - Grant, revoke, deny operations
- DATABASE_ROLE_MEMBER_CHANGE_GROUP - Role membership changes
- DATABASE_PRINCIPAL_CHANGE_GROUP - User creation and modification

Data access:

- BATCH_COMPLETED_GROUP - All completed batches (high volume)
- SELECT on specific objects - Targeted read access monitoring
- INSERT, UPDATE, DELETE on specific objects - Data modification tracking

Schema changes:

- SCHEMA_OBJECT_CHANGE_GROUP - Table and object modifications
- DATABASE_OBJECT_CHANGE_GROUP - DDL statements

Important

Start with a focused set of audit actions rather than capturing everything. High-volume auditing impacts performance and generates logs that are difficult to analyze.

Query and analyze audit logs

For SQL Server file-based audits, use the fn_get_audit_file function to query your logs:

SQL

```
SELECT
  event_time,
  action_id,
  succeeded,
  session_server_principal_name AS UserName,
  database_name,
  object_name,
  statement
FROM fn_get_audit_file('C:\AuditLogs\*.sqlaudit', DEFAULT, DEFAULT)
WHERE event_time > DATEADD(day, -7, GETUTCDATE())
ORDER BY event_time DESC;
```

If you're using Azure SQL auditing with Log Analytics, you can query using KQL:

Kusto

```
AzureDiagnostics
| where Category == "SQLSecurityAuditEvents"
| where TimeGenerated > ago(7d)
| where action_name_s == "SELECT"
| summarize count() by client_ip_s, server_principal_name_s
| order by count_ desc
```

This query identifies which users and IP addresses generated the most SELECT queries over the past week, helping identify unusual access patterns.

Implement audit retention and protection

Your audit logs need protection from tampering, and you need to retain them according to compliance requirements.

For SQL Server, configure immutable storage for audit files and use Windows file system permissions to prevent deletion. For Azure SQL, configure storage with immutable blob storage and set retention policies in Azure Storage lifecycle management.

Important

Store audit logs separately from the databases they monitor. This ensures that even if attackers compromise a database, they can't tamper with the audit trail.

For compliance scenarios, consider these retention practices:

- Define retention periods based on regulatory requirements (often seven years for financial data)
- Use immutable storage to prevent log deletion
- Implement log archival to cold storage for cost management
- Establish processes for audit log review and alerting

Regular review of audit data helps you identify security issues before they become incidents. Create alerts for failed sign-in attempts, permission changes outside maintenance windows, and unusual data access patterns.

Next unit: Configure secure access to AI services

Configure secure access to AI services

Completed100 XP

- 10 minutes

Choose your preferred content format

AI-enabled database solutions often expose model endpoints that applications call to generate predictions, embeddings, or other AI-powered responses. Securing these endpoints protects both your AI models and the data they process. Managed Identity provides a secure, credential-free way to authenticate applications to model endpoints.

When applications call Azure OpenAI, Azure Machine Learning, or other AI services from within your database environment, you need to control who can invoke these

calls and protect the communication channel. Let's look at strategies for securing model endpoints in SQL Server, Azure SQL, and SQL databases in Microsoft Fabric.

Understand model endpoint security concerns

Model endpoints present unique security challenges compared to traditional database operations. These endpoints often process sensitive data, can incur significant costs per call, and may expose intellectual property embedded in fine-tuned models.

What happens if someone gains unauthorized access? They could exfiltrate data through carefully crafted prompts, run up unexpected costs with excessive API calls, or probe your proprietary model behaviors. That's why protecting these endpoints requires authentication, authorization, and monitoring.

In Azure SQL and SQL databases in Microsoft Fabric, you can call external AI services using stored procedures or functions that make HTTP requests. These calls must authenticate to the AI service, typically using API keys or Managed Identity.

Configure Managed Identity for AI services

Managed Identity eliminates the need to store API keys in your database or application code. Instead, Azure manages the identity credentials automatically, and you grant that identity access to AI services.

For an Azure SQL Database that needs to call Azure OpenAI, start by enabling system-assigned managed identity:

SQL

```
-- The managed identity is enabled in Azure portal or via Azure CLI

-- Verify the identity exists

SELECT * FROM sys.dm_external_provider_certificate_store;
```

Next, grant the managed identity access to Azure OpenAI using Azure role-based access control:

PowerShell

```
# Get the managed identity principal ID from Azure SQL

$sqlIdentity = (Get-AzSqlServer -ServerName myserver -ResourceGroupName myrg).Identity.PrincipalId


# Assign Cognitive Services User role on the Azure OpenAI resource
```

```
New-AzRoleAssignment -ObjectId $sqlIdentity `
  -RoleDefinitionName "Cognitive Services User" `
  -Scope "/subscriptions/{sub-id}/resourceGroups/{rg}/providers/Microsoft.CognitiveServices/accounts/{aoai-name}"
```

Create a database-scoped credential that uses Managed Identity:

SQL

```
CREATE DATABASE SCOPED CREDENTIAL AzureOpenAICredential
WITH IDENTITY = 'Managed Identity',
SECRET = '{"resourceId": "https://cognitiveservices.azure.com/"}';
```

This credential tells SQL to obtain a token for the Cognitive Services resource scope using its managed identity.

Implement secure endpoint calls

Now let's put it all together. Use external REST endpoint invocation to call AI services securely. First, create an external data source that references your AI endpoint:

SQL

```
CREATE EXTERNAL DATA SOURCE AzureOpenAI
WITH (
  TYPE = REST,
  LOCATION = 'https://myopenai.openai.azure.com',
  CREDENTIAL = AzureOpenAICredential
);
```

Then create a stored procedure that calls the AI endpoint:

SQL

```
CREATE PROCEDURE dbo.GetEmbedding
  @InputText nvarchar(max),
  @Embedding nvarchar(max) OUTPUT
AS
BEGIN
```

```
    DECLARE @payload nvarchar(max) = JSON_OBJECT('input': @InputText);
    DECLARE @response nvarchar(max);

    EXEC sp_invoke_external_rest_endpoint
        @url = 'https://myopenai.openai.azure.com/openai/deployments/text-embedding/embeddings?api-version=2024-02-01',
        @method = 'POST',
        @payload = @payload,
        @credential = AzureOpenAICredential,
        @response = @response OUTPUT;

    SET @Embedding = JSON_VALUE(@response, '$.data[0].embedding');
END;
```

Grant execute permission only to users who should access AI capabilities:

SQL

```
-- Create a role for AI feature access
CREATE ROLE AIFeatureUsers;
GRANT EXECUTE ON dbo.GetEmbedding TO AIFeatureUsers;

-- Add specific users to the role
ALTER ROLE AIFeatureUsers ADD MEMBER [app-service-identity];
```

Important

Limit which database users can invoke AI endpoints. AI calls often incur costs and can process sensitive data. Use role-based access to control which applications and users can use these features.

Secure Azure Machine Learning endpoints

Azure Machine Learning model endpoints follow similar patterns. Configure Managed Identity access and create credentials for authentication.

Here's how to set up access for real-time inference endpoints:

SQL

```
CREATE DATABASE SCOPED CREDENTIAL AMLCredential
WITH IDENTITY = 'Managed Identity',
SECRET = '{"resourceId": "https://ml.azure.com/"}';

CREATE EXTERNAL DATA SOURCE AMLEndpoint
WITH (
  TYPE = REST,
  LOCATION = 'https://myworkspace.region.inference.ml.azure.com',
  CREDENTIAL = AMLCredential
);
```

For batch inference endpoints, you typically submit data to storage and trigger a pipeline. Make sure to secure that storage access using Managed Identity.

Implement endpoint monitoring

You can monitor AI endpoint usage to detect anomalies and keep costs under control. Here's a simple logging approach:

SQL

```
-- Create a logging table for AI calls
CREATE TABLE dbo.AIEndpointLog (
  LogID int IDENTITY PRIMARY KEY,
  CallTimestamp datetime2 DEFAULT SYSUTCDATETIME(),
  CallerPrincipal nvarchar(128) DEFAULT ORIGINAL_LOGIN(),
  EndpointName nvarchar(256),
  InputLength int,
  ResponseStatus int,
  DurationMs int
);

-- Modify procedures to log calls
```

```
CREATE PROCEDURE dbo.GetEmbeddingWithLogging
  @InputText nvarchar(max),
  @Embedding nvarchar(max) OUTPUT
AS
BEGIN
  DECLARE @StartTime datetime2 = SYSUTCDATETIME();
  DECLARE @response nvarchar(max);
  DECLARE @status int;

  -- Make the AI call
  EXEC sp_invoke_external_rest_endpoint
    @url = 'https://myopenai.openai.azure.com/openai/deployments/text-embedding/embeddings?api-version=2024-02-01',
    @method = 'POST',
    @payload = JSON_OBJECT('input': @InputText),
    @credential = AzureOpenAICredential,
    @response = @response OUTPUT;

  -- Log the call
  INSERT INTO dbo.AIEndpointLog (EndpointName, InputLength, ResponseStatus, DurationMs)
  VALUES ('text-embedding', LEN(@InputText), 200, DATEDIFF(millisecond, @StartTime, SYSUTCDATETIME()));

  SET @Embedding = JSON_VALUE(@response, '$.data[0].embedding');
END;
```

With this logging in place, you can query the log to spot unusual patterns:

SQL

```
-- Find users making excessive AI calls
```

```
SELECT CallerPrincipal, COUNT(*) AS CallCount, AVG(DurationMs) AS AvgDuration
FROM dbo.AIEndpointLog
WHERE CallTimestamp > DATEADD(hour, -1, SYSUTCDATETIME())
GROUP BY CallerPrincipal
HAVING COUNT(*) > 100
ORDER BY CallCount DESC;
```

Tip

Set up alerts for unusual AI endpoint usage patterns. Sudden spikes in calls or calls from unexpected principals can indicate compromised credentials or application bugs.

Secure AI features in Microsoft Fabric

SQL databases in Microsoft Fabric handle AI endpoint access differently than Azure SQL. Instead of database-scoped credentials and Managed Identity, Fabric uses workspace roles and capacity-based access control.

In Fabric, AI capabilities are tied to the workspace. Users with **Contributor** or higher workspace roles can invoke AI features like AI skills and Copilot. Access control happens at the workspace level rather than through database permissions.

To secure AI features in Fabric:

- Assign workspace roles carefully, since they control AI access
- Monitor usage through Fabric's capacity metrics and activity logs
- Use Fabric's data protection features to control what data AI features can access

For detailed guidance on configuring AI services in Fabric, see the Fabric AI documentation.

Next unit: Secure data API endpoints

Secure data API endpoints

Completed100 XP

- 11 minutes

Choose your preferred content format

Modern database solutions expose data through various API endpoints, enabling applications to access information without writing traditional SQL queries. GraphQL, REST, and Model Context Protocol (MCP) endpoints each present unique security considerations. Properly securing these endpoints protects your data while enabling the flexibility that modern applications require.

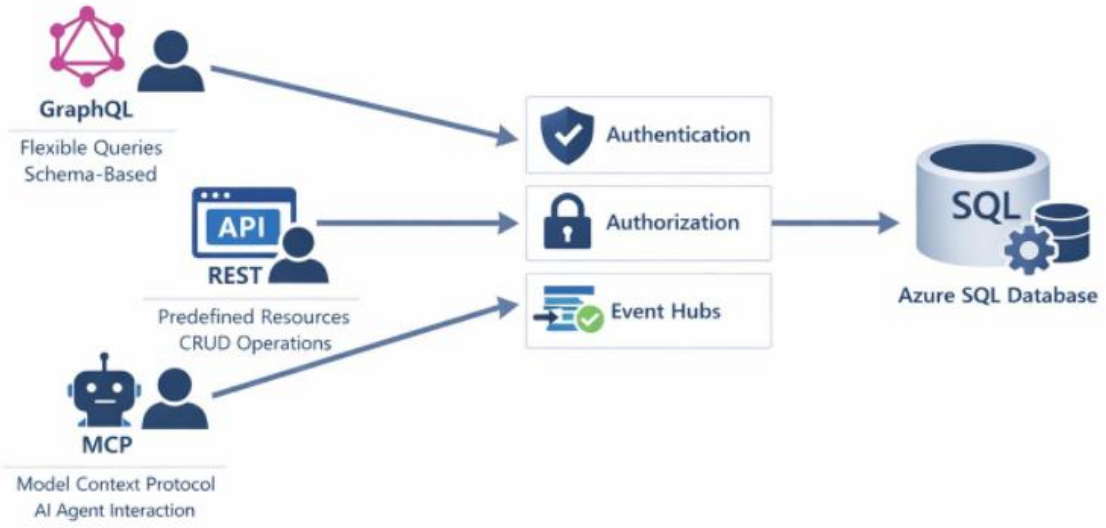
These endpoints often serve as the primary data access layer for web and mobile applications. Without appropriate security controls, they can expose sensitive data, allow unauthorized modifications, or become targets for denial-of-service attacks.

Understand endpoint types and risks

GraphQL endpoints provide flexible query capabilities, allowing clients to request exactly the data they need. This flexibility creates security challenges because clients can construct complex queries that might expose unintended data or consume excessive resources.

REST endpoints follow a more structured pattern with predefined resources and operations. Security focuses on authentication, authorization for specific endpoints, and input validation. REST APIs have been around long enough that security patterns are well established.

MCP endpoints enable AI models and agents to interact with databases, providing context and executing actions. These endpoints require careful security consideration because AI systems may attempt operations that human users wouldn't, and prompt injection attacks can manipulate AI behavior.



Secure GraphQL endpoints

Azure SQL Database and SQL databases in Microsoft Fabric support GraphQL through Data API builder and Microsoft Fabric's GraphQL API. Securing these endpoints involves authentication, authorization, and query controls.

Here's how to configure authentication requirements:

JSON

```
{
 "runtime": {
  "rest": { "enabled": false },
  "graphql": {
   "enabled": true,
   "path": "/graphql",
   "allow-introspection": false
  }
 },
 "authentication": {
  "provider": "AzureAD",
  "jwt": {
   "audience": "api://my-graphql-api",
   "issuer": "https://login.microsoftonline.com/{tenant-id}/v2.0"
  }
 }
}
```

Notice the allow-introspection: false setting. Disable introspection in production to prevent attackers from discovering your schema structure. Introspection queries reveal all available types, fields, and relationships.

Next, implement entity-level permissions to control which roles can access which data:

JSON

```
{
```

```
"entities": {
  "Customer": {
    "source": "dbo.Customers",
    "permissions": [
      {
        "role": "authenticated",
        "actions": ["read"]
      },
      {
        "role": "admin",
        "actions": ["*"]
      }
    ]
  },
  "Order": {
    "source": "dbo.Orders",
    "permissions": [
      {
        "role": "authenticated",
        "actions": ["read"],
        "fields": {
          "include": ["OrderID", "OrderDate", "Status"],
          "exclude": ["InternalNotes", "CostPrice"]
        }
      }
    ]
  }
}
```

```
}
```

See how the Order entity excludes sensitive columns like InternalNotes and CostPrice? Even authenticated users can't access those fields.

Tip

Implement query depth and complexity limits to prevent denial-of-service attacks through deeply nested queries. Data API builder includes built-in protections against excessive query complexity.

Secure REST endpoints

REST endpoints in Data API builder or custom implementations need authentication and endpoint-specific authorization. Here's an example:

JSON

```
{
  "entities": {
    "Product": {
      "source": "dbo.Products",
      "rest": {
        "enabled": true,
        "path": "/products"
      },
      "permissions": [
        {
          "role": "anonymous",
          "actions": [
            { "action": "read", "policy": { "database": "@item.IsPublic eq true" } }
          ]
        },
        {
          "role": "inventory-manager",
          "actions": ["create", "read", "update"]
```

```
    },
    {
      "role": "admin",
      "actions": ["*"]
    }
   ]
  }
 }
}
```

Notice the database policy on anonymous access? It filters results to only public products. Role-based permissions then control what actions each role can perform.

For custom REST endpoints built with stored procedures, implement security at the database level:

SQL

```
CREATE PROCEDURE api.GetCustomerOrders
  @CustomerID int
AS
BEGIN
  -- Verify the caller has access to this customer
  IF NOT EXISTS (
    SELECT 1 FROM dbo.CustomerAccess
    WHERE CustomerID = @CustomerID
    AND UserPrincipal = ORIGINAL_LOGIN()
  )
  BEGIN
    THROW 50401, 'Unauthorized access to customer data', 1;
    RETURN;
  END
```

```
  SELECT OrderID, OrderDate, TotalAmount
  FROM dbo.Orders
  WHERE CustomerID = @CustomerID;
END;
```

Secure MCP endpoints

Model Context Protocol (MCP) endpoints let AI assistants and agents interact with your database. MCP requires extra security attention because AI systems might process untrusted user input.

Here's how to configure MCP server authentication:

JSON

```
{
 "mcpServers": {
  "sqlDatabase": {
   "transport": "stdio",
   "authentication": {
    "type": "azure-identity",
    "scope": "https://database.windows.net/.default"
   },
   "security": {
    "allowedOperations": ["read"],
    "deniedTables": ["dbo.Passwords", "dbo.APIKeys"],
    "maxRowsReturned": 1000
   }
  }
 }
}
```

The key is limiting what MCP endpoints can do:

- Restrict operations to read-only when write access isn't needed

- Explicitly deny access to tables containing credentials or sensitive configuration
- Limit result set sizes to prevent data exfiltration through large queries
- Log all MCP operations for security monitoring

You'll also want to implement prompt injection defenses by validating AI-generated queries:

SQL

```
CREATE PROCEDURE mcp.ExecuteQuery
  @QueryDescription nvarchar(max),
  @GeneratedQuery nvarchar(max) OUTPUT
AS
BEGIN
  -- Validate the generated query doesn't access restricted objects
  IF @GeneratedQuery LIKE '%sys.%' OR @GeneratedQuery LIKE
'%INFORMATION_SCHEMA%'
  BEGIN
    THROW 50403, 'Access to system objects not permitted', 1;
    RETURN;
  END

  -- Ensure query is read-only
  IF @GeneratedQuery LIKE '%INSERT%' OR @GeneratedQuery LIKE '%UPDATE%'
    OR @GeneratedQuery LIKE '%DELETE%' OR @GeneratedQuery LIKE
'%DROP%'
  BEGIN
    THROW 50403, 'Write operations not permitted', 1;
    RETURN;
  END
```

```
    -- Execute the validated query
    EXEC sp_executesql @GeneratedQuery;
END;
```

Important

Never trust AI-generated queries without validation. Implement allow lists for permitted tables and operations rather than trying to block malicious patterns.

Implement network security

Regardless of endpoint type, protect the network layer:

SQL

```
-- Azure SQL: Configure firewall rules
-- Deny all public access, allow only specific IPs or virtual networks
EXECUTE sp_set_firewall_rule
    @name = 'AllowAppService',
    @start_ip_address = '10.0.0.1',
    @end_ip_address = '10.0.0.255';
```

Use Private Link or service endpoints to keep traffic on the Microsoft network:

- Configure virtual network integration for App Services hosting API endpoints
- Use Private Endpoints for Azure SQL Database
- Enable managed private endpoints in Microsoft Fabric

These network controls add defense in depth, ensuring that even if application-level security is bypassed, attackers can't reach your database from unauthorized networks.

Next unit: Exercise: Implement security features

Summary

Completed100 XP

- 2 minutes

In this module, you learned how to implement comprehensive data security and compliance features for SQL Server, Azure SQL Database, and SQL databases in Microsoft Fabric.

You explored how to:

- Implement data encryption using Always Encrypted for client-side encryption and column-level encryption for granular protection
- Configure Dynamic Data Masking to protect sensitive columns while maintaining data usability for authorized users
- Design Row-Level Security policies to filter data access based on user context and business rules
- Apply object-level permissions using roles and schemas to implement the principle of least privilege
- Enable passwordless authentication using Microsoft Entra ID and Managed Identity
- Set up auditing to track database activity and maintain compliance records
- Secure AI model endpoints using Managed Identity authentication
- Protect GraphQL, REST, and MCP endpoints from unauthorized access

Key takeaways

- Defense in depth requires combining multiple security features. Use encryption for data protection, masking for presentation-layer security, and Row-Level Security for row-level access control.
- Managed Identity eliminates credential management risks by letting Azure handle authentication automatically.
- Auditing provides accountability, but plan your retention strategy and storage location to meet compliance requirements while managing costs.

Learn more

- Always Encrypted documentation
- Dynamic Data Masking in Azure SQL Database
- Row-Level Security
- Microsoft Entra authentication with Azure SQL
- Auditing for Azure SQL Database

Secure, optimize, and deploy database solutions

Optimize database performance

Optimize Azure SQL Database performance by choosing the right service tier and managing concurrency with transaction isolation levels. Analyze queries with execution plans and DMVs. Use Query Store for plan management and diagnose blocking and deadlocks.

Introduction

A slow query during peak hours can cascade into a service outage. A poorly chosen isolation level can cause users to see stale data or block each other for seconds at a time. Performance problems in Azure SQL Database rarely have a single cause. They emerge from the interaction between hardware configuration, concurrency behavior, query execution, and workload patterns. Diagnosing these problems requires understanding each layer and knowing which tools to use at each level.

Imagine a team managing an e-commerce application on Azure SQL Database. During a holiday sale, transaction throughput drops and customers report timeouts. Is the database under-provisioned? Are queries scanning entire tables instead of seeking through indexes? Are blocking chains forming between concurrent transactions? Is the query optimizer choosing a bad plan? Each possibility requires a different investigation tool and a different resolution strategy. This module gives you the skills to work through each of those questions systematically.

After completing this module, you're able to:

- Evaluate and recommend database configurations including service tiers, compute tiers, and resource limits.
- Choose transaction isolation levels and concurrency controls that balance consistency with throughput.
- Analyze query performance using execution plans and dynamic management views.
- Monitor and tune queries with Query Store and Query Performance Insight.

- Identify and resolve blocking and deadlocks using DMVs and Extended Events.

Note

We recognize that different people like to learn in different ways. You can choose to complete this module in video-based format or you can read the content as text and images. The text contains greater detail than the videos, so in some cases you might want to refer to it as supplemental material to the video presentation.

Next unit: Recommend database configurations

Recommend database configurations

Choose your preferred content format

Before you can tune queries or troubleshoot concurrency, you need the right infrastructure underneath. Azure SQL Database gives you two resource models and multiple service tiers. The combination you choose sets the ceiling on compute, memory, I/O, and storage for your workload. Choose too little and performance suffers. Choose too much and you waste budget.

Compare resource models

Azure SQL Database supports two resource models: **vCore** and **DTU**. They measure and bill resources differently, so understanding the distinction helps you make the right choice from the start.

The **vCore model** gives you direct control over virtual cores, memory, and storage. You pick the hardware generation, service tier, and compute tier independently. If you're migrating from on-premises SQL Server, this model maps cleanly to physical CPU and memory, which makes capacity planning more straightforward. It also supports reserved instance pricing and the Azure Hybrid Benefit for cost savings.

The **DTU model** bundles CPU, memory, and I/O into a single unit called a **Database Transaction Unit** (DTU). DTU-based tiers (Basic, Standard, and Premium) offer preconfigured resource packages. This model works when you don't need fine-grained control over individual resource dimensions.

For most new deployments, Microsoft recommends the vCore model. It provides higher resource limits, greater scaling granularity, and more pricing flexibility.

Understand service tiers in the vCore model

The vCore model has three service tiers: General Purpose, Business Critical, and Hyperscale. Each tier uses a different architecture, which affects storage type, I/O performance, and availability.

General Purpose separates compute and storage. The database engine runs on a compute node while data files reside on Azure Blob Storage. Storage latency is typically between 5 milliseconds and 10 milliseconds. This architecture provides budget-friendly pricing and works well for most business workloads. If the compute node fails, Azure Service Fabric moves the process to a spare node and reattaches the remote storage files.

Consider the e-commerce application from the introduction. During normal business hours, General Purpose handles the order volume without issue. But during a holiday flash sale, I/O latency of 5 milliseconds to 10 milliseconds might not be fast enough for the checkout flow.

Business Critical integrates compute and storage on each node. The database engine and data files both use locally attached SSDs in an Always On availability group with three secondary replicas. This design gives you the lowest I/O latency (1 millisecond to 2 milliseconds on average), the highest IOPS (input/output operations per second), and a free read-only replica you can use for reporting queries. The trade-off is cost, roughly 2.7 times more than General Purpose for the same vCore count. For the e-commerce team, Business Critical makes sense if their checkout transactions need consistent sub-2-millisecond latency.

Hyperscale uses a decoupled storage architecture with independent page servers and a multi-tiered cache. It supports databases up to 128 TB, allows zero to four high-availability replicas, and scales compute up or down without copying data. You're billed only for allocated storage, not maximum storage. Hyperscale removes the practical storage and scaling limits of the other tiers and is suitable for the widest variety of workloads.

The following table summarizes the key differences:

Feature	General Purpose	Business Critical	Hyperscale
Storage type	Remote (Azure Blob Storage)	Local SSD	Decoupled with local
Max storage	4 TB	4 TB	128 TB
Max IOPS per vCore	320	4,000	5,500 (local SSD)
Availability replicas	1 (no read replicas)	3 + 1 read replica	0 to 4 (configurable)
Best for	Budget-oriented workloads	Low latency, high I/O	Large databases, flexi

Choose a compute tier

Within the vCore model, you also choose between two compute tiers: **provisioned** and **serverless**.

Provisioned compute allocates a fixed number of vCores that stay available regardless of workload activity. You pay a fixed hourly rate. This tier fits workloads with consistent or predictable resource consumption, like the e-commerce application that processes orders throughout the day.

Serverless compute automatically scales vCores based on demand and bills per second for the compute used. When the database is idle, it can autopause and eliminate compute costs entirely, though autopause is currently supported only in General Purpose. Serverless compute itself is available for both General Purpose and Hyperscale tiers. It works well for development environments, internal tools, or applications with intermittent traffic.

Match the configuration to your workload

Now that you understand the options, how do you decide? Evaluate your workload against these factors:

- **Latency requirements**: If your application needs I/O latency under 2 milliseconds consistently, choose Business Critical. For moderate latency tolerance, General Purpose is sufficient.
- **Storage size**: If your database exceeds 4 TB or you expect rapid growth, Hyperscale is the only option that accommodates up to 128 TB.
- **Read-heavy workloads**: Business Critical includes a free read-only replica. Hyperscale supports named replicas for flexible read scale-out.
- **Cost sensitivity**: General Purpose with provisioned compute offers predictable pricing. Serverless compute in General Purpose or Hyperscale reduces costs for intermittent workloads.
- **Availability requirements**: Business Critical provides the highest resiliency with three synchronous replicas and the fastest failover. Hyperscale lets you configure the number of replicas to balance resiliency with cost.

Tip

When migrating from on-premises SQL Server, use the vCore model because it maps directly to physical CPU and memory. The DTU model doesn't expose individual resource dimensions, which makes capacity planning harder for migrations.

Configure database-level settings

You picked your tier and compute model. Now look inside the database itself. Several settings affect how Azure SQL Database handles parallelism, query optimization, and recovery. You adjust these settings without changing your service tier.

Control parallelism with MAXDOP

Max degree of parallelism (MAXDOP) controls how many processor threads the engine assigns to a single query. Azure SQL Database defaults to **8**, which works for the widest variety of workloads. Before September 2020, new databases defaulted to 0, unlimited parallelism, and that caused problems. A single analytical query could consume every available thread, starving the checkout flow of CPU.

You set MAXDOP at the database level with ALTER DATABASE SCOPED CONFIGURATION SET MAXDOP. You can also set a different value for secondary replicas when your read-write and read-only workloads have different concurrency needs. For a specific query, use the OPTION (MAXDOP) hint. The one rule: avoid MAXDOP 0 in production. Unlimited parallelism leads to resource exhaustion, query timeouts, and application outages.

Let automatic tuning catch regressions

The query optimizer doesn't always pick the best plan. Statistics go stale, data distributions shift, and a plan that was fast yesterday becomes slow today. **Automatic tuning** monitors query performance and applies corrections without waiting for you to notice.

Azure SQL Database supports three options:

- **FORCE_LAST_GOOD_PLAN** detects plan regressions and forces the previous fast plan. Enabled by default.
- **CREATE_INDEX** identifies missing indexes, creates them, and verifies the improvement. Disabled by default.
- **DROP_INDEX** removes unused and duplicate indexes. Disabled by default. Unique indexes, including indexes supporting primary key and unique constraints, are never dropped.

Every change goes through a validation window, 30 minutes to 72 hours depending on query frequency. If performance gets worse, the change is automatically reverted.

SQL

```
ALTER DATABASE CURRENT
```

```
SET AUTOMATIC_TUNING (FORCE_LAST_GOOD_PLAN = ON,
        CREATE_INDEX = ON,
        DROP_INDEX = OFF);
```

Think about the e-commerce application during a holiday sale. Query patterns shift as different product pages spike in popularity. FORCE_LAST_GOOD_PLAN catches those regressions automatically, so a bad plan change at 2 AM doesn't slow down checkout until someone notices Monday morning. You probably want to leave CREATE_INDEX and DROP_INDEX off until what they propose is reviewed.

Unlock optimizer features with compatibility level

Every database has a **compatibility level** that determines which query optimizer behaviors are available. New databases in Azure SQL Database default to level **170**, or the highest available level. Each level unlocks a set of **intelligent query processing (IQP)** features:

- **Level 150**: batch mode on rowstore, table variable deferred compilation, scalar user-defined function (UDF) inlining.
- **Level 160**: parameter sensitive plan optimization (PSP), cardinality estimation feedback.
- **Level 170**: optional parameter plan optimization.

Existing databases can run at a lower compatibility level because Microsoft never automatically upgrades this setting. A database created when a lower default was in effect keeps its original level. For example, if you created an Azure SQL Database in 2024, the database is still at level 160 if the level isn't manually updated. Likewise, if you imported a database through a BACPAC file, the imported database's compatibility level is based on the source database's compatibility level. To move up:

SQL

```
ALTER DATABASE CURRENT SET COMPATIBILITY_LEVEL = 170;
```

Don't change this setting blindly in production. Use Query Store to capture a performance baseline at the current level, upgrade in a test environment, and compare. If a query regresses, you can force the old plan while you investigate.

Reduce plan cache bloat with OPTIMIZE_FOR_AD_HOC_WORKLOADS

The e-commerce application generates product search queries with dozens of filter combinations. Each unique query text gets its own compiled plan in the cache, even if that query never runs again. Over time, the plan cache fills with

thousands of single-use plans, pushing out the frequently executed plans that actually matter.

OPTIMIZE_FOR_AD_HOC_WORKLOADS solves this issue. When enabled, the engine stores a tiny **compiled plan stub** on first execution instead of the full plan. Only when the same query runs a second time does the engine compile and cache the full plan.

SQL

```
ALTER DATABASE SCOPED CONFIGURATION

SET OPTIMIZE_FOR_AD_HOC_WORKLOADS = ON;
```

This setting keeps the cache lean and ensures that plans for your most important queries stay resident in memory.

Understand accelerated database recovery

Accelerated database recovery is always enabled in Azure SQL Database. You can't turn it off, and you don't need to. Accelerated database recovery (ADR) redesigns the recovery process so that recovery time stays constant regardless of how many active transactions were running when a failure occurred. It also provides instantaneous transaction rollback and aggressive log truncation.

ADR stores row versions in a **persistent version store** (PVS) inside the database rather than in tempdb. Depending on the size of the row being modified, versions are stored either in-row on data pages or off-row in a separate internal table. Write-intensive workloads can see increased page splits and higher log generation because every row version is logged. To minimize that overhead, keep transactions short and reduce unnecessary aborted transactions.

The PVS shares your database's allocated storage, so a growing PVS reduces the space available for your data. To monitor off-row PVS overhead, query sys.dm_tran_persistent_version_store_stats and check the persistent_version_store_size_kb column, which reports the size of off-row versions only and doesn't include in-row versions stored on data pages. To establish a baseline during typical workloads, compare that value to your total database size. If PVS grows significantly beyond that baseline, look for long-running transactions or high abort rates that delay version cleanup.

Key takeaways

Azure SQL Database gives you two resource models, three service tiers, and two compute tiers. The vCore model with General Purpose covers most workloads. Business Critical adds sub-2-millisecond latency. Hyperscale removes storage and scaling limits. Inside the database, MAXDOP 8 is the safe default, automatic

tuning catches plan regressions, and upgrading your compatibility level unlocks the latest intelligent query processing (IQP) features. Enable OPTIMIZE_FOR_AD_HOC_WORKLOADS to keep the plan cache clean, and monitor PVS storage usage from ADR using sys.dm_tran_persistent_version_store_stats, especially in write-heavy scenarios. Next, you explore how isolation levels and concurrency control affect the queries running inside this infrastructure.

Next unit: Preserve data integrity with transaction isolation levels and concurrency controls

Preserve data integrity with transaction isolation levels and concurrency controls

Completed100 XP

- **10 minutes**

Choose your preferred content format

Choosing the right hardware and configuration is only part of database performance. What happens when two customers try to buy the last item in stock at the exact same moment? Or when a report reads a price that another transaction is in the middle of changing? Transaction isolation levels control how the database handles these situations.

How isolation levels work

Isolation levels define how much one transaction can see of another transaction's uncommitted work. The SQL standard frames them around three concurrency side effects, each of which maps to a real problem in your applications. Let's use an e-commerce application to illustrate them:

- **Dirty reads: A customer's order reads a discounted price that another transaction set but isn't committed yet. If the transaction that set the price rolls back, the order used a price that never existed. Dirty reads are defined as reading uncommitted data from another transaction, which can lead to incorrect behavior if that other transaction doesn't commit.**
- **Nonrepeatable reads: On the same transaction, your inventory service reads the stock count for an item. After the initial read, the transaction does some calculations, then reads the same row again. In between those steps, another transaction committed a change, so the count is**

now different. Nonrepeatable reads occur when a transaction reads the same row twice and gets different values each time.

- **Phantom reads: A report queries all orders placed in the last hour, calculates a total, then runs the same query moments later within the same transaction. Another transaction inserted new orders in between, so the totals don't match. Phantom reads happen when a transaction re-executes a query and sees new rows that weren't there before.**

Consider our e-commerce application during a flash sale with hundreds of concurrent customers. All three side effects can show up at once. The isolation level you choose determines which ones your application tolerates and how much blocking it accepts in return.

Notice that each of these problems involves a transaction that performs multiple operations: read a row, do some work, then read again or make a decision. A single-statement transaction usually completes so quickly that these conflicts rarely surface. Isolation levels matter most when your transactions span multiple statements, which is common in business logic that reads data, applies rules, and then writes results.

Here's a typical pattern where isolation levels come into play. Imagine two customers both trying to buy the last unit of the same product:

SQL

```
-- Transaction A (Customer 1)        -- Transaction B (Customer 2)

BEGIN TRANSACTION;

                                      BEGIN TRANSACTION;

SELECT @Stock = StockCount

 FROM Products

 WHERE ProductID = 42;

-- @Stock = 1, proceed with order

                                      SELECT @Stock = StockCount

                                       FROM Products

                                       WHERE ProductID = 42;

                                      -- @Stock = 1, proceed with order
```

```
UPDATE Products
 SET StockCount = StockCount - 1
 WHERE ProductID = 42;
INSERT INTO Orders (ProductID, Quantity)
 VALUES (42, 1);
COMMIT TRANSACTION;
```

```
UPDATE Products
 SET StockCount = StockCount - 1
 WHERE ProductID = 42;
INSERT INTO Orders (ProductID, Quantity)
 VALUES (42, 1);
COMMIT TRANSACTION;
```

-- StockCount is now -1. Two orders placed for one item.

Both transactions read a stock count of 1 and both decided to proceed. The isolation level determines whether Transaction B sees the original value or waits for Transaction A to finish first. If the isolation level allows dirty reads, Transaction B reads the uncommitted value of 1 and proceeds, which leads to overselling. If the isolation level is more restrictive, Transaction B waits until Transaction A commits. If Transaction A commits successfully, Transaction B sees the updated stock count of 0 and can prevent the second order from going through. If Transaction A rolls back for some reason, Transaction B sees the original value of 1 and can still proceed without ever reading uncommitted data.

SQL Server and Azure SQL Database support six isolation levels. The first four use pessimistic concurrency (locking). The last two use optimistic concurrency (row versioning).

Lock-based isolation levels

With lock-based isolation, the database engine uses shared and exclusive locks to keep transactions from interfering with each other. The more restrictive the level, the longer those locks are held. Fewer side effects get through, but more transactions end up waiting.

READ UNCOMMITTED is the fastest and the riskiest isolation level. It skips shared locks on reads entirely, so there's no read-write blocking. But that also

means a transaction can read another transaction's uncommitted changes. If that other transaction rolls back, your application just acted on data that never existed. This level makes sense only when approximate results are acceptable, like a dashboard showing rough order counts.

READ COMMITTED is the default isolation level in SQL Server and strikes a basic balance. Each read acquires a shared lock, grabs the data, and releases the lock right away. Releasing the lock quickly is enough to prevent dirty reads. However, nothing stops another transaction from changing the row between your two reads, so nonrepeatable reads and phantom reads can still happen.

REPEATABLE READ goes a step further than READ COMMITTED. It holds shared locks on every row you read until your transaction completes. Now those rows can't change while the transaction is active, which prevents both dirty reads and nonrepeatable reads. But other transactions can still insert new rows that match your query filter, so phantom reads remain a possibility.

SERIALIZABLE takes care of everything previous levels do and more. It takes range locks that cover not just the rows you read but also the gaps between key values, blocking inserts into those ranges. Dirty reads, nonrepeatable reads, and phantom reads are all prevented. The cost is real, though: range locks cause significantly more blocking and increase the chance of deadlocks. Reserve this level for scenarios like financial reconciliation or inventory reservation where phantom reads would cause calculation errors.

Row-versioning isolation levels

What if readers didn't have to wait for writers at all? That question is what row-versioning isolation addresses. Instead of blocking behind locks, the database engine keeps previous versions of rows in a version store. When a transaction needs to read data that another transaction is modifying, it reads from the version store instead of waiting. In Azure SQL Database, accelerated database recovery (ADR) is always enabled, so the version store resides in the database itself using the persistent version store (PVS).

Read Committed Snapshot Isolation changes the behavior of READ COMMITTED at the database level. With Read Committed Snapshot Isolation (RCSI) enabled, each read operation sees a snapshot of the data as it existed at the start of that *statement*. Writers still take locks on the rows they modify, but readers never block behind them. The important detail: RCSI is enabled by default in Azure SQL Database, so you get this behavior out of the box without any code changes.

SNAPSHOT isolation takes this solution a step further. Instead of a per-statement snapshot, each read sees the data as it existed at the start of the

entire *transaction*. If your application needs to run multiple queries within a single transaction and get consistent results across all of them, SNAPSHOT isolation provides that guarantee. To use it, you must enable ALLOW_SNAPSHOT_ISOLATION on the database and set the isolation level explicitly in the session. One thing to watch for: if two transactions try to modify the same row, SNAPSHOT isolation raises an update conflict rather than letting one silently overwrite the other.

The following table summarizes the behavior of each isolation level:

Isolation level	Dirty reads	Nonrepeatable reads	Phantom reads	Blocking between re writers
READ UNCOMMITTED	Yes	Yes	Yes	No
READ COMMITTED	No	Yes	Yes	Yes
REPEATABLE READ	No	No	Yes	Yes
SERIALIZABLE	No	No	No	Yes (range locks)
READ COMMITTED SNAPSHOT	No	Yes	Yes	No
SNAPSHOT	No	No	No	No (update conflicts

Reduce blocking with optimized locking

RCSI eliminates read-write blocking. But what about write-write blocking? Traditional locking holds row and page locks for every modified row until the transaction commits. Under heavy write concurrency, those held locks add up fast.

Azure SQL Database addresses this issue with optimized locking, which is always enabled and works alongside RCSI. It uses two mechanisms:

- **Transaction ID (TID) locking: Instead of holding individual key or row locks for every modified row, the engine takes a single exclusive lock on the transaction ID (TID). Other transactions that need to access the same row acquire a shared lock on the TID and wait for the modifying transaction to complete. The result is far fewer locks held in memory, which also makes lock escalation much less likely.**
- **Lock after qualification: Before a row is modified, the engine reads the latest committed version without acquiring a lock and checks whether the row matches the query predicate. Only rows that actually qualify get locked, and that exclusive lock is released as soon as the row**

update is complete, not at the end of the transaction. Lock after qualification (LAQ) requires RCSI to be enabled.

The practical effect is significant. Page and row locks for modifications are released as soon as each row is updated rather than held until the transaction commits. Concurrent writes that touch different rows rarely block each other, even under peak load.

Choose the right isolation level

With six isolation levels available, how do you decide? Start with the defaults. For most Azure SQL Database workloads, the combination of RCSI and optimized locking already provides the best balance of consistency and performance. Readers don't block writers. Writers don't block readers. Lock memory stays low. You don't need to change anything.

Step up to SNAPSHOT isolation when your application runs multiple queries within a single transaction and needs consistent results across all of them. Think of a financial report that queries account balances and then totals them: you need both queries to see the same point-in-time data.

Reserve SERIALIZABLE for scenarios where phantom reads would cause real business errors, like double-booking inventory or miscalculating a reconciliation. Accept that throughput drops and deadlocks become more likely.

Important

Regardless of which level you choose, keep transactions as short as possible. Long-running transactions hold locks for extended periods, increase blocking, and consume version store space. A transaction that stays open while waiting for user input or an external API call can bring concurrency to a halt.

Key takeaways

Transaction isolation levels control the trade-off between data consistency and concurrency. Higher isolation prevents more side effects but increases blocking. Azure SQL Database enables RCSI by default, which eliminates read-write blocking by using row versioning instead of shared locks. Optimized locking further reduces blocking by releasing row and page locks as soon as each row is updated. For most workloads, the default combination of RCSI and optimized locking provides the best balance. Reserve SERIALIZABLE for scenarios where phantom reads would cause business logic errors.

Next unit: Evaluate query performance with execution plans and DMVs

Evaluate query performance with execution plans and DMVs

Completed100 XP

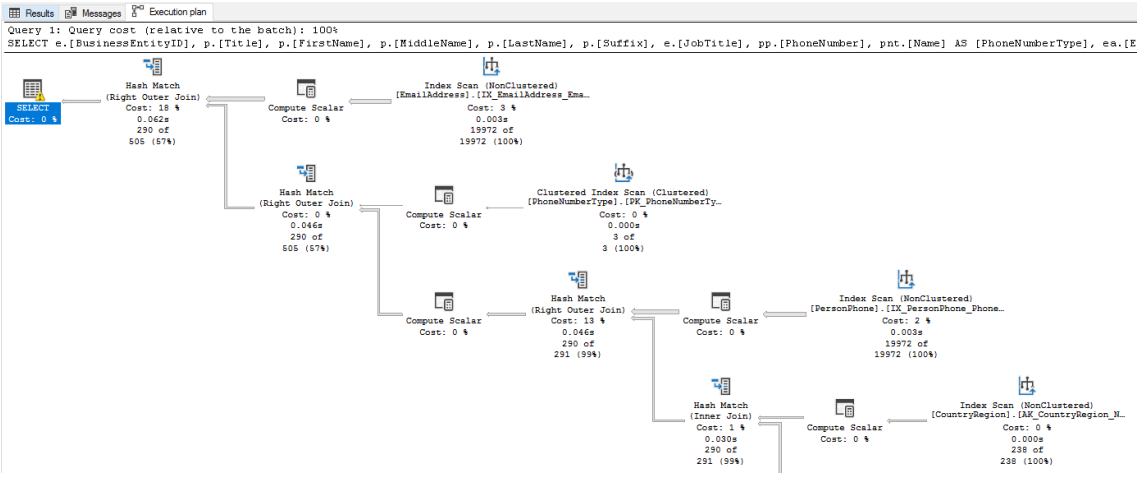
- **10 minutes**

Choose your preferred content format

When a query runs slower than expected, the first step is understanding *how* the database engine executes it. Execution plans show you the exact operators, data access methods, and resource costs the optimizer chose for a query. Dynamic management views (DMVs) complement that by exposing runtime performance data across all queries in the database, so you can find the most expensive ones before diving into any single plan.

Read execution plans

An execution plan is the set of instructions the query optimizer produces to retrieve and process data. It defines which tables to access first, whether to use indexes or scan tables, and how to join, filter, sort, and aggregate results. The optimizer evaluates multiple candidate plans and selects the one with the lowest estimated cost.



There are two types of execution plans:

- **Estimated execution plan: Generated without running the query. It shows the planned operators and estimated row counts based on statistics. Use estimated plans for quick analysis without affecting the database.**
- **Actual execution plan: Captured during query execution. It includes the estimated plan plus real row counts, actual execution times, memory grants, and warnings. The actual plan reveals discrepancies between what the optimizer expected and what actually happened.**

To display an estimated plan, run SET SHOWPLAN_XML ON before the query or select Display Estimated Execution Plan in SQL Server Management Studio (SSMS). To capture an actual plan, run SET STATISTICS XML ON or select Include Actual Execution Plan in SSMS before executing the query.

Even though the estimated and actual plans look similar, the actual plan's runtime metrics are crucial for diagnosing performance issues. For example, if the estimated row count for a table scan is 100 but the actual row count is 10,000, that could indicate outdated statistics leading to a bad plan choice. The optimizer compiles the plan based on statistics the first time it encounters a query. If those statistics don't reflect the current data distribution, the plan may perform poorly.

Identify common issues in execution plans

Execution plans are read left to right, top to bottom. The first operators access the base tables, and the final operator produces the query result. Look for these common issues:

Operator types tell you how the engine accesses data. There are many operator types, each representing a different method of retrieving or processing data. For example, an Index Seek operator represents a highly efficient method that targets specific rows using index keys. A Table Scan or Index Scan operator on the other hand, represents a less efficient method that reads every row. If you see a scan on a large table, you likely need an index. For example, if the e-commerce application queries orders by date and the plan shows a Clustered Index Scan on the Orders table, adding a nonclustered index on the OrderDate column could change that scan into a seek. Do note that not all scans are bad. If a table is small, or the search condition returns most rows in a table, a scan might be the most efficient access method. Always consider the context of the query and the size of the data. Know your data and use execution plans to confirm whether the access method makes sense.

Estimated versus actual row counts reveal whether the optimizer's assumptions match reality. The optimizer bases its plan on statistics, metadata that describes the distribution and density of data in your tables. If those statistics are stale, the estimated and actual row counts diverge. When the optimizer underestimates row counts, it might choose a nested loop join (which processes one row at a time from the inner table of a join) when a hash join (which builds a hash table in memory for fast lookups) would be faster, or allocate too little memory for a sort operation. Statistics can become stale after significant data changes, so updating statistics with UPDATE STATISTICS

or enabling automatic statistics updates can help the optimizer make better decisions.

Key Lookup operators appear when the engine finds rows through a nonclustered index but needs extra columns from the clustered index. For every matching row, the engine performs an extra round trip to the clustered index to retrieve those columns. If the filter returns many rows, those extra lookups add up fast. For example, if the e-commerce application filters orders by CustomerID but also selects OrderDate, TotalAmount, and ShippingAddress, and the nonclustered index on CustomerID doesn't include those columns, the plan shows a Key Lookup for each matching order. You can eliminate Key Lookups by adding the missing columns as included columns in the index. Keep in mind that included columns increase index size, which can slow down writes, so weigh the read performance benefit against the write overhead.

Thick arrows between operators represent the number of rows flowing between them. An unexpectedly thick arrow early in the plan (reading from left to right, top to bottom) often means a missing filter or index is letting too many rows through.

Missing index suggestions appear as green highlighted text at the top of the graphical execution plan in SSMS. When the optimizer detects that an index could significantly reduce the cost of a query, it surfaces a recommendation directly in the plan. Right-click the suggestion and select Missing Index Details to generate a CREATE INDEX statement you can review and run. These suggestions are one of the easiest wins you can get from reading an execution plan.

Warnings appear as a yellow triangle with an exclamation mark (⚠) on the affected operator. Each warning points to an optimization opportunity. Common warnings include:

- **Missing statistics: The optimizer couldn't find statistics for a column, so it guessed at row counts instead of using actual data distribution. To fix this issue, create statistics on the columns used in your queries or update existing statistics if they're stale.**
- **Excessive memory grant: The query requested more memory than it needed, wasting resources that other queries could use. This issue often happens when the optimizer overestimates row counts. Updating statistics or rewriting the query to filter rows earlier can help reduce memory grants.**

- **No Join Predicate: Two tables are joined without a proper condition, producing a Cartesian product that returns every possible row combination. Check your query for a missing or incorrect ON clause.**
- **Implicit conversion: A data type mismatch forces the engine to convert values at runtime, which can turn an index seek into a scan. For example, if a WHERE clause compares an nvarchar parameter to a varchar column, the engine converts every row in the column to nvarchar before comparing. To fix implicit conversions, match the data types in your query parameters to the column definitions.**
- **Sort or Hash spill: A sort or hash operation ran out of its granted memory and spilled intermediate results to tempdb. These operations are the second most common driver of high CPU after scans. If you see a spill warning, the optimizer likely underestimated row counts and requested too little memory. Running UPDATE STATISTICS to refresh the table's statistics or rewriting the query to reduce the number of rows before the sort can often eliminate the spill.**

Execution plans are a powerful tool for understanding query performance. They show you exactly how the engine executes a query and where the bottlenecks are. By learning to read execution plans effectively, you can quickly identify and fix performance issues in your database queries.

Query DMVs for runtime performance data

DMVs expose real-time and accumulated performance data from the database engine. Azure SQL Database requires VIEW DATABASE STATE permission to query them. While execution plans show you how a single query runs, DMVs show you what's happening across all queries, which helps you find the most expensive ones first.

Find the most expensive queries

CPU time, logical reads, and execution count are the most common metrics to identify expensive queries. High CPU time or logical reads indicate a query is resource-intensive, while a high execution count means even a moderately expensive query can have a large impact on overall performance. Start by reviewing the top queries by average CPU time or logical reads to find candidates for optimization.

sys.dm_exec_query_stats returns aggregate performance statistics for cached query plans. Join it with sys.dm_exec_sql_text to see the query text and sys.dm_exec_query_plan to retrieve the execution plan.

The following query finds the top 10 queries by average CPU time:

SQL

```
SELECT TOP 10
  qs.total_worker_time / qs.execution_count AS avg_cpu_time,
  qs.execution_count,
  qs.total_logical_reads / qs.execution_count AS avg_logical_reads,
  SUBSTRING(st.text, (qs.statement_start_offset / 2) + 1,
    ((CASE qs.statement_end_offset
      WHEN -1 THEN DATALENGTH(st.text)
      ELSE qs.statement_end_offset
    END - qs.statement_start_offset) / 2) + 1) AS query_text
FROM sys.dm_exec_query_stats AS qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) AS st
ORDER BY avg_cpu_time DESC;
```

This script helps you identify which queries deserve your attention. High avg_logical_reads relative to the result set size often points to missing indexes or inefficient plans. However, be cautious when interpreting these results. A query with high average CPU time that only runs once a day might matter less than a moderate query that runs thousands of times per hour. Always consider both the average cost and the execution count when you prioritize. You can also order by avg_logical_reads to find queries that are heavy on I/O, which often indicates missing indexes or inefficient access methods.

Check currently executing queries

While the previous query shows you the most expensive historic queries in the plan cache, sys.dm_exec_requests gives you a snapshot of every request that is currently running. It includes columns for CPU time, reads, writes, wait type, wait time, and blocking session ID. Use this view to spot active queries that are consuming too many resources or stuck waiting on locks. This view is one of the most important DMVs for real-time performance monitoring and troubleshooting.

SQL

```
SELECT
  r.session_id,
```

```
    r.status,
    r.command,
    r.wait_type,
    r.wait_time,
    r.blocking_session_id,
    r.cpu_time,
    r.logical_reads,
    t.text AS query_text
FROM sys.dm_exec_requests AS r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) AS t
WHERE r.session_id > 50
ORDER BY r.cpu_time DESC;
```

This query filters out system sessions (session IDs 1-50) and orders by CPU time. You can also order by logical_reads to find I/O-heavy queries. The wait_type and wait_time columns help you identify whether a query is waiting on locks, I/O, or other resources.

Discover missing indexes

Earlier, we saw how execution plans can show you missing index suggestions for a single query. The missing index DMVs give you a broader view of which indexes the optimizer would use across all queries if they existed. These views are a great way to find optimization opportunities that affect multiple queries. sys.dm_db_missing_index_details shows the table, equality and inequality columns, and included columns. sys.dm_db_missing_index_group_stats provides an improvement measure that estimates the cost reduction.

SQL

```
SELECT
    mid.statement AS table_name,
    mid.equality_columns,
    mid.inequality_columns,
    mid.included_columns,
    migs.avg_total_user_cost * migs.avg_user_impact *
```

```
    (migs.user_seeks + migs.user_scans) AS improvement_measure
FROM sys.dm_db_missing_index_groups AS mig
INNER JOIN sys.dm_db_missing_index_group_stats AS migs
  ON migs.group_handle = mig.index_group_handle
INNER JOIN sys.dm_db_missing_index_details AS mid
  ON mig.index_handle = mid.index_handle
ORDER BY improvement_measure DESC;
```

This query calculates an improvement_measure for each missing index recommendation, which is a product of the average cost of queries that would benefit from the index, the average percentage improvement, and the number of times those queries were executed. Sorting by this measure helps you prioritize which missing indexes to create first. However, remember that these results are just recommendations based on the queries currently in the plan cache. Always review the suggested index columns and test their impact on both query performance and write overhead before adding them to production.

Note

Missing index recommendations are suggestions, not directives. Always test the impact of a new index on both query performance and write overhead before adding it to production.

Monitor active sessions and waiting tasks

sys.dm_exec_sessions gives you information about all authenticated sessions, including sign in time, host name, program name, and cumulative CPU and reads. Combine it with sys.dm_os_waiting_tasks to see which tasks are waiting and what resources they're waiting on. These views become essential when you diagnose blocking and resource contention in a later unit.

Put it all together

Execution plans and DMVs give you a complete picture of query behavior. Start with DMVs to identify the most expensive queries. Then drill into their execution plans to understand *why* they're expensive. Is it a missing index causing a scan? Outdated statistics causing row estimate errors? A Key Lookup you can eliminate? This systematic approach, from system-wide view to individual query analysis, is the most efficient way to find and fix performance bottlenecks.

Key takeaways

Execution plans reveal the optimizer's strategy for a query, and actual plans include runtime metrics that expose discrepancies between estimated and actual row counts. When you read a plan, focus on operator types (seek vs. scan), row count estimates, warnings, and Key Lookup operators. DMVs provide system-wide performance data: use sys.dm_exec_query_stats to find the most expensive queries, sys.dm_exec_requests for currently running queries, and the missing index DMVs for optimization opportunities. Start broad with DMVs to identify where the biggest problems are, then drill into individual execution plans to understand why.

Next unit: Monitor and tune queries with query store and query performance insight

Monitor and tune queries with query store and query performance insight

Completed100 XP

- **10 minutes**

Choose your preferred content format

Real-time monitoring tells you what's happening now, but what about yesterday? Last week? After a deployment? You need a historical perspective to diagnose performance regressions, because you can only understand what changed by comparing current behavior against a baseline. Query Store gives you exactly this capability.

Understand the query store architecture

Query Store captures query text, execution plans, and runtime statistics directly inside the database engine. It stores this data in three internal stores:

- **Plan store: Persists execution plans for each query. A single query can have multiple plans over time due to statistics updates, schema changes, or index modifications.**
- **Runtime stats store: Records aggregate execution statistics (CPU time, duration, logical reads, row counts) for each plan, grouped into configurable time intervals.**
- **Wait stats store: Captures wait statistics per query and plan, categorized by wait type. This links query performance directly to the resources the query waits on.**

Query Store is enabled by default in Azure SQL Database. Data is written asynchronously to minimize overhead on the production workload. You can configure the retention period, maximum storage size, and capture mode using ALTER DATABASE SET QUERY_STORE options.

Consider a common scenario: your team deploys a code update on Tuesday. By Thursday, users report that a key page is slow. Without Query Store, you'd need to reproduce the issue and compare current execution plans against plans you that were never saved. With Query Store, you open the Regressed Queries view and immediately see which queries switched to slower plans after Tuesday's deployment.

Detect regressed queries

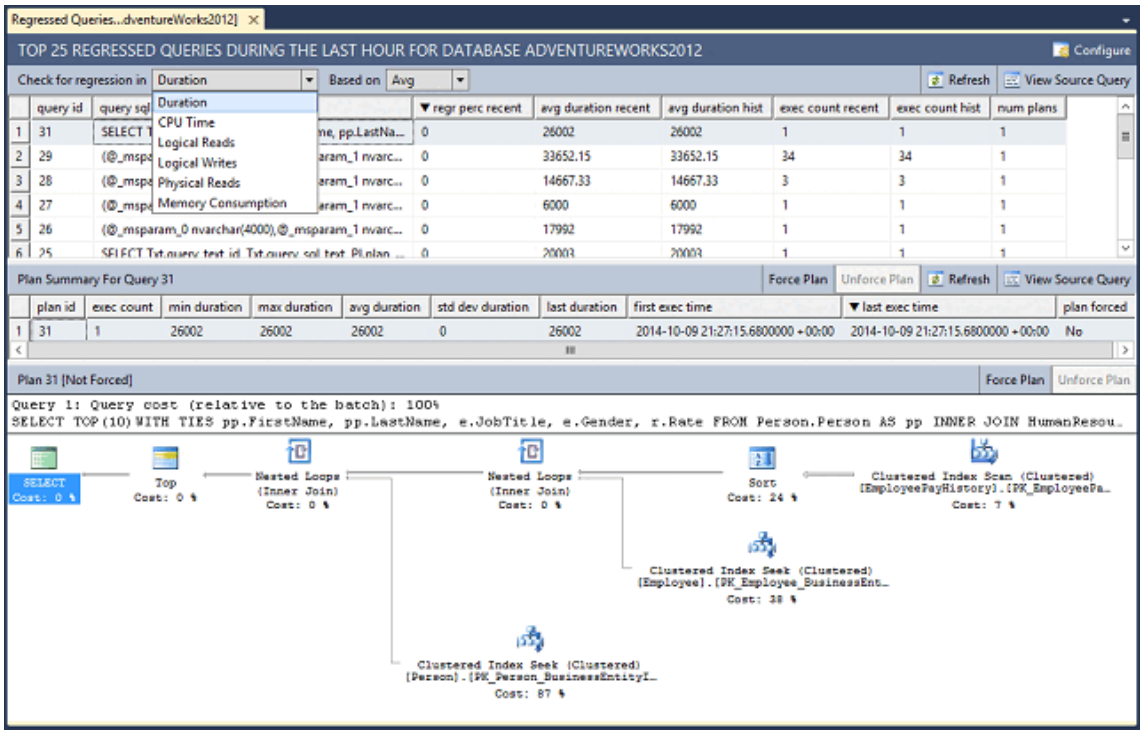
One of the most valuable capabilities of Query Store is detecting queries whose performance degraded after a plan change. The Regressed Queries view in SQL Server Management Studio (SSMS) surfaces queries that recently switched to a slower execution plan.

To use this view:

1. **In SSMS, expand the database node in Object Explorer.**
2. **Expand the Query Store folder.**
3. **Select Regressed Queries.**

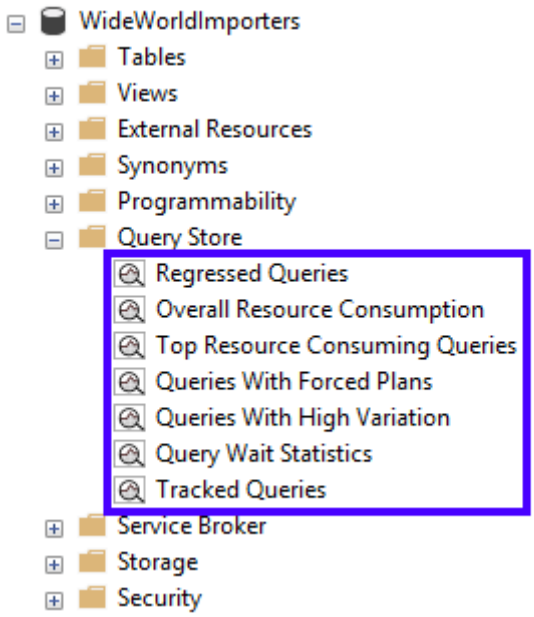
This view displays queries where runtime metrics are worse than in a previous time interval. Use the dropdown lists at the top to filter by metric (CPU Time, Duration, Logical Reads, Memory Consumption, Execution Count, and more), time range, and aggregation method (Average, Maximum, Total).

Selecting a query shows its plan history as a set of points in a chart. Each point represents a different execution plan. Select two plans to compare them side by side. This comparison is where the real investigation happens. Look for operators that changed between the fast plan and the slow plan. Common patterns include a new plan switching from an index seek to a scan, losing a key lookup that was previously efficient, or introducing a sort operator that spills to disk.



The Query Store folder in Object Explorer contains several other views beyond Regressed Queries:

- **Top Resource Consuming Queries: Shows the queries with the highest resource usage for a chosen metric and time range. This report is the most common starting point for day-to-day performance tuning.**
- **Queries With High Variation: Surfaces queries whose performance fluctuates significantly, which often indicates parameter sensitivity or varying data distributions.**
- **Queries With Forced Plans: Lists all currently forced plans so you can review and manage them.**
- **Query Wait Statistics: Groups *wait statistics* by category and show which queries contribute to each wait type.**



Query the Query Store with T-SQL

You can also query Query Store data directly using its catalog views. The following query finds the top 10 queries by average duration in the last hour:

SQL

```
SELECT TOP 10
  qt.query_sql_text,
  q.query_id,
  p.plan_id,
  ROUND(CONVERT(FLOAT, SUM(rs.avg_duration * rs.count_executions))
    / NULLIF(SUM(rs.count_executions), 0), 2) AS avg_duration,
  SUM(rs.count_executions) AS total_executions
FROM sys.query_store_query_text AS qt
INNER JOIN sys.query_store_query AS q
  ON qt.query_text_id = q.query_text_id
INNER JOIN sys.query_store_plan AS p
  ON q.query_id = p.query_id
INNER JOIN sys.query_store_runtime_stats AS rs
  ON p.plan_id = rs.plan_id
WHERE rs.last_execution_time > DATEADD(HOUR, -1, GETUTCDATE())
```

GROUP BY qt.query_sql_text, q.query_id, p.plan_id

ORDER BY avg_duration DESC;

This query joins four catalog views: sys.query_store_query_text (query text), sys.query_store_query (query metadata), sys.query_store_plan (execution plans), and sys.query_store_runtime_stats (runtime statistics). You can adapt it to find queries by CPU time, logical reads, or execution count by changing the metric in the SELECT and ORDER BY clauses.

Force a plan

When you identify that a previous plan was better, you can tell the optimizer to use that specific plan for future executions. This approach gives you an immediate fix without modifying the query or adding hints to the application code.

To force a plan:

SQL

EXEC sp_query_store_force_plan

@query_id = 42,

@plan_id = 17;

To unforce a plan, and let the optimizer choose freely again:

SQL

EXEC sp_query_store_unforce_plan

@query_id = 42,

@plan_id = 17;

Plan forcing is useful during deployments. If a statistics update or schema change causes a plan regression, you can force the previously working plan immediately while you investigate the root cause. Think of it as a performance rollback that doesn't require an application rollback.

Apply Query Store hints

Sometimes you need to shape query execution without modifying application code. Query Store hints let you attach a query hint to a specific query through Query Store, and the optimizer applies it during compilation.

For example, to limit a query to a single thread:

SQL

EXEC sp_query_store_set_hints

@query_id = 42,

@query_hints = N'OPTION (MAXDOP 1)';

You can also force a recompile on every execution, which is useful for queries with parameter sensitivity issues:

SQL

EXEC sp_query_store_set_hints

@query_id = 42,

@query_hints = N'OPTION (RECOMPILE)';

You can even combine multiple hints in a single call:

SQL

EXEC sp_query_store_set_hints

@query_id = 42,

@query_hints = N'OPTION (MAXDOP 1, MAX_GRANT_PERCENT = 10)';

To remove a hint:

SQL

EXEC sp_query_store_clear_hints @query_id = 42;

Query Store hints override hard-coded statement-level hints and existing plan guides, so they give you full control without touching the application code.

Analyze wait statistics per query

Query Store captures wait statistics per query, not just at the server level. Enable wait stats capture with:

SQL

ALTER DATABASE CURRENT

SET QUERY_STORE (WAIT_STATS_CAPTURE_MODE = ON);

The Query Wait Statistics view in SSMS groups waits into categories such as CPU, Lock, Buffer IO, and Memory. Selecting a wait category reveals which queries contribute the most to that wait type.

This connection between waits and queries is a significant advantage over server-level wait statistics, which can't attribute waits to specific queries. For

example, if you see high Lock waits, you can drill into the specific queries causing them. You can then examine their execution plans, check their isolation levels, or evaluate whether missing indexes are causing long-held locks.

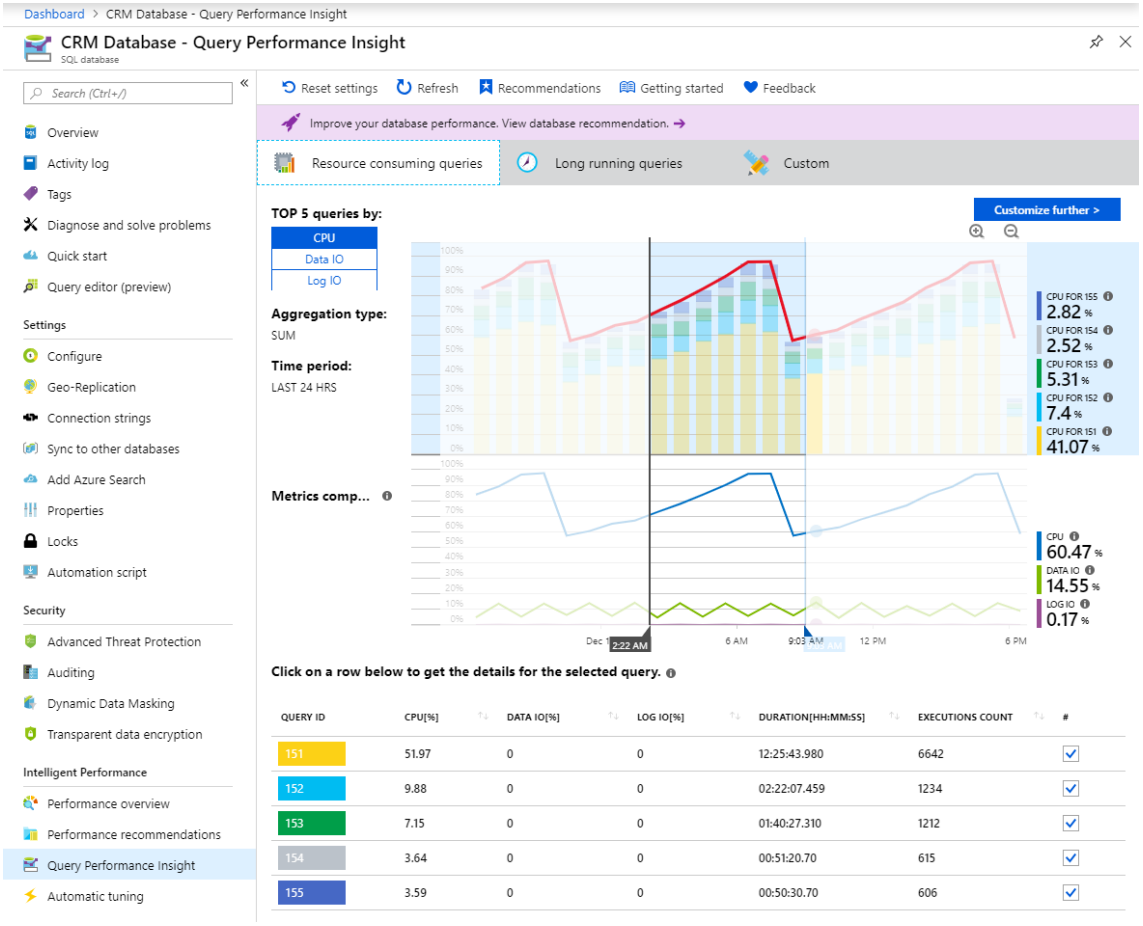
Monitor with query performance insight

Query Performance Insight (QPI) is an Azure portal feature that visualizes Query Store data. It gives you a graphical view of the top resource-consuming queries without requiring SSMS.

To access QPI:

1. **Open your database in the Azure portal.**
2. **In the left menu, select Intelligent Performance > Query Performance Insight.**

By default, QPI shows the top five CPU-consuming queries. The top line in the chart represents overall Database Transaction Unit (DTU) percentage for the database. The following bars show CPU percentage consumed by each selected query during the selected interval. This layout lets you see at a glance whether a single query dominates resource usage or whether the load is spread across many queries.



Select the Custom tab to change the view. You can switch the metric to Duration or Execution count, adjust the time interval (last 24 hours, past week, or past month), change the number of queries displayed (5, 10, or 20), and choose an aggregation function (Sum, Max, Min, or Avg). This flexibility lets you investigate different dimensions of the same workload without leaving the portal.

Selecting an individual query opens a detail view with three charts: CPU consumption over time, duration over time, and execution count over time. This drill-down helps you distinguish between a query that's slow on every execution and one that's fast most of the time but spikes occasionally.

QPI also integrates with Database Advisor recommendations. Performance annotations appear on the chart as icons with a vertical line. Hover over an annotation to see a summary of the recommendation, such as creating an index or parameterizing queries. Select the icon to open the full recommendation detail and apply it directly from the portal.

Note

Query Performance Insight is limited to displaying the top 5 to 20 queries. If your workload includes many smaller queries that collectively consume

significant resources, they don't appear in QPI. For deeper analysis, query the Query Store catalog views directly or use SSMS. Additionally, also note that Query Performance Insight requires Query Store to be active. If Query Store runs out of space and enters read-only mode, QPI can't display new data. Increase the Query Store maximum size or clear old data to restore normal operation.

Follow best practices

The default capture mode, Auto, is the right choice for most workloads. It filters out infrequent queries and queries with negligible resource consumption, which keeps storage usage manageable. Pair this setting with size-based cleanup set to Auto, so the database automatically removes the oldest data when Query Store approaches its maximum size.

Build a habit of checking the Regressed Queries view after every deployment, statistics update, or index change. Plan regressions often appear within hours and are easier to fix when you catch them early. When you do find a regression, use plan forcing as a short-term fix while you investigate the root cause. Long-term, address the underlying issue through index tuning, statistics updates, or query rewrites.

The most common operational problem with Query Store is running out of space. When that happens, Query Store silently switches to read-only mode and stops collecting new data. You can detect this issue by checking sys.database_query_store_options:

SQL

```
SELECT actual_state_desc, desired_state_desc,
  current_storage_size_mb, max_storage_size_mb,
  readonly_reason
FROM sys.database_query_store_options;
```

If actual_state_desc shows READ_ONLY while desired_state_desc shows READ_WRITE, Query Store switched itself. The readonly_reason column tells you why. To restore normal operation, increase the maximum storage size or clear old data:

SQL

```
ALTER DATABASE CURRENT
SET QUERY_STORE (MAX_STORAGE_SIZE_MB = 1024);
```

Key takeaways

Query Store captures query text, execution plans, and runtime statistics over time, and it's enabled by default in Azure SQL Database. The Regressed Queries view surfaces queries whose performance degraded after a plan change, making it essential to check after every deployment or schema change. When you identify a regression, plan forcing gives you an immediate fix without modifying application code, while Query Store hints let you attach query-level hints to shape execution behavior. For teams working in the Azure portal rather than SSMS, Query Performance Insight provides a graphical view of the same Query Store data.

Next unit: Identify and resolve blocking and deadlocks

Identify and resolve blocking and deadlocks

Completed100 XP

- **10 minutes**

Choose your preferred content format

Blocking is normal in a database that uses locking. One transaction holds a lock. Another transaction waits. Brief blocking that lasts a few milliseconds is expected. It becomes a problem when it lasts long enough to affect users. Deadlocks are the more severe form: two transactions permanently block each other, and the database engine has to terminate one to break the cycle.

Blocking

Blocking occurs when one session holds a lock on a resource and another session requests a conflicting lock on the same resource. The requesting session waits until the first session releases its lock.

Resources can be rows, pages, or even entire tables. Locks can be shared (for reads) or exclusive (for writes). When a session requests a lock that conflicts with an existing lock, it becomes blocked until the first session commits or rolls back its transaction and releases its locks.

Identify blocking chains

In Azure SQL Database, Read Committed Snapshot Isolation (RCSI) is enabled by default, so read operations use row versioning instead of shared locks. This setting significantly reduces blocking between readers and writers. However, blocking between two writers, or blocking caused by explicit transactions with higher isolation levels, still happens.

The session at the top of a blocking chain is called the head blocker. All other blocked sessions are waiting, directly or indirectly, for the head blocker to release its locks. To find the head blocker, query sys.dm_exec_requests and look for sessions where blocking_session_id is nonzero:

SQL

```
SELECT
  r.session_id,
  r.blocking_session_id,
  r.wait_type,
  r.wait_time,
  r.wait_resource,
  t.text AS query_text,
  r.status,
  r.command
FROM sys.dm_exec_requests AS r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) AS t
WHERE r.blocking_session_id <> 0
ORDER BY r.wait_time DESC;
```

To find the head blocker in those results, look for the session ID that blocks others but isn't blocked itself. For example, suppose the query returns these rows:

session_id	blocking_session_id
55	52
60	52
52	0

Sessions 55 and 60 are both blocked by session 52, and session 52 has a blocking_session_id of 0, which means nothing is blocking it. Session 52 is the head blocker. Once you identify it, query sys.dm_exec_sessions and sys.dm_exec_requests filtered to that session ID to see what it's running and why it's holding locks.

Keep in mind that RCSI eliminates blocking between readers and writers, but not between two writers. Consider a scenario where session 52 runs a batch update inside an explicit transaction:

SQL

```
-- Session 52
BEGIN TRANSACTION;
UPDATE Orders SET Status = 'Processing' WHERE Region = 'West';
-- Transaction stays open while application does other work
```

This update acquires exclusive locks on every matching row. Now session 55 tries to update one of those same rows:

SQL

```
-- Session 55
UPDATE Orders SET Priority = 1 WHERE OrderId = 4820;
```

Session 55 waits because session 52 already holds an exclusive lock on that row and hasn't committed. A SELECT query against those rows would still succeed under RCSI because it reads a row version instead of requesting a shared lock. With RCSI removing reader-writer blocking by default, the blocking you encounter in Azure SQL Database typically involves two sessions that both need to write to the same rows.

Recognize common blocking scenarios

Understanding *why* blocking happens helps you prevent it. The Azure SQL Database engine automatically manages locks, but certain patterns lead to longer blocking:

A long-running query that holds locks for an extended period. The query is actively executing and making progress, but it holds locks the entire time. Other sessions that need conflicting locks on the same resources wait until the query finishes. To resolve this issue, look for ways to optimize the query, such as adding indexes or rewriting it to touch fewer rows.

A sleeping session with an uncommitted transaction. A session executes a statement within an explicit transaction, then stops executing but never commits or rolls back. The locks from the transaction remain held indefinitely. This issue often happens when an application experiences a query timeout or cancellation but doesn't issue a corresponding ROLLBACK. Use SET XACT_ABORT ON so that runtime errors automatically roll back the transaction.

A session that didn't fetch all result rows. An application sends a query but doesn't retrieve all the rows from the result set. Locks can remain held on rows that haven't been fetched yet. Make sure your application fetches all result rows to completion.

A session in a rollback state. A query that was terminated (with KILL or by a deadlock) is rolling back its changes. Rollback can take significant time for large modifications, and the session continues to hold locks during this process. Wait for the rollback to complete, and avoid large batch modifications during busy periods.

An orphaned connection. A client application crashes or a workstation restarts without cleanly closing the database connection. The server doesn't immediately detect the disconnection, so locks from that session remain held. Terminate the orphaned session with KILL <session_id>;.

Note

Two of these scenarios are mitigated in Azure SQL Database by default. RCSI reduces the impact of unfetched result rows because SELECT queries don't acquire shared locks under row versioning, so unfetched rows don't block writers. Accelerated Database Recovery (ADR) makes lengthy rollbacks rare because it can undo changes almost instantaneously regardless of transaction size. The remaining three scenarios (long-running queries, sleeping sessions with uncommitted transactions, and orphaned connections) remain fully relevant because they involve exclusive write locks that RCSI and ADR can't release early.

Resolve active blocking

When you find active blocking:

1. **Identify the head blocker using the DMV query shown earlier.**
2. **Determine whether the blocking session's transaction can finish on its own or whether it's waiting on external input.**

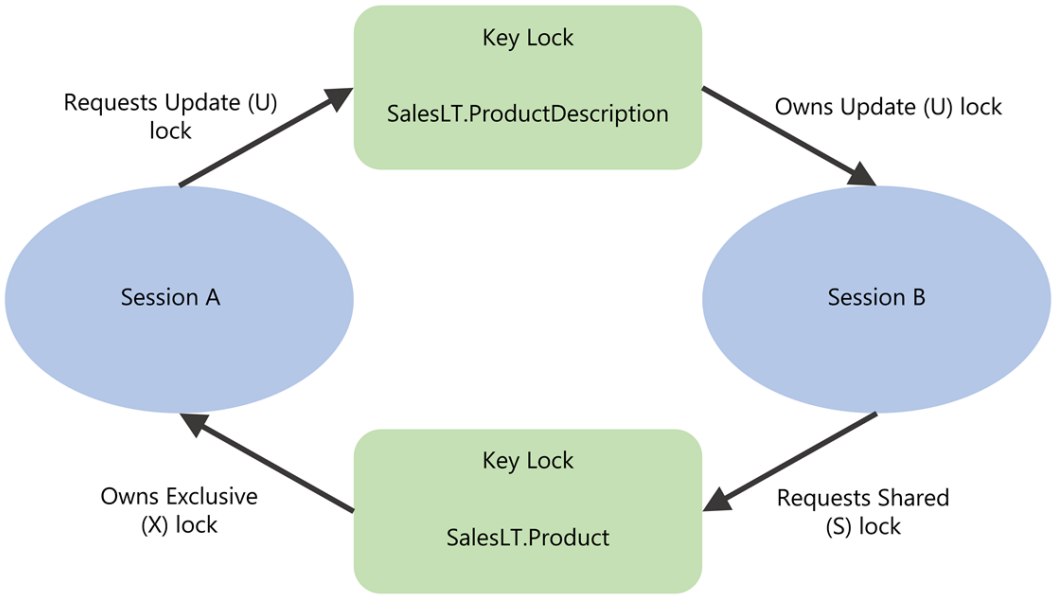
3. **If the blocking session is an orphaned or abandoned connection, terminate it with KILL <session_id>;**
4. **Review the blocking query's execution plan for optimization opportunities such as missing indexes.**

To prevent blocking from recurring, keep transactions short. Execute only the minimum required statements within a transaction and commit immediately. Use SET XACT_ABORT ON in your application code so that any runtime error automatically rolls back the entire transaction, which prevents half-completed transactions from holding locks indefinitely. Move all user-facing logic outside transaction boundaries.

Deadlocks

A deadlock occurs when two or more transactions form a circular dependency. Each transaction holds a lock that the other needs, and neither can proceed. Here's a concrete example:

1. **Transaction A updates row 1 and acquires an exclusive lock.**
2. **Transaction B updates row 2 and acquires an exclusive lock.**
3. **Transaction A tries to update row 2 and is blocked by Transaction B.**
4. **Transaction B tries to update row 1 and is blocked by Transaction A.**



Neither transaction can finish. The database engine's deadlock monitor periodically checks for these cycles, with a default interval of five seconds that drops to as low as 100 milliseconds when deadlocks are frequent. When it detects a cycle, it chooses the transaction that's least expensive to roll back

as the victim, rolls it back, and returns error 1205 to the application. This rollback allows the other transaction to complete.

Capture deadlock information

In SQL Server and Azure SQL Managed Instance, the system_health Extended Events session captures deadlock events by default. You can query the deadlock report from its ring buffer using sys.dm_xe_session_targets and sys.dm_xe_sessions.

In Azure SQL Database, the approach is different. You create a custom Extended Events session that captures the sqlserver.database_xml_deadlock_report event, and query it using the database-scoped DMVs sys.dm_xe_database_sessions and sys.dm_xe_database_session_targets. The following example creates a deadlock capture session and queries its ring buffer:

SQL

```
-- Create and start the session

CREATE EVENT SESSION [deadlocks] ON DATABASE

ADD EVENT sqlserver.database_xml_deadlock_report

ADD TARGET package0.ring_buffer

WITH (STARTUP_STATE = ON, MAX_MEMORY = 4 MB);

GO


ALTER EVENT SESSION [deadlocks] ON DATABASE STATE = START;

GO


-- Query deadlock events from the ring buffer

DECLARE @tracename sysname = N'deadlocks';


SELECT

  d.value('(/event/@timestamp)[1]', 'datetime2') AS deadlock_time,

  d.query('/event/data[@name=''xml_report'']/value/deadlock') AS
deadlock_xml
```

```
FROM (
  SELECT CAST(target_data AS XML) AS rb
  FROM sys.dm_xe_database_sessions AS s
  INNER JOIN sys.dm_xe_database_session_targets AS t
    ON CAST(t.event_session_address AS BINARY(8)) = CAST(s.address AS
BINARY(8))
  WHERE s.name = @tracename
    AND t.target_name = N'ring_buffer'
) AS ring_buffer
CROSS APPLY rb.nodes(
  '/RingBufferTarget/event[@name=''database_xml_deadlock_report'']'
) AS xevent(d)
ORDER BY deadlock_time DESC;
```

The deadlock graph includes three sections. The victim-list identifies which transaction was terminated. The process-list shows each process involved, including the query text, isolation level, and lock mode. The resource-list shows the locked resources and which process owns and waits on each one.

In Azure SQL Database, you can also configure deadlock alerts through the Azure portal to receive notifications when deadlocks occur.

Prevent deadlocks

You can't eliminate all deadlocks, but you can significantly reduce how often they occur:

- Access objects in a consistent order: If all transactions modify Table A before Table B, circular dependencies can't form. Standardize access patterns through stored procedures.
- Keep transactions short: Shorter transactions hold locks for less time, which reduces the window for circular dependencies.
- Use row-versioning isolation levels: RCSI eliminates shared locks for read operations, which removes one common source of deadlock cycles. Optimized locking in Azure SQL Database further reduces deadlock likelihood.

- **Add appropriate indexes: When queries scan many rows, they acquire locks across a wide range of data. Adding indexes that narrow the scan to fewer rows reduces lock conflicts.**
- **Use plan forcing with Query Store: If a plan change caused a query to scan more rows and acquire more locks, forcing the previous plan can reduce deadlocks while you investigate.**

Handle deadlocks in application code

Applications should always include retry logic for deadlock errors. When a transaction is chosen as the deadlock victim, the database engine rolls it back and returns error 1205. Your application should catch this error, pause briefly, and resubmit the transaction.

SQL

```
BEGIN TRY
  BEGIN TRANSACTION;
  -- Your data modification statements
  COMMIT TRANSACTION;
END TRY
BEGIN CATCH
  IF ERROR_NUMBER() = 1205
  BEGIN
    ROLLBACK TRANSACTION;
    WAITFOR DELAY '00:00:01';  -- Brief pause before retry
    -- Retry logic here
  END
  ELSE
  BEGIN
    ROLLBACK TRANSACTION;
    THROW;
  END
END CATCH;
```

Tip

Randomize the retry delay between attempts to prevent the same two transactions from deadlocking each other again immediately. A common pattern is to wait between one and three seconds with a random component.

Key takeaways

Blocking is normal, but extended blocking affects users, so you use sys.dm_exec_requests to find the head blocker and understand what it's doing. Common scenarios include long-running queries, sleeping sessions with uncommitted transactions, and orphaned connections, all of which you address by keeping transactions short, using SET XACT_ABORT ON, and ensuring applications properly manage connections and result sets. Deadlocks form when transactions create circular lock dependencies, and the database engine resolves them automatically by terminating the least expensive transaction and returning error 1205. You reduce deadlock frequency by accessing objects in a consistent order, keeping transactions short, using row-versioning isolation levels, and adding appropriate indexes. Your application code should always include retry logic for error 1205 so it can recover automatically when chosen as a deadlock victim.

Next unit: Exercise: Optimize query performance

Summary

Completed100 XP

- **3 minutes**

Performance optimization in Azure SQL Database is a systematic process. You start with infrastructure decisions, work through concurrency controls, and apply diagnostic tools to find and fix the problems that affect your users.

In this module, you learned how to:

- **Recommend database configurations: Evaluate vCore versus DTU resource models. Choose among General Purpose, Business Critical, and Hyperscale service tiers based on I/O latency, storage, and availability needs. Select provisioned or serverless compute to match workload patterns.**

- **Preserve data integrity with isolation levels: Understand the trade-off between consistency and concurrency across six isolation levels. Use RCSI and optimized locking (both enabled by default in Azure SQL Database) to minimize blocking.**
- **Evaluate query performance: Read execution plans to identify scans, row estimate errors, Key Lookups, and warnings. Query DMVs to find the most expensive queries, currently running requests, and missing indexes.**
- **Monitor and tune with Query Store: Force previous plans for immediate fixes. Apply Query Store hints without modifying application code. Visualize performance in the Azure portal with Query Performance Insight.**
- **Identify and resolve blocking and deadlocks: Find head blockers with sys.dm_exec_requests. Capture deadlock graphs through Extended Events. Prevent concurrency issues by keeping transactions short, accessing objects in consistent order, and implementing retry logic for error 1205.**

Implement CI/CD by using SQL database projects

Implement CI/CD for SQL Database Projects with source control, branching, schema drift detection, automated pipelines, and testing strategies using GitHub Actions and Azure DevOps.

Introduction

Choose your preferred content format

A manual deployment process for database changes is slow and error-prone. Someone writes a script, emails it to the database administrator, and waits for it to be run during a maintenance window. If something goes wrong, rolling back is hard because no one tracked the exact state of the schema before the change. Meanwhile, developers working on different features overwrite each other's changes because there's no structured workflow for database code.

Imagine a team managing an e-commerce application on Azure SQL Database. Three developers are adding features that require schema changes. One adds a new product category table, another modifies the order processing stored procedures, and a third updates the customer lookup views. Without source control, CI/CD pipelines, and automated testing, these changes collide. A

hotfix applied directly to production falls out of sync with the development environment. A deployment overwrites the hotfix, causing a production outage during peak hours.

SQL Database Projects and CI/CD pipelines solve these problems by treating database code with the same rigor as application code. You version-control every object, automate builds and deployments, detect when the live database drifts from the project, and test changes before they reach production.

After completing this module, you're able to:

- **Create, build, and validate database models by using SQL Database Projects, including SDK-style.**
- **Configure source control for SQL Database Projects and manage reference data with predeployment and post-deployment scripts.**
- **Manage branching, pull requests, and conflict resolution for database code.**
- **Detect schema drift by using schema comparison tools and SqlPackage.**
- **Implement CI/CD pipelines with GitHub Actions and Azure DevOps, including secrets management and deployment controls.**
- **Design and implement a testing strategy with unit tests and integration tests.**

Note

We recognize that different people like to learn in different ways. You can choose to complete this module in video-based format or you can read the content as text and images. The text contains greater detail than the videos, so in some cases you might want to refer to it as supplemental material to the video presentation.

Next unit: Create, build, and validate SQL database projects

Create, build, and validate SQL database projects

Completed100 XP

- **10 minutes**

Choose your preferred content format

Before you can automate deployments, run tests, or detect schema drift, you need something a pipeline can actually build. That something is a SQL database project.

Understand SQL database projects

Think of a SQL database project as your database in file form. Every table, stored procedure, view, and function lives in its own .sql file. You declare each object once, and when you need to add a column or change a data type, you edit that single file.

If you worked with migration scripts before, the difference is significant. Migration scripts are sequential. You write an ALTER statement, hope it runs in the right order, and accumulate a growing chain of changes over time. A SQL database project takes the opposite approach: you maintain the desired end state of each object, and the tooling calculates the difference between the project and the target database at deployment time.

When you build the project, the output is a .dacpac file, a compiled model of your entire database schema. You publish that .dacpac to create a new database or update an existing one. The build process gives you two things that matter for CI/CD:

- **Validation: object references and T-SQL syntax are checked against a specific version of SQL before anything gets deployed.**
- **A deployable artifact: the .dacpac becomes the single package that flows through your pipeline to every environment.**

Choose between original and SDK-style projects

SQL database projects come in two formats. The original format is built on MSBuild (.NET Framework) and ships with SQL Server Data Tools (SSDT) in Visual Studio. The SDK-style format is built on the Microsoft.Build.Sql project SDK and is the format used by the SQL Database Projects extension for Visual Studio Code.

For new projects, go with SDK-style. It brings capabilities the original format doesn't have:

- **.NET 8+ support, so you can build cross-platform on Windows, Linux, and macOS. This support matters when your CI runners aren't Windows machines.**

- **NuGet package references for database references, so dependency management follows the same patterns as the rest of the .NET ecosystem.**
- **Default globbing for .sql files. Drop a file in the project folder and it automatically is included in the build. No manual file entries needed.**

If you have an existing original project, you can convert it to SDK-style by modifying the .sqlproj file. Before you convert the project, back up the project file and archive a .dacpac from the current project. Compare a "before" and "after" .dacpac to confirm the conversion preserved everything.

Note

SDK-style SQL projects are generally available in Visual Studio Code and in preview in Visual Studio 2022. Visual Studio 2026 supports only the original SQL project format.

Create and populate a project

You can create a SQL database project from Visual Studio Code, Visual Studio, or the command line. The command line is especially handy for CI/CD scenarios where no graphical environment is available.

To create a new SDK-style project:

Bash

```
dotnet new sqlproj -n MyDatabaseProject
```

📝 This generates a .sqlproj file with the SDK-style format. From here, you add database objects by dropping .sql files into the project folder. The default globbing pattern picks them up automatically.

For example, to define a table, create Tables/Customers.sql:

SQL

```
CREATE TABLE [dbo].[Customers]
(
  [CustomerID] INT NOT NULL PRIMARY KEY,
  [FirstName] NVARCHAR(50) NOT NULL,
  [LastName] NVARCHAR(50) NOT NULL,
  [Email] NVARCHAR(100) NULL
);
```

Starting with an existing database? Schema comparison tools let you compare a live database against an empty project and import the differences, so you don't have to recreate every object by hand.

Build and validate the project

Building is where the safety net kicks in. The build process validates every object reference and checks T-SQL syntax against the target platform. If a view references a column that doesn't exist, the build fails. If you use a vector function added in SQL Server 2025 but your project targets SQL Server 2017 (Sql140), the build catches it.

To build from the command line:

Bash

```
dotnet build MyDatabaseProject.sqlproj
```

📝 The build produces a .dacpac file in the bin/Debug folder by default.

Build output includes errors (which block the build) and warnings (such as inconsistent casing in object names). Warnings don't stop the build, but they're worth paying attention to. You can also enable SQL code analysis rules to flag best practice violations during the build, catching issues like deprecated join syntax, SELECT * in views, or unindexed columns in IN predicates.

The target platform is set in the .sqlproj file and controls which T-SQL features pass validation. Set it to match your deployment target, whether that's SQL Server 2025 or Azure SQL Database.

Deploy the project

Once you have a .dacpac, SqlPackage handles the deployment. Install it as a .NET global tool:

Bash

```
dotnet tool install --global microsoft.sqlpackage
```

Then publish to a target database:

Bash

```
sqlpackage /Action:Publish
/SourceFile:bin/Debug/MyDatabaseProject.dacpac
/TargetConnectionString:"Server=myserver.database.windows.net;Database=
mydb;Authentication=Active Directory Default"
```

📤 **SqlPackage is cross-platform and runs on Windows, Linux, and macOS.**

What happens during deployment depends on the target. For a new database, SqlPackage navigates the object dependency graph and creates each object in the correct order, such as referenced tables before foreign keys. For an existing database, it calculates the diff between the .dacpac and the live schema, then generates only the ALTER statements needed to close the gap. Missing two columns? You get one ALTER TABLE with both additions. The process is idempotent. Deploy the same .dacpac five times and the fifth run changes nothing. You can also fan out a single .dacpac across a fleet of databases when upgrading multiple tenants.

Before deploying directly, you can preview the planned changes:

Bash

```
sqlpackage /Action:Script /SourceFile:bin/Debug/MyDatabaseProject.dacpac /TargetConnectionString:"..." /OutputPath:deploy-script.sql

sqlpackage /Action:DeployReport /SourceFile:bin/Debug/MyDatabaseProject.dacpac /TargetConnectionString:"..." /OutputPath:report.xml
```

The Script action produces the exact T-SQL that would run. The DeployReport action produces an XML summary of every CREATE, ALTER, and DROP. Review either one before pushing changes to production.

Key takeaways

A SQL database project stores every database object as a declarative .sql file, and the build compiles them into a single .dacpac artifact. SDK-style projects (Microsoft.Build.Sql) support .NET 8+, cross-platform builds, NuGet package references, and default globbing for .sql files. The build process validates object references and T-SQL syntax against the target platform before anything reaches a database. SqlPackage calculates the difference between the .dacpac and the target database, generating only the statements needed to close the gap. Use the Script and DeployReport actions to preview planned changes before deploying to production. The SQL database project, its .dacpac artifact, and SqlPackage form the foundation for everything ahead, including source control, CI/CD pipelines, schema drift detection, and testing.

Next unit: Configure source control and manage reference data

Configure source control and manage reference data

Completed100 XP

- **10 minutes**

Choose your preferred content format

A SQL database project without source control is just a folder of files on someone's machine. The moment that person's laptop breaks, or two developers edit the same stored procedure, you're back to square one. Placing the project in Git gives you change history, collaboration through branching and pull requests, and the foundation every CI/CD pipeline needs.

But source control alone doesn't solve everything. Your database probably has lookup tables, status codes, and default configurations that live alongside the schema. You need a strategy for that data too.

Set up source control for a SQL database project

Because a SQL database project is just .sql files and a .sqlproj file, it fits naturally into Git. Each database object lives in its own file, so Git tracks changes at the object level. A commit that modifies the Customers table and updates a stored procedure shows exactly those two files in the diff, not a monolithic migration script with dozens of unrelated changes.

To get started, initialize a Git repository in the project folder:

Bash

cd MyDatabaseProject

git init

git add .

git commit -m "Initial commit of database project"

Organize the project folder

A consistent folder structure makes the project navigable at a glance. The most common pattern groups files by object type:

MyDatabaseProject/

├── MyDatabaseProject.sqlproj

├── Tables/

│ ├── Customers.sql

│ └── Orders.sql

├── Views/

│ └── vw_ActiveCustomers.sql

├── StoredProcedures/

│ └── usp_GetCustomerOrders.sql

├── Scripts/

│ ├── PostDeployment/

│ │ └── seed-data.sql

│ └── PreDeployment/

│ └── prep-db.sql

└── PostDeploy.sql

With SDK-style projects, the default globbing pattern picks up every .sql file in the folder automatically. No manual file entries needed.

Add a .gitignore file

Keep the repository focused on source files by excluding build outputs and user-specific settings:

bin/

obj/

***.dacpac**

***.user**

The .dacpac gets recreated from the project on every CI/CD build, so there's no reason to track it in Git.

Manage reference data with predeployment and post-deployment scripts

The declarative model handles schema objects like tables, views, and stored procedures, but some data is as essential as the schema itself. Status codes, lookup tables, default configurations, region lists. If that data disappears, the application breaks. It belongs in the project, versioned alongside the objects that depend on it.

Predeployment and post-deployment scripts solve this problem. They're SQL scripts that execute during deployment but sit outside the compiled database model:

- **A pre-deployment script runs before the deployment plan. Use it for tasks that must complete before schema changes, such as dropping constraints or migrating data.**
- **A post-deployment script runs after the deployment plan completes. Use it to populate reference data, seed lookup tables, or set application defaults.**

Add scripts to the project

A project supports exactly one predeployment script and one post-deployment script. You declare them in the .sqlproj file with PreDeploy and PostDeploy item entries:

XML

```
<ItemGroup>
  <PreDeploy Include="prep-db.sql" />
</ItemGroup>
<ItemGroup>
  <PostDeploy Include="PostDeploy.sql" />
</ItemGroup>
```

Use SQLCMD includes for multiple data files

One script file doesn't mean one giant file. Use SQLCMD :r syntax to pull in multiple files from a single entry point. A typical PostDeploy.sql looks like this:

SQL

```
:r .\Scripts\PostDeployment\seed-statuses.sql
:r .\Scripts\PostDeployment\seed-regions.sql
:r .\Scripts\PostDeployment\seed-app-settings.sql
```

Each referenced file needs to be excluded from the build, otherwise the build process tries to compile it as a schema object and fails. In the .sqlproj file, use Build Remove to prevent compilation and None Include to keep the file visible in the project:

XML

<ItemGroup>

** <Build Remove="Scripts\PostDeployment\seed-statuses.sql" />**

** <None Include="Scripts\PostDeployment\seed-statuses.sql" />**

</ItemGroup>

📝 **Repeat this pattern for each referenced file in your predeployment or post-deployment scripts.**

Write idempotent reference data scripts

Post-deployment scripts run on every deployment, not just the first one. If you use plain INSERT statements, the second deployment fails with duplicate key violations. Use MERGE statements instead to make the scripts safe to run repeatedly:

SQL

```
MERGE INTO [dbo].[OrderStatuses] AS target
USING (VALUES
  (1, N'Pending'),
  (2, N'Processing'),
  (3, N'Shipped'),
  (4, N'Delivered'),
  (5, N'Cancelled')
) AS source ([StatusID], [StatusName])
ON target.[StatusID] = source.[StatusID]
WHEN MATCHED THEN
  UPDATE SET [StatusName] = source.[StatusName]
WHEN NOT MATCHED THEN
  INSERT ([StatusID], [StatusName])
  VALUES (source.[StatusID], source.[StatusName]);
```

Tip

You can validate the predeployment and post-deployment scripts after a project build by changing the .dacpac file extension to .zip and extracting it. A

single .sql file for each script type contains the combined T-SQL content from all referenced files.

Key takeaways

Initialize a Git repository at the solution level and use a .gitignore file to exclude build outputs like bin/, obj/, and .dacpac files. Organize schema objects into folders that mirror the database structure, with one file per object. Place reference data in post-deployment scripts and use SQLCMD :r includes to keep each script focused on a single table. To prevent data scripts from being compiled as schema objects, use Build Remove and None Include in the .sqlproj file. Write MERGE statements instead of plain INSERT statements so reference data scripts are idempotent and safe to run on every deployment. Next, you put this repository to work with branching and pull request workflows.

Next unit: Manage branching, pull requests, and conflict resolution

Manage branching, pull requests, and conflict resolution

Completed100 XP

- **8 minutes**

Choose your preferred content format

Two developers add columns to the same table on the same day. One pushes to production, the other follows an hour later and overwrites the first change. Sound familiar? Branching, pull requests, and conflict resolution exist to prevent exactly this kind of collision.

Use feature branches for database changes

A simple branching strategy for SQL database projects follows three principles:

- **Creates a feature branch for every change, whether it's new tables, bug fixes, or stored procedure updates.**
- **Merges feature branches into main through pull requests.**
- **Always keep main in a deployable state.**

When you start a database change, branch off main. Your work is isolated from everything else in progress. You modify the relevant .sql files, and because each object has its own file, a commit that adds a column to Customers touches only Tables/Customers.sql and nothing else.

Bash

git checkout -b feature/add-customer-email

After completing the changes, push the branch to the remote repository:

Bash

git add .

git commit -m "Add Email column to Customers table"

git push origin feature/add-customer-email

Review changes with pull requests

A pull request signals that your changes are ready for a second pair of eyes. Because SQL database projects store each object in a separate file, the diff shows the precise T-SQL that changed. Separate files make it far easier to review than a 500-line migration script that mixes unrelated modifications.

Pull requests for database changes should address questions like:

- **Does the schema change break existing queries or stored procedures?**
- **Are the naming conventions consistent with the rest of the project?**
- **Does the change require a corresponding update to a post-deployment script?**
- **Does the change cause data loss or require data migration?**

In Azure DevOps, create a pull request from a feature branch to main in the Repos section. In GitHub, use the Pull requests tab.

Connect pull requests to CI builds

A pull request with a green build gives reviewers confidence. Configure the PR to trigger a CI build that compiles the SQL database project. If the build fails because of a broken reference, syntax error, or unresolved dependency, the problem gets caught before it reaches main.

In Azure DevOps, you configure a CI build trigger through a Build Validation branch policy. In GitHub, you set up a workflow that triggers on pull_request events targeting the main branch.

Resolve merge conflicts in SQL database projects

Conflicts happen when two branches modify the same file in ways Git can't reconcile on its own. In database projects, a common scenario is two developers editing the same CREATE TABLE statement. One adds a Phone column, the other adds Address, and both touch the same region of the file.

To resolve the conflict:

1. **Pull the latest changes from main into your feature branch:**

Bash

```
git checkout feature/add-customer-phone

git pull origin main
```

2. **Git marks the conflicting sections in the file. Open the file and review both versions.**
3. **Edit the file to include both changes in the correct syntax.**
4. **Mark the conflict as resolved and commit:**

Bash

```
git add Tables/Customers.sql

git commit -m "Resolve merge conflict: include both Phone and Address columns"
```

Tip

Reduce the frequency of merge conflicts by keeping feature branches short-lived. Merge them back to main as soon as the change is complete and reviewed. Long-running branches that diverge significantly from main are harder to merge.

Validate after resolving conflicts

A clean text merge doesn't guarantee a valid schema. Suppose one branch renames a column while another branch adds a stored procedure that references the old column name. Git merges the files without complaint, but the project is broken. Always rebuild after resolving conflicts:

Bash

```
dotnet build MyDatabaseProject.sqlproj
```

📝 **A successful build confirms that object references are intact and the T-SQL syntax is correct after the merge.**

Key takeaways

Use short-lived feature branches to isolate database changes, and merge them back to main through pull requests. Configure CI builds as pull request checks so a successful dotnet build validates every proposed change before merging. Merge conflicts in .sql files are typically straightforward because each file contains a single object declaration. Always rebuild the project after resolving merge conflicts to verify that all object references are intact. Next, you learn how to detect when the live database falls out of sync with your project.

Next unit: Detect and resolve schema drift

Detect and resolve schema drift

Completed100 XP

- **10 minutes**

Choose your preferred content format

Your SQL database project says the Customers table has 12 columns. Production has 13. Someone added a LoyaltyTier column directly through SQL Server Management Studio (SSMS) last Thursday during an incident. Your next deployment will quietly drop that column because the project doesn't know it exists. This type of situation is known as schema drift, and it's one of the ways CI/CD pipelines break production without warning.

Understand schema drift

Schema drift is the gap between what your project defines and what actually exists in the live database. Common causes include:

- **Manual changes: someone opens a query window and runs an ALTER TABLE outside the normal workflow.**
- **Emergency hotfixes: a production issue gets patched at 2 AM, and the fix never makes it back into the project.**
- **Third-party tools: monitoring agents or Object-Relational Mapping (ORM) frameworks that create or modify objects behind the scenes.**

The danger isn't the drift itself. It's what happens next. Your pipeline deploys the .dacpac, SqlPackage calculates the diff, and every object the project doesn't know about gets dropped. Detecting drift before that deployment runs is critical.

Detect drift with schema comparison

Schema comparison tools let you hold two database definitions side by side. You can compare any combination of a live database, a SQL database project, or a .dacpac file. When you compare a live database against your project, the results surface every difference, grouped by action type (Create, Alter, Delete) with the source and target definitions shown for each object.

In Visual Studio or Visual Studio Code, launch schema compare from the SQL Server menu or the Database Projects view. Point the source at the live database and the target at the project. The comparison grid shows what's different and what would change if you brought them into alignment.

Note

The direction is reversible. Compare database-to-project to find drift. Compare project-to-database to preview what a deployment would change. Same tool, opposite perspective.

Schema comparison options

Not every difference matters. You can tune the comparison to focus on what's meaningful:

- Ignore whitespace to skip formatting-only differences.
- Ignore column order when column position is irrelevant to your application.
- Block on possible data loss to flag destructive operations like dropping columns that contain data.

Save your comparison settings as an .scmp file and commit it to your repository. It stores the source, target, comparison options, and excluded object types, making drift checks repeatable across the team.

Import changes from the database into the project

Once you spotted a drift, you need to decide: does the live database have the right version, or does the project? If production's hotfix was correct, bring it into the project so your source of truth stays accurate.

Use schema compare to apply changes

In the comparison results, select the specific differences you want to import and apply them. The graphical interface in Visual Studio and Visual Studio Code lets you cherry-pick. Accept the hotfix index but ignore the monitoring agent's temp table, for example.

Automate with SqlPackage extract

For CI/CD scenarios or scheduled drift checks, use SqlPackage Extract to pull the live schema into files:

Bash

```
sqlpackage /Action:Extract /SourceConnectionString:"{connection string}" /TargetFile:MyDatabaseProject /p:ExtractTarget=SchemaObjectType
```

📝 **This extracts database objects into files organized by schema and object type. With the project folder under Git, running git status after the extract reveals exactly which files changed. That's your automated drift report.**

Three commands give you a drift count:

Bash

```
rm -rf MyDatabaseProject

sqlpackage /Action:Extract /SourceConnectionString:"{connection string}" /TargetFile:MyDatabaseProject /p:ExtractTarget=SchemaObjectType

git status --porcelain | wc -l
```

📝 **The output is the number of files that changed, giving you an automated drift count.**

Review deployment changes before applying them

You saw the Script and DeployReport actions in the earlier unit on building and deploying. In a drift-detection context, these same commands help you verify what SqlPackage would do before you run it against production.

Generate a deployment script

The Script action produces the exact T-SQL that would execute, without actually running it:

Bash

```
dotnet build MyProject.sqlproj

sqlpackage /Action:Script /SourceFile:bin/Debug/MyProject.dacpac /TargetConnectionString:"{connection string}" /DeployScriptPath:Deployment.sql
```

Review Deployment.sql to see every ALTER, CREATE, and DROP that the deployment would execute. Store the script as a pipeline artifact for approval before running it against production.

Generate a deployment report

The DeployReport action produces an XML summary of planned operations:

Bash

```
sqlpackage /Action:DeployReport /SourceFile:bin/Debug/MyProject.dacpac /TargetConnectionString:"{connection string}" /OutputPath:report.xml
```

The report lists operations like Create, Alter, Drop, and TableDataMotion. Parse the XML in your pipeline to trigger alerts on high-impact events like data motion, clustered index rebuilds, or column drops:

XML

```
<DeploymentReport xmlns="http://schemas.microsoft.com/sqlserver/dac/DeployReport/2012/02" >
  <Alerts />
  <Operations>
    <Operation Name="Create">
      <Item Value="[dbo].[Products].[IX_Products_CategorySlug]" Type="SqlIndex" />
    </Operation>
    <Operation Name="Alter">
      <Item Value="[dbo].[Customers]" Type="SqlTable" />
    </Operation>
  </Operations>
</DeploymentReport>
```

Verify .dacpac consistency with DacpacVerify

DacpacVerify compares two .dacpac files and reports differences between them, including predeployment scripts, post-deployment scripts, SQLCMD variables, database references, properties, and database objects. This comparison is useful when converting from an original SQL Server Data Tools (SSDT) project to SDK-style, or when validating that a pipeline produces the expected artifact.

Install it as a .NET global tool and run it against two .dacpac files:

Bash

dotnet tool install -g microsoft.dacpacverify

dacpacverify before.dacpac after.dacpac

Key takeaways

Schema drift occurs when the live database diverges from the SQL project, often caused by hotfixes, manual changes, or direct production edits. Schema comparison tools detect differences between a database and a project, letting you choose which changes to pull into the project or push to the database. Save schema comparison settings in .scmp files so the team uses consistent options every time. Automate drift detection by running SqlPackage /Action:Extract on a schedule and comparing the extracted project against the repository. Use SqlPackage /Action:DeployReport to preview every planned CREATE, ALTER, and DROP before applying changes. The goal isn't to prevent drift entirely, but to detect it early and resolve it before your next deployment turns a hotfix into a rollback. Next, you build the CI/CD pipelines that automate the entire build-and-deploy cycle.

Next unit: Implement CI/CD pipelines

mplement CI/CD pipelines

Completed100 XP

- 12 minutes

Choose your preferred content format

Up to this point, every step in the process was manual. Build the project, run SqlPackage, check the deployment report. A CI/CD pipeline turns that sequence into something that happens automatically on every commit, with the same steps, in the same order, every time. No forgotten flags, no typos in connection strings, no "it worked on my machine."

Design the pipeline structure

Most database pipelines follow a two-stage pattern:

- **Build**: compile the SQL database project and produce the .dacpac artifact.
- **Deploy**: publish that .dacpac to target databases, moving through environments (development, staging, production).

Building once and deploying the same artifact everywhere eliminates the "works in dev, breaks in prod" problem. The .dacpac that passed validation in staging is the same file that gets deployed to production.

Implement a CI/CD pipeline with GitHub Actions

GitHub Actions workflows live in .github/workflows/ and define your pipeline as YAML. The azure/sql-action action handles .dacpac deployment to Azure SQL Database.

Here's a workflow that builds on every push to main and deploys to production:

YAML

```
# .github/workflows/sql-deploy.yml

name: Build and Deploy SQL Project

on:

 push:

  branches: [main]


jobs:

 build:

  runs-on: ubuntu-latest

  steps:

  - uses: actions/checkout@v4


  - name: Build SQL project

   run: dotnet build ./Database.sqlproj -o ./output


  - name: Upload dacpac artifact

   uses: actions/upload-artifact@v4

   with:
```

```
    name: dacpac
    path: ./output/Database.dacpac

deploy:
 needs: build
 runs-on: ubuntu-latest
 environment: production
 steps:
 - name: Download dacpac artifact
  uses: actions/download-artifact@v4
  with:
   name: dacpac

 - name: Install SqlPackage
  run: dotnet tool install -g microsoft.sqlpackage

 - uses: azure/login@v2
  with:
   client-id: ${{ secrets.AZURE_CLIENT_ID }}
   tenant-id: ${{ secrets.AZURE_TENANT_ID }}
   subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}

 - uses: azure/sql-action@v2.3
  with:
   connection-string: ${{ secrets.AZURE_SQL_CONNECTION_STRING }}
   path: './Database.dacpac'
   action: 'publish'
```

📝 The azure/sql-action supports .dacpac, .sqlproj, and .sql scripts. It works with SQL authentication, Microsoft Entra ID, and service principal authentication.

If your Azure SQL Database has firewall rules enabled, the GitHub Actions runner's IP needs access. When you pair azure/login with azure/sql-action, the action adds a temporary firewall rule for the runner's IP during deployment and removes it afterward.

Implement a CI/CD pipeline with Azure DevOps

Azure DevOps provides the SqlAzureDacpacDeployment task for .dacpac deployment. Here's the equivalent pipeline:

YAML

```
# azure-pipelines.yml

trigger:
 - main


pool:
 vmImage: 'windows-latest'


steps:
- task: DotNetCoreCLI@2
 inputs:
  command: 'build'
  projects: './Database.sqlproj'
  arguments: '-o $(Build.ArtifactStagingDirectory)'


- task: PublishBuildArtifacts@1
 inputs:
  PathtoPublish: '$(Build.ArtifactStagingDirectory)/Database.dacpac'
  ArtifactName: 'dacpac'
```

```
- task: SqlAzureDacpacDeployment@1
  inputs:
    azureSubscription: 'your-service-connection'
    AuthenticationType: 'server'
    ServerName: 'your-server.database.windows.net'
    DatabaseName: 'your-database'
    SqlUsername: '$(SqlUser)'
    SqlPassword: '$(SqlPassword)'
    deployType: 'DacpacTask'
    DeploymentAction: 'Publish'
    DacpacFile: '$(Build.ArtifactStagingDirectory)/Database.dacpac'
```

For Linux agents or more control over the process, install and use SqlPackage directly:

YAML

```
steps:
- script: dotnet tool install --global microsoft.sqlpackage
  displayName: 'Install SqlPackage'

- script: |
    sqlpackage /Action:Publish \
      /SourceFile:$(Build.ArtifactStagingDirectory)/Database.dacpac \
      /TargetConnectionString:"$(ConnectionString)"
  displayName: 'Deploy database'
```

Whether you use GitHub Actions or Azure DevOps, both pipelines need credentials to connect to your database. The next step is making sure those credentials stay out of your YAML files.

A connection string hardcoded in a YAML file is a breach waiting to happen. Both GitHub Actions and Azure DevOps provide mechanisms to store and inject secrets at runtime without exposing them in source control.

GitHub repository and environment secrets

Store sensitive values as **repository secrets** or **environment secrets**. In your GitHub repository on github.com:

1. Select **Settings** > **Secrets and variables** > **Actions**.
2. Select **New repository secret**.
3. Add a name (like AZURE_SQL_CONNECTION_STRING) and the value.

Reference secrets in your workflow with ${{ secrets.SECRET_NAME }}. Environment secrets are scoped to specific deployment environments, so production credentials are accessible only to jobs targeting production.

Service principal and OpenID Connect authentication

Better yet, skip passwords entirely. **OpenID Connect (OIDC)** with azure/login authenticates using federated credentials, with no client secret stored anywhere. You need three values:

- AZURE_CLIENT_ID: The application (client) ID of the service principal.
- AZURE_TENANT_ID: Your Microsoft Entra ID tenant ID.
- AZURE_SUBSCRIPTION_ID: Your Azure subscription ID.

The service principal authenticates through federated credentials, so there's no password to rotate or leak.

Azure Key Vault integration

For centralized secrets management, store connection strings and credentials in **Azure Key Vault** and have your pipeline pull them at deployment time. In Azure DevOps, the **Azure Key Vault** task fetches secrets into pipeline variables. In GitHub Actions, use azure/login followed by Azure CLI commands to read from the vault.

Important

Rotate database credentials regularly. Azure Key Vault can integrate with Azure Functions to automate secret rotation, reducing the risk of compromised credentials.

Implement deployment pipeline controls

A pipeline that deploys to production on every push with no review step is risky for database changes. You need controls that prevent unreviewed changes from reaching production.

Environment protection rules

Both GitHub and Azure DevOps support **environments** with protection rules that gate deployments:

- **Required reviewers**: one or more team members must approve before the deploy job runs. In GitHub, configure this setting under **Settings** > **Environments** > **Protection rules**.
- **Wait timers**: add a delay between approval and execution, giving the team a window to reconsider.
- **Deployment branches**: restrict which branches can target an environment. For example, only main deploys to production.

Branch policies

Branch policies protect your main branch and serve as the first-line of defense. In Azure DevOps, configure:

- **Minimum number of reviewers** for pull requests.
- **Build validation**, which runs the SQL project build as a PR check before merging.
- **Comment resolution**, requiring all review comments to be addressed.
- **Automatically included reviewers**, so specific people are added to PRs that touch database files.

GitHub provides similar protections through **branch protection rules** or **rulesets**.

Code owners

A CODEOWNERS file (GitHub) or automatically included reviewers policy (Azure DevOps) makes sure the right people review database changes. No SQL file gets merged without the database team seeing it:

.github/CODEOWNERS

/Database/ @db-team

*.sql @db-team @dba-lead

This rule requires members of the db-team to review any pull request that modifies SQL files or the database project folder.

Branch control checks

In Azure DevOps, a **branch control check** on service connections locks down which pipelines can access production credentials. Only pipelines running in the

context of main get access. Even if someone modifies a pipeline on a feature branch to target production, the service connection refuses the request.

Key takeaways

Use azure/sql-action (GitHub Actions) or SqlAzureDacpacDeployment (Azure DevOps) to deploy .dacpac files from your CI/CD pipeline. Store connection strings and credentials as repository secrets, environment secrets, or in Azure Key Vault, and never hardcode them in YAML files. To authenticate without storing passwords, use OpenID Connect (OIDC) with federated credentials. Gate production deployments with environment protection rules, required reviewers, and deployment branch restrictions. Use CODEOWNERS files or automatically included reviewers to ensure that the right people review the database changes.

Next unit: Design and implement a testing strategy

Design and implement a testing strategy

Completed100 XP

- 10 minutes

Choose your preferred content format

A SQL database project that builds successfully doesn't mean the stored procedures inside it return the right results. A procedure can compile with no errors and still calculate the wrong totals or skip a validation check. Testing catches these problems before they reach production.

Understand database testing levels

Database testing works in layers, each catching a different class of problem:

- **Build validation** is the dotnet build step you already have. It catches syntax errors and broken object references. Fast, but it only proves the schema is structurally valid.
- **Unit tests** verify that individual stored procedures and functions produce the right results for a given set of inputs. They catch logic errors.
- **Integration tests** exercise end-to-end scenarios against a deployed database. They prove that objects work together correctly, but they require a running instance and take the most time.

Each layer is slower and more expensive than the one before it, but also catches problems the previous layer can't.

Create SQL Server unit tests

SQL Server Data Tools (SSDT) in Visual Studio includes a built-in framework for database unit tests. Each test executes T-SQL against a live database and validates the results using test conditions.

Structure of a unit test

Each test has three sections that follow a setup-execute-cleanup pattern:

- **Pre-test**: set up the data the test needs. Insert customer records, clear leftover data from previous runs.
- **Test**: execute the operation you're testing. Call the stored procedure and query the view.
- **Post-test**: clean up after the test so it doesn't contaminate the next run.

Test conditions

After the test T-SQL runs, **test conditions** validate what came back. The most commonly used conditions are:

- **Row Count**: verifies that the result set contains the expected number of rows.
- **Scalar Value**: verifies that a specific cell in the result set contains the expected value.
- **Expected Schema**: verifies that the result set has the expected column names and data types.

Other built-in conditions include **Data Checksum**, **Empty ResultSet**, **Not Empty ResultSet**, and **Execution Time**.

Create a unit test project

Setting up SQL Server unit tests in Visual Studio takes a few steps:

1. Open your SQL database project.
2. In **SQL Server Object Explorer**, find the stored procedure or function you want to test.
3. Right-click the object and select **Create Unit Tests**.
4. Choose or create a C# test project.

5. Set the test connection to your development database.
6. Select **Automatically deploy the database project before unit tests are run**. This option keeps the test database in sync with your latest project changes.

The designer opens with a T-SQL template where you write your test logic and attach test conditions.

Write effective database unit tests

A good unit test answers a specific question: "Does this procedure do what it supposed to for a known input?" Here's an example that tests uspPlaceNewOrder, verifying that placing an order correctly updates the customer's year-to-date total:

SQL

```
-- Pre-test: Set up customer and clear previous data

DECLARE @CustomerID INT;

INSERT INTO [Sales].[Customer] (CustomerName) VALUES (N'Test Customer');

SET @CustomerID = SCOPE_IDENTITY();

DELETE FROM [Sales].[Orders] WHERE [CustomerID] = @CustomerID;


-- Test: Place an order and verify YTDOrders updated correctly

DECLARE @RC INT;

EXECUTE @RC = [Sales].[uspPlaceNewOrder] @CustomerID, 100, GETDATE(), 'O';


SELECT [YTDOrders]

FROM [Sales].[Customer]

WHERE [CustomerID] = @CustomerID;
```

📝 Pair this test T-SQL with a **Scalar Value** test condition that expects the YTDOrders value to be 100.

Negative tests

You also need to verify that stored procedures fail correctly when they receive invalid inputs. For example, canceling an order that was already shipped should raise an error, not succeed silently. A **negative test** confirms that the expected error occurs.

When you select **Create Unit Tests** in step 3 of the previous section, Visual Studio generates a C# test project in **Solution Explorer** with a .cs file containing a test for each stored procedure you selected. For example, if you created a test for uspCancelOrder, the generated file includes a section named Sales_uspCancelOrder_Test. To mark that test as a negative test, open the .cs file in **Solution Explorer** and add the ExpectedSqlException attribute directly above it and save the file:

C#

```
[TestMethod()]

[ExpectedSqlException(Severity = 16, MatchFirstError = false, State = 1)]

public void Sales_uspCancelOrder_FilledOrder_Test()
```

The test passes only when the stored procedure raises an error matching the specified severity and state. If the procedure succeeds silently, the test fails, which is exactly what you want.

Design an integration testing approach

Unit tests isolate individual objects. Integration tests go further and test scenarios that span multiple operations. They answer questions like:

- Does a sequence of stored procedure calls leave the database in the expected state?
- Do triggers fire correctly when data arrives through the application layer?
- Do views return accurate results after a series of related table changes?

Integration tests need a dedicated database. Deploy the SQL database project to a test instance before each run using the **Automatically deploy the database project before unit tests are run** setting. This setting keeps the test schema current.

Considerations for test databases

Both unit tests and integration tests run T-SQL against a live database, so the test environment needs careful setup to produce reliable results.

- **Isolate tests from production**. Use a separate instance or dedicated test database. Running tests against production is never recommended.
- **Reset to a known state** before each run. Post-deployment scripts or test cleanup scripts handle this issue.

- **Externalize connection strings** in configuration files so local development and CI pipelines each point to the right database without code changes.

Integrate tests into CI/CD pipelines

Add a test step after build and deployment so tests run automatically on every commit. In Azure DevOps, use the VSTest task:

YAML

```
- task: VSTest@2
  inputs:
    testAssembly: '**\*Tests.dll'
    searchFolder: '$(System.DefaultWorkingDirectory)'
```

In GitHub Actions, run the tests using dotnet test:

YAML

```
- name: Run database unit tests
  run: dotnet test ./DatabaseTests/DatabaseTests.csproj
```

Tip

Configure the test project to automatically deploy the database project before tests run. This setting ensures that the test database schema matches the project at the time of testing.

When a test fails, the pipeline stops and the change doesn't reach staging or production. This delay gives the team time to fix the issue before it affects any environment.

Key takeaways

Database testing spans three levels: build validation for structural correctness, unit tests for logic verification, and integration tests for end-to-end workflows. SQL Server unit tests follow a three-phase structure of test preactions, test actions, and test post-actions. To verify stored procedure and function behavior, use test conditions like Scalar Value, Row Count, and Expected Schema. To test methods that should validate error handling paths, add the ExpectedSqlException attribute. Wire test projects into your CI/CD pipeline so a failing test blocks the deployment before changes reach staging or production. Together, these three layers form a safety net that lets your team deploy database changes with confidence instead of anxiety.

Next unit: Exercise: implement CI/CD by using SQL database projects

Summary

Completed100 XP

- 3 minutes

CI/CD for SQL Database Projects brings the same automation, consistency, and safety nets to database development that application teams rely on for their code.

In this module, you learned how to:

- **Create, build, and validate SQL Database Projects**: Define database objects in declarative T-SQL files, build them into .dacpac artifacts, and validate references and syntax against a target platform using the SDK-style Microsoft.Build.Sql project format.
- **Configure source control and manage reference data**: Place SQL database projects in Git, organize files by object type, and use predeployment and post-deployment scripts with SQLCMD :r includes to manage reference data alongside the schema.
- **Manage branching, pull requests, and conflict resolution**: Use feature branches for database changes, review T-SQL diffs in pull requests, resolve merge conflicts at the object level, and validate merged results with a project build.
- **Detect and resolve schema drift**: Compare live databases against SQL database projects using schema comparison tools, automate drift detection with SqlPackage Extract, and review planned changes with deployment reports and scripts.
- **Implement CI/CD pipelines with deployment controls**: Build and deploy .dacpac files with GitHub Actions (azure/sql-action) and Azure DevOps (SqlAzureDacpacDeployment), manage secrets through repository secrets and Azure Key Vault, and protect production with environment approvals, branch policies, and code owners.
- **Design and implement a testing strategy**: Create SQL Server unit tests with test conditions (Row Count, Scalar Value, Expected Schema), write negative tests for error handling, and integrate tests into CI/CD pipelines to catch logic errors before deployment.

Learn more

- What are SQL database projects?
- Get started with SQL database projects
- SQL Server Data Tools, SDK-style (preview)
- Predeployment and post-deployment scripts
- Schema comparison overview
- Compare a database and a project
- SQL projects automation
- About branches and branch policies
- Verify database code by using SQL Server unit tests
- Azure SQL Deploy action (GitHub)
- SqlAzureDacpacDeployment task reference

Get started with Azure

Choose the Azure account that's right for you. Pay as you go or try Azure free for up to 30 days.Sign up.

Integrate SQL solutions with Azure services

Create REST and GraphQL APIs for SQL databases using Data API Builder, deploy to Azure hosting services, and implement monitoring and event-driven change patterns.

Introduction

Choose your preferred content format

Modern applications rarely interact with databases through direct SQL connections. Instead, they consume data through REST and GraphQL APIs that provide flexibility, security, and standardization. Building these API layers traditionally requires significant backend development effort—writing controllers, mapping data models, handling authentication, and managing deployment infrastructure.

Data API Builder solves this challenge by automatically generating REST and GraphQL endpoints from your database schema. With a single configuration file, you can expose tables, views, and stored procedures through secure, scalable APIs without writing backend code.

In this module, you learn how to:

- Create and configure Data API Builder configuration files for SQL databases
- Define entities with field mappings, caching, pagination, and filtering
- Configure REST and GraphQL endpoints for different client needs
- Expose views, stored procedures, and define GraphQL relationships
- Explore deployment options for Data API Builder, including Azure Container Apps, App Service, and Static Web Apps
- Set up Azure Monitor with Application Insights for API observability
- Handle database changes using Change Data Capture, Azure Functions, and Change Event Streaming

Prerequisites

- Experience with SQL Server, Azure SQL Database, or SQL databases in Microsoft Fabric
- Familiarity with T-SQL for creating tables, views, and stored procedures
- Basic understanding of REST APIs and JSON
- Access to an Azure subscription for deployment exercises
- Docker Desktop installed for local development (optional)

Note

We recognize that different people like to learn in different ways. You can choose to complete this module in video-based format or you can read the content as text and images. The text contains greater detail than the videos, so in some cases you might want to refer to it as supplemental material to the video presentation.

Next unit: Create configuration files for data API builder

Create configuration files for data API builder

Completed100 XP

- 10 minutes

Choose your preferred content format

Data API Builder (DAB) is a cross-platform, open-source engine that creates modern REST and GraphQL endpoints for your database without requiring you to write custom code. With a single configuration file, you can expose your database objects through secure, scalable APIs that support authentication, authorization, and caching.

If you work with Azure SQL or SQL databases in Microsoft Fabric, you've likely faced requests to make your data accessible to application developers. Building custom API layers takes time and adds maintenance overhead. Data API Builder eliminates that work by generating APIs directly from your database schema.

Understand the Data API Builder configuration file

Everything in Data API Builder starts with a configuration file. This JSON file tells DAB how to connect to your database, which objects to expose, how to handle authentication, and how to behave at runtime. When DAB starts, it reads this configuration and generates your REST and GraphQL endpoints automatically.

A typical configuration file has four main sections:

- **$schema** - References the JSON schema for validation and IntelliSense support
- **data-source** - Defines the database connection string and type
- **runtime** - Controls runtime behavior like host settings, authentication, and caching
- **entities** - Specifies which database objects to expose and how to configure them

Here's a basic configuration file structure:

JSON

```
{
 "$schema": "https://github.com/Azure/data-api-builder/releases/download/v1.1.7/dab.draft.schema.json",
 "data-source": {
  "database-type": "mssql",
  "connection-string": "@env('DATABASE_CONNECTION_STRING')"
 },
 "runtime": {
```

```
  "rest": {
    "enabled": true,
    "path": "/api"
  },
  "graphql": {
    "enabled": true,
    "path": "/graphql"
  },
  "host": {
    "cors": {
      "origins": ["http://localhost:3000"],
      "allow-credentials": false
    },
    "authentication": {
      "provider": "StaticWebApps"
    }
  }
 },
 "entities": {}
}
```

Notice the $schema property at the top? That gives your editor autocompletion and validation as you type. Always use the schema version that matches your DAB installation.

Configure the data source connection

The data-source section tells DAB how to connect to your database. For SQL Server, Azure SQL Database, and SQL databases in Microsoft Fabric, you use mssql as the database type.

JSON

```
"data-source": {
```

```
 "database-type": "mssql",
 "connection-string": "@env('DATABASE_CONNECTION_STRING')",
 "options": {
  "set-session-context": true
 }
}
```

You could put your connection string directly in the configuration file, but don't. Use the @env() syntax instead to reference environment variables. This keeps credentials out of your configuration files and source control.

The set-session-context option is worth noting. When enabled, DAB passes user identity information to SQL Server through session context. Your stored procedures and functions can then read this information using SESSION_CONTEXT(), which is helpful for implementing row-level security or audit logging.

Tip

For local development, you can use the dab init command with the --connection-string parameter to set up your initial configuration file quickly.

Set up runtime options

The runtime section controls how DAB operates when serving requests. Here you configure REST and GraphQL endpoints separately, along with global host settings like CORS and caching.

JSON

```
"runtime": {
 "rest": {
  "enabled": true,
  "path": "/api",
  "request-body-strict": true
 },
 "graphql": {
  "enabled": true,
  "path": "/graphql",
  "allow-introspection": true
```

```
  },
  "host": {
    "cors": {
      "origins": ["https://myapp.azurewebsites.net"],
      "allow-credentials": true
    },
    "mode": "production"
  },
  "cache": {
    "enabled": true,
    "ttl-seconds": 5
  }
}
```

The path properties define the base URL path for each API type. REST endpoints appear under /api/EntityName, while GraphQL queries go to /graphql. The allow-introspection setting for GraphQL determines whether clients can query the schema itself, which is helpful during development but should often be disabled in production for security.

CORS (Cross-Origin Resource Sharing) settings in the host section specify which domains can call your API from browser-based applications. The mode property switches between development and production behavior, affecting features like detailed error messages.

Use the DAB CLI for configuration management

The Data API Builder CLI provides commands for creating and modifying configuration files without manual JSON editing. This approach reduces errors and ensures valid configuration syntax.

To create a new configuration file:

Bash

```
dab init --database-type mssql --connection-string "@env('DATABASE_CONNECTION_STRING')" --host-mode development
```

This command creates a dab-config.json file in your current directory with baseline settings. From there, you can add entities using CLI commands or edit the file directly.

To add an entity for a table:

Bash

```
dab add Product --source dbo.Products --permissions "anonymous:read"
```

The CLI handles JSON formatting and validates your configuration against the schema automatically. If you work in a team, using CLI commands also makes pull request reviews easier since the formatting stays consistent.

Important

Always store your configuration files in source control, but never commit actual connection strings. Use environment variable references and manage credentials through secure mechanisms like Azure Key Vault or deployment pipeline secrets.

Organize configuration for multiple environments

Most projects need different settings for development, staging, and production. You might want introspection enabled during development but disabled in production, or different CORS origins for each environment.

DAB supports this through configuration merging. Start with a base dab-config.json file containing shared settings, then create environment-specific overrides:

- dab-config.development.json - Development-specific settings
- dab-config.staging.json - Staging environment settings
- dab-config.production.json - Production settings

When you start DAB with the --config parameter, it merges that file's settings with your base configuration:

Bash

```
dab start --config dab-config.production.json
```

Environment-specific files only need to contain settings that differ from the base. DAB handles the merge automatically, with environment-specific values taking precedence over base settings.

Next unit: Define entities for REST and GraphQL

Define entities for REST and GraphQL

Completed100 XP

- **12 minutes**

Choose your preferred content format

Entities are the core building blocks of Data API Builder. Each entity maps to a database object and defines how that object appears through your REST and GraphQL APIs. When you define entities properly, you control what data clients can access, what operations they can perform, and how the data appears in API responses.

Building on the configuration file you created, the entities section is where you specify the tables, views, and other database objects that DAB exposes. You configure each entity with source mappings, field customizations, and relationship definitions that shape your API surface.

The examples throughout this unit use the following sample database tables. Keep these definitions in mind as you work through each configuration:

SQL

```
CREATE TABLE dbo.Categories (
  CategoryID INT PRIMARY KEY,
  CategoryName NVARCHAR(50) NOT NULL
);


CREATE TABLE dbo.Products (
  ProductID INT PRIMARY KEY,
  ProductName NVARCHAR(100) NOT NULL,
  UnitPrice DECIMAL(10, 2) NOT NULL,
  UnitsInStock INT NOT NULL,
  CategoryID INT FOREIGN KEY REFERENCES dbo.Categories(CategoryID)
);
```

Map database objects to entities

An entity definition starts with a name and a source reference. The entity name becomes the identifier used in API endpoints and GraphQL queries, while the source points to the actual database object.

JSON

```
"entities": {
  "Product": {
    "source": {
      "object": "dbo.Products",
      "type": "table"
    },
    "permissions": [
      {
        "role": "anonymous",
        "actions": ["read"]
      }
    ]
  }
}
```

This configuration exposes the dbo.Products table as a Product entity. REST clients access it at /api/Product, while GraphQL clients query it as product (singular) or products (plural). DAB automatically generates these naming conventions based on your entity name.

The type property specifies the database object type: table, view, or stored-procedure. Each type has different capabilities. Tables support full CRUD operations (create, read, update, delete), views typically support read operations, and stored procedures execute custom logic.

Configure field mappings and aliases

Sometimes your database column names don't match the naming conventions you want in your API. Field mappings let you rename columns and control which fields appear in API responses.

JSON

```
"Product": {
 "source": {
  "object": "dbo.Products",
  "type": "table"
 },
 "mappings": {
  "ProductID": "id",
  "ProductName": "name",
  "UnitPrice": "price",
  "UnitsInStock": "stockQuantity"
 },
 "permissions": [
  {
   "role": "anonymous",
   "actions": ["read"]
  }
 ]
}
```

With these mappings, clients see id, name, price, and stockQuantity in their API responses instead of the original database column names. This approach lets you present a clean, consistent API while keeping your existing database schema unchanged.

Note

When you use field mappings, clients must use the mapped names in their queries and mutations. The original database column names become internal implementation details.

Enable caching for improved performance

Data API Builder caching reduces database load by storing query results temporarily. You can configure caching at both the global level (in the runtime section) and per-entity for fine-grained control.

JSON

```
"Product": {
 "source": {
  "object": "dbo.Products",
  "type": "table"
 },
 "rest": {
  "enabled": true
 },
 "graphql": {
  "enabled": true,
  "cache": {
   "enabled": true,
   "ttl-seconds": 60
  }
 },
 "permissions": [...]
}
```

The ttl-seconds property (time-to-live) determines how long cached data remains valid. For product catalogs that change infrequently, longer cache durations improve performance. For rapidly changing data like inventory counts, use shorter durations or disable caching.

Caching works differently for REST and GraphQL. REST caching considers the complete URL including query parameters, while GraphQL caching examines the query structure. Both respect the TTL you configure.

Implement pagination for large datasets

When entities contain many records, returning all of them in a single response creates performance and usability problems. Data API Builder provides built-in pagination support that you can customize per entity.

JSON

```
"Product": {
 "source": {
  "object": "dbo.Products",
  "type": "table"
 },
 "rest": {
  "enabled": true,
  "path": "/products"
 },
 "graphql": {
  "enabled": true,
  "type": {
   "singular": "product",
   "plural": "products"
  }
 },
 "permissions": [...]
}
```

REST clients use query parameters like $first and $after to navigate through pages:

HTTP

```
GET /api/products?$first=10
GET /api/products?$first=10&$after=eyJQcm9kdWN0SUQiOjEwfQ==
```

The $after parameter contains a cursor that points to the last item from the previous page. DAB generates these cursors automatically based on your entity's primary key.

GraphQL clients use similar pagination through the first and after arguments in their queries:

GraphQL

```
query {
 products(first: 10, after: "eyJQcm9kdWN0SUQiOjEwfQ==") {
  items {
   id
   name
   price
  }
  hasNextPage
  endCursor
 }
}
```

The response includes hasNextPage and endCursor fields that clients use to determine if more data exists and how to fetch it.

Configure filtering and searching

Data API Builder generates filtering capabilities automatically based on your entity's fields. Clients can filter results using standard comparison operators.

For REST endpoints, filtering uses OData-style query syntax:

HTTP

```
GET /api/products?$filter=price gt 100 and stockQuantity gt 0
```

For GraphQL, filtering uses typed input objects:

GraphQL

```
query {
 products(filter: { price: { gt: 100 }, stockQuantity: { gt: 0 } }) {
  items {
   id
   name
   price
  }
 }
```

}

DAB generates filter types for each field based on its data type. Numeric fields support gt, gte, lt, lte, eq, and neq. String fields add contains, startsWith, and endsWith. You don't need to configure these capabilities explicitly; they're available automatically for all exposed fields.

Tip

For complex filtering scenarios that can't be expressed through standard operators, consider creating a view or stored procedure that encapsulates the filtering logic, then expose that as a separate entity.

Define entity relationships for GraphQL

GraphQL's strength lies in traversing relationships between entities in a single query. Data API Builder supports defining relationships that map to your database's foreign key relationships.

JSON

```
"entities": {
 "Product": {
  "source": { "object": "dbo.Products", "type": "table" },
  "relationships": {
   "category": {
    "cardinality": "one",
    "target.entity": "Category",
    "source.fields": ["CategoryID"],
    "target.fields": ["CategoryID"]
   }
  },
  "permissions": [...]
 },
 "Category": {
  "source": { "object": "dbo.Categories", "type": "table" },
  "relationships": {
```

```
    "products": {
      "cardinality": "many",
      "target.entity": "Product",
      "source.fields": ["CategoryID"],
      "target.fields": ["CategoryID"]
    }
  },
  "permissions": [...]
 }
}
```

With these relationships configured, GraphQL clients can query across entities:

GraphQL

```
query {
 products {
  items {
   name
   price
   category {
    name
   }
  }
 }
}
```

The cardinality property indicates whether the relationship returns one item or many. Use one for many-to-one relationships (product to category) and many for one-to-many relationships (category to products).

Understand REST endpoint generation

When you define an entity, DAB automatically creates REST endpoints following standard conventions. Each entity gets a base URL path, and HTTP verbs map to CRUD operations.

For an entity named Product with the default configuration:

HTTP Method	URL Pattern	Operation
GET	/api/Product	List all products (paginated)
GET	/api/Product/id/123	Get product with ID 123
POST	/api/Product	Create a new product
PUT	/api/Product/id/123	Update or create product 123
PATCH	/api/Product/id/123	Partially update product 123
DELETE	/api/Product/id/123	Delete product 123

The /id/ segment in URLs indicates a primary key lookup. For composite keys, chain multiple key segments: /api/OrderDetail/orderId/100/productId/50.

Customize REST endpoint paths

You can modify the default URL patterns through entity configuration. The rest section controls REST-specific behavior:

JSON

```
"Product": {
 "source": { "object": "dbo.Products", "type": "table" },
 "rest": {
  "enabled": true,
  "path": "/products",
  "methods": {
   "get": true,
   "post": true,
   "put": false,
   "patch": true,
   "delete": true
  }
```

```
  },
  "permissions": [...]
}
```

The path property changes the URL from /api/Product to /api/products. This setting is useful when you want lowercase, plural endpoint names that differ from your entity names.

The methods object lets you enable or disable specific HTTP verbs. In this example, PUT is disabled, which prevents full record replacement while still allowing PATCH for partial updates. This configuration pattern protects against accidental data loss from clients that might send incomplete records.

Tip

Disabling methods at the endpoint level provides defense in depth. Even if permissions allow an action, disabling the method prevents it entirely.

Set up GraphQL endpoint configuration

GraphQL operates through a single endpoint, typically at /graphql. All queries, mutations, and subscriptions go through this endpoint, with the request body specifying the operation.

JSON

```
"runtime": {
  "graphql": {
    "enabled": true,
    "path": "/graphql",
    "allow-introspection": true,
    "depth-limit": 8,
    "multiple-mutations": {
      "create": { "enabled": true }
    }
  }
}
```

The allow-introspection setting controls whether clients can query the GraphQL schema itself. During development, introspection helps tools like

GraphiQL and Apollo Studio understand your API. In production, consider disabling it to hide implementation details.

The depth-limit setting prevents overly complex nested queries that could strain your database. A depth of 8 allows reasonable relationship traversal while blocking potential abuse through deeply nested queries.

Configure GraphQL-specific entity settings

Each entity can have GraphQL-specific configuration that differs from REST:

JSON

```
"Product": {
 "source": { "object": "dbo.Products", "type": "table" },
 "graphql": {
  "enabled": true,
  "type": {
   "singular": "product",
   "plural": "products"
  },
  "operation": "query"
 },
 "rest": {
  "enabled": true
 },
 "permissions": [...]
}
```

The type property customizes the GraphQL type names. By default, DAB uses the entity name, but you can specify different singular and plural forms.

The operation property determines where mutations appear. Use query for read-only entities (like views) or mutation for entities that support write operations.

Next unit: Expose database objects, stored procedures, and views

Expose database objects, stored procedures, and views

Completed100 XP

- **12 minutes**

Choose your preferred content format

Beyond simple table mappings, Data API Builder can expose views and stored procedures as API endpoints. These database objects let you encapsulate complex business logic, aggregate data from multiple tables, or implement operations that go beyond standard CRUD patterns.

Views provide read-only access to computed or joined data, while stored procedures enable custom operations with input parameters and multiple result sets. GraphQL relationships add another dimension, allowing clients to traverse connections between entities in single queries.

Expose views as read-only entities

Database views aggregate and transform data without duplicating storage. When you expose a view through Data API Builder, clients access the computed results through familiar REST and GraphQL patterns.

Consider a view that combines product information with inventory status:

SQL

```
CREATE VIEW dbo.ProductInventory AS
SELECT
  p.ProductID,
  p.ProductName,
  p.UnitPrice,
  p.UnitsInStock,
  c.CategoryName,
  CASE
    WHEN p.UnitsInStock = 0 THEN 'Out of Stock'
    WHEN p.UnitsInStock < 10 THEN 'Low Stock'
    ELSE 'Available'
  END AS StockStatus
```

FROM dbo.Products p

INNER JOIN dbo.Categories c ON p.CategoryID = c.CategoryID;

Expose this view in your configuration:

JSON

```
"ProductInventory": {
 "source": {
  "object": "dbo.ProductInventory",
  "type": "view",
  "key-fields": ["ProductID"]
 },
 "rest": {
  "enabled": true,
  "path": "/inventory"
 },
 "graphql": {
  "enabled": true,
  "operation": "query"
 },
 "permissions": [
  {
   "role": "anonymous",
   "actions": ["read"]
  }
 ]
}
```

The key-fields property is required for views because DAB can't automatically detect primary keys. Specify the columns that uniquely identify each row.

Setting operation to query in the GraphQL section indicates this entity supports only read operations. DAB won't generate mutations for this entity.

Note

Views can be updatable in SQL Server if they meet certain criteria. However, exposing updatable views through DAB requires careful consideration of the underlying table permissions and constraints.

Configure stored procedures for custom operations

Stored procedures provide the most flexibility for custom database operations. They can accept parameters, perform complex logic, and return multiple result sets.

SQL

```
CREATE PROCEDURE dbo.GetProductsByPriceRange
  @MinPrice DECIMAL(10,2),
  @MaxPrice DECIMAL(10,2)
AS
BEGIN
  SELECT ProductID, ProductName, UnitPrice, UnitsInStock
  FROM dbo.Products
  WHERE UnitPrice BETWEEN @MinPrice AND @MaxPrice
  ORDER BY UnitPrice;
END;
```

Configure the stored procedure as an entity:

JSON

```
"GetProductsByPriceRange": {
 "source": {
  "object": "dbo.GetProductsByPriceRange",
  "type": "stored-procedure",
  "parameters": {
   "MinPrice": "minPrice",
   "MaxPrice": "maxPrice"
  }
```

```
 },
 "rest": {
  "enabled": true,
  "path": "/products/by-price-range",
  "methods": {
   "get": true,
   "post": true
  }
 },
 "graphql": {
  "enabled": true,
  "operation": "query"
 },
 "permissions": [
  {
   "role": "authenticated",
   "actions": ["execute"]
  }
 ]
}
```

The parameters object maps stored procedure parameters to API parameter names. REST clients call the procedure using query parameters or request body:

```
HTTP

GET /api/products/by-price-range?minPrice=10&maxPrice=50

HTTP

POST /api/products/by-price-range

Content-Type: application/json
```

```
{
 "minPrice": 10,
 "maxPrice": 50
}
```

GraphQL clients use the procedure through a generated query:

```
GraphQL
query {
 executeGetProductsByPriceRange(minPrice: 10, maxPrice: 50) {
  ProductID
  ProductName
  UnitPrice
 }
}
```

Expose stored procedures that modify data

For stored procedures that insert, update, or delete data, configure them with mutation operations:

```
SQL
CREATE PROCEDURE dbo.CreateOrder
  @CustomerID INT,
  @ProductID INT,
  @Quantity INT
AS
BEGIN
  DECLARE @OrderID INT;

  INSERT INTO dbo.Orders (CustomerID, OrderDate, Status)
  VALUES (@CustomerID, GETDATE(), 'Pending');
```

```
    SET @OrderID = SCOPE_IDENTITY();

    INSERT INTO dbo.OrderDetails (OrderID, ProductID, Quantity, UnitPrice)
    SELECT @OrderID, @ProductID, @Quantity, UnitPrice
    FROM dbo.Products
    WHERE ProductID = @ProductID;

    UPDATE dbo.Products
    SET UnitsInStock = UnitsInStock - @Quantity
    WHERE ProductID = @ProductID;

    SELECT @OrderID AS OrderID;
END;
```

```
JSON
"CreateOrder": {
  "source": {
    "object": "dbo.CreateOrder",
    "type": "stored-procedure",
    "parameters": {
      "CustomerID": "customerId",
      "ProductID": "productId",
      "Quantity": "quantity"
    }
  },
  "rest": {
    "enabled": true,
    "methods": {
      "post": true
```

```
    }
  },
  "graphql": {
    "enabled": true,
    "operation": "mutation"
  },
  "permissions": [
    {
      "role": "authenticated",
      "actions": ["execute"]
    }
  ]
}
```

Setting operation to mutation places this procedure under GraphQL mutations rather than queries, indicating it modifies data.

Important

Stored procedures execute with the permissions of the database connection, not the calling user. Implement appropriate validation and authorization logic within the procedure or use session context for row-level security.

Define GraphQL relationships between entities

GraphQL relationships enable clients to traverse connections between entities without multiple round trips. DAB supports both one-to-one and one-to-many relationships.

JSON

```
"entities": {
  "Order": {
    "source": { "object": "dbo.Orders", "type": "table" },
    "relationships": {
      "customer": {
        "cardinality": "one",
```

```
      "target.entity": "Customer",
      "source.fields": ["CustomerID"],
      "target.fields": ["CustomerID"]
    },
    "orderDetails": {
      "cardinality": "many",
      "target.entity": "OrderDetail",
      "source.fields": ["OrderID"],
      "target.fields": ["OrderID"]
    }
  },
  "permissions": [...]
},
"OrderDetail": {
  "source": { "object": "dbo.OrderDetails", "type": "table" },
  "relationships": {
    "order": {
      "cardinality": "one",
      "target.entity": "Order",
      "source.fields": ["OrderID"],
      "target.fields": ["OrderID"]
    },
    "product": {
      "cardinality": "one",
      "target.entity": "Product",
      "source.fields": ["ProductID"],
      "target.fields": ["ProductID"]
    }
```

```
  },
  "permissions": [...]
 }
}
```

With these relationships, GraphQL clients can build rich queries:

GraphQL

```
query {
 orders(first: 10) {
  items {
   orderDate
   customer {
    companyName
    contactName
   }
   orderDetails {
    items {
     quantity
     product {
      productName
      unitPrice
     }
    }
   }
  }
 }
}
```

This single query retrieves orders with customer information and line items including product details. Without relationships, clients would need multiple separate requests.

Handle complex relationship scenarios

Some database designs use junction tables for many-to-many relationships. DAB handles these through explicit relationship chains.

For a products-to-suppliers many-to-many relationship through a ProductSuppliers junction table:

JSON

```
"Product": {
 "relationships": {
  "productSuppliers": {
   "cardinality": "many",
   "target.entity": "ProductSupplier",
   "source.fields": ["ProductID"],
   "target.fields": ["ProductID"]
  }
 }
},
"ProductSupplier": {
 "source": { "object": "dbo.ProductSuppliers", "type": "table" },
 "relationships": {
  "supplier": {
   "cardinality": "one",
   "target.entity": "Supplier",
   "source.fields": ["SupplierID"],
   "target.fields": ["SupplierID"]
  }
 }
}
```

Clients traverse the relationship chain: Product → ProductSupplier → Supplier. While this requires one extra level in queries, it accurately represents the database structure and maintains referential integrity.

Next unit: Explore deployment options for data API builder

Explore deployment options for data API builder

Completed100 XP

- **10 minutes**

Choose your preferred content format

Moving Data API Builder from local development to Azure production environments requires choosing the right hosting platform and configuring deployment automation. Azure offers several options for running DAB, each with different characteristics for scalability, management overhead, and integration with other Azure services.

Your deployment strategy affects not only where DAB runs but also how you manage configuration, handle secrets, and scale to meet demand. Understanding these deployment options helps you select the approach that best fits your application architecture and operational requirements.

Deploy to Azure Container Apps

Azure Container Apps provides a serverless container platform that handles infrastructure management automatically. It's well-suited for Data API Builder because it can scale from zero when idle and automatically scale up based on request volume.

To deploy DAB to Container Apps, first create a container image with your configuration:

Dockerfile

```
FROM mcr.microsoft.com/azure-databases/data-api-builder:latest

COPY dab-config.json /App/dab-config.json

ENV DATABASE_CONNECTION_STRING=""
```

ENV ASPNETCORE_URLS="http://+:5000"

EXPOSE 5000

This Dockerfile starts from the official DAB image and adds your configuration file. Environment variables handle the connection string and port configuration.

Deploy using the Azure CLI:

Bash

```
az containerapp create \
 --name dab-api \
 --resource-group myResourceGroup \
 --environment myContainerAppEnv \
 --image myregistry.azurecr.io/dab-api:v1 \
 --target-port 5000 \
 --ingress external \
 --secrets database-conn="<connection-string>" \
 --env-vars DATABASE_CONNECTION_STRING=secretref:database-conn
```

Container Apps integrates with Azure Key Vault for secret management. Reference secrets in your environment variables rather than storing credentials in configuration files or container images.

Tip

Enable Azure Container Apps managed identity and grant it access to your Azure SQL Database. This approach eliminates stored credentials entirely by using Microsoft Entra authentication.

Deploy to Azure App Service

Azure App Service offers a mature platform with built-in deployment slots, custom domains, and comprehensive monitoring. For teams already using App Service, adding DAB follows familiar patterns.

Configure App Service to run the DAB container:

Bash

```
az webapp create \
```

**--name dab-api **

**--resource-group myResourceGroup **

**--plan myAppServicePlan **

--deployment-container-image-name mcr.microsoft.com/azure-databases/data-api-builder:latest

Set environment variables through App Service configuration:

Bash

**az webapp config appsettings set **

**--name dab-api **

**--resource-group myResourceGroup **

--settings DATABASE_CONNECTION_STRING="@Microsoft.KeyVault(SecretUri=https://myvault.vault.azure.net/secrets/db-connection)"

The Key Vault reference syntax lets App Service pull secrets at runtime without exposing them in configuration. Enable system-assigned managed identity on the App Service and grant it access to Key Vault.

Deploy to Azure Static Web Apps

Azure Static Web Apps includes Data API Builder as a built-in feature called database connections. This integration provides the simplest deployment experience when your API complements a static frontend.

Link your database through the Azure portal or CLI:

Bash

**az staticwebapp dbconnection create **

**--name myStaticWebApp **

**--resource-group myResourceGroup **

**--db-type mssql **

--db-resource-id /subscriptions/.../servers/myserver/databases/mydb

The staticwebapps.database.config.json file in your repository defines entity configurations using the same structure as standalone DAB:

JSON

{

```
 "data-source": {
  "database-type": "mssql"
 },
 "entities": {
  "Product": {
   "source": "dbo.Products",
   "permissions": [
    { "role": "anonymous", "actions": ["read"] }
   ]
  }
 }
}
```

Static Web Apps automatically handles authentication integration with its built-in auth providers. API routes appear under /.auth/ and /data-api/ paths by default.

Note

Database connections in Static Web Apps have some limitations compared to standalone DAB, including restricted stored procedure support. Review the documentation for current feature availability.

Implement CI/CD deployment pipelines

Automated deployment pipelines ensure consistent configuration across environments and enable rapid iteration. GitHub Actions provides a straightforward approach for DAB deployments.

Create a workflow file at .github/workflows/deploy-dab.yml:

YAML

```
name: Deploy Data API Builder

on:
 push:
  branches: [main]
```

```
    paths:
      - 'dab-config*.json'
      - 'Dockerfile'

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Login to Azure Container Registry
        uses: azure/docker-login@v1
        with:
          login-server: ${{ secrets.ACR_LOGIN_SERVER }}
          username: ${{ secrets.ACR_USERNAME }}
          password: ${{ secrets.ACR_PASSWORD }}

      - name: Build and push container
        run: |
          docker build -t ${{ secrets.ACR_LOGIN_SERVER }}/dab-api:${{ github.sha }}
.
          docker push ${{ secrets.ACR_LOGIN_SERVER }}/dab-api:${{ github.sha }}

      - name: Deploy to Container Apps
        uses: azure/container-apps-deploy-action@v1
        with:
          resourceGroup: myResourceGroup
          containerAppName: dab-api
```

imageToDeploy: ${{ secrets.ACR_LOGIN_SERVER }}/dab-api:${{ github.sha }}

This workflow triggers on changes to configuration files or the Dockerfile. It builds a new container image, pushes it to Azure Container Registry, and updates the Container App.

Configure environment-specific deployments

Production deployments require careful separation between environments. Use environment-specific configuration files combined with deployment variables.

Structure your repository with multiple configuration files:

/

├── dab-config.json # Base configuration (shared settings)

├── dab-config.development.json

├── dab-config.staging.json

├── dab-config.production.json

└── Dockerfile

Modify your Dockerfile to accept a build argument:

Dockerfile

FROM mcr.microsoft.com/azure-databases/data-api-builder:latest

ARG ENVIRONMENT=production

COPY dab-config.json /App/dab-config.json

COPY dab-config.${ENVIRONMENT}.json /App/dab-config.${ENVIRONMENT}.json

ENV DAB_ENVIRONMENT=${ENVIRONMENT}

Build environment-specific images:

Bash

docker build --build-arg ENVIRONMENT=production -t dab-api:production .

Important

Never include actual connection strings or secrets in configuration files committed to source control. Use environment variable references (@env()) and manage secrets through Azure Key Vault or your deployment pipeline's secret management.

Monitor deployment health

After deployment, configure health checks to verify DAB is operating correctly. Container Apps and App Service support health probes:

JSON

```
{
 "probes": [
  {
   "type": "liveness",
   "httpGet": {
    "path": "/",
    "port": 5000
   },
   "periodSeconds": 30
  },
  {
   "type": "readiness",
   "httpGet": {
    "path": "/api/Product?$first=1",
    "port": 5000
   },
   "periodSeconds": 10
  }
 ]
}
```

The liveness probe checks that DAB is responding. The readiness probe verifies database connectivity by making an actual API request. Configure your orchestration platform to restart unhealthy instances automatically.

Next unit: Recommend Azure monitor configurations

Recommend Azure monitor configurations

Completed100 XP

- **10 minutes**

Choose your preferred content format

Running Data API Builder in production requires monitoring to understand performance, diagnose issues, and optimize resource usage. Azure Monitor provides the comprehensive observability platform that integrates with DAB through Application Insights and Log Analytics.

Effective monitoring goes beyond simple uptime checks. You need visibility into request latencies, error rates, database query performance, and resource consumption patterns. This type of data helps you identify bottlenecks before they affect users and provides data for capacity planning.

Configure Application Insights integration

Application Insights captures detailed information from your Data API Builder instance, including request traces, dependency calls to your database, and exception details.

Enable Application Insights by setting the connection string as an environment variable:

Bash

```
APPLICATIONINSIGHTS_CONNECTION_STRING="InstrumentationKey=your-key;IngestionEndpoint=https://region.in.applicationinsights.azure.com/"
```

For Azure Container Apps or App Service deployments, configure this through application settings. DAB automatically detects the connection string and begins sending telemetry.

Once enabled, Application Insights tracks:

- **Request telemetry - Every REST and GraphQL request with timing, status code, and response size**

- **Dependency telemetry - Database queries with execution time and success/failure status**
- **Exception telemetry - Unhandled errors with stack traces**
- **Custom metrics - Cache hit rates, connection pool usage**

Tip

Set a sampling rate for high-volume production deployments to control costs while maintaining visibility. Application Insights adaptive sampling automatically adjusts based on traffic volume.

Create custom dashboards for API monitoring

Application Insights data powers custom Azure dashboards that display key performance indicators for your API.

Create a dashboard with these essential visualizations:

Request metrics:

Kusto

```
requests
| where timestamp > ago(24h)
| summarize
  RequestCount = count(),
  AvgDuration = avg(duration),
  P95Duration = percentile(duration, 95),
  FailureRate = countif(success == false) * 100.0 / count()
| project RequestCount, AvgDuration, P95Duration, FailureRate
```

This query summarizes 24-hour request volume, average response time, 95th percentile latency, and failure rate. These metrics quickly indicate API health.

Top slow endpoints:

Kusto

```
requests
| where timestamp > ago(1h)
| summarize
  AvgDuration = avg(duration),
```

RequestCount = count()

by name

| top 10 by AvgDuration desc

| project name, AvgDuration, RequestCount

Identifying slow endpoints helps prioritize optimization efforts. Slow queries might indicate missing indexes, inefficient entity configurations, or database contention.

Set up Log Analytics for detailed diagnostics

Log Analytics provides a central repository for logs from all Azure resources. Configure your DAB deployment to send logs to a Log Analytics workspace.

For Container Apps, enable logging through the environment configuration:

Bash

```
az containerapp env update \
 --name myContainerAppEnv \
 --resource-group myResourceGroup \
 --logs-workspace-id <workspace-id>
```

Query DAB container logs to investigate issues:

Kusto

```
ContainerAppConsoleLogs_CL
| where ContainerName_s == "dab-api"
| where Log_s contains "error" or Log_s contains "exception"
| project TimeGenerated, Log_s
| order by TimeGenerated desc
```

Combine container logs with Application Insights telemetry for complete request traces. Correlate errors with specific requests using operation IDs that flow through both data sources.

Configure alert rules for proactive monitoring

Alert rules notify your team when API behavior deviates from normal patterns. Set up alerts for critical conditions that require immediate attention.

High error rate alert:

Bash

```
az monitor metrics alert create \
 --name "DAB-HighErrorRate" \
 --resource-group myResourceGroup \
 --scopes "/subscriptions/.../components/myAppInsights" \
 --condition "avg requests/failed > 5" \
 --window-size 5m \
 --evaluation-frequency 1m \
 --action-group myActionGroup
```

This alert triggers when the failure rate exceeds 5% over a 5-minute window. Action groups define notification channels like email, SMS, or webhooks to your incident management system.

Response time degradation alert:

Kusto

```
requests
| where timestamp > ago(5m)
| summarize P95 = percentile(duration, 95) by bin(timestamp, 1m)
| where P95 > 2000
```

Create a log-based alert that fires when 95th percentile latency exceeds 2 seconds. This catches performance degradation even when requests technically succeed.

Important

Configure alerts thoughtfully to avoid alert fatigue. Start with high-severity conditions and refine thresholds based on normal operating patterns.

Monitor database query performance

Application Insights dependency tracking captures database queries issued by DAB. Use this telemetry to identify slow or frequently executed queries.

Kusto

```
dependencies
| where timestamp > ago(1h)
```

| where type == "SQL"

| summarize

CallCount = count(),

AvgDuration = avg(duration),

FailureCount = countif(success == false)

by data

| top 20 by AvgDuration desc

The data field contains the SQL query text. Review slow queries to determine if indexes would help, if entity configurations need optimization, or if certain operations should use stored procedures instead.

Compare database metrics across time periods to identify trends:

Kusto

dependencies

| where type == "SQL"

| summarize

AvgDuration = avg(duration)

by bin(timestamp, 1h)

| render timechart

Gradual increases in query duration might indicate growing data volumes, index fragmentation, or changing query patterns that need attention.

Implement distributed tracing

When DAB serves requests from multiple client applications, distributed tracing connects the complete request flow from client through API to database.

Enable correlation by including trace context headers in client requests:

HTTP

GET /api/Product HTTP/1.1

traceparent: 00-0af7651916cd43dd8448eb211c80319c-b7ad6b7169203331-01

Application Insights correlates these traces, allowing you to see end-to-end latency breakdown in the Transaction Search view. This visibility is valuable for diagnosing intermittent issues that span multiple services.

For GraphQL queries that resolve multiple entities, traces show the timing of each database call, helping identify which relationships or entities contribute most to response time.

Review resource utilization metrics

Beyond application telemetry, monitor the underlying compute resources running DAB. Container Apps and App Service provide resource metrics through Azure Monitor.

Key metrics to track:

- **CPU percentage - Sustained high CPU indicates compute-bound operations or need for scaling**
- **Memory percentage - Memory pressure can cause container restarts or degraded performance**
- **Network bytes in/out - Unusual network patterns might indicate large result sets or denial-of-service attempts**
- **Replica count - For scaled deployments, verify autoscaling responds appropriately to load**

Set up autoscaling rules based on these metrics to maintain responsive APIs during traffic spikes while controlling costs during quiet periods.

Next unit: Handle changes with event-driven patterns

Handle changes with event-driven patterns

Completed100 XP

- 12 minutes

Choose your preferred content format

Modern applications often need to respond to database changes in real time. When a customer places an order, updates their profile, or when inventory levels change, downstream systems need notification. SQL Server and Azure SQL provide several mechanisms for capturing and streaming these changes, each with different characteristics for latency, throughput, and implementation complexity.

Data API Builder integrates with these change mechanisms through Azure Functions SQL trigger bindings. Understanding when to use each approach helps you build responsive, event-driven architectures that react to data changes efficiently.

Understand change capture mechanisms

SQL platforms offer multiple approaches for tracking data changes, each suited to different scenarios:

- Change Data Capture (CDC) records insert, update, and delete operations to special change tables. It captures the complete before-and-after state of changed rows, making it ideal for audit requirements and data synchronization.
- Change Tracking provides a lighter-weight alternative that records which rows changed without capturing the actual values. Applications query change information to determine what data needs synchronization.
- Change Event Streaming (CES) is a newer feature that pushes changes to event streams without polling. It integrates directly with Azure Event Hubs for high-throughput scenarios.

Feature	CDC	Change Tracking	CES
Captures changed values	Yes	No	Yes
Polling required	Yes	Yes	No
Historical data	Yes	Limited	Stream
Setup complexity	Moderate	Low	Modera

Enable Change Data Capture for audit scenarios

CDC is appropriate when you need a complete history of changes for compliance or debugging. Enable it on specific tables:

SQL

```
-- Enable CDC at the database level

EXEC sys.sp_cdc_enable_db;


-- Enable CDC on a specific table

EXEC sys.sp_cdc_enable_table
```

```
  @source_schema = 'dbo',
  @source_name = 'Orders',
  @role_name = NULL,
  @capture_instance = 'dbo_Orders',
  @supports_net_changes = 1;
```

Once enabled, SQL Server creates change tables that store historical records. Query these tables to retrieve changes within a time range:

SQL

```
DECLARE @from_lsn binary(10), @to_lsn binary(10);
SET @from_lsn = sys.fn_cdc_get_min_lsn('dbo_Orders');
SET @to_lsn = sys.fn_cdc_get_max_lsn();

SELECT *
FROM cdc.fn_cdc_get_all_changes_dbo_Orders(@from_lsn, @to_lsn, 'all');
```

This query returns all changes between the minimum and maximum log sequence numbers, including the operation type (__$operation) indicating insert, update, or delete.

Note

CDC requires SQL Server Agent to be running. The capture and cleanup jobs run on schedules you can configure. Monitor job performance in high-transaction environments.

Configure Azure Functions SQL trigger binding

Azure Functions SQL trigger binding provides the most direct integration for responding to database changes. The trigger monitors tables using Change Tracking and invokes your function when rows change.

First, enable Change Tracking on your database and table:

SQL

```
-- Enable Change Tracking at database level
ALTER DATABASE [YourDatabase]
SET CHANGE_TRACKING = ON
```

```
(CHANGE_RETENTION = 2 DAYS, AUTO_CLEANUP = ON);

-- Enable Change Tracking on the table
ALTER TABLE dbo.Orders
ENABLE CHANGE_TRACKING;
```

Create an Azure Function with the SQL trigger:

C#

```
[FunctionName("OrderChangedTrigger")]
public static void Run(
    [SqlTrigger("[dbo].[Orders]", "SqlConnectionString")]
    IReadOnlyList<SqlChange<Order>> changes,
    ILogger log)
{
    foreach (var change in changes)
    {
        log.LogInformation($"Change operation: {change.Operation}");
        log.LogInformation($"Order ID: {change.Item.OrderId}");

        if (change.Operation == SqlChangeOperation.Update)
        {
            // Process order update
            ProcessOrderUpdate(change.Item);
        }
    }
}

public class Order
{
```

```
    public int OrderId { get; set; }
    public int CustomerId { get; set; }
    public DateTime OrderDate { get; set; }
    public string Status { get; set; }
}
```

The SqlChange<T> type wraps each changed row with its operation type. Your function receives batches of changes, which helps efficiency when multiple rows change simultaneously.

Configure the connection string in your function app settings:

JSON

```
{
  "Values": {
    "SqlConnectionString": "@Microsoft.KeyVault(SecretUri=https://myvault.vault.azure.net/secrets/sql-connection)"
  }
}
```

Important

The SQL trigger binding requires Change Tracking, not CDC. If you need CDC's historical capabilities alongside real-time triggers, enable both features on your tables.

Stream changes with Change Event Streaming

For high-throughput scenarios where polling latency is unacceptable, Change Event Streaming pushes changes directly to Azure Event Hubs. This approach eliminates polling delays and scales to millions of events per second.

Configure CES on your database:

SQL

```
-- Create an event streaming group
CREATE EVENT STREAMING GROUP MyOrderStream
WITH (
    TARGET_TYPE = 'eventhub',
```

```
  TARGET_NAME = 'orders-eventhub',
  TARGET_CONNECTION = '<event-hub-connection-string>'
);

-- Add tables to the streaming group
ALTER EVENT STREAMING GROUP MyOrderStream
ADD TABLE dbo.Orders;
```

Changes flow to Event Hubs immediately as transactions commit. Downstream consumers process events using Azure Functions Event Hubs triggers, Stream Analytics, or custom applications.

The Event Hubs message contains:

- Operation type (insert, update, delete)
- Timestamp
- Full row data for inserts and updates
- Key values for deletes

Choose the right change pattern

Selecting the appropriate change mechanism depends on your requirements:

Use **CDC** when:

- You need complete before-and-after values
- Compliance requires historical change records
- Batch synchronization is acceptable

Use **change tracking** with Functions when:

- You need real-time response to changes
- You only need current values, not history
- You're building event-driven microservices

Use **change event streaming** when:

- You need sub-second latency
- High transaction volumes require streaming
- You're building real-time analytics pipelines

Consider combining approaches. Use CDC for audit and compliance while using Functions triggers for real-time notifications. This pattern provides both historical records and immediate responses.

Next unit: Exercise - Configure data API builder for a product catalog

Summary

Completed100 XP

- 2 minutes

In this module, you learned how to:

- Create Data API Builder configuration files with data source connections and runtime settings
- Define entities for tables with field mappings, caching, pagination, and filtering capabilities
- Configure REST and GraphQL endpoints with customized paths and enabled operations
- Expose views as read-only entities and stored procedures for custom operations
- Explore deployment options for Data API Builder, including Azure Container Apps, App Service, and Static Web Apps
- Set up Azure Monitor with Application Insights for request telemetry and database performance tracking
- Implement change capture patterns using CDC, Change Tracking, Azure Functions triggers, and Change Event Streaming

Key takeaways

- Data API Builder eliminates custom API code by generating REST and GraphQL endpoints from a single JSON configuration file
- Entity relationships enable GraphQL clients to traverse connections between tables in single queries, reducing round trips
- Azure Container Apps provides serverless hosting with automatic scaling and managed identity support for production deployments

- Application Insights integration gives visibility into request performance, database queries, and error rates
- Change capture mechanisms enable event-driven architectures that respond to database modifications in real time

Learn more

- Data API Builder documentation
- Data API Builder configuration reference
- Deploy Data API Builder to Azure Container Apps
- Azure Functions SQL trigger binding
- Change Data Capture in SQL Server
- Application Insights overview

Implement AI capabilities in database solutions

Design and implement models and embeddings with SQL

Integrate AI models with Azure SQL Database using external models and built-in AI functions. Design effective embedding strategies and implement maintenance patterns to keep embeddings aligned with source data

Introduction

Completed100 XP

- 3 minutes

Choose your preferred content format

Building intelligent applications with Azure SQL Database involves more than storing and querying relational data. Developers need to integrate AI models, generate embeddings from text, and perform vector searches to enable features like semantic search and Retrieval Augmented Generation (RAG). Azure SQL Database and Fabric SQL database support these capabilities through external models, the vector data type, and built-in AI functions.

Together, external models and embedding workflows allow developers to add AI capabilities directly inside the database. This approach keeps AI processing close to the data and reduces the need to move information between systems for tasks like similarity search or content retrieval.

Imagine a retail development team that builds applications using Azure SQL Database to manage product catalogs, customer reviews, and support documentation. Their work includes generating embeddings from product descriptions, enabling semantic search across support articles, and keeping embeddings current as content changes. By using external models and built-in AI functions in Azure SQL Database, the team can design, generate, and maintain embeddings while working entirely within Transact-SQL.

After completing this module, you'll be able to:

- Evaluate AI models for SQL database workloads based on capabilities and performance requirements.
- Create and manage external models to reference AI endpoints from Transact-SQL.
- Design embeddings with appropriate chunking strategies.
- Generate and store embeddings using built-in SQL AI functions.
- Choose maintenance approaches to keep embeddings aligned with source data.

Note

We recognize that different people like to learn in different ways. You can choose to complete this module in video-based format or you can read the content as text and images. The text contains greater detail than the videos, so in some cases you might want to refer to it as supplemental material to the video presentation.

Understand and evaluate models for SQL database workloads

Completed100 XP

- 8 minutes

Choose your preferred content format

Large language models (LLMs) enable applications to generate responses, summarize information, and reason over user input. Their usefulness increases when they can access application data stored in a database.

Azure SQL Database and Fabric SQL database support building intelligent applications by integrating AI capabilities such as embeddings, vector data types,

and vector search. These features allow models to work directly with relational data, enabling common patterns like semantic search and Retrieval Augmented Generation (RAG).

Before integrating a model into a SQL-based solution, it's important to understand how different models behave and how their characteristics affect application design.

Identify model characteristics for SQL database workloads

Models differ in their capabilities, performance characteristics, and supported input and output formats. When evaluating models for use with Azure SQL Database or Fabric SQL database, consider the following factors.

Modalities
Some models process only text, while others support other inputs such as images or structured data. The required modality depends on the type of data stored in the database and the intended application scenario.

Language support
Multilingual support is important when applications serve users across regions or when stored content spans multiple languages.

Model size and capacity
Larger models typically provide stronger reasoning and more nuanced output, but they also consume more tokens and can introduce higher latency and cost. Smaller models might be more suitable for focused tasks such as embedding generation.

Structured output
Models that can produce structured output, such as JSON, are easier to integrate into SQL-based workflows where responses must be processed programmatically.

These characteristics influence whether a model is a good fit for generating embeddings, supporting RAG patterns, or enabling conversational access to database content.

Describe how models interact with Azure SQL data

Azure SQL Database and Fabric SQL database support intelligent application patterns by combining relational storage with AI features such as vector data types and vector functions.

A common pattern is Retrieval Augmented Generation (RAG), where relevant data is retrieved from the database and supplied to a model as context before generating a response. This step allows responses to be grounded in application data rather than relying only on a model’s pretrained knowledge.

Several concepts affect how models interact with database data:

- **Tokens**, which are the units models use to process text
- **Embeddings**, which represent data as vectors
- **Vector search**, which compares embeddings to identify semantic similarity

Because vectors live alongside relational data in Azure SQL Database, you can combine vector similarity search with any standard SQL capability in a single query. For example, you might narrow vector search results with a WHERE clause, join them to related tables, or blend vector cosine rankings with full-text BM25 scores. This combination of vector search with regular SQL operations is known as **hybrid search**. Instead of sending requests to a separate search service and reconciling the results, you query one database that handles both semantic similarity and relational filtering together.

Understanding these concepts helps you design SQL-based applications that effectively use AI capabilities while managing performance and cost.

Explain how tokens affect cost and design

Models don't process text as raw characters. Instead, they break text into tokens. Tokens are small chunks that might be words, parts of words, or punctuation. For example, the word "hamburger" might become three tokens: "ham", "bur", and "ger", while a common word like "the" is typically a single token.

Token counts matter for two reasons. First, models have input limits. A model might accept a maximum of 8,000 or 128,000 tokens in a single request. This limit constrains how much database content you can include as context in a RAG pattern. Second, model providers typically charge based on tokens processed. More tokens mean higher costs, so efficient text handling directly affects operating expenses.

When you're designing SQL-based AI solutions, token limits influence how you chunk content for embeddings and how much context you can pass to a model during generation.

Because models impose token limits and differ in how embeddings are generated, these characteristics affect schema design, chunking strategies, and query behavior.

Explore models with Microsoft Foundry

Microsoft Foundry Models provides a catalog of AI models that can be used with Azure services. The catalog includes models that support tasks such as text processing, embedding generation, reasoning, and multimodal input.

For SQL database workloads, Foundry helps you evaluate which models are appropriate for integration with Azure SQL Database or Fabric SQL database. Supported input types, language coverage, and deployment options all influence which model fits your scenario.

The model catalog exposes information such as model capabilities, benchmarks, version details, and lifecycle status. This information helps developers understand performance and operational constraints before connecting a model to database workflows.

Using Foundry during design helps ensure that the selected model aligns with SQL-based application requirements and can be integrated predictably with relational data.

Select a model for your solution

Selecting a model is a design decision that affects performance, cost, and maintainability. When choosing a model for use with Azure SQL Database or Fabric SQL database, consider:

- The type and format of data stored in the database
- Performance and scalability requirements
- Language or modality requirements
- Deployment and lifecycle considerations

Understanding these trade-offs helps ensure that AI capabilities are integrated into SQL database solutions in a way that's predictable, scalable, and aligned with application goals.

Key takeaways

Models differ in modalities, language support, size, and structured output capabilities, and these differences affect how they integrate with SQL database workloads. RAG retrieves relevant database content and supplies it to a model as context, while tokens determine how input and output text is processed. Embeddings represent data as vectors that enable semantic similarity comparisons. Microsoft Foundry Models provides a catalog for evaluating and selecting models that align with your SQL-based application requirements.

Create and manage external models in SQL

Completed100 XP

- 7 minutes

Choose your preferred content format

External models provide a way to reference AI models from within SQL Server and Azure SQL using Transact-SQL. An external model represents a connection to a model endpoint and defines how SQL built-in AI functions invoke that model.

By creating external models, you make AI capabilities available directly inside the database engine without embedding model-specific logic in application code.

What is an external model

An external model is a database object that stores metadata about an AI model endpoint. This metadata includes information such as the endpoint URL, authentication details, and model-specific configuration.

External models aren't models that run inside SQL Server. Instead, they act as a managed reference that allows SQL Server to call an external AI service when executing AI functions.

Once created, external models can be reused across queries and workloads, providing a consistent and centrally managed way to access AI capabilities from SQL.

Create an external model

You create an external model by using the CREATE EXTERNAL MODEL statement. This statement defines the model name and associates it with an external endpoint.

Creating the external model doesn't invoke the model or generate output. It establishes a reusable definition that AI functions such as embedding generation or other AI-assisted operations can later reference.

Because external models are database objects, they can be managed using standard SQL practices, including naming conventions, permissions, and deployment through scripts.

Modify an external model

External models can change over time as endpoints, credentials, or configuration settings are updated. To reflect these changes, you can use the ALTER EXTERNAL MODEL statement.

Altering an external model updates its metadata without requiring dependent queries or functions to be rewritten. This update allows you to adjust model configuration centrally while keeping existing SQL logic intact.

Managing changes at the external model level helps reduce coupling between application logic and AI service configuration.

Use external models with AI functions

SQL Server provides built-in AI functions that reference an external model to perform AI-related operations. These functions handle tasks such as generating embeddings by calling the external model endpoint defined in the database.

AI functions use the external model definition to determine which endpoint to call and how to authenticate the request. This separation allows Transact-SQL code to focus on data processing rather than connection or credential details.

Before creating an external model, you need a database scoped credential that stores the authentication details for the AI endpoint. Azure OpenAI endpoints support two authentication options:

- **Managed Identity** (recommended for Azure SQL Database): Your database's managed identity authenticates directly with Azure OpenAI. Grant it the Cognitive Services OpenAI User role on the Azure OpenAI resource.

SQL

```
CREATE DATABASE SCOPED CREDENTIAL [https://<resource-name>.cognitiveservices.azure.com/]
  WITH IDENTITY = 'Managed Identity',
  SECRET = '{"resourceid":"https://cognitiveservices.azure.com"}';
```

- **API key**: Store the key in HTTP headers. This approach works for both Azure SQL Database and SQL Server 2025.

SQL

```
CREATE DATABASE SCOPED CREDENTIAL [https://<resource-name>.cognitiveservices.azure.com/]
  WITH IDENTITY = 'HTTPEndpointHeaders',
  SECRET = '{"api-key":"<your-api-key>"}';
```

Avoid hardcoding API keys in your T-SQL code. Use managed identities when possible for better security and easier key rotation.

Important

Grant the EXECUTE ANY EXTERNAL ENDPOINT permission to users or roles that need to call external endpoints: GRANT EXECUTE ANY EXTERNAL ENDPOINT TO [user_or_role];

With the credential in place, you can create the external model. The following example shows a minimal workflow.

SQL

```
CREATE EXTERNAL MODEL my_external_model
WITH (
  LOCATION   = 'https://<resource-name>.cognitiveservices.azure.com/openai/deployments/<deployment-name>/embeddings?api-version=<api-version>',
  API_FORMAT = 'Azure OpenAI',
  MODEL_TYPE = EMBEDDINGS,
  MODEL      = 'text-embedding-3-small',
  CREDENTIAL = [https://<resource-name>.cognitiveservices.azure.com/],
  PARAMETERS = '{"dimensions":<n>}'
);
```

Once the external model is created, AI functions can reference it in queries.

SQL

```
SELECT
  id,
  AI_GENERATE_EMBEDDINGS(
    description USE MODEL my_external_model
  ) AS embedding
FROM dbo.documents;
```

In this example, the external model defines how SQL Server connects to the AI service. The AI_GENERATE_EMBEDDINGS function uses that definition to generate embeddings for the description column without embedding endpoint or authentication details in the query.

When you combine external models with AI functions, SQL Server enables AI workflows that are defined, executed, and maintained entirely through Transact-SQL.

Managing external models as database objects

Because external models are database-scoped objects, they fit naturally into existing database management practices. You can control access through permissions, include them in deployment pipelines, and manage their lifecycle alongside other schema objects.

Treating external models as first-class database objects helps ensure consistency, maintainability, and predictable behavior when integrating AI capabilities into SQL-based solutions.

Key takeaways

External models are database objects that store metadata about AI model endpoints, allowing SQL Server to call external AI services without embedding connection details in application code. You create them with CREATE EXTERNAL MODEL, update them with ALTER EXTERNAL MODEL, and reference them in built-in AI functions like AI_GENERATE_EMBEDDINGS. Because they're database-scoped, you can manage permissions, lifecycle, and deployment alongside your other schema objects.

Design embeddings for SQL database workloads

Completed100 XP

- 8 minutes

Choose your preferred content format

Embeddings represent data as vectors so that similarity between pieces of text can be compared. How you design embeddings affects relevance, performance, and cost when vectors are later generated and queried.

SQL Server provides built-in AI functions that support embedding workflows. Common vector search patterns help guide how text should be prepared before embeddings are generated.

Understand how vectors are created

But what does the term ***vector*** mean in the context of AI integration with a SQL database?

An AI model creates a vector, not SQL itself. The model was trained to read text and return a list of numbers that represents the meaning of that text.

When SQL sends text to an embedding model, the model returns a vector, which SQL stores for later comparison with other vectors.

For example, when the text *"lightweight hiking backpack"* is sent to an embedding model, the model might return:

[0.12, -0.87, 0.45, 0.31, ...]

A related sentence such as *"compact backpack for day hikes"* would produce a different list of numbers that looks similar:

[0.10, -0.85, 0.47, 0.29, ...]

Because the vectors look similar, SQL can treat the two pieces of text as related, even though the wording is different.

Identify data to include in embeddings

Embeddings work best when they represent text that carries semantic meaning, such as descriptions, titles, or other free-form text fields.

Columns that store identifiers, numeric values, or operational metadata usually don't add semantic value and should be excluded. Limiting embeddings to meaningful text reduces token usage and improves similarity results.

Design choices at this stage determine what information embeddings capture and how well they represent the underlying data.

Control input size and structure

Embedding models operate on tokenized input and impose limits on how much text can be processed in a single request. A token is a small piece of text, such as a word or part of a word, that the model processes as a unit. Long text values often require division into smaller units.

SQL Server supports this pattern through built-in AI functions that help prepare text for embedding workflows. By controlling input size, you can keep text within model limits and ensure each embedding represents a clear, focused piece of content.

Well structured input also helps avoid embedding unrelated ideas together, which can reduce the quality of similarity results.

Design chunking strategies

Chunking defines how larger text values are divided into smaller segments. A chunking strategy balances context and precision.

Chunks that are too large may exceed token limits or dilute semantic focus. Chunks that are too small may lose important context. The goal is to preserve meaning while keeping chunks efficient to process.

In practice, chunking is defined by the rules you apply when splitting text in SQL. These rules typically control how much text goes into each chunk, such as a maximum number of characters, and where splits are allowed to occur.

SQL

```
AI_GENERATE_CHUNKS(SOURCE = description, CHUNK_TYPE = FIXED, CHUNK_SIZE = 500)
```

By defining chunking rules directly in SQL, near the source tables, you can change chunk size or splitting behavior by adjusting the query instead of modifying application code.

Apply embedding design with SQL

The following example shows a conceptual pattern that prepares text for embedding generation by splitting it into smaller units.

SQL

```
SELECT
  id,
  c.chunk
FROM dbo.documents
CROSS APPLY
  AI_GENERATE_CHUNKS(SOURCE = description, CHUNK_TYPE = FIXED, CHUNK_SIZE = 500) AS c;
```

In this example, the text in the description column is divided into chunks of up to 500 characters. Each chunk can later be passed to an embedding function, ensuring embeddings represent focused portions of the original content.

For example, if a row contains the following text in the description column:

"Lightweight backpack designed for long day hikes in warm weather."

The chunking function might produce multiple chunks such as:

- "Lightweight backpack designed for long day hikes"

- "in warm weather."

Each chunk is returned as a separate row by the query. These smaller pieces can then be passed individually to an embedding function so that each embedding represents a focused portion of the original text.

Tip

One design pattern is to store vectors in a separate table from the source text data. A dedicated embeddings table makes it easier to track how much space vectors consume and to rebuild embeddings without affecting the original data.

Key takeaways

Embedding quality depends on the decisions you make before any model runs. Choosing the right columns and chunking long text into focused segments with AI_GENERATE_CHUNKS determine how useful your vector search results are. Getting these design choices right early prevents costly rework when you move to embedding generation and storage.

Generate and maintain embeddings for SQL database workloads

Completed100 XP

- 8 minutes

Choose your preferred content format

After you design how embeddings represent your data, you need to generate them and keep them in sync as data changes.

SQL Server provides built-in AI functions to generate embeddings from text stored in database columns. Generating embeddings is typically not a one-time operation. When source data changes, embeddings may need to be regenerated so they continue to reflect the current state of the data.

Generate embeddings with SQL

SQL Server provides the AI_GENERATE_EMBEDDINGS function to generate embeddings directly from text stored in database columns. This function uses an external model to convert text into a vector that can be stored and later compared.

A common pattern is to generate embeddings during an initial load or as part of a batch process. The resulting vectors are stored alongside the source data or in a related table so they can be queried efficiently.

The following example shows a simple end-to-end pattern, from table definition to embedding generation.

First, create a table that stores both the source text and it's embedding.

SQL

```
CREATE TABLE dbo.documents
(
  id INT PRIMARY KEY,
  description NVARCHAR(MAX),
  embedding VECTOR(1536)
);
```

Next, generate embeddings from the text and store them in the table.

SQL

```
UPDATE dbo.documents
SET embedding = AI_GENERATE_EMBEDDINGS(description USE MODEL
my_embedding_model);
```

In this example, each row's description value is sent to the embedding model. The function returns a vector, which is stored in the embedding column. These stored vectors can later be queried or compared without regenerating them. You might want to include extra logic to handle chunking or filtering based on your embedding design.

Embedding generation determines how vectors are created. Maintenance strategies determine when those vectors need to be refreshed.

Understand embedding maintenance

Embedding maintenance keeps stored embeddings aligned with changes to the underlying data. When text values are inserted, updated, or deleted, the corresponding embeddings may no longer reflect the current content.

Different maintenance approaches can be used depending on how often data changes, how quickly embeddings must be updated, and where embedding generation runs.

Choose an embedding maintenance method

Embeddings need to stay aligned with the source text as data changes. Several options can be used to detect changes and decide when embeddings should be regenerated. These options differ in where the work happens and how quickly changes are reflected.

- **Table triggers**
  Triggers run automatically when rows are inserted or updated. For embedding maintenance, a trigger can mark rows that need new embeddings or initiate regeneration immediately. This approach reflects changes quickly but adds work to write operations.

- **Change Tracking**
  Change Tracking records that a row changed since a given point in time. A background process can use this information to identify which rows need their embeddings refreshed and process them in batches. This approach balances latency and performance.

- **Change Data Capture (CDC)**
  CDC records detailed information about data changes, including before and after values. Embedding maintenance can use CDC tables to identify which text values changed and regenerate embeddings asynchronously. This approach is suitable for high-volume workloads.

- **Azure Functions with SQL trigger binding**
  Azure Functions can react to database changes using SQL trigger bindings. This feature allows embedding generation to run outside the database engine while still responding to data changes. This approach offloads work from the database and can scale independently.

- **Azure Logic Apps**
  Logic Apps can orchestrate embedding updates as part of a workflow. For example, a Logic App might periodically check for changed rows and call an embedding service, coordinating updates with other systems. This approach is low-code and integrates well with other Azure services.

- **Change Event Streaming**
  Change event streaming (CES) streams DML changes directly into Azure Event Hubs in near real-time. Downstream systems can consume these events and regenerate embeddings as changes occur without adding work to database transactions. This approach decouples embedding generation from the database and supports multiple consumers processing the same change stream.

- **Microsoft Foundry**
  Microsoft Foundry can be used to manage and evaluate the models that generate embeddings. In a maintenance workflow, Foundry typically supports model selection or hosting while another process handles change detection and database updates. This approach centralizes model

management while embedding generation occurs in response to data changes.

Choose an appropriate maintenance approach

There's no single correct way to maintain embeddings. The right approach depends on factors such as data volume, update frequency, latency requirements, and where embedding generation fits within the overall solution.

Some solutions favor immediate updates, while others prioritize batching or external processing. Understanding these trade-offs helps you choose a maintenance strategy that fits your SQL application.

Key takeaways

Generating embeddings with AI_GENERATE_EMBEDDINGS is only the first step. As source data changes, stored vectors can fall out of sync, so you need a maintenance strategy. Options range from triggers and Change Tracking for tightly coupled updates to Change Data Capture, Change Event Streaming, and Azure Functions for asynchronous or decoupled approaches. The right choice depends on your data volume, latency requirements, and where embedding generation fits in your overall architecture.

Summary

Completed100 XP

- 2 minutes

This module covered designing and implementing models and embeddings for Azure SQL Database and Fabric SQL database. You learned how to evaluate AI models, create external models to reference AI endpoints from Transact-SQL, and design effective chunking strategies. You also explored approaches for generating and maintaining embeddings as data changes.

After completing this module, you can:

- Evaluate AI models for SQL database workloads based on capabilities and performance requirements.
- Create and manage external models to reference AI endpoints from Transact-SQL.
- Design embeddings with appropriate chunking strategies.
- Generate and store embeddings using built-in SQL AI functions.

- Choose maintenance approaches to keep embeddings aligned with source data.

Design and implement intelligent search with SQL

Implement intelligent search capabilities in SQL Server and Azure SQL by combining traditional full-text search with semantic vector search. Establish a mental model for different search approaches, prepare SQL for vector-based search, and implement vector, hybrid, and ranking-based search patterns with performance considerations.

Introduction

Completed100 XP

- 3 minutes

Choose your preferred content format

Finding relevant information in a database often requires more than matching keywords. Users search for concepts, not just words. A customer looking for "lightweight hiking backpack" might find nothing if the product is listed as "ultralight trail bag." SQL Server and Azure SQL Database provide multiple search approaches to handle these challenges: full-text search for keyword matching, vector search for semantic similarity, and hybrid search that combines both.

Full-text search excels when users know the specific terms they want. Vector search uses embeddings to find content based on meaning even when the exact words differ. Hybrid search runs both approaches and merges the results, giving users the best of both worlds. Choosing the right approach depends on how your users search and what trade-offs you can accept between precision, recall, and performance.

Imagine a retail team building a product search feature. They have embeddings stored for their product descriptions and support articles. Now they need search capabilities that help customers find products and answers quickly. Some customers type exact product names. Others describe what they need in natural language. The team decides to implement hybrid search so their application handles both patterns effectively, using full-text indexes for keyword matching and vector search for semantic similarity.

After completing this module, you'll be able to:

- To select the right approach for different query types, evaluate full-text search, vector search, and hybrid search.

- Implement full-text search using full-text indexes and query predicates.
- Prepare SQL databases for vector search by storing embeddings and choosing distance metrics.
- Write vector search queries using VECTOR_DISTANCE and VECTOR_SEARCH functions.
- Combine full-text and vector search results using Reciprocal Rank Fusion (RRF).

Note

We recognize that different people like to learn in different ways. You can choose to complete this module in video-based format or you can read the content as text and images. The text contains greater detail than the videos, so in some cases you might want to refer to it as supplemental material to the video presentation.

Choose an intelligent search approach

Completed100 XP

- 8 minutes

Choose your preferred content format

SQL Server and Azure SQL Database provide you with three ways to search text data, full-text search, vector search, and hybrid search. Each works differently, and choosing the right one depends on what your users need to find.

To see why the right search approach matters, imagine a customer searching your product catalog. They type "lightweight hiking backpack" but your best-selling item is listed as "ultralight trail bag." A traditional keyword search finds nothing. A smarter search option understands that both descriptions mean the same thing. The search approach you choose determines whether your users find what they need or not.

Understand the three search approaches

Let's look at how each approach works and when you'd use it.

Full-text search looks for words and phrases. When a customer searches for "mountain bike helmet," full-text search finds products containing those exact terms. It understands language rules, so searching for "ride" can also find "riding" or "rode." Use full-text search when users know the specific words they want.

Semantic vector search looks for meaning. Instead of matching words, it compares the mathematical representation of concepts. When a customer searches for "something to keep me visible on evening rides," vector search can

find products described as "reflective cycling vest" or "LED bike lights" even though those words don't appear in the search. This approach works well when users describe what they need rather than name it.

Hybrid search uses both. It runs a full-text search and a vector search, then combines the results. A customer searching for "comfortable seat for long tours" gets products matching those keywords and products like "touring saddle" or "gel bike seat" that vector search identifies as semantically related.

Match the approach to the query intent

Your users express different intentions when they search. Some know exactly what they want. Others are exploring.

Consider these two searches against a product database:

- "Model XR-500" - The user wants that specific product. Full-text search handles this search perfectly.
- "something to keep drinks cold on a hike" - The user describes a need. Vector search understands this means "insulated water bottle" or "portable cooler."

When you can't predict how users search, hybrid search covers both cases. The user who types "XR-500 portable cooler" gets results matching the model number and results matching the concept.

Evaluate the trade-offs

Choosing a search approach involves trade-offs. Consider these factors as you design your solution.

Precision versus recall - Full-text search returns fewer results, but they closely match the query terms. Vector search returns more results, including items that are conceptually related but use different words. If your application needs exact matches, lean toward full-text. If it needs discovery, lean toward vector search.

Data requirements - Full-text search needs a full-text index on your text columns. Vector search needs embeddings stored in vector columns. You might need to prepare your data differently depending on which approach you choose.

Performance characteristics - Full-text indexes are optimized for fast keyword lookup. Vector search performance depends on how many vectors you're comparing and whether you use approximate or exact methods. Hybrid search runs both, so it takes longer than either one alone.

These trade-offs aren't about which approach is better. The right choice depends on which approach fits your scenario.

Combine approaches with hybrid search

Full-text or vector search alone doesn't handle every situation. Full-text search misses documents that express the same idea with different words. Vector search might return conceptually related items that don't contain important terms the user specified.

Hybrid search solves this issue by running both approaches and merging the results. A user searching for "Azure SQL backup strategies" gets documents containing those keywords and documents about "database recovery planning" that vector search identifies as related.

The challenge is combining two ranked lists into one. How do you decide which result should appear first when full-text search ranks it #3 and vector search ranks it #7?

Merge results with Reciprocal Rank Fusion

Reciprocal Rank Fusion (RRF) is an algorithm for combining ranked results from different searches. Instead of comparing raw scores, which use different scales, RRF calculates a score based on each document's position (the **rank**) using the formula 1/(rank + k). The constant **k** is 60, a value established by the original RRF research and used by Azure AI Search. Documents appearing in multiple result sets have their scores summed, which is where RRF really shines.

Here's how it works. Suppose a customer searches for "comfortable bike seat" and you run both full-text and vector search:

Rank	Full-text results	Vector search results
1	Bike Seat Comfort Pro	Ergonomic Touring Saddle
2	Comfortable Handlebar Grips	Gel Cushion Saddle
3	Racing Bike Seat	Bike Seat Comfort Pro

Full-text search finds products containing the words *comfortable*, *bike*, and *seat*. Vector search finds semantically similar products, including saddles that don't use the word *seat* at all.

Notice that "Bike Seat Comfort Pro" appears in both lists: rank 1 in full-text and rank 3 in vector search. RRF calculates its combined score by summing:

- Full-text rank 1: 1/(60+1) = 0.0164
- Vector rank 3: 1/(60+3) = 0.0159
- **Combined: 0.0323**

Compare that to "Ergonomic Touring Saddle," which only appears in vector search at rank 1:

- Vector rank 1: 1/(60+1) = 0.0164
- **Combined: 0.0164**

Even though "Ergonomic Touring Saddle" ranked #1 in vector search, "Bike Seat Comfort Pro" wins the combined ranking because appearing in both lists doubled its score. The merged results:

Rank	Combined results	RRF Score
1	Bike Seat Comfort Pro	0.0323
2	Ergonomic Touring Saddle	0.0164
3	Comfortable Handlebar Grips	0.0161
4	Gel Cushion Saddle	0.0161
5	Racing Bike Seat	0.0159

Notice that ranks 3 and 4 have identical scores. When RRF produces ties, the ordering between those results depends on the implementation and might be arbitrary or alphabetical. In practice, tied results are equally relevant, so the exact order between them matters less than ensuring they all appear above results with lower scores.

RRF rewards documents that perform well across multiple search methods, surfacing results that match both keywords and meaning.

Key takeaways

Full-text search, vector search, and hybrid search each serve different purposes. Full-text search excels at finding exact keywords and phrases, vector search captures semantic meaning across different phrasings, and hybrid search combines both to maximize relevance. Your choice depends on how your users search and what trade-offs you can accept between precision, recall, and performance. When you implement hybrid search, Reciprocal Rank Fusion (RRF) is a powerful technique for merging ranked results from full-text and vector searches, ensuring that items relevant to both methods rise to the top of the results list.

Implement full-text search

Completed100 XP

- 8 minutes

Choose your preferred content format

Full-text search lets you run linguistic searches against text data in SQL Server and Azure SQL Database. Unlike the LIKE operator, which only matches character patterns, full-text search understands words, phrases, and language rules.

When a customer searches your product catalog for "ride," they expect to find results that say "riding," "rides," or "rode." Full-text search handles these scenarios by working with language-aware indexes that understand word forms. It doesn't automatically match synonyms or abbreviations (like "MTB" for "mountain bike") unless you configure a thesaurus, but it handles inflections out of the box.

When to use full-text search

Full-text search works best when your users search using specific words or phrases. Consider these scenarios where full-text search is the right choice:

- A customer types a product name or part number
- A support agent searches for tickets containing specific error messages
- A user looks for documents with exact technical terms

If users instead describe what they need without using specific terms, vector search might be more appropriate. But when users know the words they want, full-text search delivers fast, precise results.

Understand full-text indexes and predicates

Full-text search requires two things: a full-text index on your text columns, and predicates to query that index.

A full-text index stores information about words and their locations within your text columns. SQL Server breaks down the text into individual tokens, removes common words like "the" and "is" (called stopwords), and builds an inverted index that maps words to the rows containing them. This structure allows fast lookups without scanning every row.

To query a full-text index, you use predicates in your WHERE clause. SQL Server provides two main predicates:

- **CONTAINS** searches for exact matches of words and phrases. Use it when you need precise control over what matches.
- **FREETEXT** searches for meaning rather than exact words. It automatically looks for inflectional forms of your search terms.

SQL Server also provides table-valued functions **CONTAINSTABLE** and **FREETEXTTABLE** that return ranked results with relevance scores, which is useful when you want to sort results by how well they match.

Query a full-text index

The following example shows how to search a product catalog using both CONTAINS and FREETEXT. Assume the Production.Product table has a full-text index on the Name column.

Use CONTAINS when you need exact word matching:

SQL

```
SELECT ProductID, Name, ListPrice

FROM Production.Product

WHERE CONTAINS(Name, 'mountain')

  AND ListPrice < 500;
```

This query returns products with "mountain" in the name under 500. The CONTAINS predicate checks the full-text index while the ListPrice filter uses the regular column.

Use FREETEXT when you want SQL Server to find related word forms automatically:

SQL

```
SELECT ProductID, Name

FROM Production.Product

WHERE FREETEXT(Name, 'riding bikes');
```

This query finds products related to "riding bikes," including matches for "ride," "rides," and "biking." FREETEXT handles inflections automatically without requiring you to specify them.

When you need ranked results, use CONTAINSTABLE to get full-text relevance scores. Full-text search assigns a rank to each result based on how well it matches the query. This rank is useful for sorting results by relevance:

SQL

```
SELECT p.ProductID, p.Name, ft.RANK

FROM Production.Product AS p

INNER JOIN CONTAINSTABLE(Production.Product, Name,
```

```
  'NEAR((mountain, bike))') AS ft
 ON p.ProductID = ft.[KEY]
ORDER BY ft.RANK DESC;
```

This query finds products where "mountain" and "bike" appear near each other, sorted by full-text relevance. The RANK column indicates how well each row matches the search criteria. Later, when you combine full-text search with vector search, you use these full-text ranks alongside vector similarity scores to produce merged results.

Common query patterns

Full-text search supports several query patterns. Each addresses a different search need.

Term search finds rows containing a specific word. A search for CONTAINS(Description, 'aluminum') returns all products with "aluminum" in the description.

Phrase search finds rows containing words in a specific order. Wrapping the search in quotes like CONTAINS(Description, '"mountain bike"') matches only that exact phrase, not rows where "mountain" and "bike" appear separately.

Prefix search finds words that start with specific characters. Searching for CONTAINS(Description, '"light*"') matches "light," "lights," "lighter," "lightweight," and any other word beginning with "light."

Inflectional search finds different forms of a word. Using CONTAINS(Description, 'FORMSOF(INFLECTIONAL, "ride")') matches "ride," "rides," "riding," and "rode."

Proximity search finds words that appear near each other. Searching for CONTAINS(Description, 'NEAR((light, aluminum))') finds products where "light" and "aluminum" appear close together, which often indicates a lightweight aluminum product.

Evaluate full-text search results

After implementing full-text search, evaluate whether it meets your users' needs. Watch for these signals:

Precision problems occur when results include irrelevant items. If searching for "brake" returns results about "brake-resistant" coatings when users want bicycle brakes, your search is too broad. Consider using phrase searches or more specific terms.

Noise happens when common words dilute results. Full-text search uses stoplists to filter out words like "the" and "and," but domain-specific noise might require custom stoplists. If searching for "the mountain" returns too many results because "mountain" appears everywhere, you might need to adjust your approach.

Query intent mismatch appears when users search for concepts rather than words. If customers search for "something for rainy commutes" and expect to find waterproof gear, full-text search doesn't help because it matches words, not meaning. This signal suggests you might need vector search or hybrid search instead.

Key takeaways

Full-text search excels when users search with specific words and phrases. It uses full-text indexes to enable fast linguistic searches and provides predicates like CONTAINS and FREETEXT to query those indexes. Different query patterns (term, phrase, prefix, inflectional, and proximity) address different search needs. When full-text search returns too many irrelevant results, or when users search for concepts instead of specific words, the next search method to consider is vector search.

Prepare SQL for vector search

Completed100 XP

- 10 minutes

Choose your preferred content format

Before you can run vector searches, you need to store vectors in your database and decide how those searches execute. This unit covers the design decisions you make before writing your first vector query.

Vector search finds rows based on mathematical similarity rather than exact matches. To make this work, you store embeddings as vectors, choose how to measure similarity, and decide whether to search all vectors exactly or use an index for faster approximate results.

Store vectors with the vector data type

SQL Server and Azure SQL Database provide a native **vector** data type designed for storing embeddings. Each vector is an array of floating-point numbers, stored in an optimized binary format but exposed as JSON arrays for convenience.

When you define a vector column, you specify the number of dimensions:

SQL

```
CREATE TABLE dbo.Products
(
  ProductID INT PRIMARY KEY,
  Name NVARCHAR(100),
  Description NVARCHAR(MAX),
  DescriptionVector VECTOR(1536) NOT NULL
);
```

The number in parentheses matches the dimensions your embedding model produces. For example, OpenAI's text-embedding-3-small model produces 1,536 dimensions, so your column would be VECTOR(1536). The maximum supported is 1,998 dimensions.

Each element is stored as a single-precision (4-byte) float. A 1,536-dimension vector uses about 6 KB per row. When designing your table, consider how this column affects storage and memory as your data grows.

Understand distance metrics

Vector search works by calculating the distance between vectors. Vectors that are closer together represent more similar concepts. SQL Server supports three distance metrics:

Cosine distance measures the angle between vectors, ignoring their magnitude. Two vectors pointing in the same direction have a cosine distance of 0, regardless of their length. This metric works well when you care about the direction of meaning, not the intensity. Cosine distance ranges from 0 (identical) to 2 (opposite).

Euclidean distance measures the straight-line distance between two points in vector space. It considers both direction and magnitude. Euclidean distance ranges from 0 (identical) to infinity.

Dot product calculates the sum of element-wise products. SQL Server returns the negative dot product so that smaller values indicate more similar vectors, consistent with the other metrics.

Most embedding models are optimized for cosine similarity, making cosine distance the common choice. However, the best metric depends on how your embeddings were trained.

Choose between exact and approximate search

When you search for similar vectors, you have two options: exact nearest neighbor (ENN) search and approximate nearest neighbor (ANN) search.

Exact nearest neighbor search compares your query vector against every vector in the table. It guarantees the most accurate results but requires calculating the distance to every row. Use the VECTOR_DISTANCE function for exact search:

SQL

```
DECLARE @query VECTOR(1536) = '[0.1, 0.2, ...]';


SELECT TOP 10 ProductID, Name,
  VECTOR_DISTANCE('cosine', @query, DescriptionVector) AS Distance
FROM dbo.Products
ORDER BY Distance;
```

This query calculates the cosine distance between @query and every row's DescriptionVector, then returns the 10 closest matches. For small tables (under 50,000 vectors as a general guideline), exact search performs well.

Approximate nearest neighbor search uses a vector index to find similar vectors without scanning every row. It trades perfect accuracy for speed, returning results that are very close to exact but not guaranteed to be the absolute nearest neighbors.

Create a vector index using CREATE VECTOR INDEX:

SQL

```
CREATE VECTOR INDEX idx_Products_DescriptionVector
ON dbo.Products(DescriptionVector)
WITH (METRIC = 'cosine', TYPE = 'DiskANN');
```

SQL Server uses the DiskANN algorithm, which builds a graph structure that allows fast navigation to nearby vectors. Once the index exists, use the VECTOR_SEARCH function for approximate search:

SQL

```
DECLARE @query VECTOR(1536) = '[0.1, 0.2, ...]';


SELECT t.ProductID, t.Name, s.distance
```

```
FROM VECTOR_SEARCH(
  TABLE = dbo.Products AS t,
  COLUMN = DescriptionVector,
  SIMILAR_TO = @query,
  METRIC = 'cosine',
  TOP_N = 10
) AS s
ORDER BY s.distance;
```

Decide when to use each approach

The choice between exact and approximate search depends on your dataset size and accuracy requirements.

Use exact search (VECTOR_DISTANCE) when:

- Your table has fewer than 50,000 vectors
- Your query filters reduce the candidate set to a few rows
- You need guaranteed accurate results and can accept longer query times

Use approximate search (VECTOR_SEARCH with an index) when:

- Your table has hundreds of thousands or millions of vectors
- Query speed matters more than perfect accuracy
- A recall close to 1 (meaning most true nearest neighbors are found) is acceptable

Recall measures how many of the true nearest neighbors the approximate search returns compared to an exact search. DiskANN typically achieves high recall, meaning the results are very close to what exact search would return.

Consider index limitations

Vector indexes in SQL Server have specific requirements to be aware of:

- The table must have a single-column integer primary key with a clustered index
- Tables with vector indexes become read-only while the index exists—you must drop the index to modify data, then recreate it
- Vector indexes can't be partitioned

Note

Vector indexes are currently in preview. In Azure SQL Database and SQL database in Microsoft Fabric, you can set the ALLOW_STALE_VECTOR_INDEX database scoped configuration to ON to allow writes, but the index doesn't reflect new data until you rebuild it. This option isn't currently available in SQL Server 2025. Check the current documentation for the latest on these limitations.

These limitations affect how you design your tables and maintenance workflows. For tables that change frequently, you might use exact search until the data stabilizes, then add a vector index.

Key takeaways

Preparing SQL for vector search means making three decisions: what data type and dimensions to use, which distance metric matches your embedding model, and whether exact or approximate search fits your dataset size. Exact search with VECTOR_DISTANCE works well for smaller datasets or filtered queries, while approximate search with VECTOR_SEARCH and a DiskANN index handles larger datasets efficiently. In the next unit, you'll learn the specific functions and query patterns for running vector searches.

Implement vector search query patterns

Completed100 XP

- 8 minutes

Choose your preferred content format

With vectors stored and a search strategy chosen, you're ready to write queries. This unit covers the T-SQL functions that power vector search and shows you how to combine them effectively.

Vector search queries follow a common pattern: generate or retrieve a query vector, calculate distances to stored vectors, and return the closest matches. SQL Server provides four functions that work together to support this pattern: VECTOR_DISTANCE, VECTOR_SEARCH, VECTOR_NORMALIZE, and VECTORPROPERTY.

Calculate distances with VECTOR_DISTANCE

The VECTOR_DISTANCE function calculates the distance between two vectors using the metric you specify. This function performs exact nearest neighbor search, computing the distance to every qualifying row. In practical terms, it compares your search embedding against every stored embedding to find which

ones are most similar, like checking every product in a catalog to find the best matches.

SQL

```
VECTOR_DISTANCE(metric, vector1, vector2)
```

The metric must be 'cosine', 'euclidean', or 'dot'. Both vectors must have the same number of dimensions.

Here's a typical pattern for finding similar products based on a description embedding:

SQL

```
-- Generate an embedding for the user's search phrase

DECLARE @searchVector VECTOR(1536);

SELECT @searchVector = AI_GENERATE_EMBEDDINGS('lightweight hiking boots' USE MODEL MyEmbeddingModel);


SELECT TOP 10

  ProductID,

  Name,

  VECTOR_DISTANCE('cosine', @searchVector, DescriptionVector) AS Distance

FROM dbo.Products

ORDER BY Distance;
```

This query works in two steps. First, it converts the search phrase "lightweight hiking boots" into an embedding using the same model that created the stored DescriptionVector values. Second, it calculates the cosine distance between @searchVector and every row's DescriptionVector. The ORDER BY Distance clause sorts the results from smallest distance (most similar) to largest, and TOP 10 returns only the first 10 rows. Since smaller distances mean more similar vectors, this combination gives you the 10 products most semantically related to "lightweight hiking boots."

You can also use VECTOR_DISTANCE in a WHERE clause to find all vectors within a specific distance threshold:

SQL

```
SELECT ProductID, Name
```

```
FROM dbo.Products

WHERE VECTOR_DISTANCE('cosine', @searchVector, DescriptionVector) < 0.3

ORDER BY VECTOR_DISTANCE('cosine', @searchVector, DescriptionVector);
```

This approach finds all products within a similarity threshold rather than returning a fixed count. The 0.3 value here's just an example. Cosine distance ranges from 0 (identical vectors) to 2 (completely opposite), but what counts as "similar enough" depends entirely on your data and embedding model. To find the right threshold, run test queries, examine the distance values in your results, and identify where relevant results stop and the noise begins. Use this pattern when you want everything meeting a quality bar rather than a fixed number of results.

Use VECTOR_SEARCH for approximate retrieval

When your table grows beyond tens of thousands of rows, calculating distances to every row becomes slow. The VECTOR_SEARCH function uses a vector index to find approximate nearest neighbors quickly.

SQL

```
VECTOR_SEARCH(
  TABLE = object AS alias,
  COLUMN = vector_column,
  SIMILAR_TO = query_vector,
  METRIC = 'cosine' | 'euclidean' | 'dot',
  TOP_N = k
)
```

This function returns a result set containing all columns from the source table plus a distance column. Here's how you'd use it:

SQL

```
DECLARE @searchVector VECTOR(1536);

SELECT @searchVector = AI_GENERATE_EMBEDDINGS('lightweight hiking boots'
USE MODEL MyEmbeddingModel);


SELECT t.ProductID, t.Name, s.distance

FROM VECTOR_SEARCH(
```

```
  TABLE = dbo.Products AS t,
  COLUMN = DescriptionVector,
  SIMILAR_TO = @searchVector,
  METRIC = 'cosine',
  TOP_N = 10
) AS s
ORDER BY s.distance;
```

The function returns the approximate 10 nearest neighbors. If a matching vector index exists on DescriptionVector with the same metric, VECTOR_SEARCH uses it for fast retrieval. Without an index, it falls back to exact search and raises a warning.

Handle post-filtering carefully

VECTOR_SEARCH applies any WHERE clause conditions *after* finding the nearest neighbors, not before. Consider this query:

SQL

```
SELECT t.ProductID, t.Name, s.distance
FROM VECTOR_SEARCH(
  TABLE = dbo.Products AS t,
  COLUMN = DescriptionVector,
  SIMILAR_TO = @searchVector,
  METRIC = 'cosine',
  TOP_N = 10
) AS s
WHERE t.CategoryID = 5
ORDER BY s.distance;
```

This query first finds the 10 nearest vectors across all categories, then filters to only category 5. If none of the 10 nearest products are in category 5, you get no results. To work around this issue, request more candidates than you need:

SQL

```
SELECT TOP 10 t.ProductID, t.Name, s.distance
```

```
FROM VECTOR_SEARCH(
  TABLE = dbo.Products AS t,
  COLUMN = DescriptionVector,
  SIMILAR_TO = @searchVector,
  METRIC = 'cosine',
  TOP_N = 50
) AS s
WHERE t.CategoryID = 5
ORDER BY s.distance;
```

By requesting 50 candidates from VECTOR_SEARCH and then taking the top 10 after filtering, you're more likely to have enough results in the target category.

Normalize vectors for consistent comparisons

Different embedding models produce vectors with different lengths (magnitudes). When comparing vectors from multiple sources or when your model doesn't normalize outputs, you can use VECTOR_NORMALIZE to scale vectors to unit length.

SQL

```
VECTOR_NORMALIZE(vector, norm_type)
```

The norm_type can be:

- 'norm2': Euclidean norm (most common)
- 'norm1': Sum of absolute values
- 'norminf': Maximum absolute value

SQL

```
DECLARE @v VECTOR(3) = '[3, 4, 0]';
SELECT VECTOR_NORMALIZE(@v, 'norm2') AS NormalizedVector;
-- Returns: [0.6, 0.8, 0.0]
```

Normalizing ensures that cosine distance and dot product behave consistently. Most modern embedding models (like those from OpenAI) produce normalized vectors, so you often don't need this step. But if you're working with embeddings from models that don't normalize, applying VECTOR_NORMALIZE before storing or comparing ensures consistent similarity scores.

You can calculate a vector's magnitude using the related VECTOR_NORM function:

SQL

```
DECLARE @v VECTOR(3) = '[3, 4, 0]';

SELECT VECTOR_NORM(@v, 'norm2') AS Magnitude;

-- Returns: 5.0
```

Inspect vectors with VECTORPROPERTY

The VECTORPROPERTY function returns metadata about a vector. This metadata is useful for validating data or debugging dimension mismatches.

SQL

```
VECTORPROPERTY(vector, property)
```

Two properties are supported:

- 'Dimensions': Returns the number of dimensions as an integer
- 'BaseType': Returns the data type name (currently always float)

SQL

```
DECLARE @v VECTOR(1536) = '[0.1, 0.2, ...]';

SELECT VECTORPROPERTY(@v, 'Dimensions') AS Dims;

-- Returns: 1536
```

This function helps when troubleshooting queries where vectors might have mismatched dimensions—a common issue when different embedding models are used.

Choose the right function for your scenario

Each function serves a distinct purpose in vector search:

Scenario	Function	When to use
Small dataset or filtered queries	VECTOR_DISTANCE	Under 50,000 vectors, or when a WHERE clau significantly reduces candidates
Large dataset with vector index	VECTOR_SEARCH	Hundreds of thousands of vectors or more, w matters
Comparing vectors from different models	VECTOR_NORMALIZE	When embedding sources vary or model does normalized vectors

Scenario	Function	When to use
Validating vector data	VECTORPROPERTY	Checking dimensions during troubleshooting validation

Key takeaways

Vector search queries in SQL follow a predictable pattern: prepare a query vector, find neighbors using either VECTOR_DISTANCE (exact) or VECTOR_SEARCH (approximate), and order by distance. VECTOR_NORMALIZE and VECTORPROPERTY support data quality and consistency. Remember that VECTOR_SEARCH applies filters after finding nearest neighbors, so request more candidates than you need when combining with WHERE clauses. In the next unit, you'll learn how to combine vector search with full-text search in hybrid queries.

Implement hybrid search and ranking

Completed100 XP

- 10 minutes

Choose your preferred content format

Full-text search and vector search each have strengths, but neither alone handles every query. Full-text search excels at finding exact keywords but misses documents that express the same idea differently. Vector search captures semantic meaning but might return conceptually related items that don't contain important terms the user specified. Hybrid search combines both approaches to maximize relevance.

In this unit, you learn how to implement hybrid search in SQL by running full-text and vector searches in parallel, then merging the results using Reciprocal Rank Fusion (RRF).

Understand the hybrid search pattern

A hybrid search query runs two searches against the same data:

1. **Full-text search** uses FREETEXTTABLE or CONTAINSTABLE to find documents matching keywords, returning results ranked by BM25 relevance.
2. **Vector search** uses VECTOR_DISTANCE or VECTOR_SEARCH to find documents with similar embeddings, returning results ranked by distance.

Each search produces a ranked list. The challenge is combining these two lists into a single result set that benefits from both ranking signals.

Consider a user searching for "Azure SQL backup strategies." Full-text search finds documents containing those exact words. Vector search finds documents about "database recovery planning" or "disaster recovery for SQL databases" that are semantically related but use different terms. A hybrid approach returns both, giving users the best of both worlds.

Merge results with Reciprocal Rank Fusion

Reciprocal Rank Fusion (RRF) is a technique for combining ranked lists from different sources. Instead of comparing raw scores, which might use different scales, RRF focuses on rank positions.

The formula for RRF is:

RRF_score = 1/(k + rank_from_source_1) + 1/(k + rank_from_source_2)

The constant k (typically 60) prevents high-ranked items from dominating. A document ranked #1 in both lists receives a higher combined score than one ranked #1 in only one list.

This approach has two key advantages:

- **Score normalization isn't needed.** Full-text BM25 scores and cosine distances use completely different scales. RRF sidesteps these differences by using ranks instead.
- **Neither source dominates.** If vector search consistently returns higher scores than full-text search, raw score fusion would favor vector results. RRF treats both sources equally.

Implement hybrid search in SQL

Here's a complete hybrid search implementation using Common Table Expressions (CTEs) to structure the query:

SQL

```
DECLARE @searchText NVARCHAR(1000) = 'lightweight hiking boots';

DECLARE @searchVector VECTOR(1536);

DECLARE @topN INT = 50;

DECLARE @rrfK INT = 60;


-- Generate embedding for the search phrase
```

```
SELECT @searchVector = AI_GENERATE_EMBEDDINGS(@searchText USE MODEL MyEmbeddingModel);


-- Run hybrid search with RRF
WITH keyword_search AS (
  SELECT TOP(@topN)
    p.ProductID,
    RANK() OVER (ORDER BY ftt.[RANK] DESC) AS keyword_rank
  FROM dbo.Products p
  INNER JOIN FREETEXTTABLE(dbo.Products, Description, @searchText) AS ftt
    ON p.ProductID = ftt.[KEY]
),
vector_search AS (
  SELECT TOP(@topN)
    ProductID,
    RANK() OVER (ORDER BY distance) AS vector_rank
  FROM (
    SELECT
      ProductID,
      VECTOR_DISTANCE('cosine', @searchVector, DescriptionVector) AS distance
    FROM dbo.Products
  ) AS similar_products
),
combined AS (
  SELECT TOP(@topN)
    COALESCE(ks.ProductID, vs.ProductID) AS ProductID,
    ks.keyword_rank,
    vs.vector_rank,
```

```
    COALESCE(1.0 / (@rrfK + ks.keyword_rank), 0.0) +
    COALESCE(1.0 / (@rrfK + vs.vector_rank), 0.0) AS rrf_score
  FROM keyword_search ks
  FULL OUTER JOIN vector_search vs ON ks.ProductID = vs.ProductID
)
SELECT
  p.ProductID,
  p.Name,
  p.Description,
  c.keyword_rank,
  c.vector_rank,
  c.rrf_score
FROM combined c
INNER JOIN dbo.Products p ON c.ProductID = p.ProductID
ORDER BY c.rrf_score DESC;
```

Let's break down what each CTE does:

1. **keyword_search**: Runs a full-text search using FREETEXTTABLE, which returns BM25-ranked results. The RANK() function assigns position numbers.
2. **vector_search**: Calculates cosine distance for all products and assigns rank positions based on distance (smallest first).
3. **combined**: Joins the two result sets using FULL OUTER JOIN so products appearing in either list are included. The RRF formula calculates the combined score. COALESCE handles products that appear in only one list by treating the missing rank as contributing zero.
4. **Final SELECT**: Joins back to the Products table to retrieve the full product details, ordered by the combined RRF score.

Use VECTOR_SEARCH for large datasets

When your table has hundreds of thousands of rows, replace the exact VECTOR_DISTANCE calculation with VECTOR_SEARCH for better performance:

SQL

```
vector_search AS (
  SELECT TOP(@topN)
    t.ProductID,
    RANK() OVER (ORDER BY s.distance) AS vector_rank
  FROM VECTOR_SEARCH(
    TABLE = dbo.Products AS t,
    COLUMN = DescriptionVector,
    SIMILAR_TO = @searchVector,
    METRIC = 'cosine',
    TOP_N = @topN
  ) AS s
)
```

This version uses the DiskANN index for approximate nearest neighbor search, which is faster for large datasets while still providing high-quality results.

Tune hybrid search parameters

Several factors affect hybrid search quality:

Result set size (@topN): The number of candidates from each search. Larger values give RRF more candidates to work with but increase query time. Start with 50 and adjust based on your relevance testing.

The RRF constant (@rrfK): The formula uses 1/(k + rank). A larger k (like 60) smooths out differences between ranks. A smaller k amplifies the advantage of top-ranked items. The value 60 is standard and comes from the original RRF research.

Weighting: You can weight one source more than the other by multiplying its contribution:

SQL

```
-- Weight vector search 2x more than keyword search
COALESCE(1.0 / (@rrfK + ks.keyword_rank), 0.0) +
COALESCE(2.0 / (@rrfK + vs.vector_rank), 0.0) AS rrf_score
```

Filtering: Apply filters before the hybrid search when possible. Filtering after RRF might eliminate highly ranked results.

Evaluate search quality

Measuring search quality helps you tune parameters and choose between approaches. Common metrics include:

Precision: Of the results returned, what percentage are relevant? High precision means few irrelevant results.

Recall: Of all relevant documents in your database, what percentage did the search find? High recall means you're not missing good results.

Mean Reciprocal Rank (MRR): How high does the first relevant result appear? MRR rewards searches that put the best result at the top.

To evaluate, you need a test set of queries with known relevant documents. Run each approach (full-text only, vector only, hybrid) and compare metrics. Hybrid search typically improves recall compared to either approach alone, often with minimal precision loss.

Key takeaways

Hybrid search combines full-text and vector search to handle both keyword-focused and concept-focused queries. Reciprocal Rank Fusion merges the ranked results without requiring score normalization, treating both sources fairly. You can tune the result set size, RRF constant, and source weights to optimize for your data. Evaluating precision, recall, and MRR helps you measure whether hybrid search improves relevance for your specific use case.

Summary

Completed100 XP

- 2 minutes

This module covered designing and implementing intelligent search capabilities for SQL Server and Azure SQL Database. You learned how to evaluate full-text search, vector search, and hybrid search approaches for different query types. You also explored the T-SQL functions for vector operations and implemented hybrid search using Reciprocal Rank Fusion to combine ranked results from multiple sources.

After completing this module, you can:

- To select the right approach for different query types, evaluate full-text search, vector search, and hybrid search.
- Implement full-text search using full-text indexes and query predicates.

- Prepare SQL databases for vector search by storing embeddings and choosing distance metrics.
- Write vector search queries using VECTOR_DISTANCE and VECTOR_SEARCH functions.
- Combine full-text and vector search results using Reciprocal Rank Fusion (RRF).

Design and implement RAG with SQL

This module teaches you how to implement Retrieval Augmented Generation (RAG) using Azure SQL Database. You learn to identify appropriate RAG scenarios, prepare SQL results as LLM context, construct augmented prompts, and process model responses.

Introduction

Completed100 XP

- 3 minutes

Choose your preferred content format

Large language models can answer questions, summarize content, and generate text. But they only know what they learned during training. Ask about your company's products, customer orders, or compatible components, and the model has no useful answer. Retrieval Augmented Generation (RAG) solves this problem by giving the model access to your data at query time. Instead of hoping the model knows your product catalog, you retrieve the relevant products from your database and include them in the prompt. The model then generates a response grounded in actual, current information.

RAG keeps AI processing inside the database layer, where your data already lives. This approach avoids moving data between systems and lets you use Transact-SQL to control what context the model receives. The result is an application that can answer specific questions using real, up-to-date information from your tables.

Imagine a retail team building a customer support application on top of a product database. A customer asks "Which gloves work best for cold weather cycling?" The application needs to search the product catalog, find matching accessories, and generate a helpful response using an LLM. This workflow is RAG in action: retrieve relevant data, augment the prompt with that data, and generate a grounded

response. By building this workflow in SQL, the team keeps the retrieval step close to the data and avoids building a separate retrieval service.

After completing this module, you'll be able to:

- Identify when RAG is the right approach for your application.
- Convert SQL query results to JSON for Large Language Model (LLM) processing.
- Construct prompts that combine instructions with database context.
- Call LLM endpoints from SQL.
- Parse LLM responses and return answers to your application.

Note

We recognize that different people like to learn in different ways. You can choose to complete this module in video-based format or you can read the content as text and images. The text contains greater detail than the videos, so in some cases you might want to refer to it as supplemental material to the video presentation.

Identify RAG use cases and architecture

Completed100 XP

- 8 minutes

Choose your preferred content format

Large Language Models (LLM) don't know your organization. Your products, customers, policies. None of that exists in their training data. When a customer asks "Which pedals fit the mountain bike I bought last month?", the model can only guess.

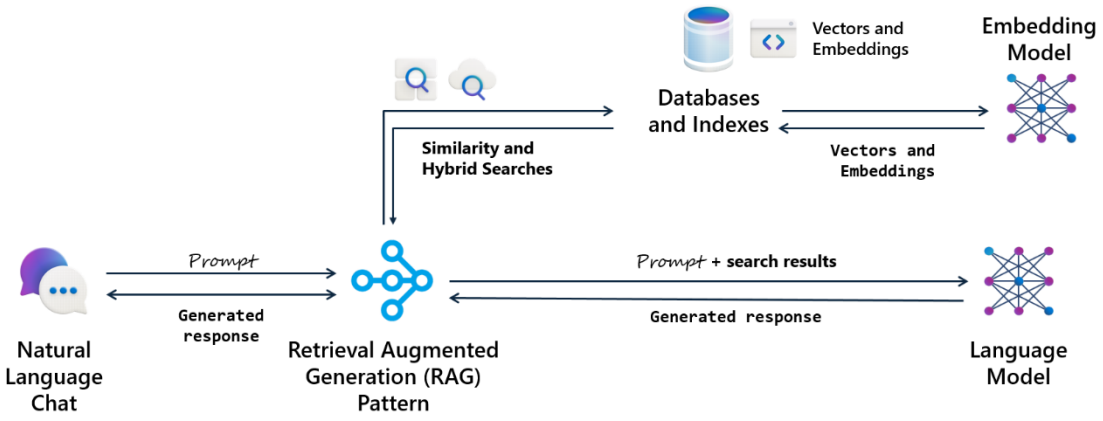
Retrieval Augmented Generation (RAG) fixes this issue by feeding the model your data when it needs to answer. You retrieve information from your database, add it to the prompt, and let the model respond based on what you provided. The model doesn't need to "know" your catalog because you give it what it needs for each question.

In previous modules, you learned to find relevant data using full-text, vector, and hybrid search. That step becomes the retrieval piece in RAG. Now you complete the pattern by augmenting prompts with retrieved data and generating grounded responses.

Understand how RAG works

RAG combines three steps. First, retrieve relevant data from your database. For the pedals question, find the customer's order, identify the bike model, and look up compatible components. Second, augment the prompt by adding that data as context. The model receives both the question and the information to answer it. Third, the model generates a response grounded in your data rather than its training.

The following diagram shows a RAG architecture with SQL Database. A natural language chat sends a prompt to a RAG orchestration layer, which runs similarity and hybrid searches against the database. The database exchanges vectors and embeddings with an embedding model. The orchestration layer forwards the prompt plus search results to a language model, which returns a grounded response.



This approach differs from fine-tuning, which retrains a model on your data. Fine-tuning bakes knowledge in permanently. RAG keeps data in your database and supplies only what each request needs. When specs change or new components arrive, answers reflect those changes immediately.

Recognize when RAG fits your scenario

RAG works when data changes frequently. Fine-tuned models can't keep pace with daily inventory updates, but RAG retrieves current data every time. RAG also fits when you need to trace answers. You control retrieval, so you know which records informed each response.

Privacy matters too. Fine-tuning sends data to the model provider for training. RAG keeps data in your database and sends only the context needed per request. For sensitive customer data or proprietary knowledge, RAG provides a cleaner separation.

Consider whether users ask domain-specific questions. General models handle common knowledge, but struggle with your products, processes, or history. RAG bridges that gap by grounding responses in your information.

Explore common RAG scenarios

RAG applies wherever users need answers that depend on your specific data. Here are some scenarios where SQL-based RAG adds value:

- **Customer support**: A customer asks "Can I return the bike I ordered three weeks ago?" The system retrieves their order history, checks your return policy, and generates an answer that cites the specific order and applicable policy terms.
- **Product configuration**: A shopper says "Build me a complete mountain biking setup under 2,000." RAG retrieves compatible frames, components, and accessories from your catalog, checks current inventory and pricing, and assembles a configuration that meets the budget.
- **Internal knowledge base**: An employee needs the expense reimbursement process for international travel. RAG searches policy documents stored in your database and summarizes the relevant steps with links to forms.
- **Sales enablement**: A sales rep preparing for a call says "Draft a follow-up email for this customer based on their recent purchases and open issues." RAG pulls order history and support tickets, then generates a personalized email grounded in the actual relationship context.
- **Technical documentation**: A developer asks "Generate a migration script to move our user table to the new schema." RAG searches your documentation and schema definitions, retrieves the relevant structures, and produces a script based on your actual table definitions.

Each scenario follows the same pattern: retrieve context from SQL, augment the prompt, and generate a grounded response. The difference is what you retrieve and how you structure it. There's an endless variety of use cases where RAG enhances LLM capabilities with your data.

Understand SQL's role in RAG architecture

Azure SQL Database and SQL Server 2025 participate in RAG without hosting models. Your database stores products, orders, documents, and embeddings. T-SQL handles retrieval using full-text, vector, or hybrid search. You format results as JSON, construct prompts, call the LLM via sp_invoke_external_rest_endpoint, and parse responses.

The LLM runs elsewhere, for example Azure OpenAI. Your T-SQL orchestrates the flow: retrieve data, build a prompt, call the model, extract the answer, return it. This flow keeps logic close to data and lets you add AI to existing apps by modifying T-SQL rather than rearchitecting.

Note

SQL Server 2025 and Azure SQL Database both support RAG. However, sp_invoke_external_rest_endpoint is enabled by default only in Azure SQL Database. In SQL Server 2025, enable it with sp_configure.

Key takeaways

RAG grounds model responses in your database by combining retrieval, augmentation, and generation. It fits when data changes often, when you need traceable answers, or when privacy requires keeping data in your control. SQL handles retrieval and orchestration; the model handles generation. Next, you prepare context as JSON, construct prompts, and call LLM endpoints.

Prepare retrieval context for augmentation

Completed100 XP

- 8 minutes

Choose your preferred content format

You now understand what RAG is and when to use it. In the following units, you learn how to implement each step of the RAG workflow in T-SQL.

Consider the following scenario: A customer asks "Which pedals fit the mountain bike I bought last month?" You query the database, find their order, identify the bike model, and locate compatible components. That step represents the "R" in RAG: retrieval. Let's retrieve and prepare that data for the language model. There are many ways to retrieve data from SQL, but for RAG, the goal is to provide the model with structured context it can actually use.

Convert relational data to JSON

Language models process text, not relational structures. If you pass raw query results, the model has no way to interpret column names, data types, or relationships. JSON solves this issue by preserving structure in a text format. Field names stay attached to values. Nested objects represent relationships. The model can read "ProductName": "Mountain-500" and reference that specific product in its response.

JSON also keeps things predictable. When the model returns an answer, you know the answer came from fields you explicitly provided. If you dumped unstructured

text instead, you'd have less control over what the model uses to formulate its response.

Format query results with FOR JSON

No need to build JSON strings by hand. Just add FOR JSON to the end of your SELECT and SQL does the work:

- FOR JSON AUTO formats output based on your query structure. Column names become field names automatically.

SQL

```
SELECT Name, ListPrice, Color
FROM Production.Product
WHERE ProductID = @ProductID
FOR JSON AUTO;
```

This query returns something like: [{"Name":"Mountain-500 Black, 48","ListPrice":564.99,"Color":"Black"}]

- FOR JSON PATH gives you explicit control over the JSON structure. You define the field names and nesting using column aliases.

SQL

```
SELECT
  Name AS 'product.name',
  ListPrice AS 'product.price',
  Size AS 'product.size'
FROM Production.Product
WHERE ProductID = @ProductID
FOR JSON PATH;
```

This query returns something like: [{"product":{"name":"Mountain-500 Black, 48","price":564.99,"size":"48"}}]

Control JSON output options

A few options let you shape the output:

- WITHOUT_ARRAY_WRAPPER removes the square brackets when you're retrieving a single record. This option is useful for RAG because you often retrieve one customer, one product, or one order at a time.

SQL

```
SELECT Name, Description, Size, Weight
FROM Production.Product
WHERE ProductID = @ProductID
FOR JSON PATH, WITHOUT_ARRAY_WRAPPER;
```

- INCLUDE_NULL_VALUES keeps null fields in the output instead of omitting them. Use this parameter when the absence of a value is meaningful.
- ROOT('name') wraps the entire output in a named root element, which can help the model understand what kind of data it's receiving.

Choose what to include

Not every column belongs in your context. For example, internal IDs, audit timestamps, and warehouse codes don't help the model answer customer questions. They just consume tokens and add noise.

For a product question, include the product name, description, specifications, and pricing. Skip the rowguid, modified date, and discontinued flag unless they're directly relevant.

Token limits matter too. Every token you send costs money and counts against the model's context window. If you're retrieving multiple products, keep each one lean. The model performs better with focused context than with everything you could possibly include.

Combine multiple sources

RAG context often comes from multiple tables. A pedal compatibility question needs product details, specifications, and related components. You might start by finding relevant products with vector search, then join other tables to build out the full picture.

Here's what that looks like in practice. First, convert the user's question into an embedding. Then use VECTOR_DISTANCE to find the closest matches, join the related tables, and format everything as JSON:

SQL

```
DECLARE @userQuestion NVARCHAR(1000) = 'Which pedals are compatible with the Mountain-500?';
DECLARE @questionVector VECTOR(1536);
DECLARE @context NVARCHAR(MAX);
```

```
-- Generate embedding for the question
SELECT @questionVector = AI_GENERATE_EMBEDDINGS(@userQuestion USE
MODEL my_embedding_model);


-- Find relevant products and format as JSON
SET @context = (
  SELECT TOP 3
    p.Name AS ProductName,
    p.Color,
    p.Size,
    pc.Name AS Category,
    pm.Name AS Model
  FROM Production.Product p
  INNER JOIN Production.ProductSubcategory ps ON p.ProductSubcategoryID =
ps.ProductSubcategoryID
  INNER JOIN Production.ProductCategory pc ON ps.ProductCategoryID =
pc.ProductCategoryID
  INNER JOIN Production.ProductModel pm ON p.ProductModelID =
pm.ProductModelID
  ORDER BY VECTOR_DISTANCE('cosine', p.DescriptionVector, @questionVector)
  FOR JSON PATH
);
```

The variable @context now holds a JSON string ready for the prompt. This JSON string is your retrieval result formatted for augmentation.

Key takeaways

The goal is to give the language model structured context it can actually use. JSON preserves the meaning of your data while keeping it readable as text. Use FOR JSON AUTO when you want quick results and FOR JSON PATH when you need control over field names and nesting. Add WITHOUT_ARRAY_WRAPPER for single-

record queries. Keep your JSON lean by including only the columns the model needs. Less noise means better answers and lower token costs.

Augment prompts with database context

Completed100 XP

- 8 minutes

Choose your preferred content format

Retrieval gets you the data. But a JSON blob on its own doesn't answer anyone's question. If a customer asks about bike pedal compatibility, you've got the product info from your database, but now you need to tell the language model what to do with it. That step represents the "A" in RAG: augmentation.

Understand the chat message structure

Different models have different APIs, but most chat-based models follow a similar pattern. We use Azure OpenAI as our example, but the concepts apply broadly.

Azure OpenAI chat models expect messages with roles. The **system** role defines how the assistant should behave. The **user** role contains the original question and the database context into which the model should ground its answer. Optionally, an **assistant** role can hold previous responses in a multi-turn conversation. Here's a simplified example of a prompt structure:

JSON

```
{
 "messages": [
  {
   "role": "system",
   "content": "You are a helpful assistant that answers questions about products. Use only the provided context to answer. If the context doesn't contain enough information, say so."
  },
  {
   "role": "user",
   "content": "Context: {retrieved_data}\n\nQuestion: {user_question}"
  }
```

]

}

In this example, {retrieved_data} would be the JSON you built from your product query, and {user_question} would be something like "Which pedals are compatible with the Mountain-500?"

The **system** message sets ground rules. The **user** message combines your database context with the actual question.

Build prompts in T-SQL

You could build this JSON in your application code, but it's often convenient to keep it all in the database. Your retrieval query, your context formatting, and your prompt construction live together, and T-SQL's JSON_OBJECT and JSON_ARRAY functions handle the JSON formatting.

SQL

```
DECLARE @userQuestion NVARCHAR(1000) = 'Which pedals are compatible with the Mountain-500?';


-- @context contains retrieved product data as JSON from the retrieval step


DECLARE @systemMessage NVARCHAR(MAX) = 'You are a customer service assistant for Adventure Works. Answer questions using only the provided product information. Be concise and helpful.';


DECLARE @userMessage NVARCHAR(MAX) = 'Product information: ' + @context + CHAR(10) + CHAR(10) + 'Customer question: ' + @userQuestion;


DECLARE @payload NVARCHAR(MAX) = JSON_OBJECT(
  'messages': JSON_ARRAY(
    JSON_OBJECT('role': 'system', 'content': @systemMessage),
    JSON_OBJECT('role': 'user', 'content': @userMessage)
  ),
  'max_tokens': 500,
```

'temperature': 0.7

);

The @payload variable now contains a complete request body ready for the Azure OpenAI API.

Ground the model in your data

Grounding tells the model to use your data as the source of truth. Without it, the model might rely on its training data, which could be outdated or wrong for your domain. A customer asking about Adventure Works bike warranties shouldn't get generic warranty information from the internet.

Good grounding instructions set scope ("use only the provided product data"), encourage honesty ("if you don't have enough information, say so"), and can specify format ("keep responses under 100 words"). Here's an example for the *system* role:

SQL

DECLARE @systemMessage NVARCHAR(MAX) =

'You are an Adventure Works product expert. Follow these rules:

1. Answer only using the product information provided

2. Do not invent features or specifications

3. If information is missing, tell the customer you''ll need to check

4. Keep responses under 100 words

5. Suggest related products when relevant';

These instructions help the model stay focused and provide useful, accurate answers.

Control model behavior

The request payload includes a couple of parameters that affect how the model generates responses:

- max_tokens limits response length. For detailed product answers, 500 to 1,000 works well.
- temperature controls creativity on a scale from 0 to 2. Lower values (0.3 to 0.5) produce more consistent, factual responses. Higher values let the model get more creative, which you usually don't want for RAG.

Controlling the tokens and temperature helps ensure the model behaves predictably when grounded in your data.

Construct the complete payload

Here's how you might build a RAG prompt for a product question using Adventure Works tables:

SQL

```
DECLARE @userQuestion NVARCHAR(1000) = 'Which pedals are compatible with the Mountain-500?';

DECLARE @questionVector VECTOR(1536);

DECLARE @context NVARCHAR(MAX);

DECLARE @payload NVARCHAR(MAX);


-- Generate embedding for the question

SELECT @questionVector = AI_GENERATE_EMBEDDINGS(@userQuestion USE MODEL my_embedding_model);


-- Get product context using vector search

SET @context = (

  SELECT TOP 3

    p.Name AS ProductName,

    p.Color,

    p.Size,

    pc.Name AS Category,

    pm.Name AS Model

  FROM Production.Product p

  INNER JOIN Production.ProductSubcategory ps ON p.ProductSubcategoryID = ps.ProductSubcategoryID

  INNER JOIN Production.ProductCategory pc ON ps.ProductCategoryID = pc.ProductCategoryID
```

```
  INNER JOIN Production.ProductModel pm ON p.ProductModelID =
pm.ProductModelID
  ORDER BY VECTOR_DISTANCE('cosine', p.DescriptionVector, @questionVector)
  FOR JSON PATH
);

-- Build the prompt
SET @payload = JSON_OBJECT(
  'messages': JSON_ARRAY(
    JSON_OBJECT(
      'role': 'system',
      'content': 'You are an Adventure Works product assistant. Use only the
provided product data to answer questions.'
    ),
    JSON_OBJECT(
      'role': 'user',
      'content': 'Product data: ' + @context + CHAR(10) + 'Question: ' +
@userQuestion
    )
  ),
  'max_tokens': 500,
  'temperature': 0.5
);
```

The @payload variable now contains everything the model needs: your grounding instructions, the retrieved product data, and the customer's question. All you need to do is send it to the model endpoint and handle the response.

Key takeaways

The prompt is where retrieval becomes useful. Your JSON context means nothing unless you tell the model how to use it. Set clear grounding rules in the system message so the model sticks to your data. Keep temperature low for factual

answers. Use JSON_OBJECT and JSON_ARRAY to build valid JSON payloads directly in T-SQL.

Generate and process RAG responses

Completed100 XP

- 10 minutes

Choose your preferred content format

A customer wants to know which pedals fit their Mountain-500 bike. You use vector search to find the relevant products, formatted them as JSON, and built a prompt with grounding instructions. The retrieval and augmentation steps are done. Now comes the "G" in RAG: generation. This step is where you send everything to a language model and get back an answer.

Think about it like this: you gathered all the ingredients (retrieved data), prepared the recipe (augmented prompt), and now you're putting it in the oven (calling the model) to bake the final dish (the response). The model uses the context you provided to generate a grounded answer.

Call the model from SQL

You might think this step requires leaving T-SQL and writing application code. But SQL Server and Azure SQL Database can call REST endpoints directly using sp_invoke_external_rest_endpoint. This stored procedure sends HTTPS requests to external services and returns the response. The stored procedure is enabled by default in Azure SQL Database and can be enabled in SQL Server 2025 using sp_configure.

Here's the basic pattern:

SQL

```
DECLARE @response NVARCHAR(MAX);

DECLARE @returnValue INT;


EXECUTE @returnValue = sp_invoke_external_rest_endpoint

  @url =
N'https://<endpoint>.openai.azure.com/openai/deployments/<model>/chat/completions?api-version=<api-version>',

  @method = 'POST',

  @payload = @payload,
```

```
  @credential = [https://<endpoint>.openai.azure.com],
  @response = @response OUTPUT;
```

The @url parameter points to your Azure OpenAI deployment endpoint. The @method is usually 'POST' for generation requests. The @payload contains the JSON prompt you previously built. The @credential references a database scoped credential that holds your authentication details. The @response output parameter captures the model's response.

When you call the deployment endpoint of your model with your augmented prompt, the model processes it and returns the result. The stored procedure returns 0 when the HTTP call succeeds with a 2xx status code, or the actual HTTP status code when it fails.

Authenticate with the endpoint

The @credential parameter references a database scoped credential that holds your authentication details. You set up these credentials when you create an external model, using either managed identity or an API key. The same credential works for both external model calls and direct REST endpoint calls with sp_invoke_external_rest_endpoint.

Extract the answer from the response

When the model finishes processing your prompt, sp_invoke_external_rest_endpoint returns the result wrapped in a standard envelope. The stored procedure adds metadata about the HTTP transaction, then nests the actual API response inside a result property:

JSON


```
{
 "response": {
  "status": {
   "http": {
    "code": 200,
    "description": "OK"
   }
  }
 },
```

```
 "result": {
  "choices": [
   {
    "message": {
     "role": "assistant",
     "content": "The Mountain-500 is compatible with several pedal options..."
    }
   }
  ]
 }
}
```


The answer you want lives at $.result.choices[0].message.content. To get there, use JSON_VALUE, which extracts scalar values from JSON:

SQL

```
IF @returnValue = 0
BEGIN
  DECLARE @answer NVARCHAR(MAX);
  SET @answer = JSON_VALUE(@response,
'$.result.choices[0].message.content');
  SELECT @answer AS AssistantResponse;
END
ELSE
BEGIN
  SELECT
    @returnValue AS HttpStatus,
    JSON_VALUE(@response, '$.response.status.http.description') AS
ErrorDescription;
END
```

If you ever need to extract a JSON object or array rather than a single value, use JSON_QUERY instead. For most RAG scenarios, JSON_VALUE on the message content is all you need.

Manage errors and retries

Calling an external service from a database can introduces failure modes you don't encounter with local queries. The endpoint might be temporarily unavailable, your request might get rate-limited, or authentication could fail. Your SQL queries need to handle these conditions gracefully.

The return value from sp_invoke_external_rest_endpoint tells you what happened. A 0 means the HTTP call succeeded with a 2xx status. For failures, the return value is the HTTP status code itself. For example, a 429 status means the service is throttling your requests. A 401 or 403 points to credential problems:

SQL

```
IF @returnValue = 0

   SET @answer = JSON_VALUE(@response,
'$.result.choices[0].message.content');

ELSE IF @returnValue = 429

   RAISERROR('Service is busy. Try again later.', 16, 1);

ELSE IF @returnValue = 401 OR @returnValue = 403

   RAISERROR('Authentication failed. Check your credential configuration.', 16, 1);

ELSE

BEGIN

   DECLARE @errorMsg NVARCHAR(500) = 'API call failed with status ' +
CAST(@returnValue AS NVARCHAR(10));

   RAISERROR(@errorMsg, 16, 1);

END
```

For transient failures like timeouts or temporary service unavailability, the stored procedure can retry automatically. Add the @retry_count parameter and in this example, it attempts the call up to three times before giving up:

SQL

```
EXECUTE @returnValue = sp_invoke_external_rest_endpoint

   @url = @url,
```

```
  @payload = @payload,
  @credential = @credentialName,
  @retry_count = 3,
  @response = @response OUTPUT;
```

This parameter handles the common case where a request fails once but succeeds on the next attempt.

Build a complete RAG stored procedure

You now know each piece of the RAG puzzle. Let's put them together into a single stored procedure that a customer support application could call. The procedure accepts a natural language question and returns an answer grounded in your product data.

Here's what the procedure does:

1. Converts the question into an embedding so you can compare it against your product descriptions.
2. Finds the most relevant products using vector distance to compare the question embedding against each product's precomputed description embedding.
3. Builds the prompt with a system message that tells the model to stick to the provided data, and a user message that combines the retrieved products with the original question.
4. Sends everything to Azure OpenAI.
5. Extracts the answer from the response and return it to the caller.

SQL

```
CREATE PROCEDURE dbo.AskProductQuestion
  @Question NVARCHAR(1000),
  @Answer NVARCHAR(MAX) OUTPUT
AS
BEGIN
  SET NOCOUNT ON;

  DECLARE @questionVector VECTOR(1536);
```

```
  DECLARE @context NVARCHAR(MAX);
  DECLARE @payload NVARCHAR(MAX);
  DECLARE @response NVARCHAR(MAX);
  DECLARE @returnValue INT;

  -- Step 1:Convert question to embedding
  SELECT @questionVector = AI_GENERATE_EMBEDDINGS(@Question USE
MODEL my_embedding_model);

  -- Step 2: Retrieve relevant products using vector search
  SET @context = (
    SELECT TOP 3
      p.Name AS ProductName,
      p.Color,
      p.Size,
      pm.Name AS Model
    FROM Production.Product p
    INNER JOIN Production.ProductModel pm ON p.ProductModelID =
pm.ProductModelID
    ORDER BY VECTOR_DISTANCE('cosine', p.DescriptionVector,
@questionVector)
    FOR JSON PATH
  );

  -- Step 3: Build augmented prompt
  SET @payload = JSON_OBJECT(
    'messages': JSON_ARRAY(
      JSON_OBJECT('role': 'system', 'content': 'You are an Adventure Works product
assistant. Answer questions using only the provided product data.'),
```

```
        JSON_OBJECT('role': 'user', 'content': 'Products: ' + @context + ' Question: ' +
@Question)
    ),
    'max_tokens': 500,
    'temperature': 0.5
  );

  -- Step 4: Call the model
  EXECUTE @returnValue = sp_invoke_external_rest_endpoint
    @url = N'https://adventureworks-
openai.openai.azure.com/openai/deployments/gpt-5.2/chat/completions?api-
version=2024-10-21',
    @method = 'POST',
    @payload = @payload,
    @credential = [https://adventureworks-openai.openai.azure.com],
    @response = @response OUTPUT;

  -- Extract and return the answer
  IF @returnValue = 0
    SET @Answer = JSON_VALUE(@response,
'$.result.choices[0].message.content');
  ELSE
    SET @Answer = 'Unable to process your question. Please try again.';
END;
```

You completed the RAG pipeline in T-SQL. A customer asks "Which pedals work with the Mountain-500?" Your procedure converts that question to a vector, finds the most relevant products, sends them to the model with grounding instructions, and returns an answer based on your actual inventory. No middleware, no external application code, just SQL calling AI and returning results.

Key takeaways

The *generation* step is where your RAG pipeline delivers value. You send the augmented prompt to Azure OpenAI using sp_invoke_external_rest_endpoint, which handles the HTTP communication without leaving T-SQL. Authentication uses the same database scoped credentials you set up when creating external models. When the response comes back, JSON_VALUE pulls out the assistant's answer. Build in error handling because network calls fail in ways that local queries don't. With all three RAG steps running in T-SQL, your database becomes more than storage. It becomes an intelligent service that answers questions using your data.

Summary

Completed100 XP

- 3 minutes

Retrieval Augmented Generation connects your database to the capabilities of large language models. Instead of relying on a model's training data, you provide current, relevant information from your own tables.

The entire RAG pattern executes in T-SQL. Your database orchestrates the flow: search, format, prompt, call, parse. You can add AI capabilities to existing applications by modifying stored procedures, without rearchitecting your application stack.

In this module, you learned how to:

- **Identify RAG use cases**: Recognize scenarios where grounding Large Language Model (LLM) responses in database content improves accuracy and relevance
- **Prepare context from SQL**: Use FOR JSON to convert query results into text that LLMs can process effectively
- **Construct augmented prompts**: Build request payloads that combine system instructions, retrieved context, and user questions
- **Execute the RAG pipeline**: Call Azure OpenAI endpoints using sp_invoke_external_rest_endpoint and parse the responses